PEARSON

Programming Techniques through C

A Beginner's Companion

M. G. Venkateshmurthy

Programming Techniques through C

A Beginner's Companion

Ths page is intentionally left blank

Programming Techniques through C

A Beginner's Companion

M. G. Venkateshmurthy
Vice-Principal
Professor and Head
Department of Computer Science and Engineering
Malnad College of Engineering
Hassan (Karnataka)

Pearson Education

Copyright © 2002 by Pearson Education (Singapore) Pte. Ltd.

All rights reserved. No part of this publication may be reproduced, stored in a database or retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher.

ISBN 81-7808-726-X

First Indian Print, 2002

Published by Pearson Education (Singapore) Pte. Ltd., Indian Branch, 482 F.I.E. Patparganj, Delhi 110 092, India

Laser typeset by Bharati Composers, Rohini, Delhi
Printed in India at Nutech Photolithographers, Delhi

Preface

Since the introduction of the PC's (Personal Computers) in the fall of 1981, computers have found their applications in all walks of life. At present they no more remain the tools of only scientists or engineers. They are being used to solve problems of varying complexities by every one. A problem could be as simple as 'finding out the area of a triangle, given its three sides' or 'converting the temperature given in degrees Fahrenheit into temperature in degrees Celsius'. Also, a problem could be as complex as 'controlling the trajectory of a missile' or 'in weather forecasting'. In any case, *the digital computer does not figure out its own solution to problems, but must be told exactly how to solve any given problem as well as how to make all decisions*. Preparing a *list of instructions*, which tells the computer how to perform its calculations and take decisions, is called *programming*. Therefore, in order to write a program one has to have sufficient 'knowledge' about how to solve the problem. In other words, one should know the 'procedure' for solving the problem otherwise. Here, *otherwise* means without using computers i.e. may be using a pen and a piece of paper. Thus knowing the procedure means knowing the **method** of solving the problem.

After having knowledge about **the method** or **the technique**, to be used to solve a problem, *the first step* is to determine the *sequence of operations* required to solve the problem. These sequences of operations are documented either by *listing the steps involved sequentially* using some natural language like English or represented *diagrammatically* as flow diagrams or **flowcharts**. As the *visual approach* makes it easy for beginners to follow the method or procedure of solving problems, flowcharts have been developed for all the examples discussed in this book. The *next step* in solving a problem using computers is to **code the method** using any of the computer languages like Java, Fortran, C, Pascal etc. Obviously, to solve problems, of whatever complexity using computers, one has to learn a *specific computer language* in order to *code* the problem solving procedure. As such,

beginners generally concentrate on learning some computer language initially. With an eagerness to pick up the syntax and semantics of the language, *less attention will be given to pick up the techniques of problem solving*. This is why, in this book, *I am concentrating on the problem solving procedures and techniques* rather than getting trapped in the *vicious circle of just learning a language*. Nevertheless, complete C programs have been presented to all the examples discussed in this book along with the *just required C details*, in the form of **C tips**. However, in latter chapters, as more details of C are required, a section called **... beyond** that discusses more C specific details has been included. Some examples, in these latter chapters, deal with *techniques of C handling* for problem solving rather than general problem solving techniques. In any case, the reader is advised to enter the code into the machine directly, without bothering much about the intricacies of the programming language, compile and run the programs. By the time a reader comes to the end of the book, I am sure that, he/she not only will be really capable to think in terms of some plausible problem solving techniques but also will be quite comfortable in coding using C. If necessary any standard textbook on ANSI C may be referred to for more details of the C language.

For many problems, several possible methods of solving them do exist. Some are more efficient than the others, some are more tolerant of unexpected data than others are and some are easier to code in a computer language or are less prone to programming errors. Only the methods, which are easily comprehendible to the beginners, have been considered in this book. No much concern has been shown towards really efficient techniques.

As the target readers of this book are beginners, most of the material presented here is elementary in nature. The material has been organized into various chapters of increasing complexity. Every chapter begins with a brief introduction that gives an idea about the problems discussed in that particular chapter and ends with a list of problems in the form of exercises.

As the first step in problem solving is to develop an algorithm or a flowchart that encapsulates the procedure of solving the problem, algorithms and flowcharts have been introduced in Chapter 1. Also the necessity and importance of dividing a problem into sub-problems and solving these sub-problems independently which helps a problem solver in obtaining the final solution of the problem is explained in this chapter.

Chapter 2 discusses the solving of very simple problems. This chapter deals with the technique of taking data values into the computer, taking out the results and the use of very popular and widely used library functions. All the steps or instructions of the programs discussed in this chapter are executed strictly sequentially. Problems that need basic *two ways* and *three way decision making* are discussed in Chapter 3.

Quite often we come across problems that need *repeated execution* of certain steps during a problem solving process. Chapter 4 discusses the techniques for *repeated execution* of certain steps or instructions using what are known as *iterative techniques*. Problems that need *multiway decision making* are dealt with in Chapter 5.

Many applications need the processing of a *collection of data items* of similar characteristics, called *arrays*. Techniques for handling arrays are discussed in Chapter 6.

Text processing is also one of the important activities that are performed with computers. Chapter 7 discusses the techniques for handling texts that are made up of characters and character strings.

As problems grow in size it is natural to divide the problem in to sub-problems and to first solve these sub-problems independently and then solve the complete problem by putting the solutions of these sub-problems together. Techniques for handling sub-problems (called *functions in C*) and tailoring them into a single unit (called *main( ) in C*) are discussed in Chapter 8.

Chapters 9 and 10 deal with *pointers* and *structures* respectively at an introductory level. Actually these topics are slightly advanced in nature and may not be easily comprehendible by beginners. However, these topics have been included for the sake of completeness and as an indicator for further study.

This book is for anyone who has *plus-two* level knowledge in mathematics (except the calculus) and is interested in using computers or keen in pursuing college or diploma studies.

M. G. Venkateshmurthy

This page is intentionally left blank

Contents

Preface *v*

1. Algorithms and Flowcharts **1**

1.1 Algorithms *1*
1.2 Flowcharts 2
1.3 Divide and Conquer Strategy *4*

2. Basic Techniques **6**

2.1 Sum of Two Given Numbers *6*
2.2 Swapping the Contents of Two Variables *8*
2.3 Simple Interest Calculation *10*
2.4 Computing the Area of a Circle Given its Radius *11*
2.5 Computing the Area of a Triangle Given all its Sides *13*
2.6 Extracting the Unit's Digit of a Given Integer Number *14*
Exercises *16*

3. Decision Making **17**

3.1 Largest of Two Given Numbers *18*
3.2 Determining Whether a Given Integer Number is Positive or Negative *20*
3.3 Determining Whether a Given Integer Number is Odd or Even *21*
3.4 Largest of Given Three Numbers *23*
3.5 To Find Out Whether a Given Year is a Leap Year or Not *25*
3.6 Solving a Given Quadratic Equation *27*
Exercises *30*

4. Looping Techniques 31

4.1 Sum of First *N* Natural Numbers 31
4.2 Sum of the Squares of all Integers From 1 to *N* *33*
4.3 Factorial of a Given Integer *35*
4.4 Finding the Sum of Odd Numbers and Sum of Even Numbers from 1 to *N* *38*
4.5 Generation of Required Number of Terms of the Fibonacci Sequence *40*
4.6 Finding the GCD and LCM of Given Numbers *42*
4.7 Sum of all the Digits of a Given Integer Number *44*
4.8 Reversing a Number and to Check Whether it is a Palindrome or Not *46*
4.9 To Find out Whether a Given Integer Number is Prime or Not *50*
4.10 Prime Numbers Between 1 and *N* *52*
4.11 Sum of First *N* Terms in the Expansion of Sine Series *55*
4.12 Computing the Area of Circles Given Their Radii *58*
Exercises *61*

5. Multi-Way Decision Making 62

5.1 Solving Given Quadratic Equation Using Multi-way Decision Making Facility *62*
5.2 Awarding Grades Based on Marks Scored in an Examination *67*
5.3 A Simple Calculator *69*
5.4 Selecting an Operation Based on a Menu *72*
Exercises *75*

6. Arrays 76

6.1 Reading in an Array and to Output the Same *77*
6.2 Biggest Among Given 'N' Integer Numbers *80*
6.3 The Maximum and Minimum Among Given *N* Integer Numbers *82*
6.4 Average of a Given Set of *N* Numbers *84*
6.5 Given *N* Integers (Zero, +Ve, –Ve) to Find Out the Sum of +Ve Numbers, –Ve Numbers and the Average of all Numbers *87*
6.6 Mean, Variance and Standard Deviation of a Given Set of Numbers *89*
6.7 Classification of the Elements of a Given Array into an Array Consisting of only Odd Elements and Another Array Consisting of only Even Elements *92*
6.8 Sorting *N* Numbers in Ascending Order using Bubble Sort *95*

6.9 Evaluating a Polynomial *98*
6.10 Reading and Printing out a Given Matrix *101*
6.11 To Compute the Sum/Difference of Given Matrices *104*
6.12 Trace of a Given Matrix *107*
6.13 Norm of a Given Matrix *110*
6.14 Transpose of a Given Matrix *112*
6.15 Product of Two Given Matrices *114*
6.16 Searching a Given Element in a Given List of Integer Elements (Linear Search Algorithm) *120*
6.17 Searching for a Given Element in an Array Using Binary Search Method *123*
6.18 Sorting a Given Array of Elements Using Selection Sort *128*
Exercises *133*

7. Characters and String Handling 135

7.1 Finding the Length of A Given String *136*
7.2 Determining Whether a Given String is Palindrome or Not *138*
7.3 Reversing a Given String *141*
7.4 Copying the Contents of a Given String into Another String *144*
7.5 Concatenating Two Given Strings *147*
7.6 Converting the Uppercase Alphabets into Lowercase Alphabets in a Given String and Vice–Versa *151*
7.7 Counting the Number of Vowels and Consonants in a Given Line *154*
7.8 Sorting a Given Set of Names *156*
7.9 Counting the Number of Characters in an Input *159*
7.10 Counting the Number of Words in a Given Line *160*
7.11 Counting the Number of Lines in a Given Text *163*
Exercises *165*

8. Solving with Modules 166

8.1 Function to Print a Line Made up of Only Hash (#) Characters *167*
8.2 Function to Multiply Two Given Integer Numbers *170*
8.3 Function to Compute the GCD of Given Three Numbers *174*
8.4 Function to Sum First *N* Terms of Sine Series Expansion and to Tabulate Sine Values for Different Angles *177*
8.5 Functions to Compute Mean and Variance and Hence to Compute Standard Deviation of Given Set of Numbers *180*
8.6 Functions to Read and Write one Dimensional Array *184*

8.7 Function to Sort an Array Using Bubble Sort Method *187*
8.8 Function to Search for a Required Element Using Binary Search Method *190*
8.9 Functions to Sort a Given Unsorted Array Using Selection Sort *194*
8.10 Functions to Read, Write and Compute Trace of Given Matrix *197*
8.11 Functions to Compute Sum of Each Row, Sum of Each Column and Sum of All Elements of Given Matrix *202*
8.12 Functions to Read, Write and Compute Product of Two Given Matrices *207*
8.13 Program to Search for Required Element in a Given Unsorted Array Using Functions on Separate File *211*
8.14 Understanding the Meaning of Local and Global Variables *214*
8.15 Function to Swap Contents of Two Variables—The Wrong One *217*
Exercises 220

9. Pointers 221

9.1 Understanding Prelims of Pointers *223*
9.2 Understanding Pointer Expressions, Pointer-to-Pointer and Null Pointer *226*
9.3 Finding Sum of all Elements of Array Using Pointers *229*
9.4 Function to Swap Contents of Two Variables Using Pointers *233*
9.5 Solving Quadratic Equation—Using Functions and Pointers *238*
Exercises 243

10. Structures 244

10.1 Defining a Structure, Declaring, Structure Variables, Assigning Values to Structure Members and Printing Out Structure Member Values *249*
10.2 Array of Structure Variables: Listing Names of Students Who Have Scored More Than 60% of Total Marks in Three Subjects Using Structure Variables *251*
10.3 Sum of Two Complex Numbers–Passing Structure Variable To Function *256*
10.4 Array of Structure Variables with a Structure as Member of Another Structure *259*
Exercises 263

Index 265

CHAPTER 1

Algorithms and Flowcharts

1.1 ALGORITHMS

In order to solve a problem using a computer, we must *first express* the procedure used to solve the problem in terms of computer instructions. A computer **program** is just a collection of these instructions, that are necessary to solve a specific problem. The approach or method that is used to solve the problem is known as an *algorithm*. For example, if we wish to develop a program that tests whether a number is +ve or –ve, then the set of statements which solves the problem becomes the program. The method that is used to test if the number is +ve or –ve is the algorithm. Normally, to develop a program to solve a particular problem, we first express the solution to the problem in terms of an algorithm and then translate or code this into a program using certain programming language. So the algorithm for solving the +ve or –ve problem might be expressed as follows.

"Check whether the number is less than zero or not.

If the number is less than zero, then the number is –ve; otherwise the number is +ve"

From the above discussion an algorithm can be defined as the set of unambiguous steps precisely described in broken English or some pseudo code to solve a problem. *Though generally a set of pseudocode or broken English steps written sequentially to solve a problem is termed as an algorithm, any documentation that clearly holds the procedure of problem solving, including pictorial representations, are termed algorithms*. An algorithm must be *definite*, *complete* and *finite*. By *definite*, we mean that each step of the algorithm must be precisely defined such that there is no ambiguity or contradiction. *Finite* means the algorithm must terminate in some definite number of steps. In otherwords it should not go on forever. (Some programs known as *system programs* are designed to go on forever. These programs are developed for the purpose of helping computer operations and

operators. Such programs have nothing to do with problem solving using computers). *Complete* means the algorithm must work successfully in solving all the problems of a particular type for which it is designed. Another very important property of the algorithm to be considered seriously at final stages of program implementation is efficiency. One of the better methods of learning any new technique by a rank outsider i.e. beginner, is to get acquainted with as much *know-how* as one needs to get going, without worrying about the worst possible cases of the problem as well as efficient implementations.

1.2 FLOWCHARTS

As computers are used to solve problems of varying complexities, solving certain problems need larger algorithms to be developed. As already mentioned, generally algorithms are written in some textual form. Reading these textual form algorithms, understanding and analyzing them is practically some what difficult and irk-some as its size increases. For example, think of an architect explaining the plan of a proposed building complex to be built in words as well as actually drawing a plan and presenting it. Also, think of a TV reporter reporting about the movement of air pressure, heat wave etc., without maps being displayed as well as reporting the same weather conditions along with the map. Definitely it is the latter one which gives better understanding and hence preferred by many. Also, our experience has shown that, given a choice, one would prefer to have a look at pictures rather than reading the details available in the form of text. In other words, pictorial representations are, in general, more appealing, easily readable as well as understandable. Thus one may conclude that, for nontrivial problems, pictorial representations of the sequence of steps involved in solving a problem are better suited. These pictorial representations are called **flow charts**. All the operations to be performed and all the paths of processing to be followed while solving problem are indicated in a flow chart.

Flowcharts have several advantages. As they are pictorial representations, they could be easily read as well as the inbuilt logic could be followed. It is not necessary that a problem formulator be necessarily a programmer himself. In such cases it acts as a link between the person who formulates the problem and the programmer. Further, modifications, if necessary to a programme could be affected very easily by going through its flowchart.

1.2.1 Flowchart Symbols

On flow charts different actions are represented using different geometric shapes. These are called *flowchart symbols*. Figures below show some of these commonly used flow chart symbols.

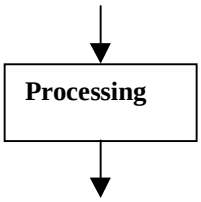


Fig. 1.1

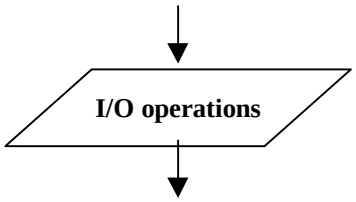


Fig. 1.2

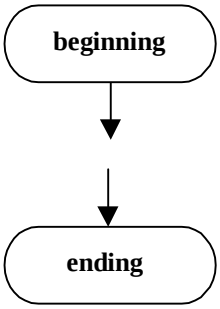


Fig. 1.3

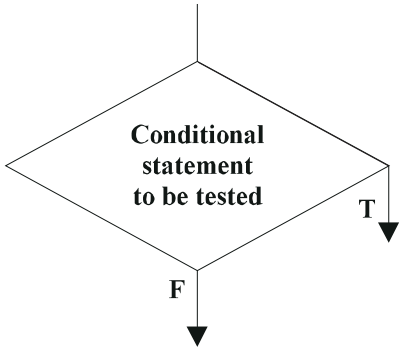


Fig. 1.4

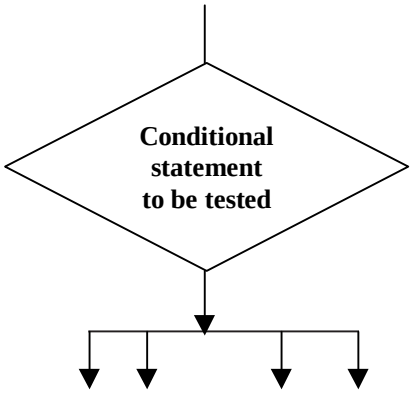


Fig. 1.5

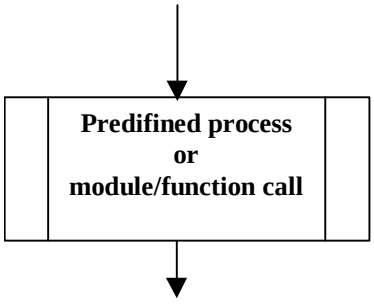


Fig. 1.6



Fig. 1.7

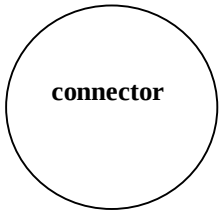


Fig. 1.8

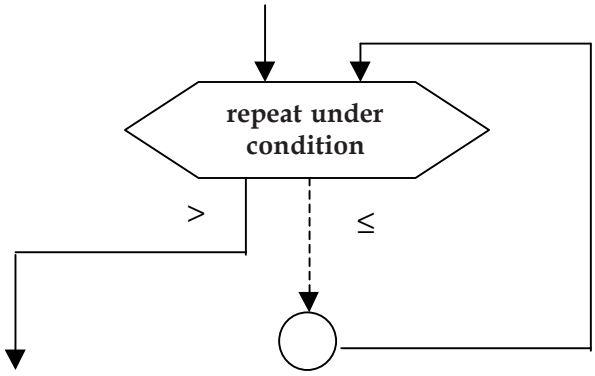


Fig. 1.9

The *processing* of a statement is shown using a rectangular box. This box may contain one or more assignment statements (Fig 1.1). The input and/or output operations are shown using a parallelogram symbol (Fig 1.2). The beginning and/or ending of an algorithm are shown by an oval shaped symbol (Fig 1.3). A diamond shaped symbol is used to represent decision making (Fig 1.4). Generally this is used to indicate two way decision making (i.e. in decision making using statements like **if** statements).

There exist many situations where decisions taken could lead to *multiple branching* (rather than simple two way decision making with *true* and *false* answers). Many computer languages do provide facilities for such multiple-branching. For example, ***computed goto*** of Fortran, ***switch*** of C constructs allow multiple-branching. A diamond box, supplemented with a system of flow lines represents multiple decision-making (Fig 1.5).

The use of a predefined process i.e. a function or a module call is indicated by using a rectangle with two vertical bars inside it (Fig 1.6).

A line with an arrow head is used as *flow line*. A flow line indicates the order or sequence of actions in a program. They connect various other symbols in a flow chart.

A small circle is used as a connector. This connector may be just used to indicate *continuation* of a process or to indicate connectivity to some other part of the program, either on the same page or on a different page.

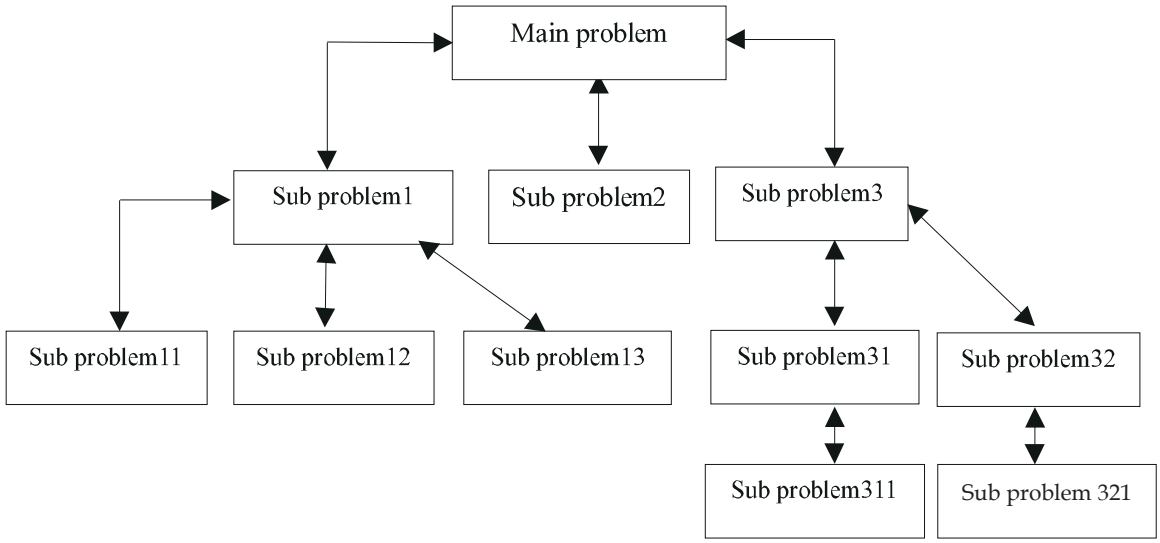
Repeated execution of certain statements for certain number of times or under certain conditions is quite a common feature. Such executions are pictorially shown using an hexagon shape symbol (Fig 1.9). The *initial value*, *test condition* and *step size* are shown inside the figure. The flowline leaving the box at the lower right (labelled ≤) indicates the repeated execution of statements following it. Statement like *for, while* and *do-while* are shown with this symbol.

Generally, declaration of variables, unless otherwise they have initial values, are not shown on a flowchart.

1.3 DIVIDE AND CONQUER STRATEGY

The discussion hitherto concentrates on developing algorithms to solve a problem on hand completely. As already mentioned one can solve problems of varying complexities using computers. A problem is considered as *big* if the complexity of the problem results in larger algorithms, which lead to large amount of coding in a single segment. It is practically very difficult to test and debug, if necessary, larger algorithms/code segments. As such it is customary as well as natural to divide larger problems to smaller, independent subproblems, as shown in the figure below, and then solve these subproblems independently and to put the results of all the subproblems together to get the final solution. Generally there will be a *central program* segment which 'puts together' all the independent small subprogram segments. Such central program segments are

called *main* programs or *main* functions. *When one starts to solve a big problem, the actual action starts from the main and ends in the main.*



As seen from the figure above a subproblem can be further subdivided into smaller problem. The bi-directional lines connecting various subproblems indicate the relation, flow of data and flow of results among various subproblems. This strategy is popularly referred to as *divide and conquer* strategy. One will study many such different problem solving techniques as he advances into the field of computer science. Infact, almost all computer languages do have facilities to develop program codes to these subproblems. However, they are referred to as *subprograms* in FORTRAN, as *functions* in C, as *procedures* in PASCAL and so on.

CHAPTER 2

Basic Techniques

Undoubtedly, present day computers are very powerful tools, which are being used to solve a **variety** of problems of varying complexities. Yet they cannot solve any problem by themselves *however trivial* the problem may be. It is the responsibility of the computer user to provide all necessary information to the computers for it to work for us. For example, even to add just two numbers the user has to provide the necessary procedure (in the form of a *program*) as well as *supply* (input) the necessary data values and *get the results* (output) of the addition out of the machine. This chapter discusses few very straightforward examples by studying which the reader gets acquainted with the tricks of *reading in* (i.e. **input** operation) the data values, performing very simple computations and to *getting out* (i.e. **output** operation) the computed results.

2.1 SUM OF TWO GIVEN NUMBERS

The Method

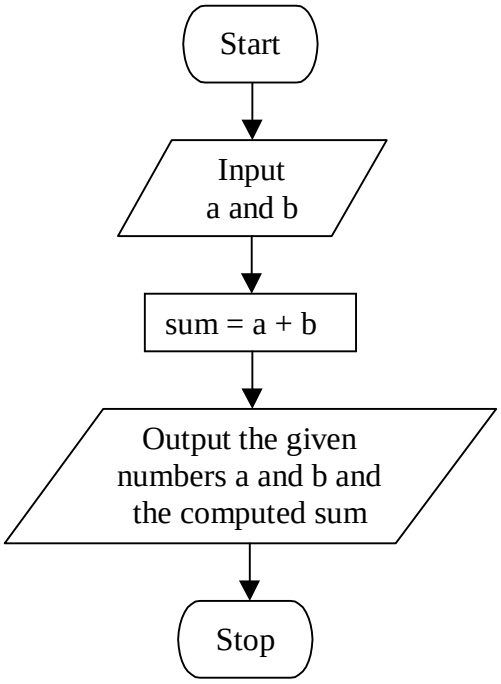
Let the two given numbers be *a* and *b*. Also, let both these numbers be integers. These two numbers are first read in (i.e. input). Then the *sum* is computed using a statement like *sum* = *a* + *b*. Observe that after actually computing the sum, the answer is stored in a variable called sum. As the required result is in *sum*, the value of sum is finally printed out or displayed (i.e. output).

C-Tips

In C the function `scanf( )` is used to input values of data items like *a* and *b*. The function `printf( )` is used to output the computed results. As such functions are available as predefined or library functions on a file called `<stdio.h>`, this file must be *included* in the

beginning. Such inclusions are made using statements called *preprocessor directives*. Here `#include` is used as preprocessor directive. It is a good practice to clear the display screen before the results of a presently running program is displayed on the screen. The C function `clrscr ( )` is used for this clearing purpose. This function is available on a file called `<conio.h>` which also must be *included* in the beginning. Files like `<stdio.h>` and `<conio.h>` which are included in the beginning, something like a heading, are called *header files*. Files with names having `.h` extension are generally identified as header files. Brief explanations about the program or some specific statement are provided using statements called comment statements. In C, comment statements are written with a `/*` pair in the beginning and a `*/` pair at the end of the statement. Since integers are handled by the `scanf( )` and `printf()` functions the format specifier `%d` is used. It may be observed that there will be a control string with both these `scanf( )` and `printf()` functions. This string will be within a pair of double quote marks (“). A `\n` produces a new line.

Flowchart



The Program

```
/* Program to find the sum of two given numbers */
#include<conio.h>                /* Including header file */
#include<stdio.h>

main()
{
       int a,b,sum;              /* Declaration of variables */
       clrscr();
       /* A message is displayed on the monitor indicating the action
          to be taken by the user */
       printf("\n\nEnter two numbers\n");
       /* The input statement follows */
```

```
        scanf("%d%d",&a,&b);
        /* The output statement follows */
        printf("\n  a = %d\t b = %d\n", a, b)  ;
        sum= a+b;                   /* Computing the sum */
        printf("sum = %d\n", sum); /* Printing the sum */
}
```

Note: It is a good programming practice to *echo* i.e. to display, the values of the data items as and when they are input. An input statement like `scanf( )` silently expects the values of the data items to be input. Therefore it is recommended to include statements which display the action to be taken by the user just prior to input statements.

2.2 SWAPPING THE CONTENTS OF TWO VARIABLES

Swapping of two variables means *exchanging the values* of two variables. For example, let ***a*** and ***b*** be two given variables with the values 5 and 10 stored in them respectively, as shown in the figures, below.



Before swapping

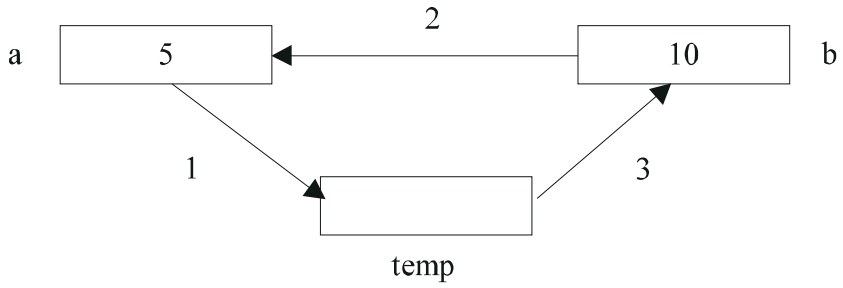
After swapping the value of ***a*** will be 10 and the value of ***b*** will be 5, as shown in the figures, below



After swapping

The Method

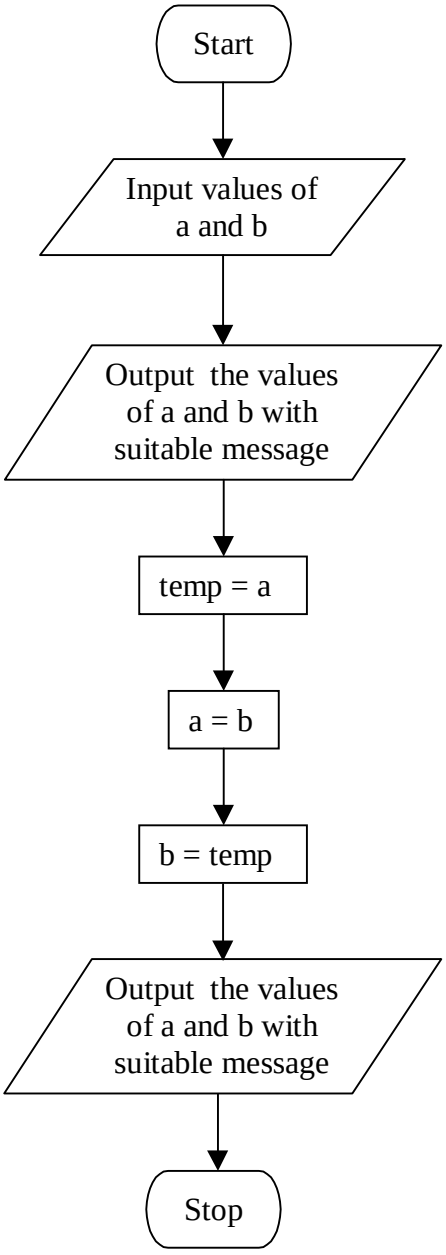
In order to exchange the values of ***a*** and ***b***, first *a copy* of A is made and stored in a temporary location (*temp*). Then the value of ***b*** is copied on to A. Finally, the contents of *temp* is copied on to the variable ***b***. These operations are illustrated in the following diagram where 1 , 2 and 3 indicate the order in which the operations are performed.



C-Tips

As already mentioned the function `scanf( )` is used to input data items. The function `printf( )` is used to output results as well as any other values. The function `clrscr( )` is used to clear the screen. Here also, the files `<stdio.h>` and `<conio.h>` are *included* in the beginning. A `\t` produces a *tab* (usually 5 blanks) in the output.

Flowchart



The Program

```
/*  Program to swap the contents of two variables */

#include<conio.h>                /* Including  header file */
#include<stdio.h>

main()
{
       int a,b,temp;             /* Declaration of variables */
       clrscr();
       printf("\n\nEnter two numbers\n");
       /*The input statement follows*/
       scanf("%d%d",&a,&b);
       /*The output statement follows*/
       printf("\n  a = %d\t b = %d\n", a, b)  ;
       temp = a;
       a = b;
       b = temp;                 /* Swapping contents of a and b */
       printf("After swapping \n a = %d \t b = %d\n", a, b);
}
```

2.3 SIMPLE INTEREST CALCULATION

The Method

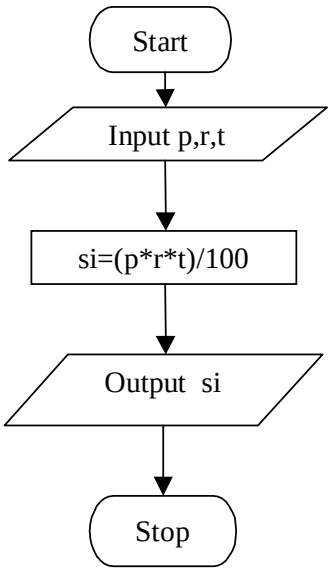
The formula used to calculate simple interest is **si = (*p r t*) / 100**

where *p* is the principal,
r is the rate of interest and
t is the time period.

Obviously, the values of *p, r* and *t* are input and the total interest for the given period is computed and output. The process involved is shown in the adjoining flowchart.

Care should be taken to avoid *integer division* while evaluating expressions like si = (*p* * *t* * *r*)/ 100.

Flowchart



C-Tips

The values of the variables like *p, r* and *t* are input using the input function ***scanf()*** statement. The result computed i.e. the value of **si** in this case is output using the output function `printf()`. These functions i.e., the `scanf()` and `printf()` are available on a C file named `<stdio.h>`. Here also `<stdio.h>` and `<conio.h>` files are included in the beginning. The format specifier `%f` is used to handle `float` type data items.

The Program

```
/* Program for simple interest calculation */

#include<conio.h>                /* Including header file */
#include<stdio.h>

main()
{
       float p,r,t,si;           /* Declaring variables */
       clrscr();
       printf("\n\nEnter principal,rate of interest and time
            \n\n\n");
       scanf("%f%f%f", &p, &r, &t);
       /* Entering principal , rate of interest and time */
       printf("\nPrincipal = %f\tRate of interest = %.2f\tTime =
            %.2f\n",p,r,t);
       si=(p*r*t)/100;           /* Computing simple interest */
       printf("\n\nSimple Interest = %.2f\n",si);
}
```

2.4 COMPUTING THE AREA OF A CIRCLE GIVEN ITS RADIUS

The Method

The relation used is **area = πr^2** where *r* is the radius of the circle. It may be observed that here the value of the radius *'r'* is to be input. The value of π is 3.14. The process involved can be shown pictorially as in the following flowchart. It should be observed that π is a constant and cannot be used as it is. Its value has been used at the proper place.

C-Tips

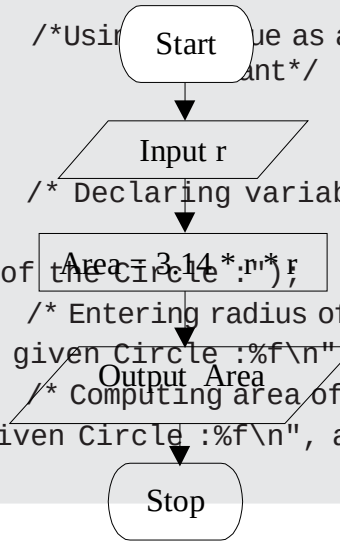
In situations where a constant like π is to be used at many number of places, it will be defined as a *symbolic constant* and this *symbolic constant* will be used at the required places. *Symbolic constants* are defined in the beginning and are put in to use using the #*define* preprocessor directive. Since a character like π is not a valid C character, it is represented by some meaning full name like **PI**. Also it may be noted that names used to represent symbolic constants are conventionally written using uppercase alphabets.

Flowchart

The Program

```
/* Program to compute the area of a circle given its radius */

#include<stdio.h>
#include<conio.h>
#define PI 3.1428                    /*Using the value as a symbolic
                                       constant*/
main()
{
        float r, area;               /* Declaring variables */
        clrscr();
        printf("\n\nEnter radius of the circle :");
        scanf("%f", &r);             /* Entering radius of the circle */
        printf("\n\nRadius of the given Circle :%f\n", r);
        area = PI * r * r;           /* Computing area of the circle */
        printf("\n\nArea of the given Circle :%f\n", area);
}
```



2.5 COMPUTING THE AREA OF A TRIANGLE GIVEN ALL ITS SIDES

The Method

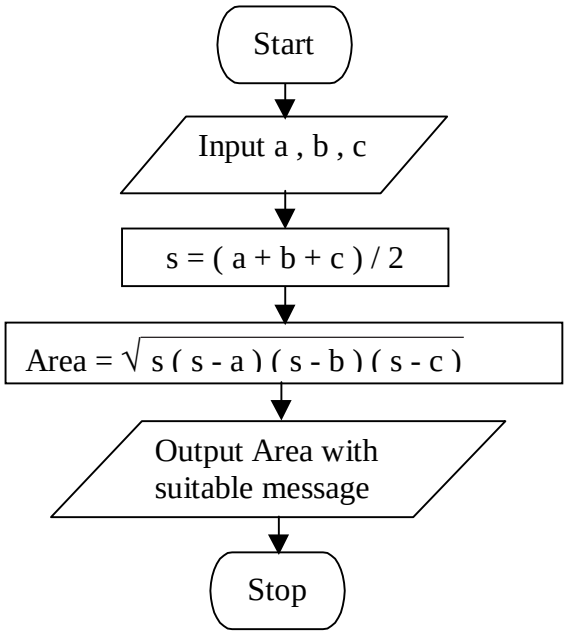
The formula used is

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}$$

where *a*, *b*, *c* are the three sides of a triangle and '*s*' is half of the perimeter of the triangle ($s = (a + b + c)/2$).

As the three sides *a*, *b* and *c* are given, they are first input. Then the half perimeter *s* is computed. Finally, the required area is computed and output. This process is shown pictorially in the adjoining flowchart.

Flowchart



C-Tips

As already mentioned both `scanf()` and `printf( )` are available as predefined or library functions on a header file called `<stdio.h>` which is also included in the beginning itself. The function `sqrt()` is also available as a library function. While using the `sqrt()` function care should be taken to see that the argument is non-negative. In other words, the argument to the function `sqrt()` should not be negative. Since `sqrt()` function is available on the file `<math.h>` it is also included. Since one likes to clear the screen before results are displayed `<conio.h>` file is also included in the beginning.

The Program

```
/*  Program to compute the area of a traingle given its sides */

#include<conio.h>                /* Including  header files */
#include<stdio.h>
#include<math.h>

main()
{
       float a,b,c,s,area;       /* Declaration of variables */
       clrscr();
```

```
        printf("\n\nEnter the three sides of the triangle\n");
        printf("such that sum of any two sides is greater than the
               third side.\n\n");
        scanf("%f%f%f",&a,&b,&c);
        /* Entering three sides of the triangle such that sum of two
        sides should be greater than the third side */
        printf("\nThree sides of the triangle are
                                    a=%f\tb=%f\tc=%f\n",a,b,c);
        s=(a+b+c)/2;                /* Computing value of s */

        area =sqrt(s*(s-a)*(s-b)*(s-c));
                                  /* Computing area of the triangle*/
        printf("\n\nArea of the triangle is %f\n",area);

}
```

2.6 EXTRACTING THE UNIT'S DIGIT OF A GIVEN INTEGER NUMBER

The Method

If 43256 is the given number then 6 is the digit we are looking for. The problem here is to extract 6, the digit in the unit's position. Since the given number is in decimal number system (i.e. base 10 system) 10 is used to move the positions of the individual digits as illustrated below.

Let the given number is: | 4 | 3 | 2 | 5 | 6 |

4	3	2	5	6	× 10

Multiplication by 10

4	3	2	5	6	0

A zero enters from RHS to the unit place

4	3	2	5	6	/ 10

Integer Division

4	3	2	5

The remainder | 6 |

The digit in the unit place has gone out

As seen from the above illustration when the given number is multiplied by 10, each digit moves to the left by one position. Therefore, the position value of each digit *increases*

10 times. A 0 (zero) gets appended to the number from RHS and the digit in the highest position goes out. When the given number is divided by 10, the position value of each digit reduces 10 times as each digit moves to the right by one position. The digit at the unit's position goes out as remainder and the size of the total number gets reduced by one digit.

In this case, we are interested in the digit that is going out when the given number is divided by 10. (i.e. the remainder). With many of the computer languages we can perform two types of divisions. The *integer division* and *modulo division.* When integer division is performed the result will be only the *quotient.* When the *modulo division* is performed the result will be only the *remainder.* Obviously, to get the unit's position digit we have to employ the 'modulo 10' division and trap the remainder by saving it in a variable. The complete procedure to extract the unit's digit may be shown diagrammatically as in the flowchart below.

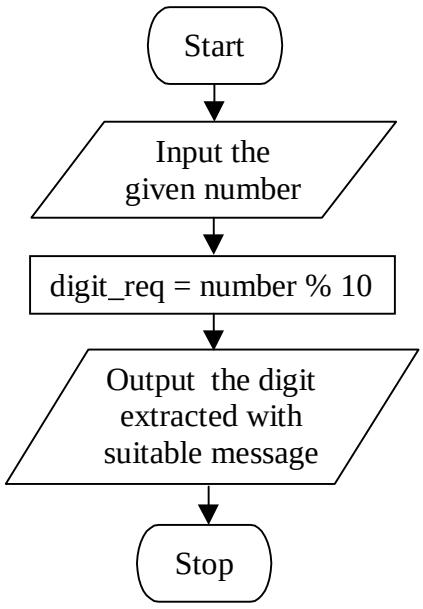
C-Tips

In C there is an operator known as the Modulo divisor, %. When this operator is used *modulo* division takes place and the remainder of the division will be returned as the answer.

For example, 254% 10 returns 4 as the result. Observe that this is the *remainder* of the % division.

While handling larger integer numbers it is advised to use the data type `long` for the declaration of the corresponding variable. With long data type `%ld` is used as the format specifier.

Flowchart



The Program

```
/* Extracting the unit's digit of a given integer number */

#include<stdio.h>
main()
{
 long number ; int digit_req;
 clrscr();
 printf("Enter a number :\n");
 scanf("%ld",&number);
 printf("Given number = %ld\n", number);

 /* The remainder in the following division operation is the unit's
    digit */

 digit_req=number % 10;

 printf("The digit at the unit's position of the given number is:
    %d",digit_req);
}
```

EXERCISES

1. Draw a flowchart to convert the temperature given in degrees Fahrenheit to degrees Celsius, using the relation $c = (5/9) \times (f - 32)$. Translate the flowchart into a C program.
2. Given the radius of a sphere, it is required to compute its volume and area. Draw a flowchart for the same. Translate the same into a C program. Use the relations: Volume = $(4\ \pi r^3)/3$ and Area = $4\ \pi r^2$.
3. Develop an algorithm to evaluate the polynomial $3x^3 - 5x^2 + 6$ for $x = 2.55$. Write a C program for the same.
4. Given that the integer variables a, b and c have the values 8, 3 and – 5 assigned to them respectively and the `float` type variables x, y and z have the values 8.8, 3.5 and – 5.2 assigned to them respectively, draw a flowchart and hence write a C program to read in the values for the different variables, perform the following operations and output the results. Comment on the results.
   (i) a**b*/*c*
   (ii) *a*% *b*
   (iii) *x*/*y*
   (iv) *x*% *y*

CHAPTER 3

Decision Making

As a beginner I was always wondering why so many examples like finding the largest of given three numbers, finding the largest among a given set of numbers, sorting a given set of numbers in a particular order, searching for a required number in a set of numbers and so on, all involving just integer numbers, are discussed. I became really restless when I came across many searching as well as sorting algorithms that have been developed and analyzed at lengths. I am sure that almost every beginner goes through such a phase of confusion during his early part of computer education. Having known the power of a computer and decided to have computer education, every one will be interested in having a go, to begin with itself. Reasons for working only with numbers will become evident as we get into the field of computer applications, particularly the one's that deal with data bases. Some of these reasons are discussed in the following paragraph.

For generations, men and materials have been identified by *unique names*. Time has seen an explosion in population and production of a **variety** of materials in large numbers. Many alternatives such as using **surnames**, aliases etc., have been employed to *identify men uniquely*. These methods of naming and addressing people have, of-late, *become inadequate* for many reasons. One of the very important reasons is that *there are too many to be named uniquely*. This is why we see '*university seat numbers*' are allotted *uniquely* for those who take up an university examination, '*unique account numbers*' are given to all the account holders in a bank, '*unique employee numbers'* are given to all the employees of a company, '*unique passport numbers*' are given to all the passport holders, '*unique numbers'* are given to files when we have large number of files, '*unique social security numbers*' are given to the citizens in certain countries like USA and so on. One can keep on listing a *very large number of contexts or situations where numbers are used to name or identify a particular person or an entity* and to *access details* pertaining to him or for some or the other type of *processing*. This is why most of the time we will be dealing with examples like the ones cited above which deal with numbers.

Beginning from this chapter all chapters discuss examples involving numbers directly or indirectly. However, different programming techniques based on different *language constructs* useful in problem solving (i.e., program writing) are presented in different chapters in the order of increasing complexity.

Decision making is an integral part of every one's life. Most of the times we will be making decisions without really knowing that we are making decisions. Depending upon the weather report we may decide to carry a raincoat or otherwise. A railway driver decides either to drive into a track or not depending upon the red or green signal. A decision is made to attend a reception or not depending on whether we are free or not. Similar decisions are also made during computations. For example, an integer number is decided to be as positive or negative depending upon whether it is greater than or lesser than zero. A number is decided as *prime* number depending upon whether it is divisible by any number other than itself and 1.

Because of the reasons cited in the previous paragraph, all the computer languages have '*language constructs*' using which decisions are made. *Two way decision* making is most general. In C the ***`if …. else`*** construct provides a two way decision making facility. Some examples which employ two way decision making are discussed in this chapter.

3.1 LARGEST OF TWO GIVEN NUMBERS

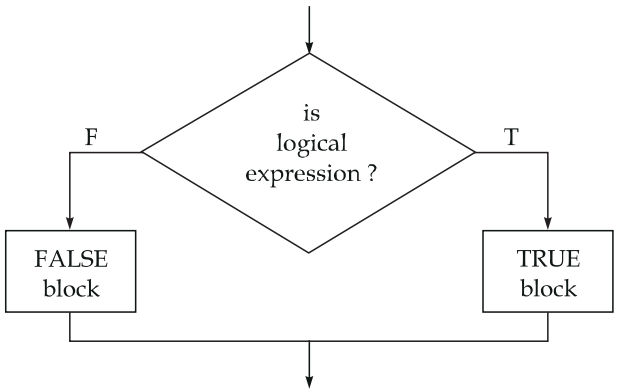
The Method

Let ***a*** and ***b*** be two given numbers. After reading in the two given numbers, they are compared**.** If ***a*** is *greater than* (or equal to) ***b***, ***a*** is declared as larger, otherwise (i.e. *else*) ***b*** is declared as larger.

C-Tips

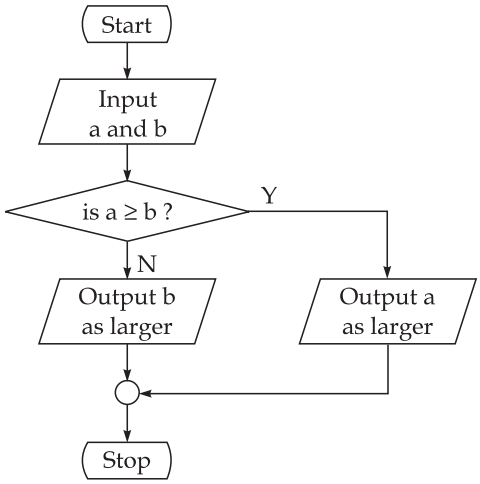
The general format of the ***`if…. else`*** statement will be as follows.

```
if (logical expression)
{
        TRUE Block
        statements
}
else
{
        FALSE Block
        statements
}
```



Here, logical expression means any expression the evaluation of which results in a logical answer like YES or NO, or TRUE or FALSE. In this case if the answer is YES or TRUE only the statements in the TRUE block will be evaluated. If the answer is NO or FALSE only the statements in the FALSE block will be evaluated. For example the expression (*a*>=*b*) is a logical expression. Also it may be noted that either the statements only in the TRUE Block will be executed or statements in the FALSE Block only will be executed, but not both. The presence of **`else`** block is optional i.e. *it may be present or not*. Whenever an **`else`** block is not present, this conditional statement is known as the *simple* ***`if`*** *statement*.

Flowchart



The Program

```
/* Finding the largest of the given two numbers */
#include<stdio.h>
#include<conio.h>
main()
{
   int a,b;
   clrscr();
   printf("Enter any two numbers\n");
   scanf("%d%d",&a,&b);
   printf("\nThe given two numbers are : a = %d , b = %d\n",a,b);
   if(a > = b)
     printf("\n a = %d is the largest number",a);
   else
   printf("\n b = %d is the largest number",b);
}
```

3.2 DETERMINING WHETHER A GIVEN INTEGER NUMBER IS POSITIVE OR NEGATIVE

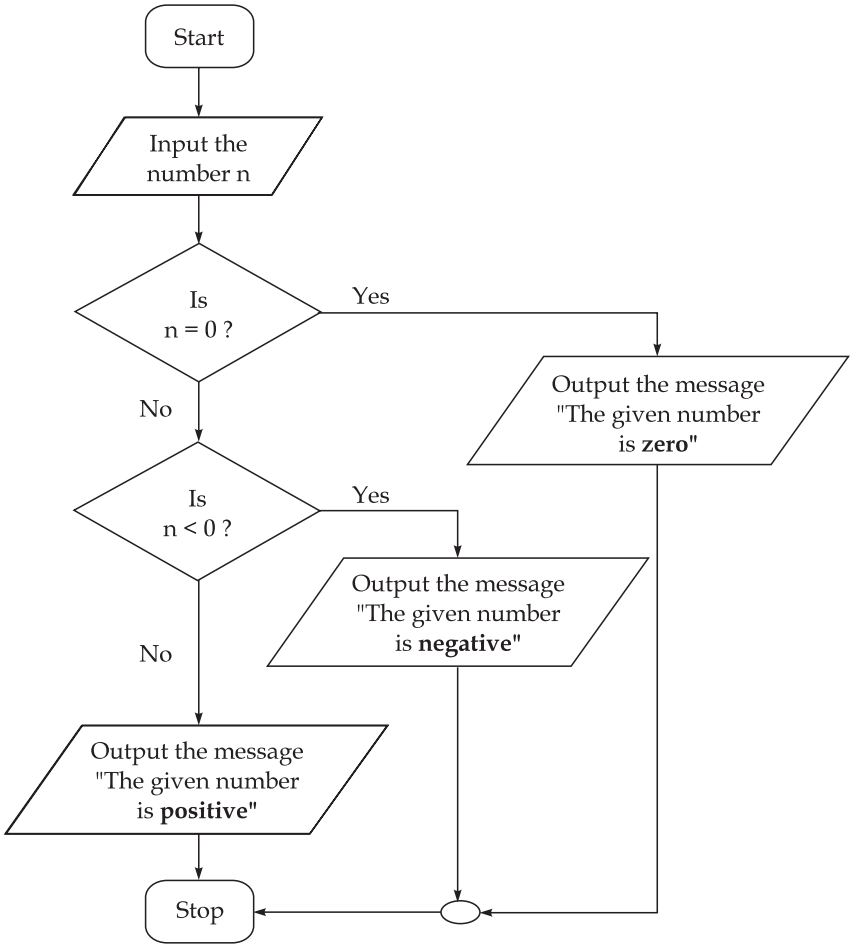
The Method

We know that a given number could be positive, negative or zero. Though some treat zero as positive, I have preferred it to be *zero* itself, so that we can demonstrate the use of *nested ifs*. The procedure here is to first verify whether a given number is zero or not. If not, the procedure verifies whether the given number is less than zero or greater than zero. If *less than zero*, then the number is *negative*. If greater than zero, then the number is *positive*. The flowchart below gives the procedure in the pictorial form.

C-Tips

The `if-else` conditional construct explained in 3.1 is used. An `if-else` construct could be used inside another `if-else` construct, as is done in this example. In such cases `if-else` constructs are said *to be nested*. The operator **==** is used to check for equality.

Flowchart



The Program

```
/* To find out whether a given integer is zero, +ve or -ve */

#include<conio.h>                /* Including header file */
#include<stdio.h>

main()
{
       int number;
       clrscr();
       printf("\n\nEnter an integer number\n\n");
       scanf("%d", &number);     /* Read a number */
       printf("\nNumber is %d\n\n", number);

       if (number == 0)
         printf ("The number is zero. \n");
       else
         if (number < 0)
             printf ("%d is negative. \n",number);
         else
             printf ("%d is positive. \n",number);

}
```

3.3 DETERMINING WHETHER A GIVEN INTEGER NUMBER IS ODD OR EVEN

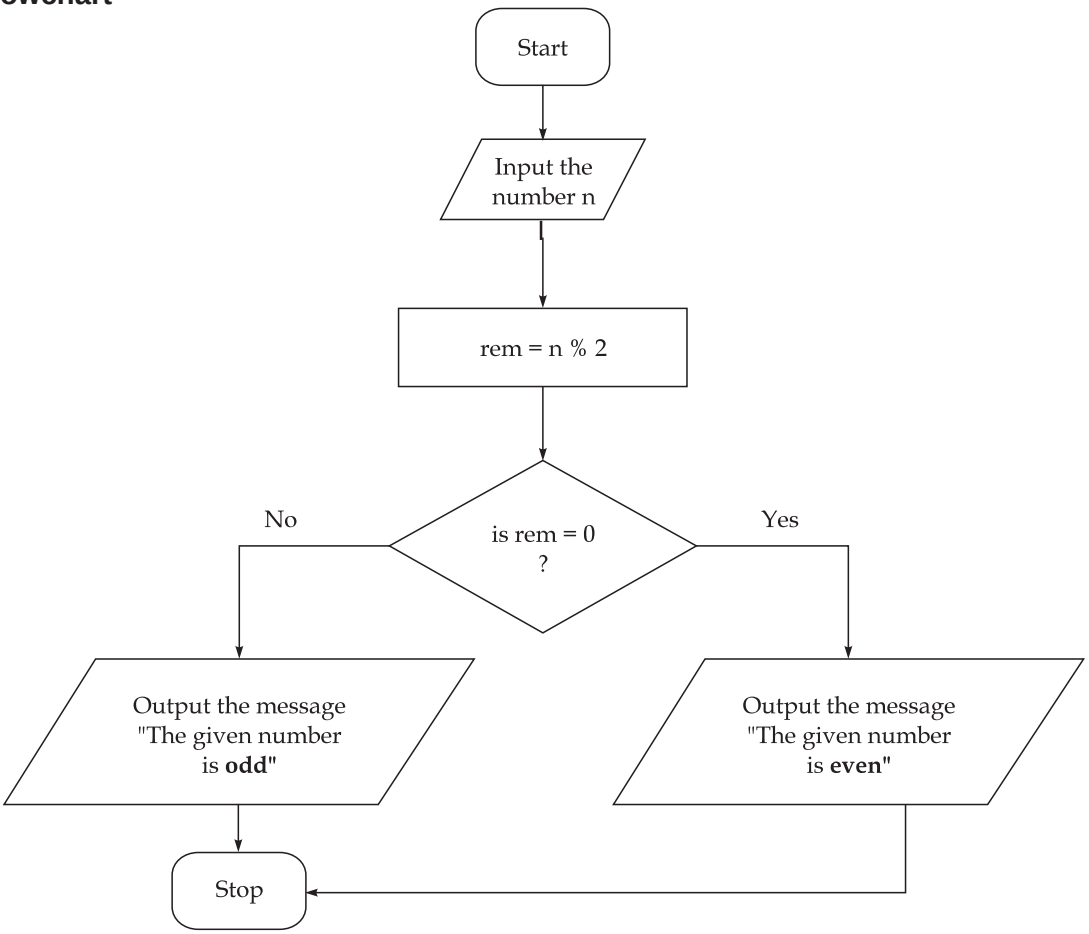
The Method

The technique used to find out whether a given integer number is *odd or even* is to *modulo divide* the given integer by 2 and to verify the *remainder*. If the *remainder is zero* then the number is even, *else* the number is odd. Below is given the flow chart, which shows the above procedure.

C-Tips

The operator % is used for modulo 2 division. The **`if.. else`** conditional statement, discussed in 3.1, is used for making decisions.

Flowchart



The Program

```
/* Determining whether a given integer is odd or even */

#include<stdio.h>
#include<conio.h>

main()
{
       int number,rem;
       clrscr();
```

```
        printf("Enter an integer number :");
        scanf("%d", &number);     /* Read an integer */
        printf("\nNumber is %d\n", number);

        /*Dividing number by 2,if the remainder is
               zero it is even otherwise it is odd*/
        rem = number % 2;

        if ( rem == 0 )
           printf("The number %d is even.\n",number);
        else
           printf("The number %d is odd.\n",number);

}
```

3.4 LARGEST OF GIVEN THREE NUMBERS

The Method

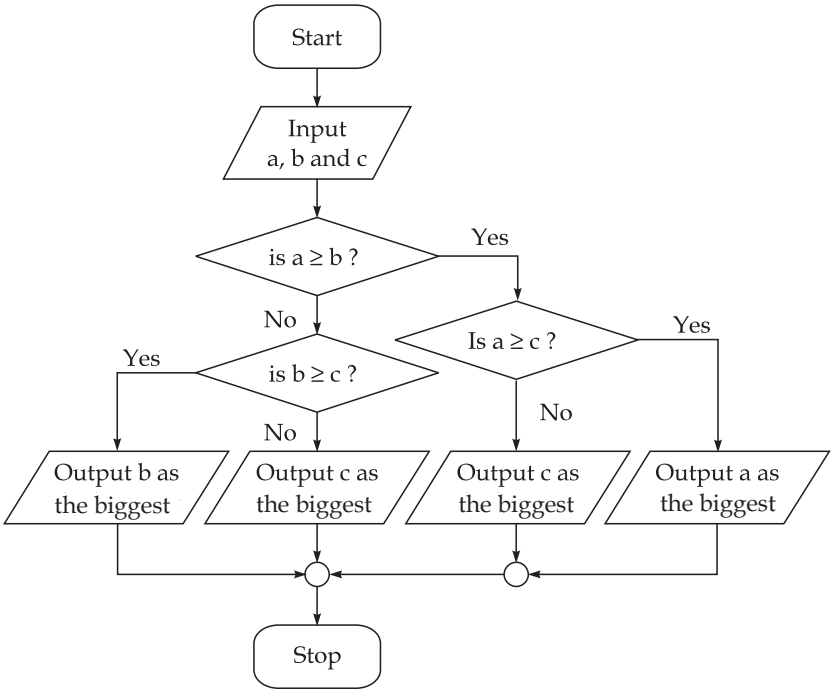
Let ***a, b*** and ***c*** are the given three integer numbers. First read in all the three numbers. Compare ***a*** with ***b*** to find out whether it is greater than (or equal to) ***b***. If it is *true* then compare ***a*** with ***c*** to find out whether it is greater than (or equal to) ***c***. Once again if it is *true*, then declare ***a*** as the largest. At this stage, if ***a*** is found to be smaller than ***c***, then declare ***c*** as the largest. Here, notice that ***a*** has been already compared with ***b*** and has been found to be larger than ***b*** and hence ***c*** has to be larger than ***b***.

If ***a*** was found to be smaller than ***b*** at the first comparison itself, then ***b*** has to be larger than ***a*** and now ***b*** has to be compared with ***c***. As such, when the comparison of ***a*** with ***b*** gives a *false* (no) answer then ***b*** is compared with ***c*** to find out whether ***b*** is greater than (or equal to) ***c***. If the answer here is *true* then ***b*** is declared as the biggest, otherwise (*else*) ***c*** is declared as the biggest.

C-Tips

As we go through the method given above, we obviously get a feeling that we have to use the `if ... else` construct. Yes, we use it. By carefully going through the method we find that we have to make a comparison (i.e. comparison of *a* and *c*) within the earlier comparison construct. The second `if... else` construct appears in the true block of the earlier `if... else` construct. Writing an `if... else` construct within the true block (or else block) is permitted. This way of using an `if... else` construct within another `if... else` construct is called **nesting** of `if...else` constructs.

Flowchart



The Program

```
/* Program to find biggest of three numbers */
/* Here num1,num2 and num3 are used instead of a,b and c */

#include<stdio.h>
#include<conio.h>
main()
{
 int num1, num2, num3;
 clrscr();
 printf("Enter three numbers :\n");

 scanf("%d%d%d", &num1, &num2, &num3); /* Reading three numbers */
 printf("\nThe three numbers are\t%d\t%d\t%d\n\n", num1, num2, num3);

 if (num1 >= num2)                    /* Comparing first with second */
```

```
 {
      if (num1 >= num3)           /* Comparing first with third */
           printf("\n%d is biggest\n", num1);
      else
           printf("\n%d is biggest\n", num3);
 }
 else
 {
      if (num2 >= num3)           /* Comparing second with third */
           printf("\n%d is biggest\n", num2);
      else
      printf("\n%d is biggest\n", num3);
 }

}
```

3.5 TO FIND OUT WHETHER A GIVEN YEAR IS A LEAP YEAR OR NOT

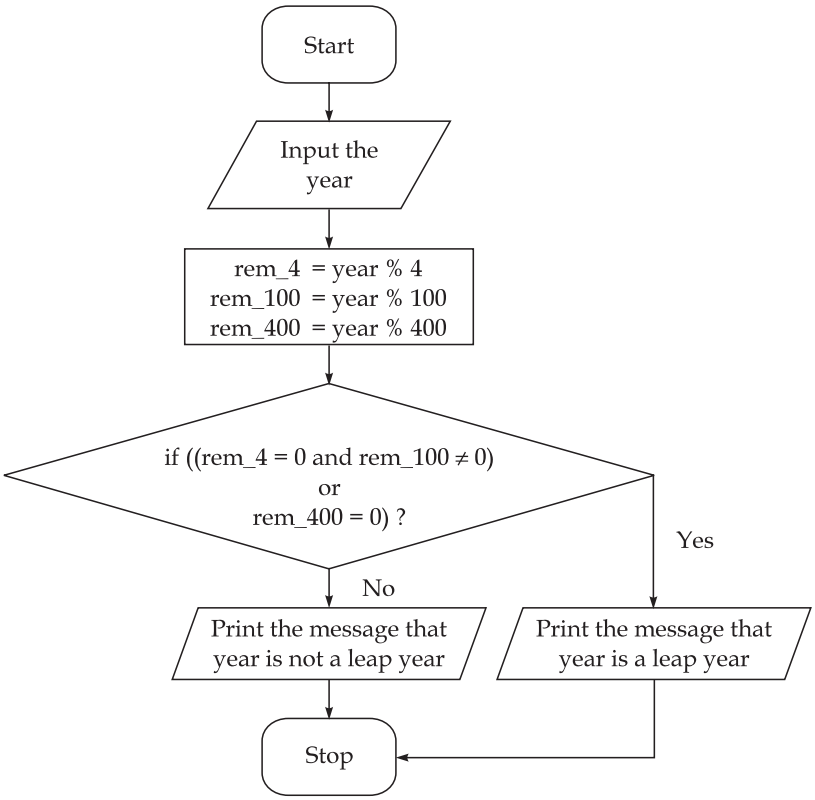
The Method

By definition, a year is said to be a *leap year* if it is "divisible by 4 and not divisible by 100" OR "divisible by 400". Divisibility is generally tested by carrying out *modulo division* by the required number. If the resultant remainder is zero, then we conclude that the number is divisible. Otherwise, it is not. Using this idea, after reading in the given year to be tested, it is modulo divided by 4, 100 and 400 respectively and the resulting remainders are stored in three separate variables, say rem_4, rem_100 and rem_400. If "(rem_4 = 0 **and** rem_100 ≠ 0) **or** (rem_400 = 0)" is true, then the given year will be a leap year, else the given year is not a leap year. The year has to be input completely (i.e. only last two digits are not sufficient).

C-Tips

The expression used to test whether the given year is a leap year or not is known as a logical expression. Here, this logical expression will be written using the logical **AND** and the logical **OR** operators. In C, double ampersands (i.e. &&) are used as logical **AND** operator and double vertical bars (i.e. **||**) are used as logical **OR** operator.

Flowchart



The Program

```
/* Program to determine if a year is a leap year or not */

#include<stdio.h>
#include<conio.h>
main()
{
    int year, rem_4, rem_100, rem_400;
    clrscr();
    printf("Enter the year to be tested\n");
    scanf("%d",&year);

    rem_4 = year % 4;
```

```
    rem_100 = year % 100;
    rem_400 = year % 400;

    if((rem_4 == 0 && rem_100 != 0) || rem_400 == 0)
       printf("The given year %d is a leap year",year);
    else
       printf("The given year %d is not a leap year",year);
}
```

3.6 SOLVING A GIVEN QUADRATIC EQUATION

A quadratic equation will be of the form

$ax^2 + bx + c = 0$, where a, b and c are called the co-efficients.

The problem here is to find the values of x for which the given equation is satisfied. These values of x which satisfy the given equation are known as its *roots*. This equation will have two roots. The procedure is to first compute the discriminant ($b^2 - 4ac$). The nature of the roots depend upon whether the value of the discriminant is zero, positive or negative.

- If the discriminant is zero, then the *two roots will be equal* and each one will be $(-b/(2a))$.
- If the discriminant is positive then the *two roots are real and distinct*.

  One root (root1) will be $\left(-b + \sqrt{b^2 - 4ac}\right) \Big/ (2a)$ and the other root

  (root2) will be $\left(-b - \sqrt{b^2 - 4ac}\right) \Big/ (2a)$.

- If the discriminant is negative, then the *roots are complex and distinct*. In this case, the real part (`part_r`) and the imaginary part (`part_i`) are *computed separately*. One of the roots is output as `part_r + i part_i` and the other root is output as `part_r – i part_i.`

The Method

As the co-efficients (a, b and c) of the given quadratic equation will be known, they are first read in. Then the discriminant ($b^2 - 4ac$) is computed and stored in a suitable variable, say *disc*. As already discussed the *disc* value could be zero, +ve, or –ve. `If... else` constructs are used to manage the decision making process and to branch out. The complete flow chart is given below.

C-Tips

As already mentioned, care should be taken to avoid computing the square root of any negative value. In other words, the argument to the `sqrt()` function must be +ve. In the output statements, 4.2 has been used as field specification along with the format specifier. Here, 4 indicates the total number of characters that will be printed/displayed and 2 indicates the number of characters that will be printed/displayed after the period.

Note: Printing out root1 and root2 when the roots are "equal" and "real and distinct" is straightforward. In the case of complex roots, first, one has to properly *decide about the layout* of the output and then write the corresponding output statement.

For example, if *real part* is equal to 3.1 and *imaginary part* is equal to 2.16 and we like to have the output printed as follows, starting from column number 5.

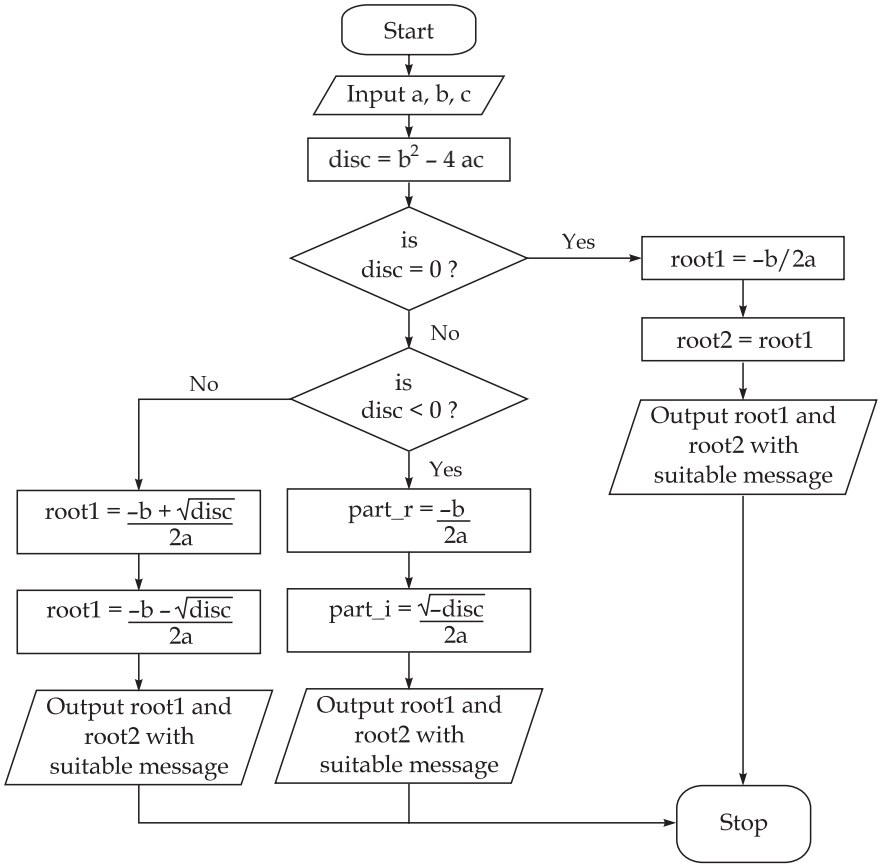
Root1=3.15+*i*2.16

Root2=3.15–*i*2.16

For the final outputs to appear as indicated above the output statements should be written as follows.

```
printf("    Root 1= %4.2f + i %4.2f \n",part_r, part_i);
printf("    Root 2= %4.2f – i %4.2f \n",part_r, part_i);
```

Flowchart



The Program

```
/* To find the roots of a given quadratic equation */

#include<conio.h>                    /* Including  header file */
#include<stdio.h>
#include<math.h>
main()
{
 float a,b,c,disc, root1, root2,part_r,part_i;/* Declaring variables*/
 clrscr();                                /* Clearing the Screen */
 printf("\n\nEnter co-efficients of the quadratic equation\n\n");
 scanf("%f%f%f", &a, &b, &c);
 printf("\nCo-efficients are a = %f\t b = %f\t c = %f\n\n", a,b,c);
 disc=b*b-4*a*c;                  /* Calculating discriminant*/

 if ( disc == 0)
 /* If discriminant is zero roots are real and equal */
 {
    printf("Roots are real and equal\n");
    root1 = -b/(2.0*a);
    root2 = root1;
    printf("\nRoot1 = %f\t Root2 = %f\n",root1,root2);
 }
 else
 {
    if ( disc < 0)
    /* If discriminant is less than zero roots are complex */
    {
          printf("Roots are complex \n");
          part_r = -b/(2.0*a);
          part_i = sqrt(-disc)/(2.0*a);
          printf("\nRoot1 = %f + i %f\tRoot2 = %f -i
          %f\n",part_r,part_i,part_r,part_i);
    }
```

```
    else
    /* As disc is greater than zero roots are real and distinct */
    {
          printf("Roots are real and distinct\n");
          root1 = (-b+sqrt(disc))/(2.0*a);
          root2 = (-b-sqrt(disc))/(2.0*a);
          printf("\nRoot1 = %f\t Root2 = %f\n",root1,root2);
    }
 }

}
```

EXERCISES

1. A worker is paid at an hourly rate of Rs. 45 for the first 36 hours of work. Thereafter overtime is paid at 1.5 times for every one hour of overtime work. Develop a flow chart to compute the net wages to be paid to a worker, taking the total number of hours worked as input. Also develop a C program for the same.
2. Develop a flowchart to obtain the absolute value of a given integer. Also develop a C program for the same.
3. Given two integers, develop a flowchart to determine whether the first number is divisible by the second one or not. A suitable message has to be displayed. Also develop a C program for the same.
4. Develop a flowchart to find the smallest among given three integer values which uses only two way decision making facility. Also develop a C program for the same.
5. Develop a flowchart to multiply an integer variable *principal* by 3 if the float type variable *deposit* is greater than 10.3. Code the same in C.

CHAPTER 4

Looping Techniques

Various computer applications need *'repeated'* execution of some set of *'computational statements'* again and again. For example, different terms of the Fibonacci series are computed by *repeatedly adding* the values of two immediate preceding terms. The sum of a *series* expansion involves the *repeated generation of next terms* and *adding it to the previous sum*, the salary of all the employees of an organization is computed using the *same set of rules again and again*. At an university level examination students results are computed using the *same set of rules again and again*, and so on. There are such innumerable number of applications where one will be *repeatedly* or ***iteratively*** executing a certain set of statements again and again. As a result almost all the computer languages provide facilities, in the form of language constructs, using which repeated executions can be carried out easily. In C, the `for, while,` and `do…while` constructs are used for repeated execution of required set of statements for required number of times. These language constructs are generally called *loop control* structures. This chapter discusses few examples that employ these structures.

4.1 SUM OF FIRST *N* NATURAL NUMBERS

Here, the problem is to find the sum of the series 1 + 2 + 3 + 4 + 5 + 6 + 7……+ *n*.

The Method

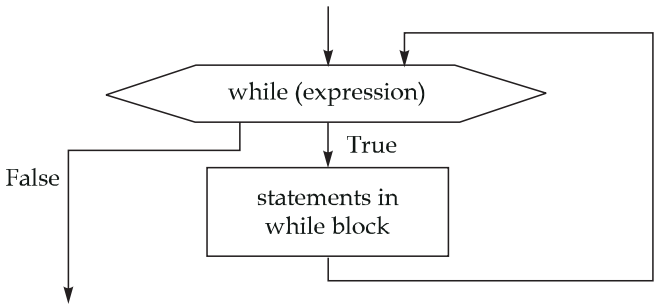
This is one of the simplest examples one comes across to compute the sum of a series. It is simple because, the first term is 1 and each of the next terms are obtained by incrementing the previous term by 1 (i.e. by adding 1 to the previous term). A variable *sum* is used to hold the entire *sum* of the series. A variable like '*i*' is used as a *counter* which counts from 1 to *n* in steps of 1. The value of this counter (i.e. variable) is added to sum at each step

using a relation like *sum = sum + i*. The only input to this program is '*n*', the value of number of terms. The flowchart for this method is given below.

C-Tips

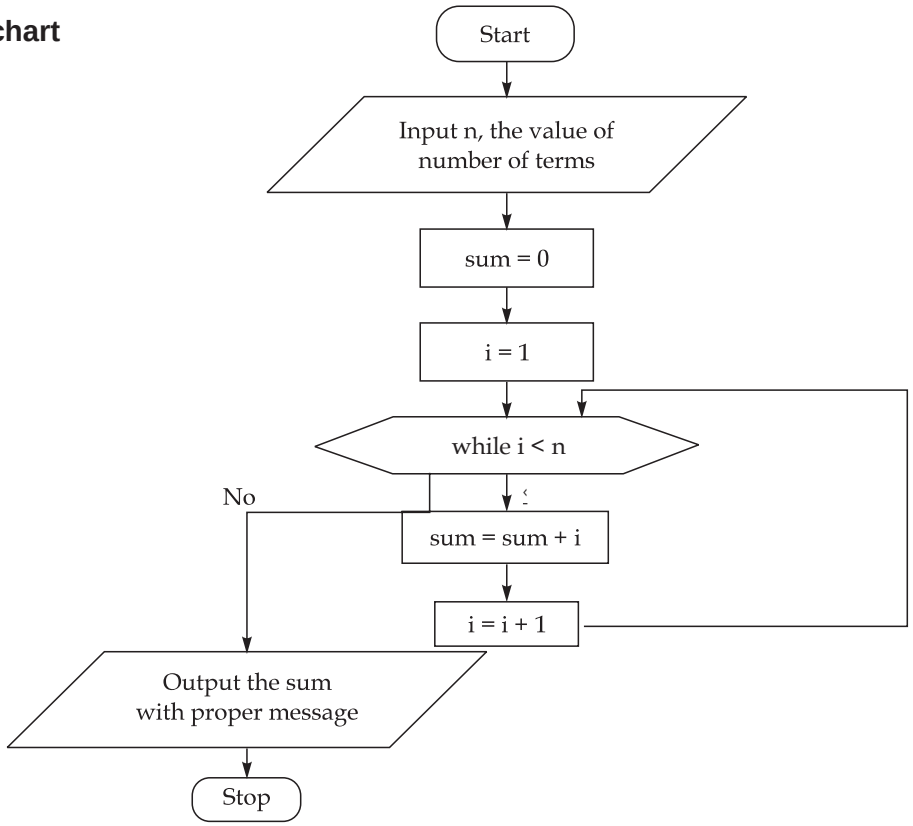
In this example *repeated execution* of certain statements is achieved using the `while` loop structure. The general structure of the `while` structure is shown below.

```
while (logical expression)
{
        statements in the
        while loop
}
```



When the control comes across a `while`, the `logical expression` is evaluated first. As long as the result of the evaluation of the logical expression is TRUE, statements in the while block are repeatedly executed. Once the evaluation of the logical expression results in FALSE, the control comes out of the `while` loop.

Flowchart



The Program

```
/* Finding the sum of first n natural numbers */

#include<conio.h>      /* Including  header file */
#include<stdio.h>

main()
{
   int n, i=1, sum=0;    /* Initializing i to one and sum to zero */
   clrscr();
   printf("\n\nEnter value of n\n");
   scanf("%d", &n);
   printf("n = %d\n",n);

    /* Repeat while i is less than or equal to n */
    while(i <= n)
   {
        sum = sum + i;            /* Adding i to sum */
        i = i + 1;                /* Incrementing i by 1 */
   }

   printf("\nSum of first %d natural numbers is %d\n", n, sum);

}
```

Note: It may be observed that the variable *sum* and *i* are initialized during type declaration itself. The C language permits this. A variable like *i* based on the value of which the loop is executed is called a *loop control variable*.

4.2 SUM OF THE SQUARES OF ALL INTEGERS FROM 1 TO N

Here, the problem is to find the sum of the series $1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 \ldots\ldots + n^2$.

The Method

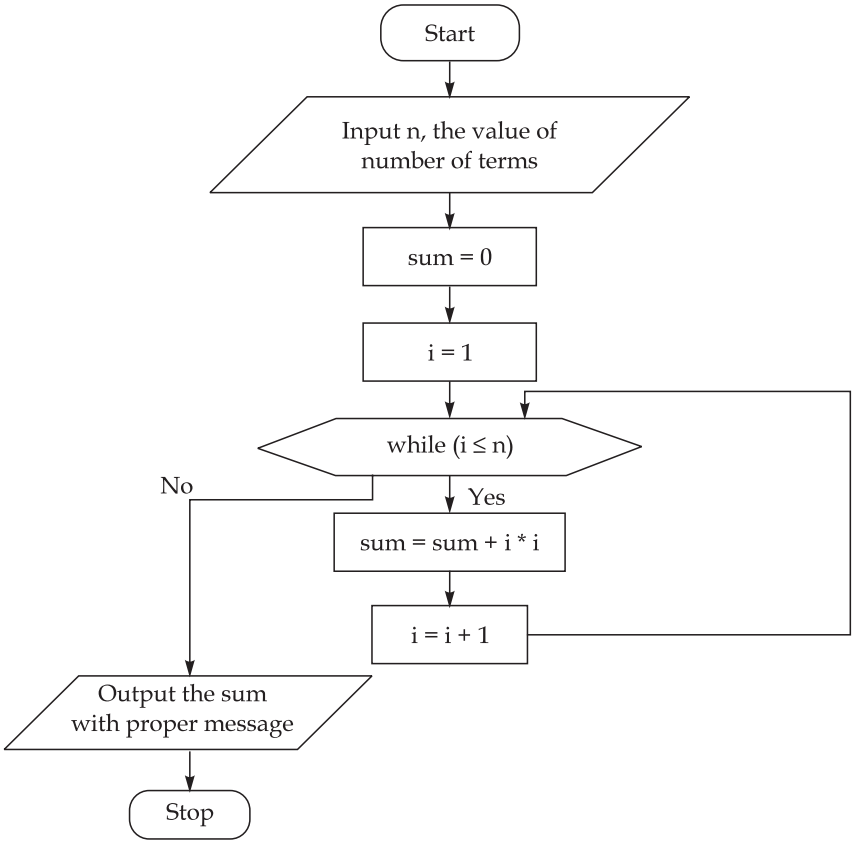
The procedure here is exactly similar to the one used in example 4.1. The only difference being, the square of the next term is added to the current sum. Of course, the loop control variable '*i*' itself is being used as the next term. Note that though the value of '*i*' is being

squared, we are not altering its value. Yes, it is important to note that we can use the value of any loop control variable but its value should not be manipulated.

C-Tips

In this example also the `while` loop structure, discussed in the previous example, is used. Notice that we have used a statement like `sum+ = i * i.` This statement is equivalent to `sum = sum + i * i.` An assignment statement of the form `sum += i * i` is called the `shorthand assignment` statement. Also, we have used a statement like `++i;` which is equivalent to `i = i + 1.` In C, `++` is known as the increment operator.

Flowchart



The Program

```
/* To find the sum of the squares of all integers from 1 to n */
#include<conio.h>                    /* Including header file */
#include<stdio.h>
```

```
main()
{
        int n, i=1, sum=0;    /* Initializing i to 1 and sum to 0 */
        clrscr();
        printf("\n\nEnter value of n\n");
        scanf("%d", &n);
        printf("n = %d\n",n);

        while(i <= n) /* Repeat while i is less than or equal to n */

        {
              sum += i * i;         /* Adding i² to sum */
              ++i;                  /* Incrementing i by 1 */
        }

        printf("\nSum of squares of numbers from 1 to %d is %d\n", n, sum);
}
```

Note: Like in the previous example, here also some of the variables are initialized during declaration itself. Also observe the use of shorthand assignment statement and increment operator.

4.3 FACTORIAL OF A GIVEN INTEGER

The factorial of a given integer n is given by the product
$1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times \ldots \times (n-1) \times n$.

The Method

It may be observed that this is also a type of series made up of many terms like 1, 2, 3, …… etc. Therefore this problem is handled in a similar manner to that of summation of series examples. It may be noted that in summation problems the initial value of the variable *sum* must be equal to *zero* (0). But in this case, since successive multiplications are used, the initial value of the variable that holds the factorial value, say *prod*, must be 1. A loop control variable like *i* itself is used for counting as well as the next term. A statement of the type `prod = prod * i` is used to get the required factorial. Note that, as already mentioned, *i* is being used as *new term* which is nothing but a loop control variable and automatically gets incremented under the control of the loop control structure used.

The process of generating the next term and multiplying with the current product is repeated till the next term exceeds the value of '*n*'.

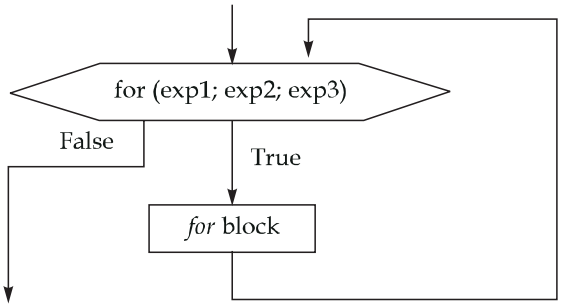
Note: Though it has been mentioned that a statement like `term = term + 1` is used to generate the next term, it is not necessary to use such a statement exclusively. Instead, the same can be managed using the loop control variable itself as it starts from 1 and runs upto and equal to *n* in steps of 1. In general only values of loop control variable, can be used, but should not be altered.

C-Tips

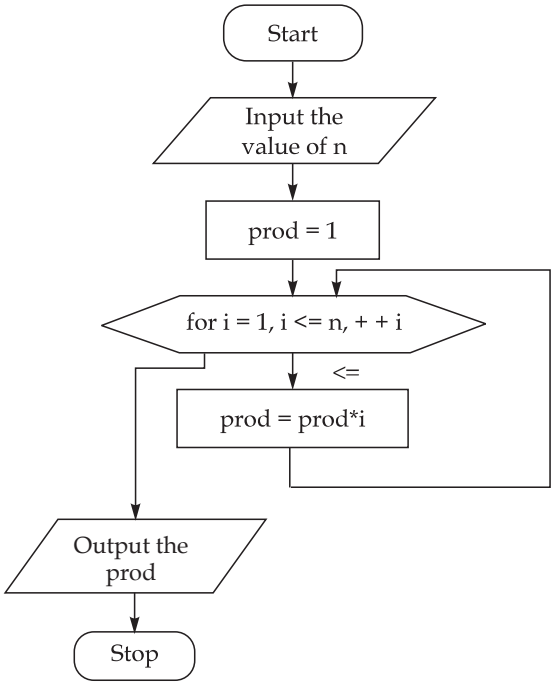
In this example the repeated execution of the required statement has been carried out using the *for* control structure of C. The general format of this loop control structure is given below.

```
for ( exp1 ; exp2 ; exp3 )
{
      for block
      statements
}
```

Here *exp1* is an expression used to hold the initial value of a variable, the value of which is incremented or decremented in the expression *exp3*. The *exp2* is generally a *logical expression*. As long as this logical expression *exp2* is TRUE, the statements in the *for* block are executed. Once *exp2* becomes FALSE control goes out of the *for* loop. The behavior of the *for* statement can be pictorially shown in a flow diagram as follows.



Flowchart



The Program

```
/* To find the factorial of a given integer */

#include<stdio.h>
#include<conio.h>

main()
{
        long n,i,prod=1;
        clrscr();

        printf("Input the value of n \n");
        scanf("%ld",&n);

   for(i=1;i<=n;++i)
   {
        prod = prod * i;
   }

   printf("\nThe value of %ld factorial is : %ld \n",n,prod);

}
```

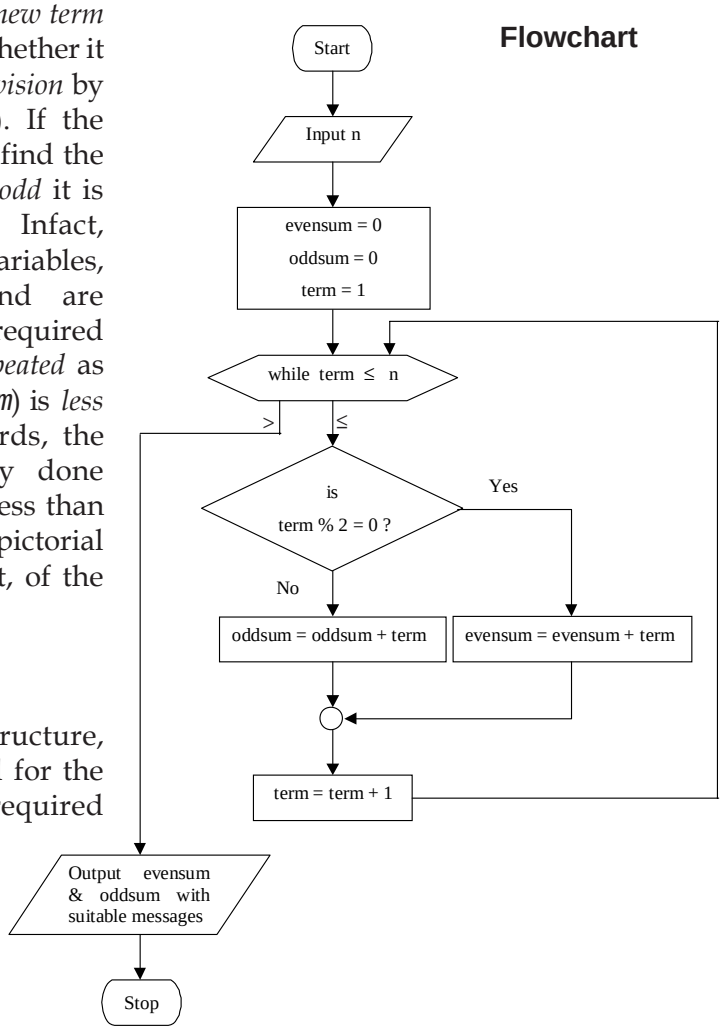
Note: As the factorial of an integer might result in a large integer value, the variable `prod` which finally holds the value of the factorial is type declared as `long`. Therefore, while handling `prod` in i/o statements the format specifier **`%ld`** is used.

4.4 FINDING THE SUM OF ODD NUMBERS AND SUM OF EVEN NUMBERS FROM 1 TO *N*

The Method

Starting from 1, numbers 2, 3, 4 ……. upto *n* are generated by adding (i.e. incrementing) a 1 to the previous number. As a sort of convention let the first number be `term` the initial value of which is 1. The next number (which is also called *term*) is generated by adding a 1 to the previous term (i.e. *term*). In other words, the relation `term = term + 1` is used. At every step, after generating a *new term* (i.e. `term`) *it is tested* to find out whether it is odd or even. For this *modulo division* by 2 is used (as discussed in 3.3). If the number is *even* then it is used to find the `evensum` and if the number is *odd* it is used to find the `oddsum`. Infact, `evensum` and `oddsum` are two variables, which are intitally zero, and are specifically used for storing the required sum. The above procedure is *repeated* as long as the number (i.e. the `term`) is *less than or equal to n*. In other words, the above procedure is repeatedly done `while` the number (or term) is less than or equal to *n*. Given below is the pictorial representation, i.e. the flowchart, of the above process.

Flowchart



C-Tips

The `while` loop control structure, already explained in 4.1, is used for the repeated execution of the required statements.

The Program

```
/*Finding the sum of odd numbers and sum of even numbers from 1 to n */

#include<stdio.h>
#include<conio.h>

main()
{
    int n,evensum=0, oddsum=0, i=1;
    /*Initializing evensum and oddsum to zero and i to 1 */
    clrscr();
    printf("Enter value of n\n");
    scanf("%d", &n);
    printf("\nn = %d\n", n);

    while(i<=n)   /*Repeat while i is less than or equal to n*/
    {
    /*If the remainder after dividing number by 2 is zero then it is
     even, add to evensum*/

       if(i%2==0)
         evensum += i;
       else

    /*If the remainder after dividing number by 2 is non zero then it
     is odd, add to oddsum*/
         oddsum += i;

       ++i;                        /*Increment i by 1*/
    }

    printf("\nSum of odd numbers from 1 to %d is %d\n", n, oddsum);
    printf("\nSum of even numbers from 1 to %d is %d\n", n, evensum);

}
```

Note: Here, observe that the loop control variable *i* is used both as the present term and the next term.

4.5 GENERATION OF REQUIRED NUMBER OF TERMS OF THE FIBONACCI SEQUENCE

The Fibonacci sequence is the sequence of integers which will be as shown below.

0 , 1 , 1, 2, 3 , 5 , 8 , 13 , 21 , 34 , ……..

The Method

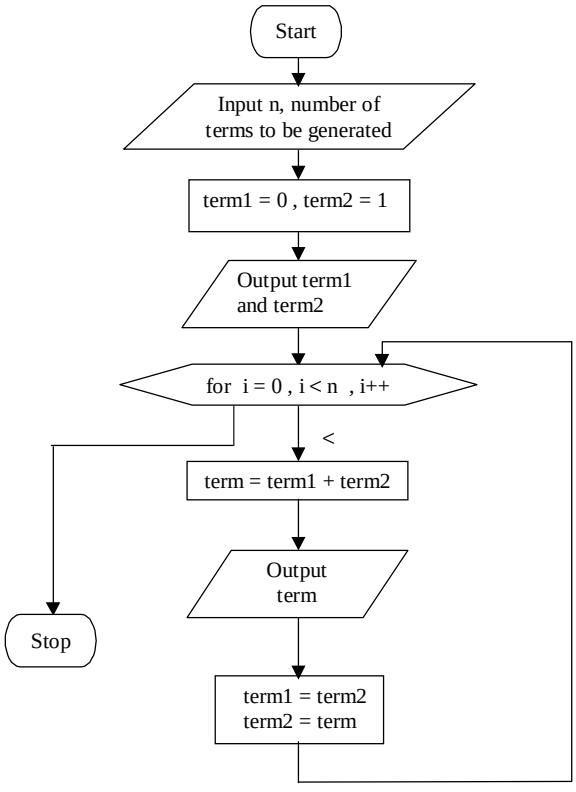
By carefully inspecting the above sequence one can see that the *third term* can be obtained by *adding* the first and second terms. The *fourth term* can be obtained by *adding* the second and the third terms. The fifth term can be obtained by adding the third and fourth terms and so on.

Let `term1` and `term2` be the first and second terms, to begin with. The *third term* is obtained using the relation `term=term1+term2`. For generating the *fourth term,* `term2` is made as `term1`, the term (i.e. third term which was generated in the recent past) is made as `term2` and the relation `term = term1 + term2` is again used. This *term* will be the *fourth term*. The *fifth term* is generated by making `term2` as `term1`, the term (i.e. fourth term which was generated in the recent past) is made as `term2` and once again using the relation `term = term1 + term2`. This procedure is totally *repeated n* times for generating *n* terms. Each *term* is output as and when generated. As two values are assumed in the beginning, total number of terms actually displayed will be (*n*+2).

C-Tips

While generating the required number of terms in the sequence, one should remember that first two terms are taken as known and *n* terms are generated. Therefore, the *loop control* variable `i` may vary from 0 to *less than n*. Note that, here, `i` is used only for the counting purpose. The repeated execution of the required statement is achieved using the `for` loop control structure. This structure has been already explained in Section 4.3.

Flowchart



The Program

```
/* Generation of required number of terms of the Fibonacci sequence
*/

#include<stdio.h>
#include<conio.h>

main()
{

    int n, term1 = 0, term2 = 1, term, i;
    /*Initializing first and second numbers to 0 and 1 respectively */
    clrscr();
    printf("Enter the value of n, the number of terms to be
                                          generated\n");
    scanf("%d", &n);
    printf("\nn = %d\n", n);
    printf("\nFirst %d Fibonacci numbers are :\n", n+2);

    /* Printing first and second number */
    printf("\n%d\n%d", term1, term2);


    /* Repeating for n times since already two numbers are printed */

    for ( i=0; i<n ; ++i)
    {
        term = term1 + term2;
        printf("\n%d",term);
        term1=term2;
        term2=term;

    }

}
```

4.6 FINDING THE GCD AND LCM OF GIVEN NUMBERS

The Method

The GCD is obtained using the Euclid's principle. In this method, the first number (*num1*) is divided by the second number (*num2*). If the remainder is zero, *num2* will be the required GCD. If the remainder is non zero, the division process is *repeated* until the *remainder becomes zero* by making *num2* as *num1* and the remainder (*rem*) as *num2*.

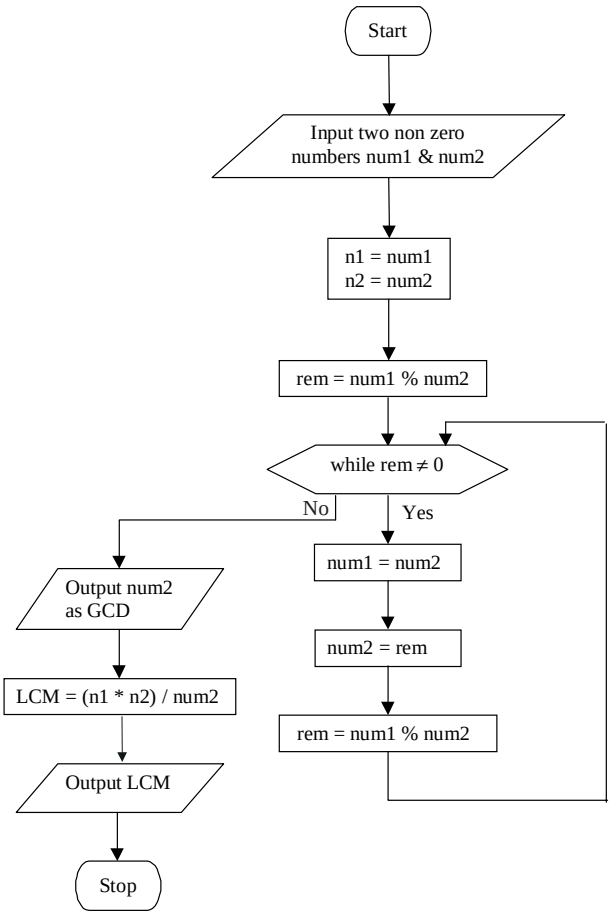
The LCM is computed using the relation

LCM = (num1 * num2) / GCD

C-Tips

The *while* loop control structure, introduced in Section 4.1 is used for repeated execution. The values of given numbers are saved in two variables, say for example in `n1` and `n2`, as they are required for LCM computation as well as for printing out the result.

Flowchart



The Program

```
/* Finding the GCD and LCM of given numbers */

#include<conio.h>          /* Including  header file */
#include<stdio.h>

main()
{
   int num1, num2, rem=0, n1, n2;
   clrscr();
   printf("\nProgram to find the GCD and LCM of 2 integer numbers");
   printf("\n\nEnter two non zero integer numbers\n");
                           /* Reading two non zero numbers */
   scanf("%d%d", &num1, &num2);
   printf("\nnum1 = %d\t num2 = %d\n",num1, num2);
   /* Keeping num1 and num2 in temporary variables to print it later
*/
   n1 = num1;
   n2 = num2;
   rem = num1 % num2;       /* Computing remainder */

   while(rem!=0)            /* Repeat while remainder is not zero */
   {
       num1=num2;           /* Store second number in first number */
       num2=rem;            /* Store remainder in second number */
       rem =num1%num2;      /* Compute remainder */
   }

   printf("\nGCD of %d and %d = %d\n", n1, n2,num2);
                           /* Computing and printing lcm */
   printf("\nLCM of %d and %d = %d\n", n1, n2, (n1*n2)/num2);
}
```

Note: Observe that the required LCM is computed within an output statement itself. Yes, this is permitted in C. Only problem with this type of computation is that, if LCM is required for any other purposes it has to be recalculated.

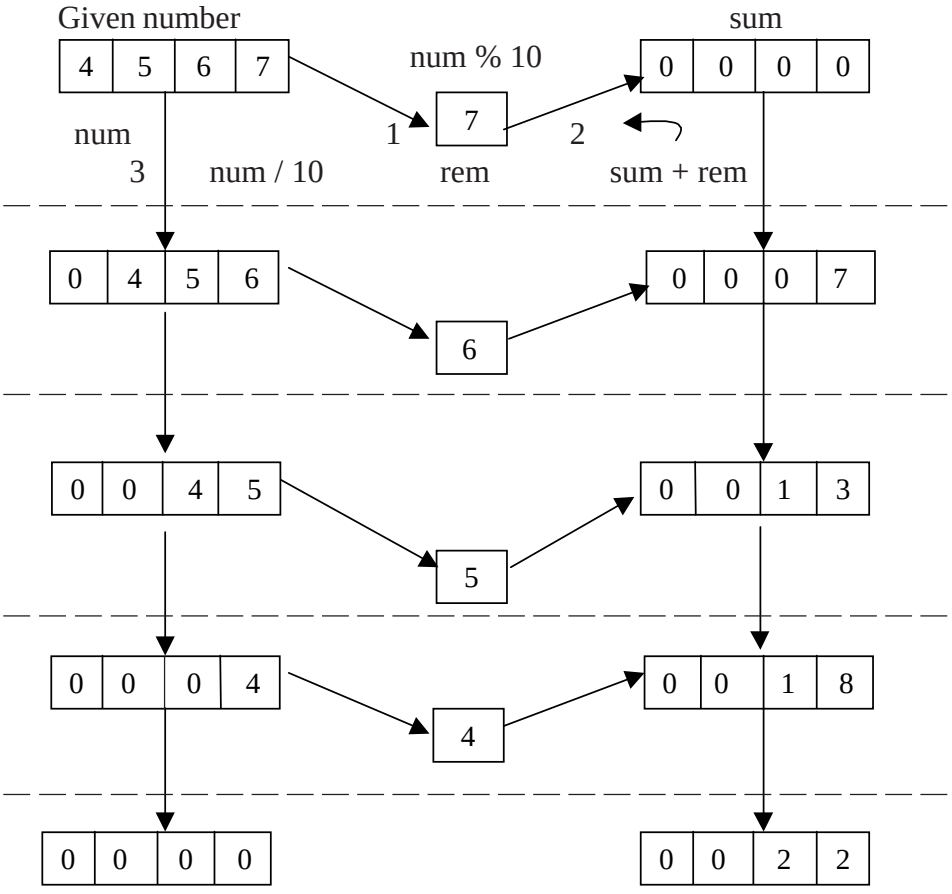
4.7 SUM OF ALL THE DIGITS OF A GIVEN INTEGER NUMBER

Given an integer number, say 4567, the problem here is to find the sum of the individual digits of the given number. That is, we have to find the sum of the digits 7, 6, 5 and 4.

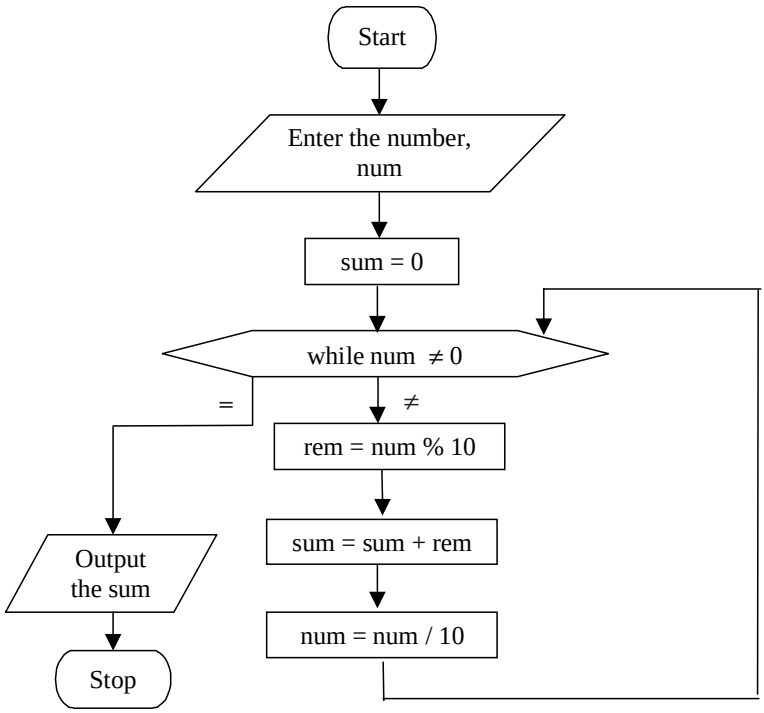
The Method

Let a variable called `sum` is used to store the *determined sum*. The digit in the unit's position is extracted using *modulo 10 division* (as discussed in Section 2.6) and added to the sum. After extracting the unit's digit, the number gets reduced. The above process of extracting the unit's digit and *summing* up is *repeated* till the given number becomes zero. The underlying principle can be well understood by going through the following illustration.

Illustration The numbers 1, 2, 3 on the lines are in the order in which the operations are performed. These set of operations are *repeatedly* done as long as i.e. `while` the number is not equal to zero.



Flowchart



The Program

```
/* To find the sum of all the digits of a given integer number */

#include < stdio.h>
#include<conio.h>
void main()
 {
       int num, rem, sum = 0;
       clrscr();
       printf("Enter a number\n");
       scanf("%d",&num);
       while ( num != 0)
           {
               rem = num % 10;
               sum = sum + rem;
               num = num / 10;
           }
       printf("The sum of the digits of a given number is %d ",sum);
}
```

4.8 REVERSING A NUMBER AND TO CHECK WHETHER IT IS A PALINDROME OR NOT

A number (in fact any string) is said to be a *palindrome*, if it and its reverse are the same. For example, 23132 is a palindrome, as it is same when it is read from right to left as well as from left to right.

The Method

The procedure involved here is to extract the least significant digit (the right most) of the given *number* and making it as current least significant digit of the reversed number. After getting the least significant digit, the remaining digits of the given number are taken as the *number* for the next step. In the next step, once again the least significant digit of the number is extracted and appended to the reversed number, as its new least significant digit, pushing the earlier least significant digit to one decimal place up. This procedure is repeated till the *number* becomes zero.

Modulo 10 division is used to extract the least significant digit, i.e. the digit at unit's position, from the *number* (Section 2.6). *Integer division by 10* is used to get the *modified number* for the next step, i.e. *next iteration*. In the first step, the reverse number is obtained by assigning the first least significant digit to the reversed number. Afterwards, the digits extracted by the *number* are added to the reversed number as its new least significant digit using the relation

```
reversednumber = reversednumber * 10 + digit
```

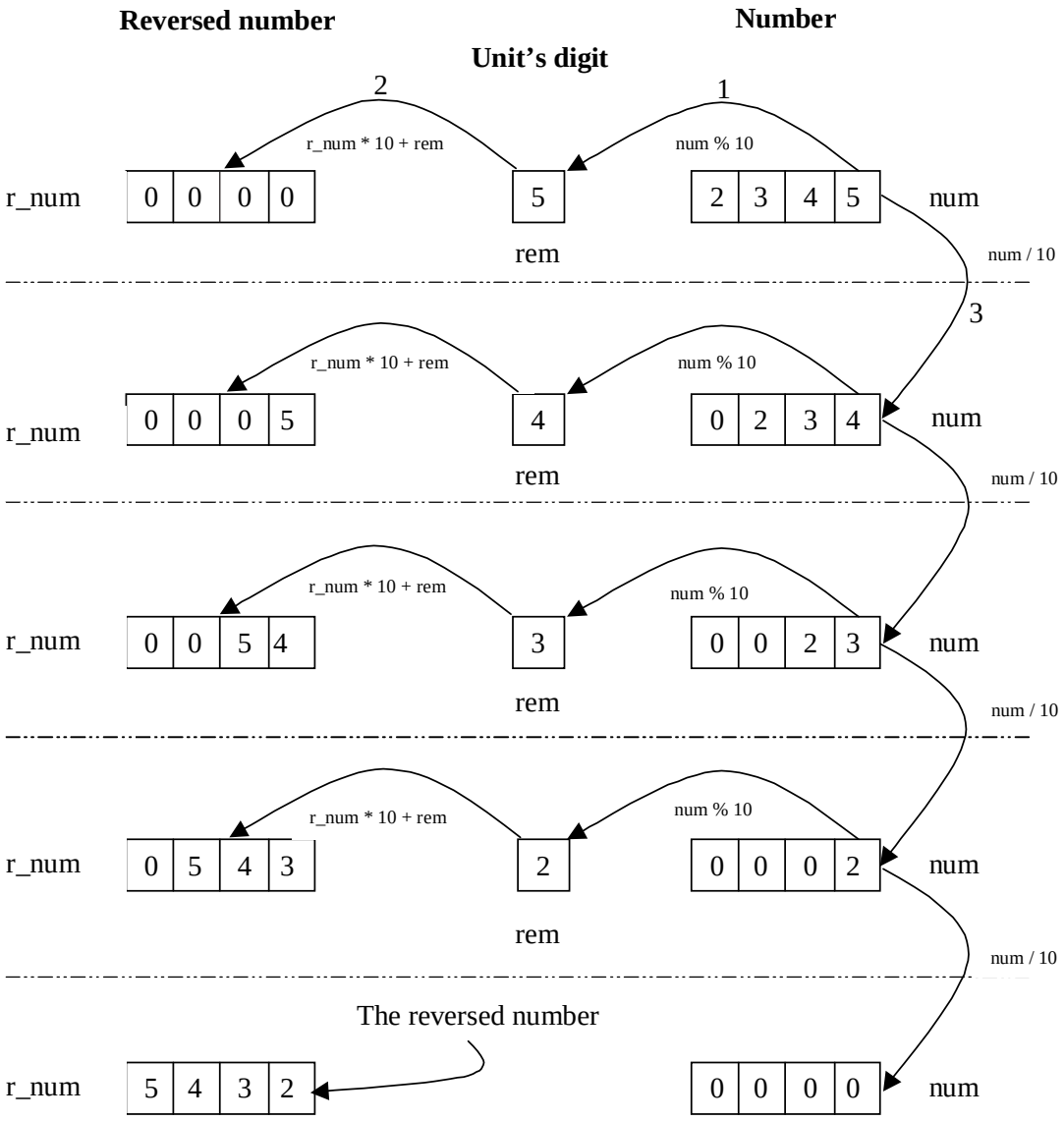
Actually, multiplying by 10 makes room for the digit to be added to the reversed number at the proper position, i.e. the unit's digit position.

C-Tips

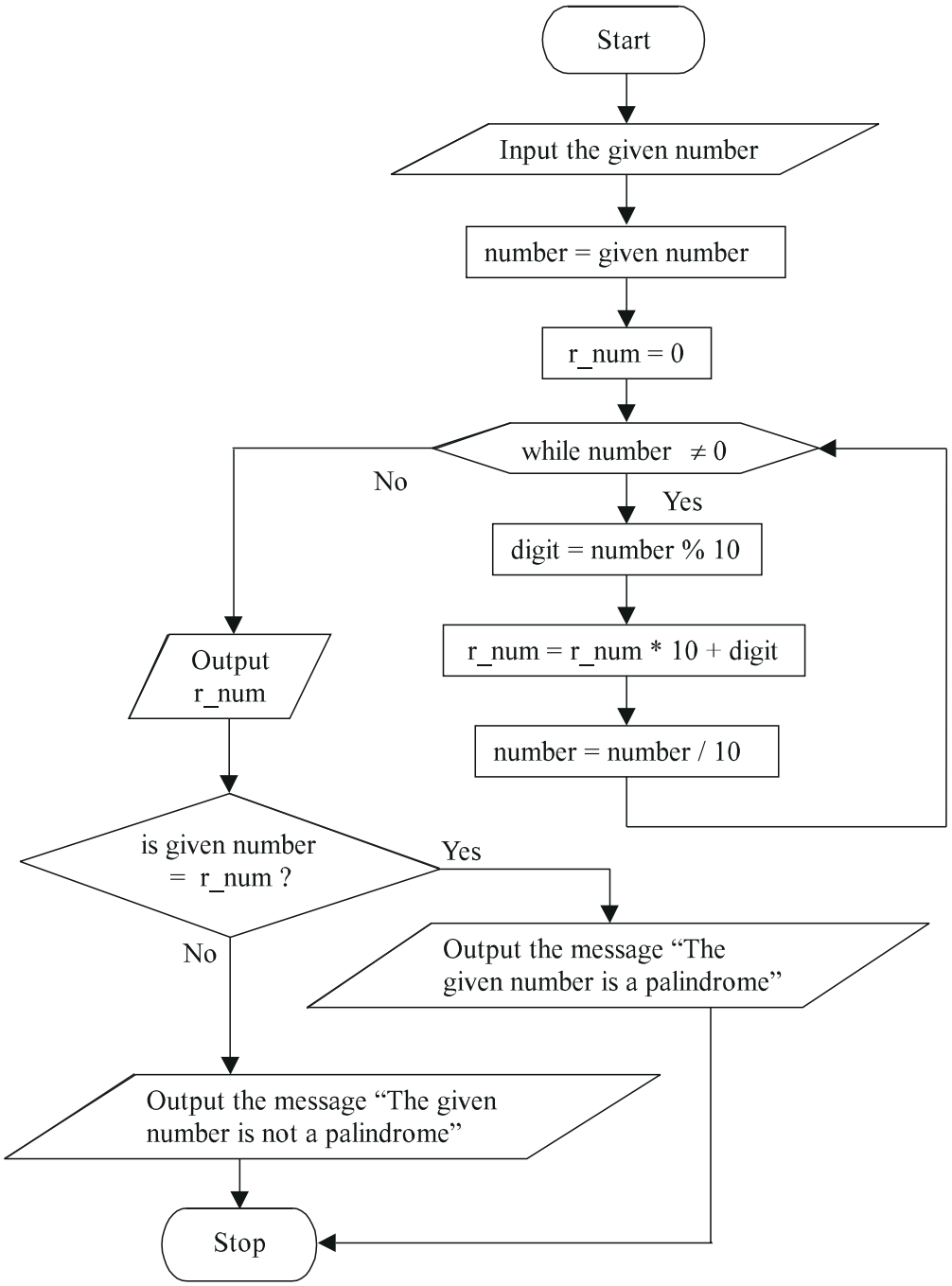
The `while` loop control structure discussed in Section 4.1 is used for repeated execution. As in Section 2.6 modulo division is done using the `%` operator. Equality of two numbers is tested using the `==` operator.

Given below is an illustration of the above process, as well as the corresponding flowchart.

The numbers on the arc indicate the order in which the operations are performed. The above set of operations are repeatedly done as long as, i.e. `while` the *number* is not equal to 0 (Zero).


Reversed number
Number
Unit's digit
2
1
r_num * 10 + rem
num % 10
r_num
0 0 0 0
5
2 3 4 5
num
rem
num / 10
3
r_num * 10 + rem
num % 10
r_num
0 0 0 5
4
0 2 3 4
num
rem
num / 10
r_num * 10 + rem
num % 10
r_num
0 0 5 4
3
0 0 2 3
num
rem
num / 10
r_num * 10 + rem
num % 10
r_num
0 5 4 3
2
0 0 0 2
num
rem
num / 10
The reversed number
r_num
5 4 3 2
0 0 0 0
num

Flowchart


Start
Input the given number
number = given number
r_num = 0
while number ≠ 0
No
Yes
digit = number % 10
r_num = r_num * 10 + digit
number = number / 10
Output
r_num
is given number
= r_num ?
Yes
No
Output the message "The
given number is a palindrome"
Output the message "The given
number is not a palindrome"
Stop

The Program

```
/* Reversing a number and to check whether it is a palindrome or not
*/

#include<conio.h>    /* Including  header file */
#include<stdio.h>

main()
{
   int num,r_num,digit,n;
   clrscr();
   printf("\n\nEnter an integer number\n");
   scanf("%d", &num);
   n=num;             /*Keep the value of num in n for later use*/
   r_num=0;
   printf("\nnumber = %d\t", num);

   while ( num != 0)   /* Repeat while num is not zero */
   {
      digit = num % 10;
                      /* Obtain the remainder by dividing number by 10,
                         so that last digit is obtained */
      r_num = r_num * 10 + digit;
                      /* r_num is multiplied by 10 and added to digit*/

      num = num/10;   /* Dividing number by 10 to get quotient */
   }

   printf("\nThe reverse of the given number %d is %d \n\n",n,r_num);

   if(n==r_num)
                      /* If number and reverse are same then
                         it is palindrome otherwise not */
      printf("reverse = %d , given number %d is a palindrome\n",r_num
           ,n);
   else
      printf("reverse = %d , given number %d is not a palindrome\n",
            r_num,n);
}
```

Note: Since the given number is directly manipulated, its value has to be saved for future usage. Hence it is saved in the variable n.

4.9 TO FIND OUT WHETHER A GIVEN INTEGER NUMBER IS PRIME OR NOT

A number ***n*** is said to be *prime* if and only if it is divisible by 1 and itself. In other words, *n* is not a prime number if it is divisible by any number in the range 2 to (*n* – 1).

The Method

The straightforward procedure is to check for the divisibility of the given number (*n*) by all numbers smaller than itself (except 1), i.e. by numbers : 2, 3, 4, 5, ……..(*n* – 1). Though one feel like to test for the divisibility of *n* by numbers 2, 3, 4, ... upto (*n* – 1), it is not necessary to do so. It is *sufficient* if divisibility is tested for numbers 2, 3, 4, . . . upto *n* / 2. This is because a number *n* is never divisible by a number greater than *n*/2. For example, 8 is NOT divisible by 5, 6, 7, all of which are greater than 8 / 2, i.e. 4. Similarly, 15 is NOT divisible by 8, 9, 10, 11, 12, 13 or 14, all of which are greater than 15 / 2, i.e. 7. (integer division).

Obviously testing for divisibility is done *repeatedly* under the control of a counted loop structure. Here the program control may *come out* of the loop ***normally*** after completing all the iterations or ***abnormally*** after a *successful* testing for division.

When the control comes out *normally*, the loop control variable will be greater than (*n*/2) and the given integer will be a *prime number* (as it was not divisible by any numbers in the range 2 to (*n*/2), rather 2 to (*n* – 1)). When the control comes out *abnormally*, the loop control variable will be less than or equal to (*n*/2) and the given integer will *not be a prime number*.

C-Tips

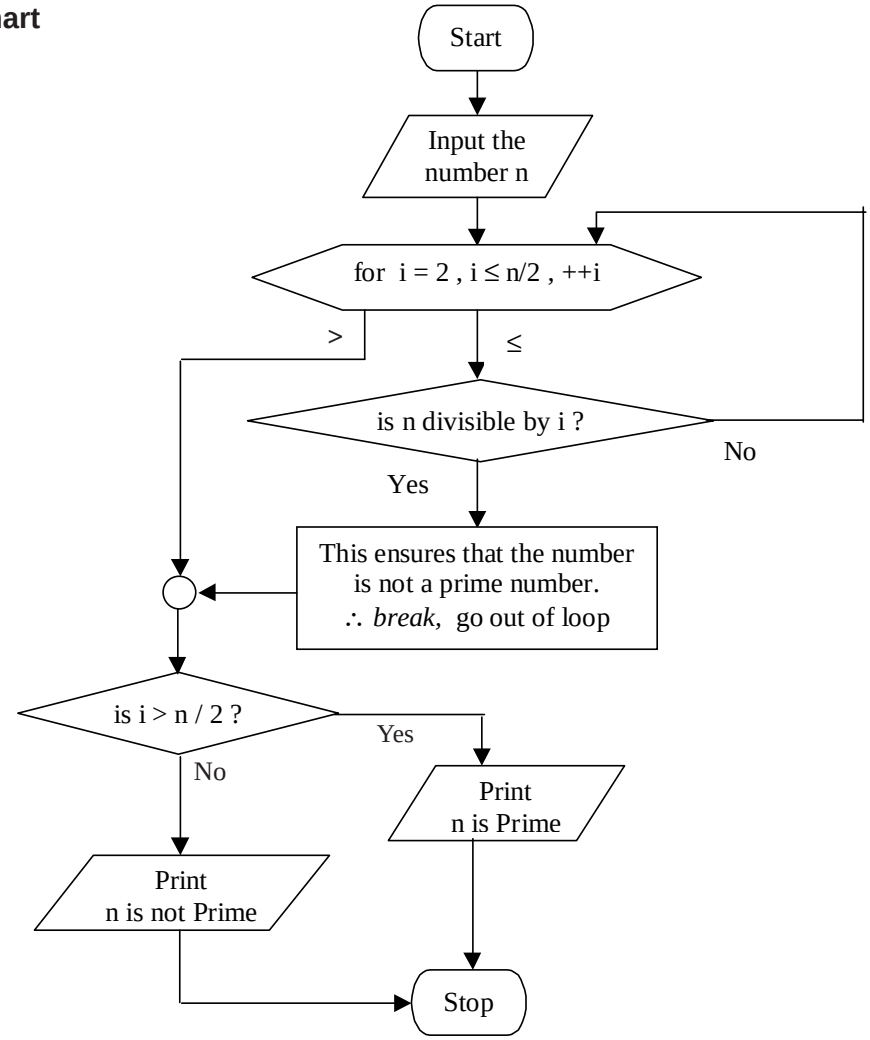
The counted loop control variable itself has been used as numbers less than *n* for the divisibility testing of *n*, i.e. the given number. As soon as the *divisibility test succeeds*, the control has to come out of the loop structure. In C this is achieved by using the *break* statement.

The general format of `break` is

```
break;
```

Execution of a `break` statement from within a `for`, `while`, `do-while` or `switch` block causes execution of that block to be *immediately terminated*. Further execution continue with the statement that immediately follows the loop or `switch` block.

Flowchart



The Program

```
/* To find out whether a given integer number is prime or not */

#include<stdio.h>
#include<conio.h>

main()
{
    int n,i;
```

```
    clrscr();
    printf("\nEnter an integer number \n");
    scanf("%d", &n);              /* Reading a number*/
    printf("Given number = %d\n", n);

    /* Initializing i to 2 (smallest factor other than one) and
       repeating   for all values of i less than or equal to n/2 */

    for(i=2; i<=n/2; ++i)
    {
      /* Dividing number n by i,If remainder is zero it is not a prime */

        if ( n%i == 0)
            break;

    }

     /* To check whether all possible numbers are tested and none of
        them are factors */

    if ( i > n/2 )
          printf("%d is prime\n", n);
    else
          printf("%d is not a prime \n", n);
}
```

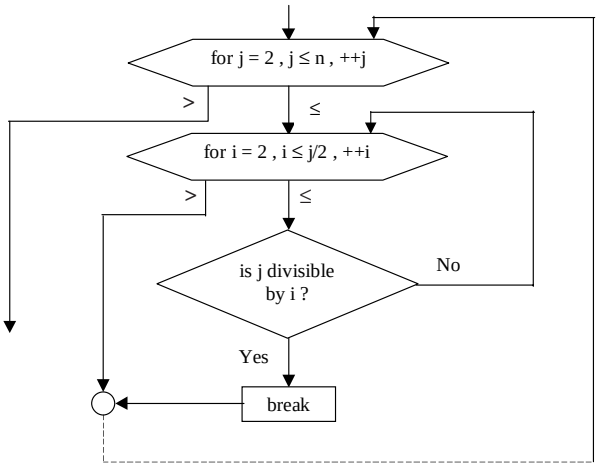
4.10 PRIME NUMBERS BETWEEN 1 AND *N*

The Method

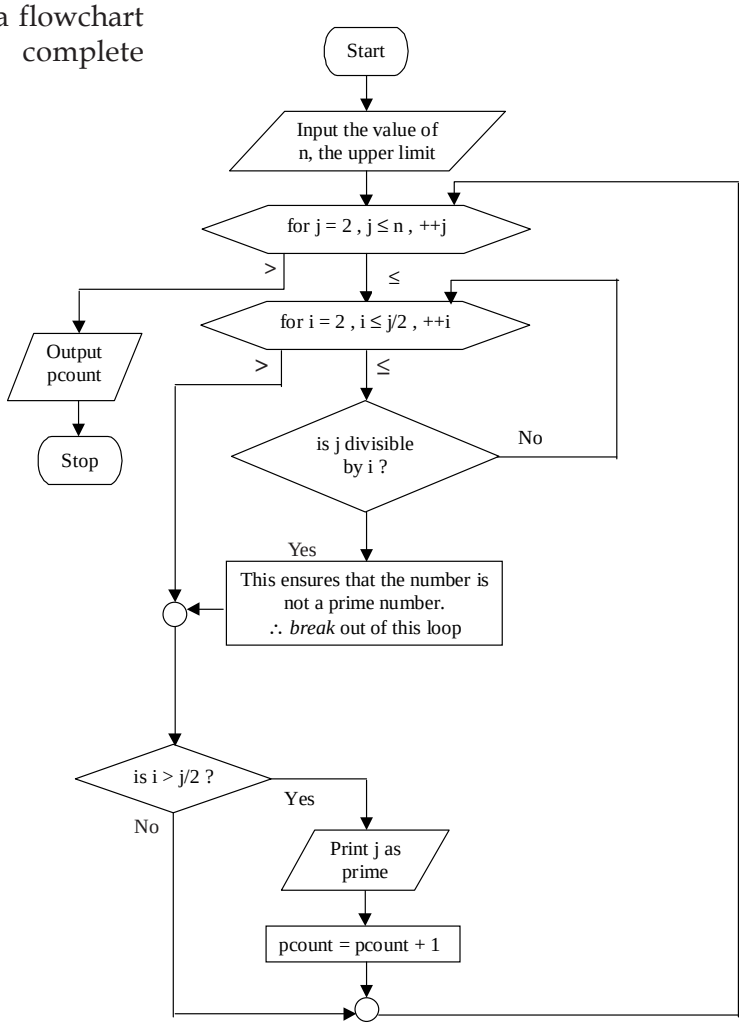
The definition of a prime number and how to find it have been already discussed (in the previous example). Using the same technique *repeatedly* for *every number* in the range 1 to *N* we can identify all the prime numbers in the required range. A counter, say `pcount` is used to count the number of prime numbers in the range as and when a prime number is identified. The underlying principle can be clearly understood by going through the following illustration.

Illustration Assume that we are interested in generating prime numbers in the range 1 to 5, both inclusive.

Since 1 is not considered as a prime number, we have to generate the numbers 2, 3, 4 and 5, and find out whether each one of these is a prime number or not. As each one of these numbers are to be considered to identify whether it is prime or not, we have to use the *prime number identification procedure* (discussed in Section 4.9) repeatedly. For this we can use the `for` loop structure with some loop control variable (say, *j*). Obviously *j* should acquire the values 2, 3, 4 and 5. Therefore *j* has to vary from 2 to 5 in steps of 1. The involved technique may be pictorially shown in a flowchart fragment as follows. The complete flowchart is also given.



Flowchart



The Program

```
/* Prime numbers between 1 and N */

#include<conio.h>            /* Including  header file */
#include<stdio.h>

main()
{
   int n,i,j,pcount = 0;
   clrscr();

   printf("\nEnter the upper limit of the range.\n");
   printf("(1 is assumed to be the lowerlimit of the range)\n");
   /* Reading upper limit */
   scanf("%d",&n);

   printf("\n\nGiven range is 1 to %d\n", n);
   for(j=2; j<=n; ++j )      /* Repeat for all numbers from 2 to n */
    {
    /* Initializing i to 2 and repeating while i is less than or equal
      to j/2. To check whether this value of j is prime or not*/

      for(i=2; i<=j/2; ++i)
    /* Dividing number j by i. If remainder is zero j is not a prime */

         if ( j%i == 0)
            break;
    /* Whether all possible numbers are  tested and none of them are
   factors */

      if ( i > j/2 )
      {
        printf("%d\t", j);  /* Then it must be prime */
        pcount++;           /* Counting number of prime numbers*/
      }
    }
```

```
    printf("\n\nNumber of prime numbers in range 1 to %d is %d.\n",n,
            pcount);
}
```

4.11 SUM OF FIRST N TERMS IN THE EXPANSION OF SINE SERIES

The Method

The *general procedure* to find the sum of a series is to

1. Find the nth term (T_n) and $(n + 1)^{th}$ term (T_{n+1}) of the series.
2. Find a general relationship between T_n and T_{n+1}.
3. Generate the T_{n+1} knowing the T_n.

Obviously, to begin with, the first term should be known. The second term is generated using the first term, the third term is generated using the second term, and the fourth term is generated using the third term and so on. It may be noted that this technique was not explicitly used in the earlier examples of series handling, i.e. in examples discussed in Sections 4.1, 4.2, 4.3. Since those series were trivial in nature and the reader could decide about the next term generating mechanism just by observation, this process was not required.

At every step, the sum is computed from the known generated terms. Thus the important step in solving this type of problem is to find a *general relationship* between the term to be generated and the previous term. Then this relation is executed repeatedly using one of the loop control structures available in the coding language. This process is stopped under the following situations:

(i) After *required number of terms are generated* and the sum is computed.

(ii) When the generated term does not significantly contribute to the sum. In other words, when the *required accuracy is achieved*.

(iii) Both the situations discussed under (i) and (ii) above are considered simultaneously and the program execution is stopped whenever either of the above two is satisfied first.

For example, consider the summation of the series

$$\text{Sin}\,(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots\ldots$$

In the given series observe that

1st term $= x$, 2nd term $= -x^3/3!$, 3rd term $= x^5/5!$, and so on.

The nth term,

$T_n = x^{2n-1}/(2n-1)!$ .... obtained by inspection.

$(n + 1)^{th}$ term, $T_{n+1} = -x^{2n+1}/(2n + 1)!$ .... also obtained by inspection.

$$= \frac{-x^{2n+1}(2n-1)!}{x^{2n-1}(2n+1)!}$$

$$\frac{T_{n+1}}{T_n} = \frac{-x^2}{(2n)(2n+1)}$$

$$T_{n+1} = \frac{-x^2 * T_n}{(2n)(2n+1)}$$

i.e. $$\boxed{\text{term} = \frac{-x^2 * \text{term}}{(2n)(2n+1)}}$$

Actually, the *term* on the right-hand side of the above expression corresponds to the *present term* and the term on the left-hand side corresponds to the *next term*.

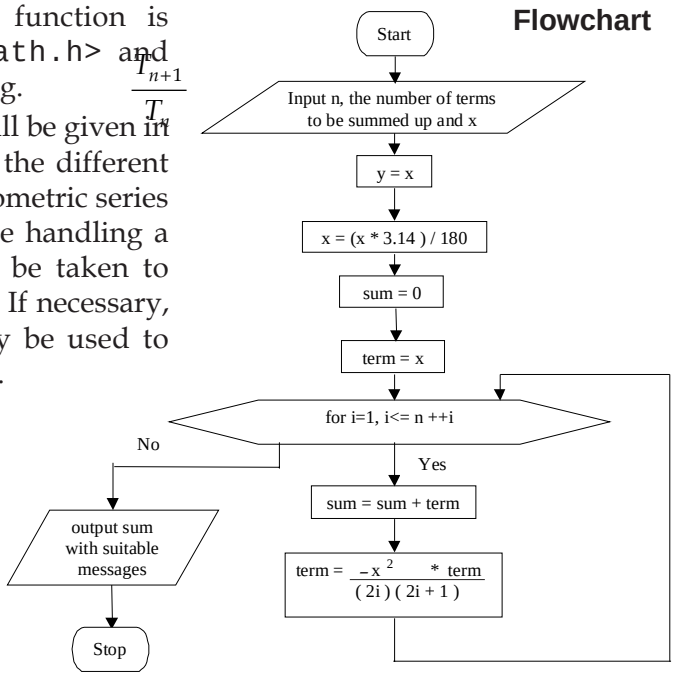
C-Tips

The `for` loop control structure is used for the repeated generation of the terms and to compute the required sum. The value of `sin(x)` can be obtained straight away using the library function. This library function is available in the header file `<math.h>` and hence it is *included* in the beginning.

$\frac{T_{n+1}}{T_n}$

Note: Normally the value of x will be given in degrees. The value to be used in the different terms of the expansion of a trigonometric series must be in radians. As such while handling a trigonometric series care should be taken to provide the value of x in radians. If necessary, the relation $180° = \pi$ *radians* may be used to carryout the necessary conversion.

Flowchart



The Program

```
/* Sum of first N terms in the expansion of Sine series */
#include<stdio.h>
#include<conio.h>
#include<math.h>
/*Including math.h header file to caculate*/
/*sin(x) value using library function */
main()
 {
   int n, i;
   /*Sum is initialized to zero, y is a temporary variable to hold x
  value in degrees*/
   float sum=0, term, x,y;
   clrscr();
   printf("Enter the value of x and n\n");

   /*Reading x in degrees and n number of terms*/
   scanf("%f%d", &x, &n);
   printf("x = %f\t n = %d\n", x,n);

   y = x;                          /*Value of x is assigned to y */
   x = x*3.14/180;                 /*Converting x value to radians*/
   term = x;                       /*x value is assigned to term*/

   /*i is intialized to 1 and loop is repeated
     for i is less than or equal to n*/

   for(i=1; i<=n; ++i)
   {
       sum += term;                /*term is added to sum*/
       term *= -x*x/(2*i*(2*i+1)); /*Next term is calculated*/
   }
   printf("Value of sin(%.0f) upto %d terms  is %f\n", y, n, sum);
   /*Calculating sin(x) value using lib function */
   printf("Value of sin(%.0f) using library function=%f\n", y, Sin(x));

 }
```

Note: When the value of a standard trigonometric function is found using its series expansion, it is customary to find the value of the function using library function also. This practice helps in comparing the results. In order to get the value using library function, the given value of x in degrees must be saved. Here, this has been accomplished by saving the value of x in y.

4.12 COMPUTING THE AREA OF CIRCLES GIVEN THEIR RADII

In all the examples discussed till now we are providing *a set of data values* and are getting the results. In a practical situation one might have *more than one set of data values* for which "required computations" are to be carried out once, for each data set. Thus the problem, here, is to compute areas of circles corresponding to different radius input interactively (i.e., input in response to the system's request).

The Method

In situations like this the program has to be *run afresh for each and every set of data values*. For example, let us assume that we have a set of data values each one corresponding to a radius. Also assume that we need to compute the area of circles corresponding to these radii. The general procedure could be

1. Have a 'program' that computes the area of a circle given its radius (like the one presented in Section 2.4)
2. The 'program' is run once with a value of the radius and the result is printed out.
3. Allow the 'program' to *interrogate* the user regarding 'whether he wants to re-run the program or not'. In other words, the program is allowed to find out whether there are some more data values (radii values, in this case) or not. Obviously there should be a facility in the program to *'trap'* the *user's response*. The 'program' is *'re-run'* or *'terminated'* depending upon the user's response.

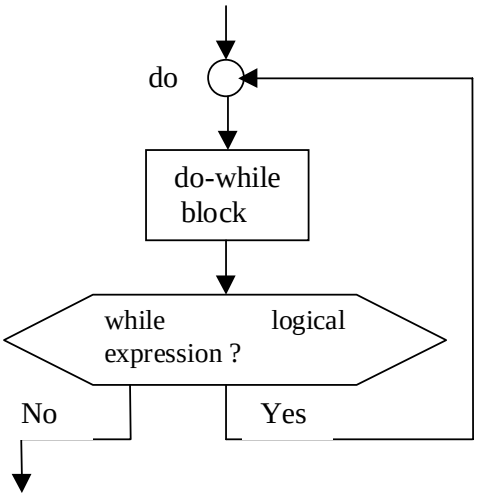
C- tips

Here, the program has to be *run at least once*. Also, depending on the number of data values, we might need to run the program *repeatedly*. Under such circumstances the use of `do….while` loop structure is recommended. The general format of the `do….while` is given below. When the system asks the user to continue processing or not, the user's response will be generally '`y`' or '`n`', meaning *yes* or *no* respectively. Therefore a character type variable, say *choice*, is used to trap the user's response. Obviously the format specifier for handling this variable has to be **`%c`**.

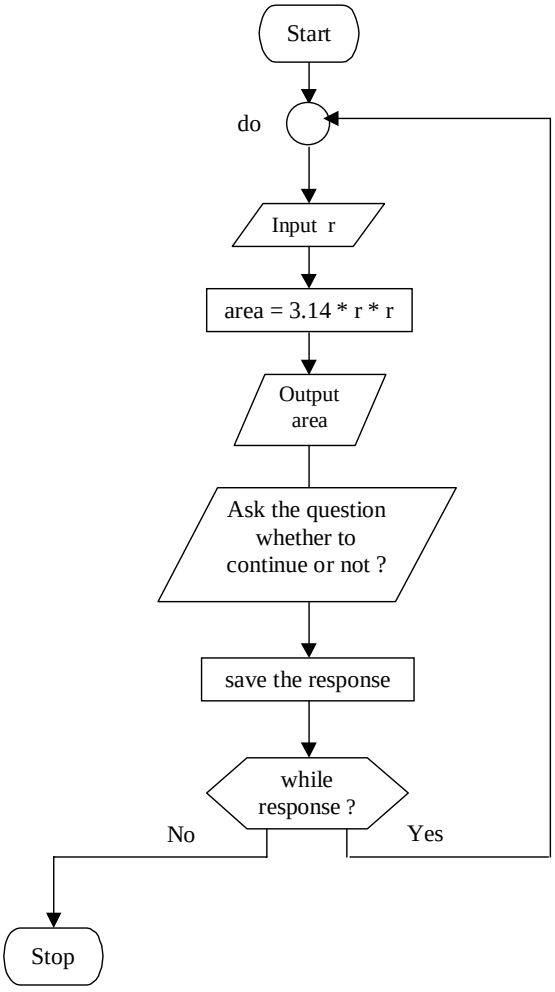
```
do
{
       do-while
       block

}while(logical expression);
```

When the control comes across **do** it enters into the *`do…while` block* directly and executes all the statements in that block once. At the end it comes across the *while statement* and evaluates the adjoining *`logical expression`*. If the evaluation of the logical expression results in a YES or TRUE value, the control *goes back* and the *`do….while` block* will be executed once more. Otherwise, the control goes beyond the 'while statement' to the statement which immediately appears after the *`while`* statement. Following is the complete flowchart.



Flowchart



The Program

```
/* computing the area of circles given their radii */

#include<stdio.h>
#include<conio.h>
main()
{
    /*declaring the variables*/
    float r,area;
    char choice,
    /* clearimg the screen*/
    clrscr();

    do
    {
        /* accepting the radius*/
        printf("Input the value of a radius\n");
        scanf("%f",&r);

        /* calculating the area of the circle*/
        area=3.142 * r * r;

        /* printing the radius and the area */
        printf("\tRadius = %5.2f\tArea = %6.2f\t\n",r,area);

        printf("\nDo you want to continue (y-YES , n - NO)\n");
        getchar();
        /*accepting the choice to continue or not*/
        scanf("%c",&choice);

    }   while(choice == 'y');

}
```

EXERCISES

1. The first few numbers of a sequence are 1 3 4 7 11 18 29 47 …….. . Develop a flowchart to generate required number of terms of the sequence. Also implement the same using C.
2. Develop a flowchart to multiply two given positive integers by the method of successive addition. {Hint: 4 X 3 = 4+ 4+ 4 : Adding three 4's}
3. Develop a flowchart to count the number of digits in a given integer.
4. The function cos(x) is expanded as cos(x) = 1 – $x^2/2!$ + $x^4/4!$ - $x^6/6!$ + … Develop a flowchart to compute the sum of the above series expansion, correct up to four decimal places.
5. Develop a flowchart to compute the sum of the exponential series: $1 + x/1! + x^2/2! + x^3/3! + \ldots$. The summation has to be done for n terms as well as for a desired accuracy. However, the computation should stop whenever either of the condition is satisfied first.
6. Develop a flowchart to find the sum of first n terms of the expansion of the sin(x) function for all values of x from 0° to 180° in steps of m^0 and to tabulate the computed sums. Implement the same in C.

CHAPTER 5

Multi-Way Decision Making

We come across many situations where we have to make decisions among many alternatives. For example, a university awards a grade to a student as either *A* or *B* or *C* or *D* or *E* based on his marks, a specific business house gives a discount depending on the amount of purchase made, a bank gives interest on the amount deposited depending on the time period for which the deposit is made and so on. A careful observation of all the examples cited above reveals the fact that one has to choose an alternative among many alternatives. The process of selecting an alternative among many alternatives is generally referred to as *multiway decision making*. Multiway decision making can be implemented using two way decision making facilities. However most of the computer languages provide an *exclusive facility* to manage multiway decision making. In C multiway decision is made by employing the `switch` construct. Some examples which employ multiway decision making are discussed in this chapter.

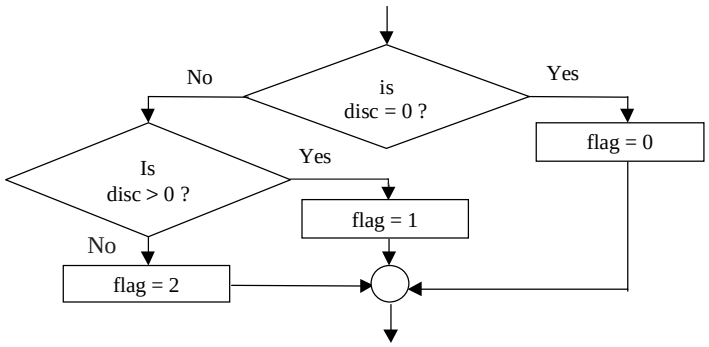
5.1 SOLVING GIVEN QUADRATIC EQUATION USING MULTI-WAY DECISION MAKING FACILITY

The Method

As already discussed in Section 3.6 an equation of the form $ax^2 + bx + c = 0$ is known as a quadratic equation. Solving a quadratic equation means finding out its roots. Every quadratic equation will have two roots. While solving a quadratic equation one of the important steps is to compute the discriminant using the relation $discriminant = (b^2 - 4ac)$. This discriminant could be zero, positive or negative. Obviously, for solving this problem we have to consider *three cases*: one for zero value of the discriminant, one for a positive value of the discriminant and another for a negative value of the discriminant. *These three cases* may be *mapped* (translated) on to some *case values* using some simple arithmetic or

logical operations and then a *multiway decision* making facility of the coding (programming) language is used. For example, *computed* `goto` of FORTRAN, `switch` of C, etc. provide the necessary multiway branching.

Let some variable (say, `flag`) is used to store the different case values. Different case values are obtained and stored in this variable `flag` using the following strategy.



C-Tips

The `switch` statement is used to get the multiway branching. The general format of this statement is

```
switch (expression)
{
       case  value-1 : ……..
                       ……..
                       break;

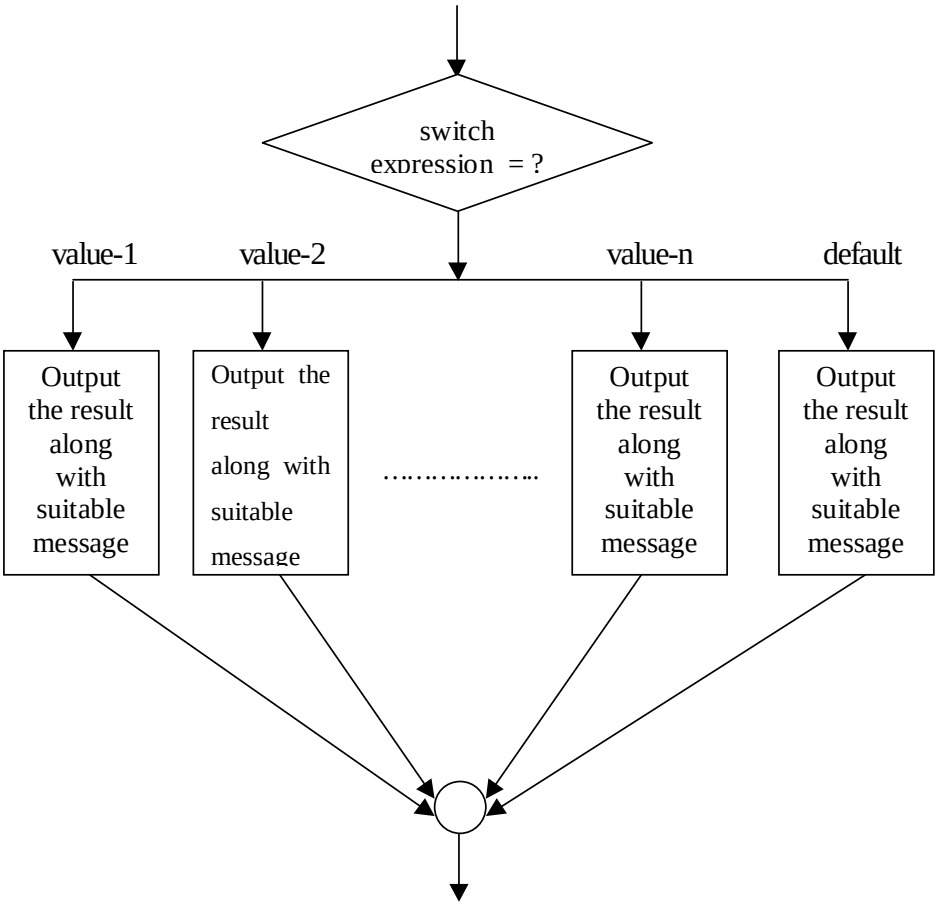
       case  value-2 : ……..
                       ……..
                       break;
             .
             .
             .

       case  value-n : ……..
                       ……..
                       break;

             default : ……..
                       ……..
                       break;
}
```

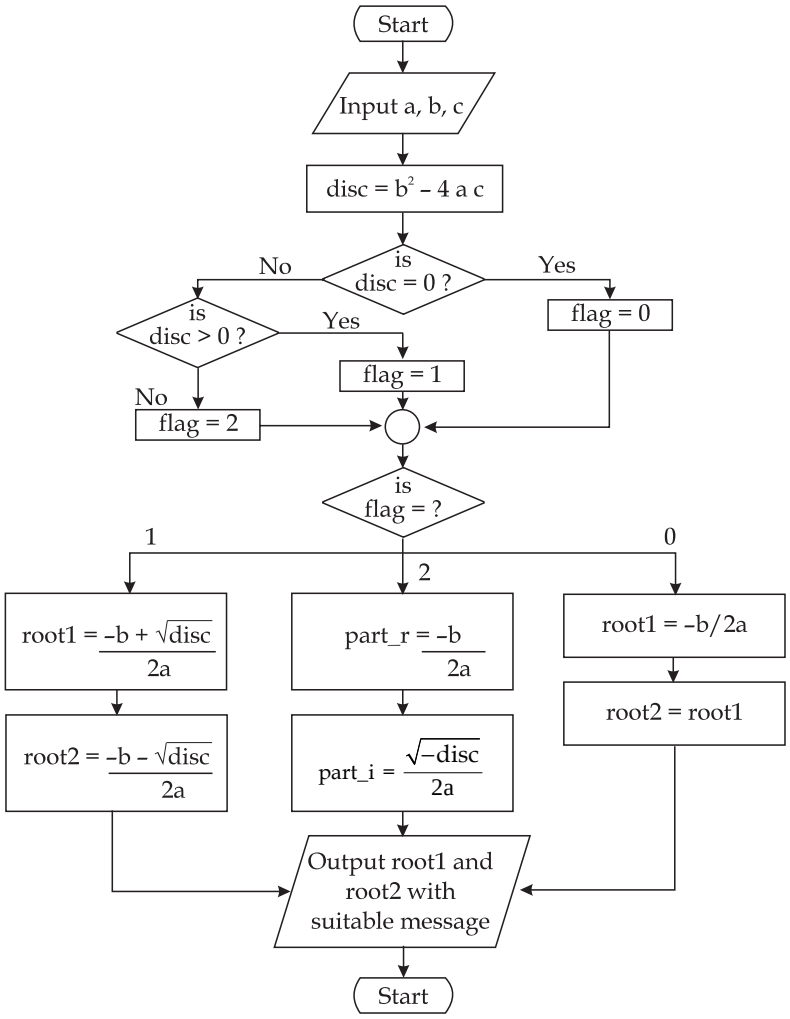
When the control comes to a switch statement the expression that appears with it is first evaluated. This evaluation must result in some integer value (`i_v`). Then the control gets into the switch block made up of *many number of cases* including an *optional* default block. The value `i_v` is compared with case values value-1, value-2, ……., one after the other. If `i_v` and case value match then that particular case block is executed. As we are generally interested only in one case, the last statement in a case block will be generally the ***`break`*** statement. Thus after executing a `case` block the control goes out of the entire `switch` block. If none of the `case` blocks are executed then the `default` block, if present, will be executed.

The entire behavior of the `switch` statement can be pictorially represented as shown in the flowchart given below.



Depending upon the value stored in the variable ***`flag`***, the roots are computed using the relations shown in the flow chart (also refer to Section 3.6) given in the next page.

Flowchart



The Program

```
/* Solving a quadratic equation using multiway decision making facility
*/

#include<conio.h>                    /* Including  header file */
#include<stdio.h>
#include<math.h>
main()
```

```
{
float a,b,c,disc,root1,root2,part_r,part_i;
int flag;                          /* Declaring variables */
clrscr();                          /* Clearing the Screen */
printf("\n\nEnter co-efficients of the quadratic equation\n\n");
scanf("%f%f%f", &a, &b, &c);
printf("\nCo-efficients are a = %f\t b = %f\t c = %f\n\n", a,b,c);
if (a==0 || b==0 || c==0)
       /* If any one co-efficent is zero roots cannot be calculated */
printf("Zero co-efficient\n");
else
{
disc=b*b-4*a*c;/* Calculating discriminant*/
if ( disc == 0)  flag = 0;
       if ( disc > 0) flag = 1;
       if ( disc < 0) flag = 2;

       switch( flag )
       {
          case 0 : printf("Root1 = b1 (2*a); are real and equal\n");
                   root2 = root1;
                   printf("\nRoot1 = %f\t Root2 = %f\n",root1,
                            root2);
                   break;

          case 1 : printf("Roots are real and distinct \n");
                   printf("\n");
                   root1 = (-b+sqrt(disc))/(2.0*a);
                   root2 = (-b-sqrt(disc))/(2.0*a);
                   printf("\nRoot1 = %f\t Root2 = %f\n",root1,
                   root2);
                   break;

          case 2 : printf("Roots are complex\n");
                   part_r = -b/(2.0*a);
                   part_i = sqrt(-disc)/(2.0*a);
```

```
                    printf("Root1 = %f + i %f\tRoot2 = %f -i %f \n"
                           ,part_r , part_i, part_r ,part_i);
                    break;

            }
        }
}
```

5.2 AWARDING GRADES BASED ON MARKS SCORED IN AN EXAMINATION

Quite often computers are used for processing examination results. This processing depends upon the policy adopted by the concerned institution or university. Typically, let us consider a situation where grades are awarded according to the following set of rules.

1. Grade *A* is awarded for all those who score 80 or more.
2. Grade *B* is awarded for all those who score 70 and above but below 80.
3. Grade *C* is awarded for all those who score 60 and above but below 70.
4. Grade *D* is awarded for all those who score 40 and above but below 60.
5. Grade *E* is awarded for all those who score less than 40.

The Method

A careful observation of the given set of rules reveals the fact that we have to employ *multiway branching technique.* One of the methods to achieve the required multiway branching is to use `if…..else` *ladder* to implement the entire set of rules. However, using `switch` (using which one can switch on to one of many branches) is simple and straight forward.

This problem can be solved by mapping or translating the entire range of marks into very few possible cases by dividing (integer division) the given marks by 10. For example, 80/10 is 8, 47/10 is 4, 51/10 is 5 and so on. Thus we get exactly 11 possible cases { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 } when marks are divided by 10. All these 11 cases can be classified into just 5 groups. Cases 10, 9 and 8 form one group, case 7 and 6 form two independent groups, case 5 and case 4 form one group. All the remaining cases, i.e. case 3, case 2, case 1 and case 0 form the last group. Now the grading can be achieved by the technique shown in the following flowchart.

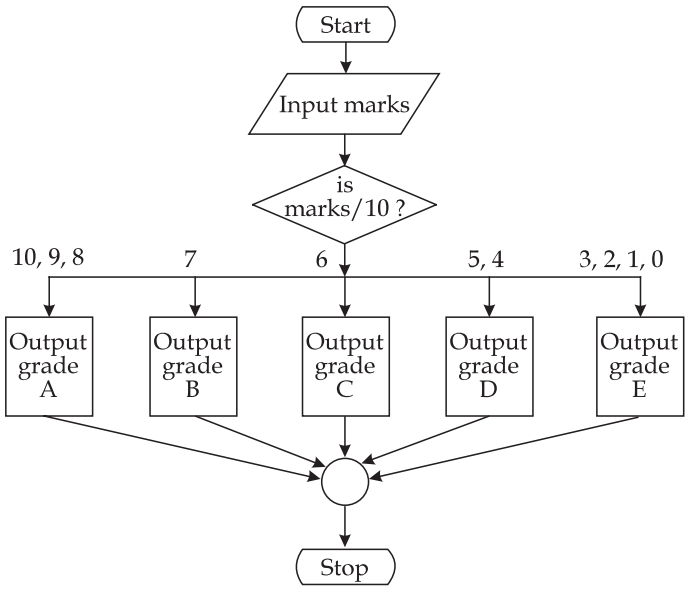
C-Tips

More than one *case* can be made to execute the *same set of statements*. For example, the program segment given below is valid.

```
— — —
— — —
case 10 :
case  9 :
case  8 : printf("\n Grade A \n");
          break;
— — —
— — —
```

The above segment will be executed for the case values of 10 and 9 as well as 8.

Flowchart



The Program

```
/* Awarding grades based on marks scored in an examination */

#include<stdio.h>
#include<conio.h>
void main( )
   {
     int marks;
     clrscr( );
```

```
      printf("Enter the marks\n");
      scanf("%d", &marks);
      if (marks > 100) exit (1);
      /* The 0 – 100 range of marks is mapped into 10 different cases
           using the marks/10 expression*/

      switch(marks/10)
       {
            case 10 :
            case  9 :
            case  8 : printf("Grade is A\n");
                      break;
            case  7 : printf("Grade is B\n");
                      break;
            case  6 : printf("Grade is C\n");
                      break;
            case  5 :
            case  4 : printf("Grade is D\n");
                      break;
           default : printf("Grade is E\n");
                      break;
   }

}
```

5.3 A SIMPLE CALCULATOR

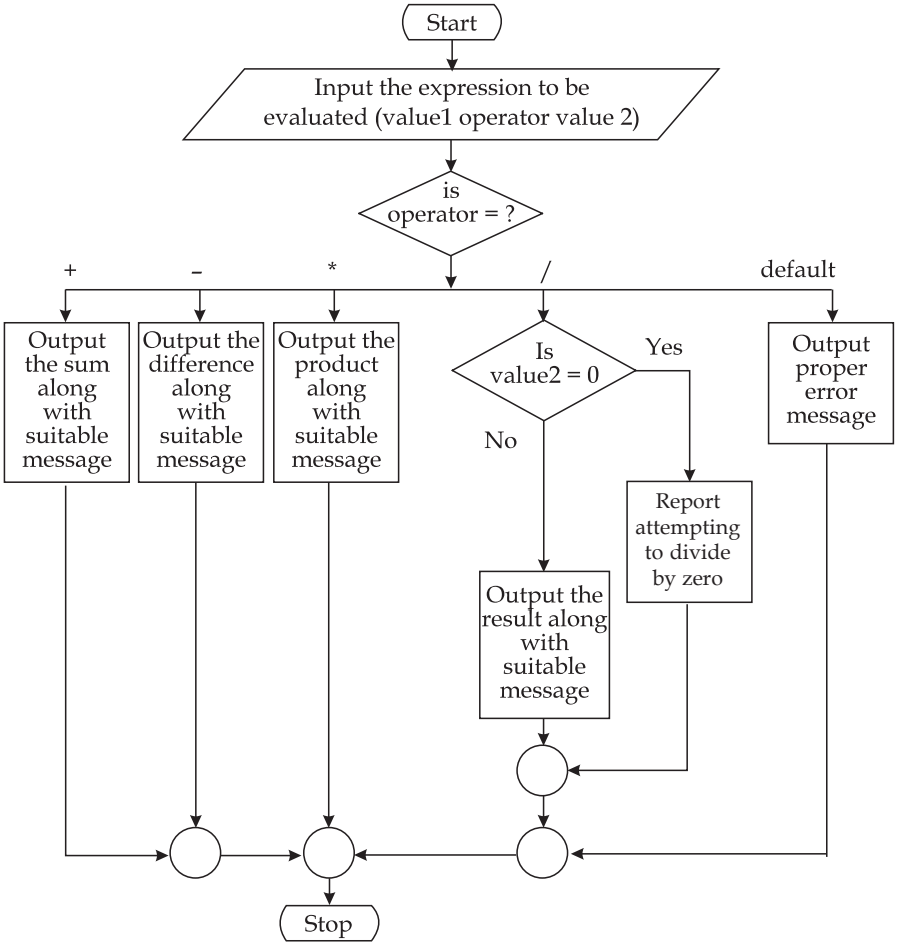
A simple calculator is the one which is used to carry basic operations like addition (+), subtraction (–), multiplication (*) and division (/). In other words, expressions like **4 + 5, 14 – 10 , 12 * 5** and **3/5** are evaluated using such a simple calculator. As the operator appears between the two operands in all the expressions given above, these expressions are said to be in **infix** form.

The problem here is to read-in an arithmetic expression given in the *infix* form, evaluate it and output the result. If any attempt is made to divide by 0 (zero) or evaluate an expression with an unidentified operator, errors must be reported.

The Method

As already mentioned, a simple arithmetic expression is first *read-in*. After *reading in* the expression to be evaluated, the *operator* has to be recognized first and then branch out to the *'evaluating section'* accordingly. For example, if the operator recognized is '+' then we have to branch out to the *add section*, if the operator recognized is '-' then we have to branch out to the *subtract section*, and so on. As there are four operators to be *taken care-of*, there will be a five-way (four for permitted operators and one for unidentified operators) branching. As multi-way branching takes place one has to use some *multi-way branching control structure* like `switch` statement available in C.

Flowchart



The Program

```
/* Program to simulate a simple calculator */

/* The input will be in infix form i.e. operand1 operator operand2 */

#include<stdio.h>
#include<conio.h>
main()
{
   float value1,value2;
   char operator;
   clrscr();

   printf("Type in your expression.\n");
   scanf("%f %c %f",&value1,&operator,&value2);

   switch(operator)
   {
      case '+': printf("The sum of %.2f and %.2f is :
                  %.2f\n",value1,value2, value1+value2);
         break;

      case '-': printf("The difference of %.2f and %.2f is : %.2f\n",
                  value1,value2,value1-value2);
           break;

      case '*': printf("The product of %.2f and %.2f is : %.2f\n",value1,
                  value2,value1*value2);
         break;

      case '/': if(value2==0)
        printf("Division by zero.\n");
          else
                 printf("The result of %.2f/%.2f is : %.2f\n",value1,
                    value2,value1/value2);
```

```
         break;

      default: printf("Unknown operator.\n");
         break;

   }

}
```

Note: Observe that evaluation of the required expressions has been done inside the `printf()` statement itself. This is permitted.

5.4 SELECTING AN OPERATION BASED ON A MENU

Assume that we are interested in some file operations. Let the different file operations be file reading, file writing, file saving and file deletion. In such circumstances a *menu* displaying the different actions that have to be performed can be used and any one action is selected.

```
        Menu
❒ W) Write a file
❒ R) Read a file
❒ S) Save a file
❒ D) Delete a file

   Enter your Choice:
```

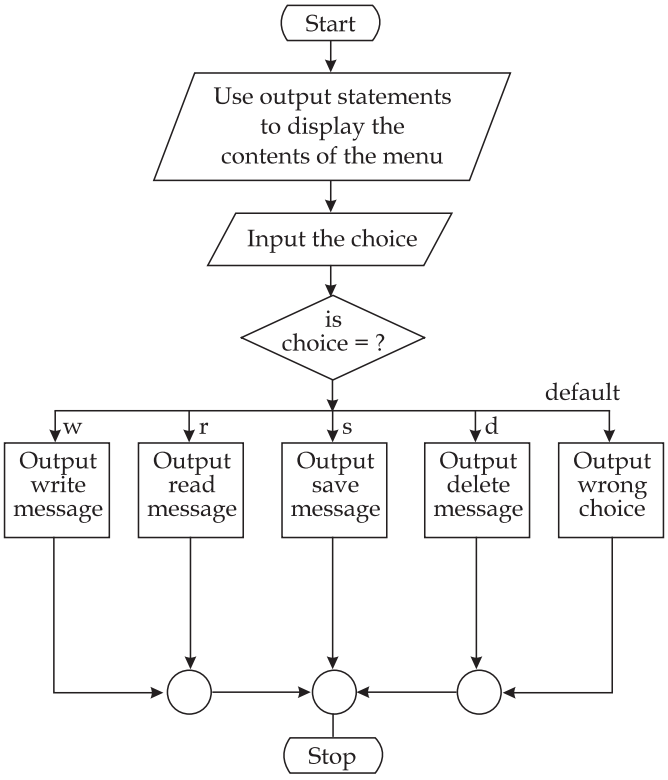
The Method

Output statements are used to display the contents of the required menu. Then an input statement is used to accept the choice and act. As there are five actions (including the wrong choice) we have to take a *multi-way decision*. Obviously for this type of decision making a control construct like `switch` is used. The flowchart below shows the technique involved

C-Tips

Since the selection is based on characters, the format specifier **`%c`** is used. It may be noted that characters are stored inside the machine in the form of integers. Hence characters can be used as case values.

Flowchart



The Program

```
/* Program for selecting an operation based on a MENU */

#include<stdio.h>
#include<conio.h>

main()
{
   char choice;
   clrscr();

   printf("\t\t\t\t  MENU\n\n");
   printf("\t\t\t  W : Writing a file \n");
   printf("\t\t\t  R : Reading a file \n");
   printf("\t\t\t  S : Saving a file \n");
```

```
   printf("\t\t\t  D : Deleting a file \n");

   printf("\t\t\t Enter your choice : ");
   scanf("%c",&choice);

   printf("\n");

   switch(choice)
   {
     case 'W':  printf("The file operation selected is : Writing a
                       file\n");
                break;

     case 'R':  printf("The file operation selected is : Reading a
                       file\n");
                break;

     case 'S':  printf("The file operation selected is : Saving a
                       file\n");
                break;

     case 'D':  printf("The file operation selected is : Deleting a
                       file\n");
                break;

     default:   printf("Invalid Input \n");
                break;

   }

}
```

EXERCISES

1. Develop a flowchart to identify and display the day of a week given an integer between 0 and 6. The first day of the week i.e. 0, may be assumed to be Sunday. Write a C program for the same.
2. Given an array of elements you are expected to determine any one of the following
   - Biggest
   - Smallest
   - Sum
   - Average of all the elements

   Develop a flow chart to solve this problem using multiway decision facility. Write a C program for the same.

CHAPTER 6

Arrays

The question *'why we deal with numbers so frequently?'* has been discussed in Chapter 3. Examples cited there dealt with the need of numerical values such as: university seat numbers, bank account numbers, file numbers etc. All register numbers will have some *definite pattern*. For example, let 76EE82 be a university seat number. Here 76 may indicate the year of admission, EE may indicate the department of study, say Electrical Engineering, and 82 may indicate the serial number of the candidate admitted during that year. Obviously, all the university seat numbers have to be and will be of the same pattern. This is true in all other (and many more) cases. In other words, all social security numbers will be of the same pattern, all file numbers will be of the same pattern and so on. Thus quite frequently, in real life situations, we come across *groups of data items having similar properties*. In the field of computers, under data processing environments *groups of data items having similar properties* are taken together and are called *arrays.* Just like individual data items every group i.e. *array*, will also have a name called ***array name***. Obviously there should be some way for the computer (i.e. the language processor or compiler) to *distinguish* between an *ordinary variable* name (which is *single valued*) and an *array name* (which is multi valued). This is generally done by *declaring* array names as *arrays* in the beginning of a program, during variables declaration itself. As an array refers to a *group of data items*, the information regarding *number of elements* in the array (i.e. size of the array) must also be provided to the compiler, during the array declaration itself. In fact, appearance of the *size information* during variables' declaration *informs* the processor that this particular variable is an *array*.

Different elements of an array are accessed using subscripts such as **i**, **j** etc. Various data items in an array may be arranged in the form of a *vector* or a *matrix* or in some other definite form. These forms are referred to as 'dimensions' of the array. An array whose data items are arranged in the form of a *vector* is called a *one-dimensional* array. An array

whose data items are arranged in the form of a matrix is called a *two-dimensional* array and so on. As in algebra, elements of a one-dimensional array are accessed using a *single subscript*, elements of a two dimensional array are accessed using *two subscripts*, and so on.

For example, If ***x*** is a *vector* (i.e. a one dimensional *array name*), then the first element is ***x*[0]**, second element is ***x*[1]**, third element is ***x*[2]** and so on. The general element of this array is represented by ***x*[*i*]**. Instead if **x** is a *matrix* (i.e. a two-dimensional *array name*), then the first element is ***x*[0][0],** second element is ***x*[0][1]**, third element is ***x*[0][2]** and so on. The general element of this array is represented by ***x*[*i*][*j*]**.

During an array declaration if no exact information regarding the total number of elements in the array is available, a suitable guess has to be made.

6.1 READING IN AN ARRAY AND TO OUTPUT THE SAME

The Method

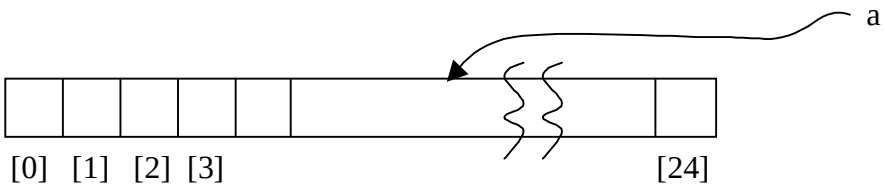
The elements of an array are input, one-by-one, repeatedly under the control of some *loop control statement* with the help of an *input function*. The elements of an array are output similarly with the help of some *output function*. During output, one has to first decide about the number of elements to be displayed or printed per line and then the output statement has to be written.

C-Tips

Let ***a*** be *used as an array consisting of 25 integer elements*. This array variable name **a** is *declared* as an array along with other variables as shown below

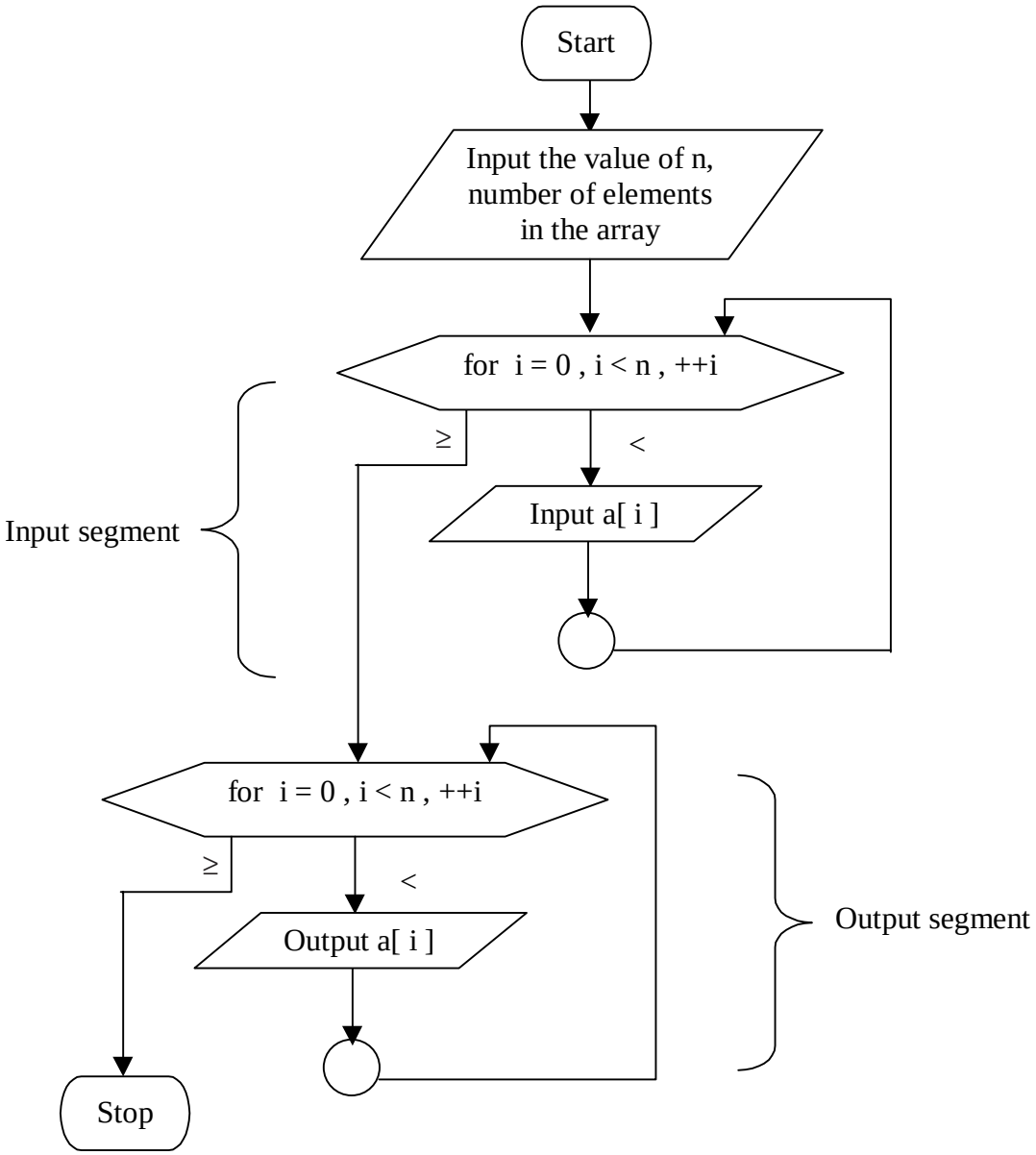
```
int a[25], x=0;
```

When an array is declared, *consecutive memory locations* are allocated or *reserved* for the array, as shown in the following figure. Number of locations allocated will be equal to the size mentioned in the array declaration. The size of each memory location depends upon the *data type* of the elements to be stored in it.



For reading or writing the different elements of an array, `for` loop structures are used along with input or output functions such as `scanf()` and `printf()` as shown in the program.

Flowchart


Start
Input the value of n, number of elements in the array
for i = 0 , i < n , ++i
≥
<
Input a[i]
Input segment
for i = 0 , i < n , ++i
≥
<
Output a[i]
Output segment
Stop

The Program

```
/* Reading in an array and to print the same (1-dimensional)*/

#include<stdio.h>
#include<conio.h>

main()
{
   int a[25],i,n;

   clrscr();
   printf("Enter the value of n\n");
   scanf("%d",&n);

   printf("\n Size of the array is %d\n\n",n);
   printf("Enter the elements of the array\n\n");

   /* Input segment */

   for(i=0;i<n;i++)
     scanf("%d",&a[i]);

   printf("\n\tThe elements of the array are\n\n");

   /* Output segment */

   for(i=0;i<n;i++)
     printf("\t%d",a[i]);

}
```

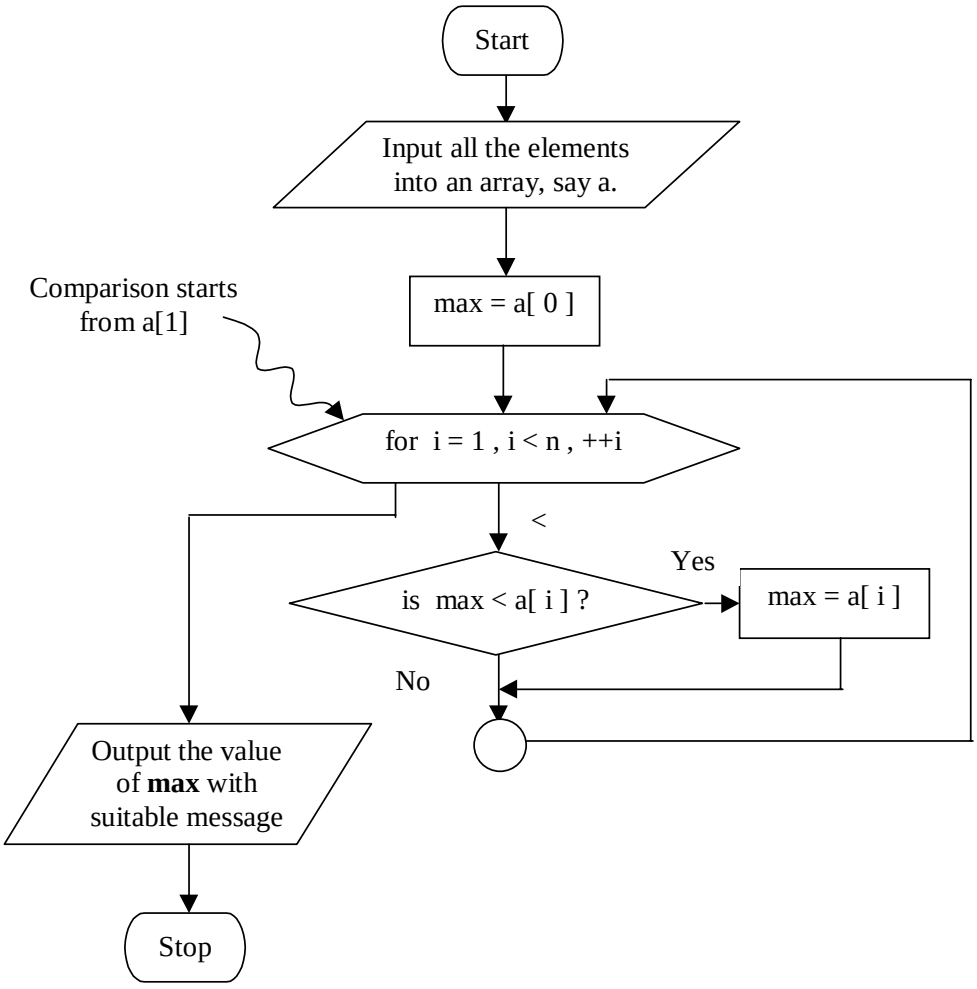
Note: It is only a practice that *i, j, k* etc. are used as subscript variables. There is no restriction that only such variables are to be used as subscripts. Infact, any integer variable or even integer expression can be used as a subscript. It may be noted that subscripts *point to* specific memory positions and position numbers can only be integers.

6.2 BIGGEST AMONG GIVEN 'N' INTEGER NUMBERS

The Method

As '*n*' numbers are to be considered, all these elements are read into an array (say ***a***) and then the *biggest* or the *maximum* among all these is found. To start with the very first element a[0] is assumed to be the maximum *(max*). Then this *max* is *compared* with the elements starting from the 2nd location i.e. starting from the elements at the locations *a*[1], *a*[2], *a*[3],..., *a*[*n* – 1], one by one. If at any comparison stage the *max is found to be smaller than a*[*i*] {*a*[*i*] means any one element in the array ***a***} then the contents of ***a***[*i*] is *copied into max* and the comparison process is continued till the end. At the end the value of the *biggest* element will be available in *max.*

Flowchart



The Program

```
/*  Biggest among given ‘n’ integer numbers */

#include<stdio.h>
#include<conio.h>

main()
{
   int a[25],i,n,max;

   clrscr();
   printf(“Input the value of n\n”);
   scanf(“%d”,&n);
   printf(“Size of the array is %d”,n);

   printf(“\nInput the array elements\n”);
   for(i=0;i<n;i++)
   scanf(“%d”,&a[i]);

   printf(“The elements of the array are\n”);
   for(i = 0;i < n; i++)
     printf(“%d\t”,a[i]);

   /* a[0] is assumed as maximum, initially */

   max=a[0];

   /* comparison starts from the 2nd element of the array i.e. with
     i =  1 */
   for(i = 1;i < n; i++)
   {
     if(max < a[i])
     {
       max = a[i];  /* value at a[i] is copied into max */
     }
   }
   printf(“\nThe biggest is %d among the given elements of the
            array”,max);


}
```

Note: If the *position* of the *maximum element* on the array is required then it can be reported by using the value of the subscript, i.e. *i*, by storing its value in a separate variable, say *pos*, whenever a value is copied into the *max*. Initially *pos* has to be zero (0), since *a*[0] is assumed to be the maximum element to begin with.

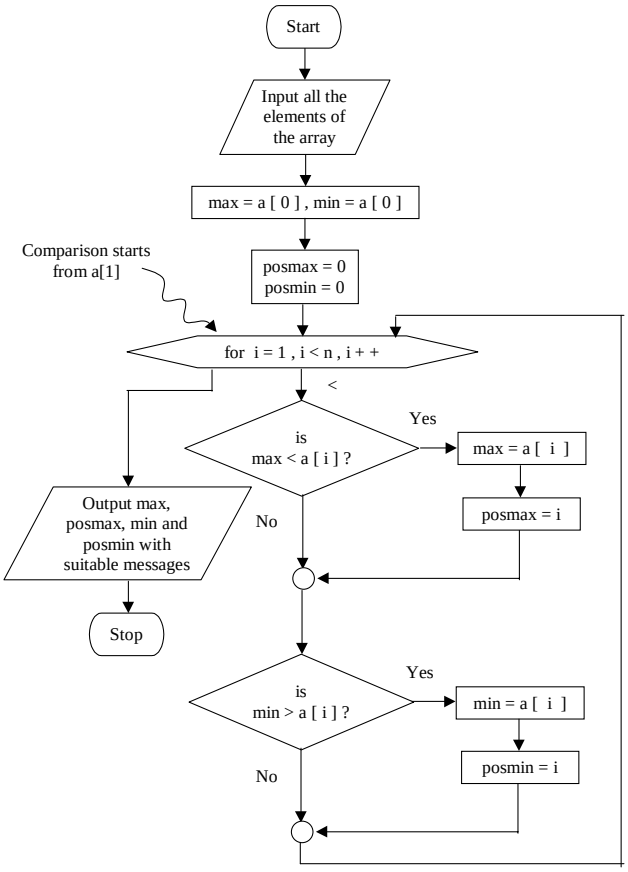
6.3 THE MAXIMUM AND MINIMUM AMONG GIVEN N INTEGER NUMBERS

The Method

Like in the previous example, here also '*n*' numbers are to be considered and are, therefore, stored in an array, say ***a***. Initially the first element {i.e. *a*[0]} is assumed to be both maximum (*max*) as well as minimum (*min*). Then these *max* and *min* terms are *compared* with all the elements of the array starting from the 2nd element (i.e. starting from *a*[1]) till the last element i.e. till a subscript value of (*n* – 1), one after the other, sequentially. At every comparison *if max is less than a*[*i*] then *a*[*i*]'s value is made maximum by storing its value in *max*. At the same time, *if min is more than a*[*i*] then element *a*[*i*]'s value is made minimum by storing its value in *min*.

Sometimes it is necessary to *identify the location or position* of the elements where maximum and minimum elements exist on the array. This can be done with the help of certain variables like *posmax* and *posmin.* As the very first element i.e. a[0], is assumed to be both maximum and minimum to begin with, values of *posmax* and *posmin* will be 0 (zero) initially. Whenever a *new maximum* or *new minimum* is found out, the subscript of that element is stored or copied into corresponding `posmax` or `posmin` variables. The complete flowchart is given in the adjoining figure.

Flowchart



C-Tips

People normally start counting from 1 where as in C the subscript value starts from zero (0). Therefore, the i's value will be 1 less than the normal human counting practice. Therefore (*i* + 1)'s value is used to display the position value.

The Program

```
/* Program to find the biggest and the smallest among the given
elements in an array*/

#include<stdio.h>
#include<conio.h>

main()
{
   int a[80],i,n,max,min,posmin=0,posmax=0;
   clrscr();
   printf("Input the value of n\n");
   scanf("%d",&n);

   printf("Size of the array is %d",n);
   printf("\nInput the elements\n");
   for(i=0;i<n;i++)
     scanf("%d",&a[i]);

   printf("The elements of the array are\n");
   for(i=0;i<n;i++)
     printf("%d\t",a[i]);

   max=a[0];
   min=a[0];

   /* Comparison starts from the 2nd element of the array i.e. with
      i=1 */

   for(i=1;i<n;i++)
   {
     if(max<a[i])
     {
       max=a[i];
       posmax=i;
```

```
    }
    if(min>a[i])
    {
      min=a[i];
      posmin=i;
    }
  }

printf("\nThe biggest is %d and is at the %d position among the given
          elements of the array",max,posmax+1);

printf("\nThe smallest is %d and is at the %d position among the given
       elements of the array",min,posmin+1);

}
```

6.4 AVERAGE OF A GIVEN SET OF N NUMBERS

The Method

As in the previous examples all the given numbers are stored in an array, say ***a***. *Average* is found out by first *summing* all the elements and then dividing the *sum* by total number of elements *'n'*. This process can be mathematically represented as

$$\text{average} = \frac{\sum_{i=0}^{n-1} a[i]}{n}$$

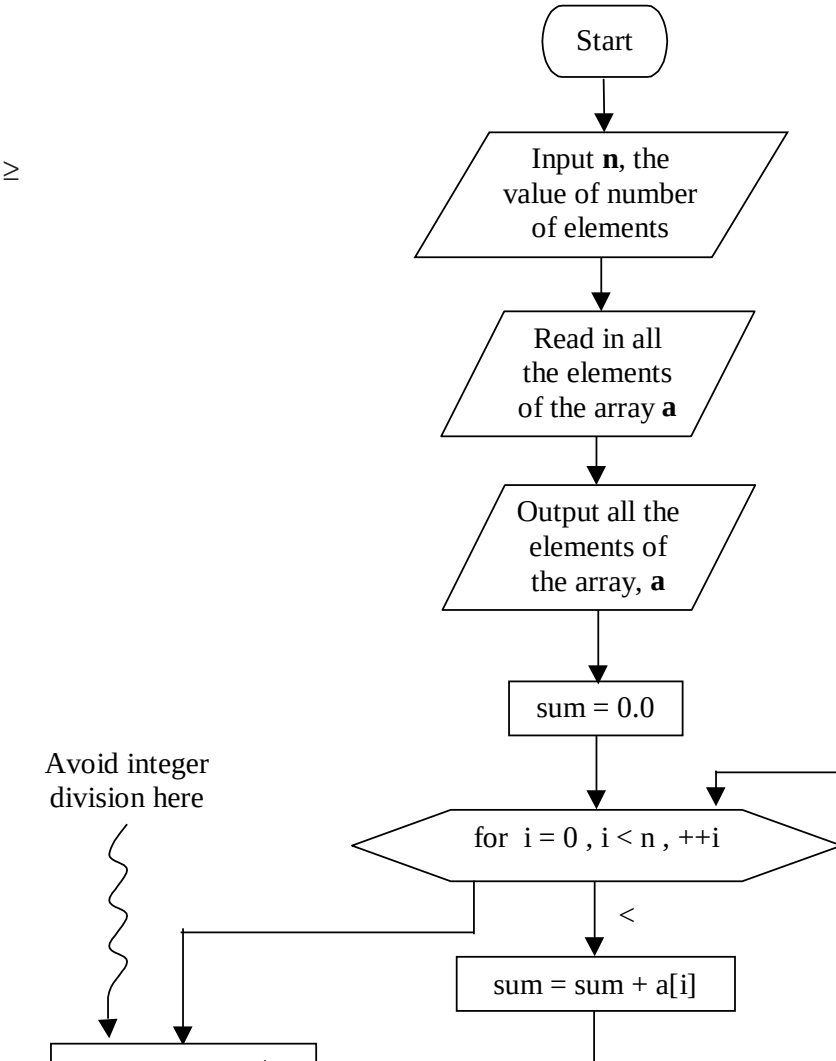
It may be noted that *average* is also termed as *mean*.

C-Tips

Care should be taken to avoid *integer division.* As already discussed one of the methods to avoid integer division is to use the *type cast* facility available in C. However, in this example the problem of integer division is overcome by declaring the variable `sum` itself as of type `float`.

The complete flowchart showing the above process is given below.

Flowchart


Start
Input **n**, the value of number of elements
Read in all the elements of the array **a**
Output all the elements of the array, **a**
sum = 0.0
Avoid integer division here
for i = 0 , i < n , ++i
<
sum = sum + a[i]

The Program

```
/* Average of a given set of n numbers */

#include<stdio.h>
#include<conio.h>
main()
{
   int a[20],i,n;
   float sum,average;

   clrscr();

   printf("How many number of elements?\n");
   scanf("%d",&n);

   printf("Input %d number of elements\n",n);
   for(i=0;i<n;++i)
   scanf("%d",&a[i]);

   printf("Elements of the given array are:\n");
   for(i=0;i<n;i++)
   printf("%d  ",a[i]);
   printf("\n\n");

   sum=0.0;

   for(i=0;i<n;i++)
   sum= sum+a[i];           /* Finding the sum of all elements */

   average=sum /n;

   printf("average=%.2f", average);

}
```

6.5 GIVEN N INTEGERS (ZERO, +VE, -VE) TO FIND OUT THE SUM OF +VE NUMBERS, -VE NUMBERS AND THE AVERAGE OF ALL NUMBERS

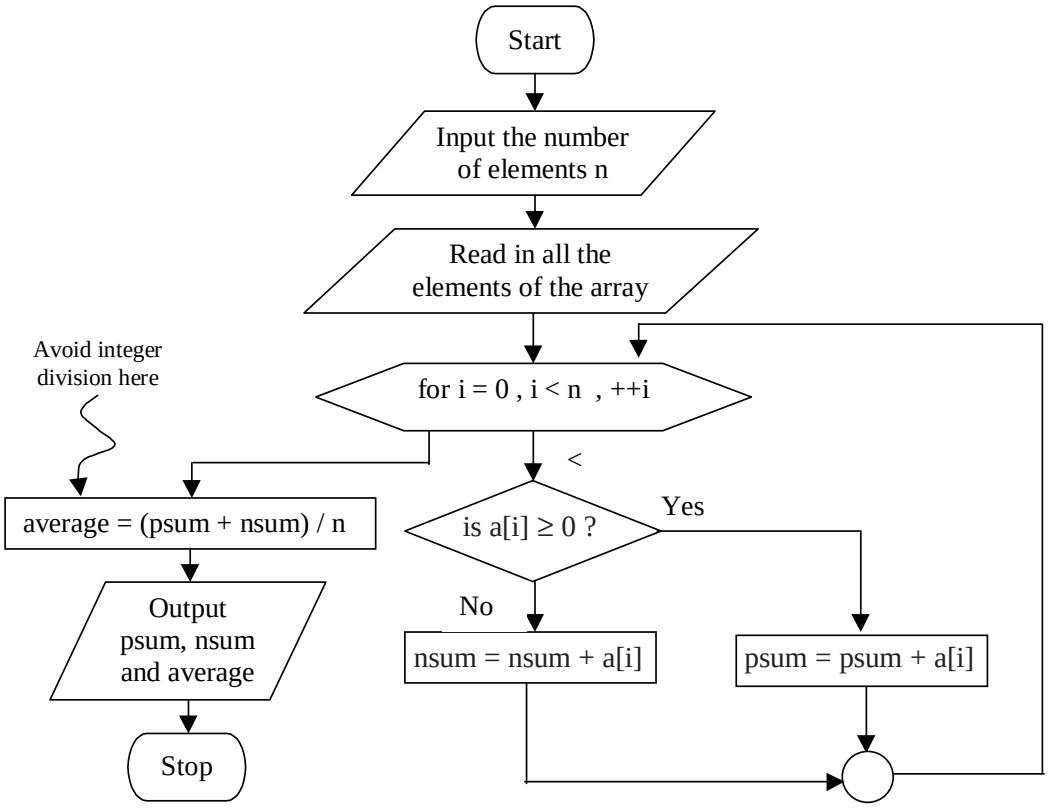
The Method

After declaring '*a*' as an array of required size, all the elements of the array are read in. Starting from the first element, each element is taken one by one and tested to determine whether it is +ve, -ve or zero. All positive numbers are added into a variable like `psum`, all negative numbers are added into a variable like *nsum* and nothing is done with zeros. Variables `psum` and `nsum` are initialized to zero. To find out the average, the ***sum*** *of* `psum` *and* `nsum` is obtained and the average is computed by dividing the ***sum*** by total number of elements, n. In this example, zero's have been considered as positive.

C-Tips

Care should be taken to avoid integer division. For this *type casting* facility of C may be used. For example, if *sum* is a *integer* variable then it can be converted into `float` type using a statement like `(float) sum`.

Flowchart



The Program

```
/* Program to read N integers(zero, +ve, -ve ) into an array a. To find
sum of -ve numbers, sum of +ve numbers and average of all input numbers
*/

/* Note: zero is taken as positive */

#include<conio.h>             /* Including  header file */
#include<stdio.h>

main()
{
   int a[20],i,psum=0,nsum=0, n;/* Declaring array a and initializing
                                   psum and nsum to zero */
   float avg;
   clrscr();

   printf("\nEnter n, the No. of elements.\n");
   scanf("%d",&n);
   printf("n = %d\n",n);

   printf("Enter the elements \n");
   for(i =0; i<n; ++i)
     scanf("%d",&a[i]);       /* Reading elements of array */

   printf("Elements are \n");
   for(i =0; i<n; ++i)
     printf("%d\t",a[i]);     /* Printing elements of array */

   for(i =0; i<n; ++i)         /* Repeating for each element */
   {
     /* If element is greater than or equal to zero it is added to psum,
        otherwise it is added to nsum */

   if (a[i] >= 0)
     psum += a[i];
```

```
   else
      nsum += a[i];

   }

   avg = (float)(psum + nsum)/n; /* Calculating average of all numbers */
                                 /* Observe that RHS is type casted */

   printf("\nSum of positive numbers = %d\n",psum);
   printf("\nSum of negative numbers = %d\n",nsum);
   printf("\nAverage of all input numbers = %.2f\n",avg);

}
```

6.6 MEAN, VARIANCE AND STANDARD DEVIATION OF A GIVEN SET OF NUMBERS

$$\text{Mean} = \frac{\sum_{i=1}^{n} x_i}{n} \qquad \text{Variance} = \frac{\sum_{i=1}^{n} (x_i - mean)^2}{n}$$

$$\text{Standard deviation} = \sqrt{\text{Variance}}$$

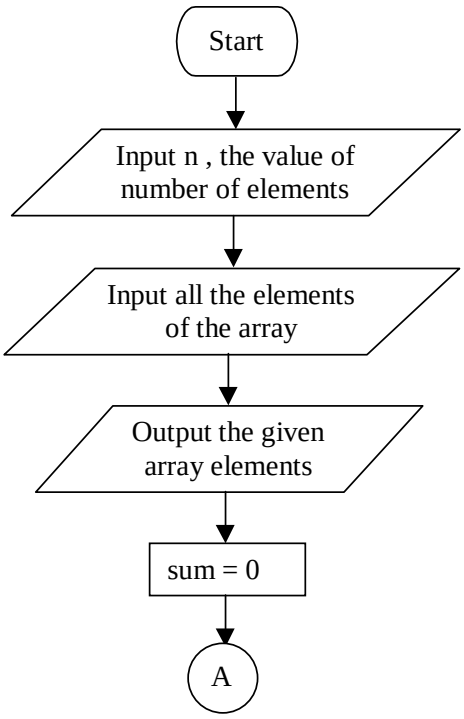
The Method

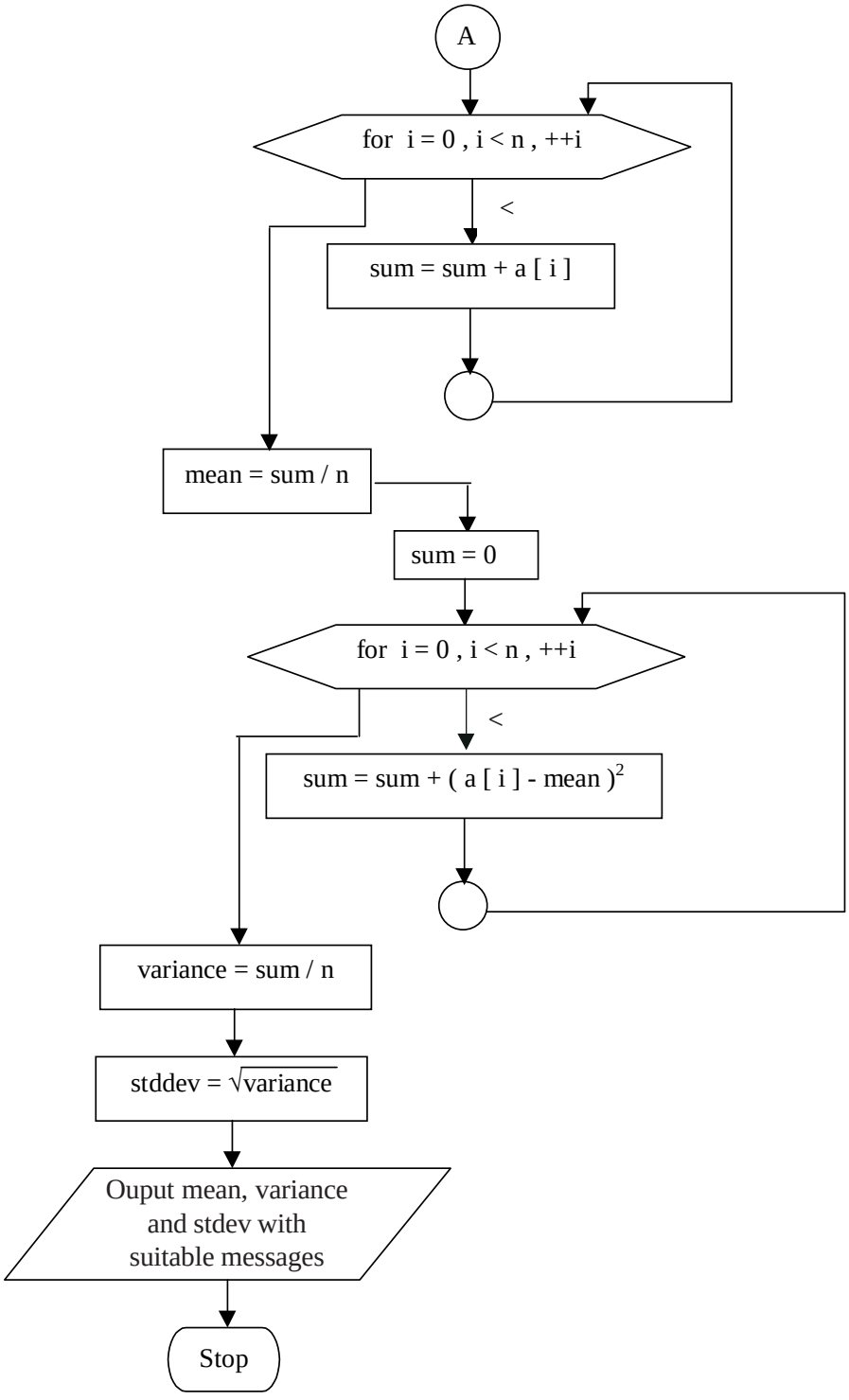
Since relations to compute the required quantities *mean, variance* and *standard deviation* are given, solving this problem is straightforward. The procedure involved is shown in the adjoining flow chart.

C-Tips

Whenever average or mean is computed care should be taken to avoid integer division. As already discussed this can be done using the *type casting* facility.

Flowchart




A
for i = 0 , i < n , ++i
<
sum = sum + a [i]
mean = sum / n
sum = 0
for i = 0 , i < n , ++i
<
sum = sum + (a [i] - mean)2
variance = sum / n
stddev = √variance
Ouput mean, variance
and stdev with
suitable messages
Stop

The Program

```
/* Mean, variance and standard deviation of a given set of numbers */

#include<stdio.h>
#include<conio.h>
#include<math.h>

main()
{
   int i, n;
   float  a[20], mean=0, sum=0, variance=0, stdev;
   clrscr();
   printf("\nEnter n, No. of elements\n");
   scanf("%d", &n);
   printf("n = %d\n", n);

   printf("\nEnter the elements \n");

   for(i = 0; i < n; ++i)           /* Reading elements of array */
     scanf("%f", &a[i]);

   printf("Elements are \n");     /* Printing elements of array */
   for(i = 0; i < n; ++i)
     printf("%.2f\n", a[i]);


   for(i = 0; i < n; ++i)           /* summing array elements */
     sum += a[i];

   mean = sum/n;                    /* computing mean */

   sum = 0;
   for(i = 0; i < n; ++i)           /* computing variance */
     sum += (mean - a[i]) * (mean - a[i]);
```

```
    variance = sum / n;

    stdev = sqrt(variance);          /* computing standard deviation */

    printf("\nMean = %.3f\nVariance = %.3f\nStandard Deviation = %.3f\n",
               mean, variance ,stdev);

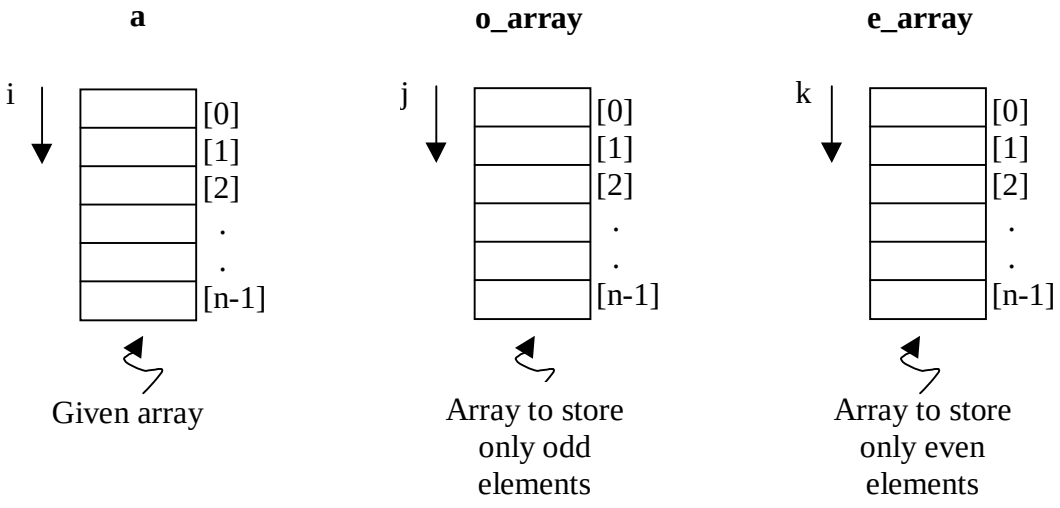

}
```

Note: In this example the variable *sum* has been used both to compute the *mean* as well as the *variance*. A careful observation of the flowchart as well as the program reveals the fact that after computing the *mean*, the value stored in *sum* is no more required. Therefore it is initialized once again to zero and employed to compute another *sum*, which is used in the computation of *variance*.

6.7 CLASSIFICATION OF THE ELEMENTS OF A GIVEN ARRAY INTO AN ARRAY CONSISTING OF ONLY ODD ELEMENTS AND ANOTHER ARRAY CONSISTING OF ONLY EVEN ELEMENTS.

The Method

Let there be '*n*' elements in the given array, say *a*. Let **o_array** and **e_array** be two arrays which will hold *only odd elements* and *only even elements* respectively.



First the given array is read into the computer. Then every element of the array ***a*** is tested to find out whether it is *odd* or *even*. This testing is done by "modulo 2" division of the element. If the "modulo 2" division results in a *zero remainder*, then the element will be *even*. If the "modulo 2" division results in a *non-zero remainder*, then the element will be *odd*. In order to verify the array elements we have to move along the array ***a.*** This movement is made using the subscript ***i*** and varying its value from 0 to $(n - 1)$. In order to store odd elements into odd-array (`o_array`) we have *to move along the* `o_array`. This movement is obtained using the subscript ***j*** the value of which has to be zero (0) to start with. In order to store even elements into even-array (`e_array`) we have to *move along the* `e_array`. This movement is obtained using the subscript **k** the value of which also has to be zero (0) to start with. Whenever required, the actual movement on an array is made by incrementing the value of the corresponding subscript variable.

If the element is *odd,* it is put into the odd array (`o_array`) and the odd count (*j*) is incremented.

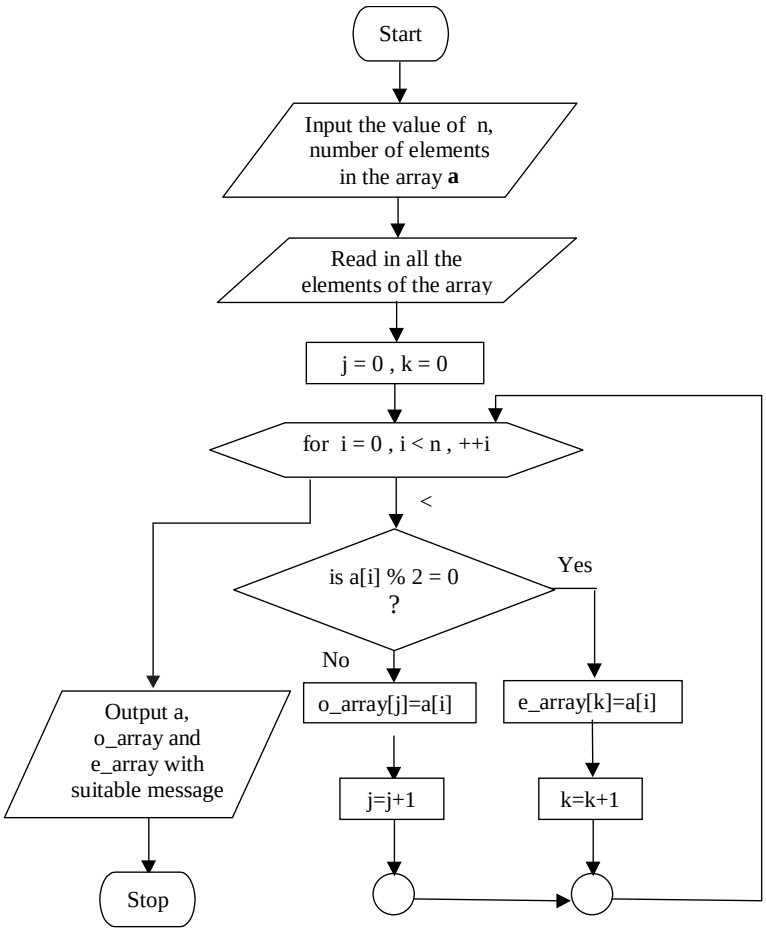
If the element is found to be *even*, it is put into the even array (`e_array`) and the even count (*k*) is incremented.

The above process is *repeated* for all the elements of the given array. Finally, the given array, odd array (`o_array`) and the even array (`e_array`) are output with suitable messages.

C-Tips

In the program all the three arrays ***a***, ***o_array*** and ***e_array*** have been declared to be of same size. This is to take care of the *worst possible situation* where all the given array elements could be either only odd or only even.

Flowchart



The Program

```
/* Classification of the elements of a given array into an array
consisting of only odd elements and another array consisting of only
even elements. */

#include <stdio.h>
main ( )
{
   int a[80] , o_array[80] , e_array[80] , n , i , k=0, j=0;
   clrscr( );

   printf( " Enter the number of elements \n");
   scanf( "%d", &n);
   printf("Size of the given array is  %d \n" , n );

   printf( "Input the elements of the array \n");
   for ( i = 0 ; i < n ; i++ )
     scanf ( "%d", &a[ i ] );

   for ( i = 0 ; i < n ; i++ )
   {
        if  ( a[ i ] % 2 == 0 )      /* Is it an even element ? */
        {
        e_array[ k ] = a [ i ];     /* Yes, even element */
          k++;
        }
        else
        {
          o_array[ j ] = a [ i ];    /* Else, odd element */
          j++;
        }
   }

   printf ( "\n The given array is \n ");
   for ( i = 0 ; i < n ; i++ )
   printf ( "%d\t", a[ i ] ) ;
```

```
   printf( "\n The array containing only even elements is \n ");
   for ( i = 0 ; i < k ; i++ )
   printf ( "%d\t", e_array[ i ] ) ;

   printf( "\n The array containing only odd elements is \n ");
   for ( i = 0 ; i < j ; i++ )
   printf ( "%d\t", o_array[ i ] ) ;

}
```

6.8 SORTING N NUMBERS IN ASCENDING ORDER USING BUBBLE SORT

The Method

The *Bubble sort* procedure can be well understood through an illustration. Below is given an illustration.

0	7
1	5
2	4
3	2

Given array A

Let there be four elements stored as an array, which are in the descending order (deliberately selected). In this method of sorting, elements in consecutive locations are *compared* from the beginning to the end of the array. If these two consecutive elements are not in the required order, they are *interchanged*. If they are in the required order, nothing is done and the comparison of next set of consecutive elements is made.

Thus, in this illustration, first 7 {i.e. *A*[0]} is compared with 5 {i.e. *A*[1]} and interchanged. Next, 7 {now *A*[1] } is compared with 4 {now *A*[2]} and are interchanged. Further, 7 {now *A*[2] } is compared with 2 {now *A*[3] } and are interchanged. No further comparisons can be made as we have already considered *A*[3], the *last element*. The array now looks as shown in the figure below.

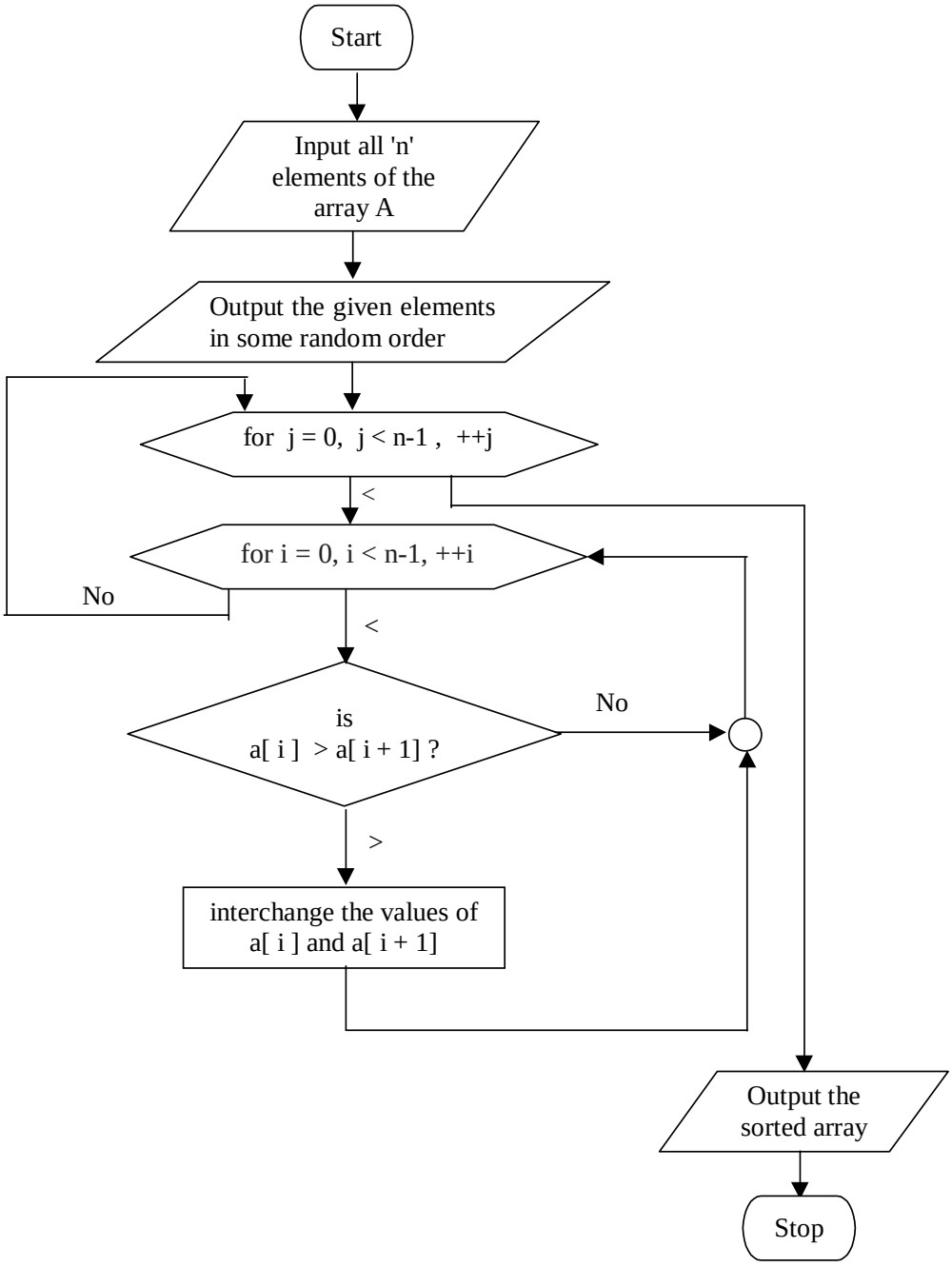
0	5
1	4
2	2
3	7

Observe that in going once from the beginning till the end there were 3 comparisons. Comparison was made between *A*[*i*] and *A*[*i* + 1] elements with *i* ranging from 0 to 2. Going once from the beginning of the array till its end or *traversing once across the entire array is known as a* ***pass***. Thus *in a pass* we will be making (*n* – 1) *comparisons,* where n is the total number of elements in the array. (Observe that from the given 4 elements, 3 comparisons were made.)

Obviously, *one pass has put the largest element in its place*. Th *second pass* will put *second largest* element in its place and so on. Thus in order to put all *n* elements in their places (*n* – 1) *passes are required*. Why (*n* – 1) passes? Why not *n* passes? This is so because after (*n* – 1) passes (*n* – 1) elements are forced to their proper positions and the last element has to be and will be in its proper position.

Thus the *Bubble sort* procedure needs ($n - 1$) *traversals* or passes *and* ($n - 1$) *comparisons* in every pass. The flow chart will be as shown below.

Flowchart



The Program

```
/* Sorting n numbers in ascending order using bubble sort */

#include<conio.h>                    /* Including header file */
#include<stdio.h>

main()
{
   int i, j, n, a[20], temp;        /* Declaring array a */
   clrscr();
   printf("\nEnter n, No. of elements\n");
   scanf("%d", &n);
   printf("n = %d\n", n);
   printf("Enter the elements \n");
   for(i =0; i<n; ++i)
     scanf("%d", &a[i]);           /* Reading elements of array */

   printf("\nElements are: \n");
   for(i =0; i<n; ++i)
        printf("%d\t", a[i]);  /* Printing elements of array */
        printf("\n");
   /* This loop performs required number of passes */
   for(j=0; j<n-1; ++j)
   {
        /* This loop performs all comparisons in a pass */
        for(i=0; i<n-1; ++i)

          if ( a[i] > a[i+1] )
        /* if first among two adjacent element is greater than the
        other then it is exchanged using temporary variable temp */
          {
                    temp  = a[i];
                    a[i]  = a[i+1];
                    a[i+1] = temp;
```

```
        }
    }

    printf("\nSorted elements are:\n");
    for(i =0; i<n; ++i)
         printf("%d\t", a[i]);  /* Printing sorted array */
         printf("\n");

}
```

Note: The analysis here is known as the *worst case analysis*. This is because initially a descending order array was assumed. It may be noted that if the given array is either *partially ordered* or *totally ordered*, still this program works. However, unnecessarily it goes through all the passes and makes all comparisons. It is possible to avoid this with the help of a *flag*. Try.

6.9 EVALUATING A POLYNOMIAL

A polynomial will be as shown below

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \ldots\ldots + a_1 x + a_0$$

Evaluating the polynomial means, finding out the value of $p(x)$, given the value of x and the coefficients $a_0, a_1, a_2, a_3, \ldots\ldots, a_{n-1}, a_n$.

The Method

A specific polynomial can be evaluated by writing an arithmetic expression for the same. For example, if $5x^3 + 4x^2 + 2x + 3$ is given, a statement of the form $p(x) = 5 * x * x * x + 4 * x * x + 2 * x + 3$ can be first written and then knowing the value of the variable x, the expression can be evaluated. However, a *general procedure* to solve any given polynomial is discussed below. Here, it may be noted that a *given polynomial* can be *rewritten* in the form of a *nested parenthesis expression* as:

$$(a_0 + x\ (a_1 + x\ (a_2 + x\ (a_3 + \ldots\ldots\ldots + x\ (a_{n-1} + x\ .\ a_n))))).$$

All the coefficients of the polynomial are *read in and stored as an array*, say ***a***. The element at 0th position in the array ***a*** refers to the coefficient $\boldsymbol{a_0}$, the element at 1st position in the array ***a*** refers to $\boldsymbol{a_1}$, and so on. A procedure for evaluating a polynomial can be formulated as discussed below. Here it may be recalled that in the case of *nested parentheses expressions the inner most parenthesis are taken care-of* first.

A verification of the above *rewritten polynomial* reveals the following facts:

First, a_n is taken as the value of a variable, say *result.* Then it is multiplied by x, and added to a_{n-1} thus generating (a_{n-1} + x * *result*). As at this stage a_n is the value of *result*, the

expression evaluated will be $(a_{n-1} + x * a_n)$. This evaluated value is made as the *new value* of the *result.*

Next $(a_{n-2} + x * result)$ is evaluated. As at this stage the *result* holds the value of $(a_{n-1} + x * a_n)$, the expression evaluated will be $(a_{n-2} + x\ (\ a_{n-1} + x * a_n\ ))$. This evaluated value is made as the *new value* of the *result.*

Further $(a_{n-3} + x * result)$ is evaluated. Since *result* already holds the value of $(a_{n-2} + x\ (a_{n-1} + x * a_n))$, the expression now evaluated will be $(a_{n-3} + x * (a_{n-2} + x\ (a_{n-1} + x * a_n)))$. Again, this evaluated value is made as the *new value* of the *result.*

Steps discussed above are to be *repeatedly executed* till all the polynomial coefficients are taken care of.

From the above discussion one can observe that

- One should have a variable like *result* which is initialized to the coefficient of the highest order term i.e., a_n.
- As a_n has been taken as the *initial value* of the variable *result* we need only a_{n-1}, a_{n-2}, a_{n-3} , a_{n-4}, …, a_2, a_1, a_0 to be considered. By using the expression $(n - 1 - i)$ as the subscript of the array ***a*** and varying ***i*** from 0 to $(n - 1)$ one can generate all the remaining coefficients i.e. a_{n-1}, a_{n-2}, a_{n-3}, a_{n-4}, …, a_2, a_1, a_0, one after the other.
- The expression $(a_{n-1-i} + x * result)$ is *repeatedly* evaluated for all the values of i varying from 0 to $(n - 1)$.

Thus the important steps in evaluating a given polynomial are:

- Read in all the coefficients into an *array*, say **a**. Initialize a variable like *result* to a_n.
- Then repeatedly execute the statement

  *result = a + x * result;*

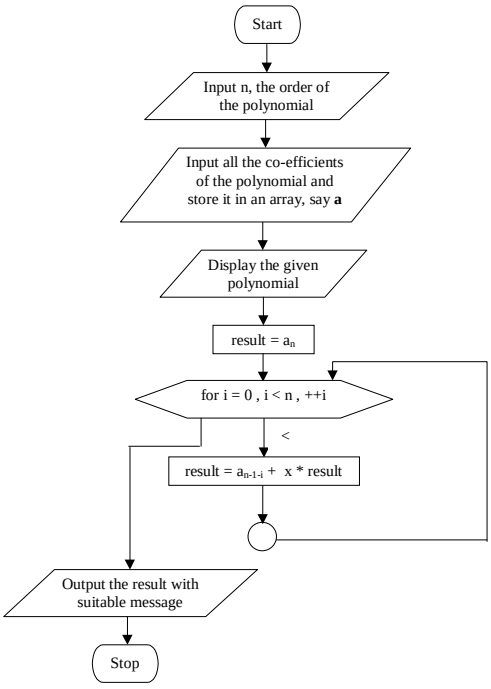
  for all values of i from 0 (zero) to $(n - 1)$.

C-Tips

In this example, for the first time a *format specifier* like **%+d** has been used in the *control string* of a `printf()` statement. Here the **+** (plus) character is called the `printf flag` *character*. The use of **+** (plus) as shown here prints out the results along *with the sign*. A **-** (minus) character can also be used as a `printf` *flag character*. The use of this flag character prints the result in the *left justified* form.

The above procedure is shown in the form of a flowchart in the adjoining figure.

Flowchart



The Program

```
/* Evaluating a polynomial expression */

#include<stdio.h>
#include<conio.h>

main()
{
 /*Declaring variables*/
 int a[10], n, i, result,x;
 /* clearing the screen*/
 clrscr();

 printf("Enter the order of the polynomial\n");
 /*Accepting the order of polynomial*/
 scanf("%d", &n);

 /* printing the order of the polynomial*/
 printf("Order of the polynomial is %d\n\n",n);

 printf("Enter the coefficients of the polynomial\n");
 for(i=0; i<=n; ++i)
 {
   printf("\n Enter a[%d] = ", i);
   /*Accepting the coefficients of the polynomial*/
   scanf("%d", &a[i]);
 }

 /*Printing the polynomial*/
 printf("\nGiven polynomial is : \n");
 printf("\np(x) =  ");
 for(i=n; i>=0; --i)
   printf(" %+d*x^%d ", a[i],i); /* Observe the use of + with the format
                                    specifier %+d with in the control
                                    string. */
```

```
  printf("\n\nEnter the value of x\n");
  /* Accepting value of x */
  scanf("%d", &x);
  printf("\nx = %d\n", x);

  /*Initializing the result with a[n]*/
  result = a[n];

  /*Evaluating the polynomial*/
  for (i=0; i< n; ++i)            /* Because a_n was first put in result */
    result = a[n-1-i] + x * result;
  printf("\np(%d) = %d\n", x, result);

}
```

6.10 READING AND PRINTING OUT A GIVEN MATRIX

The Method

This example illustrates how a matrix (a two dimensional array) could be *read* in and *printed out*. In fact, this type of program will be a segment of some other programs like "Program to transpose" a given matrix, "Program to find the sum/difference" of given matrices, "Program to compute the product" of given matrices, and so on. Let ***a*** be the given matrix, having ***m*** rows and ***n*** columns, i.e. a $m \times n$ matrix.

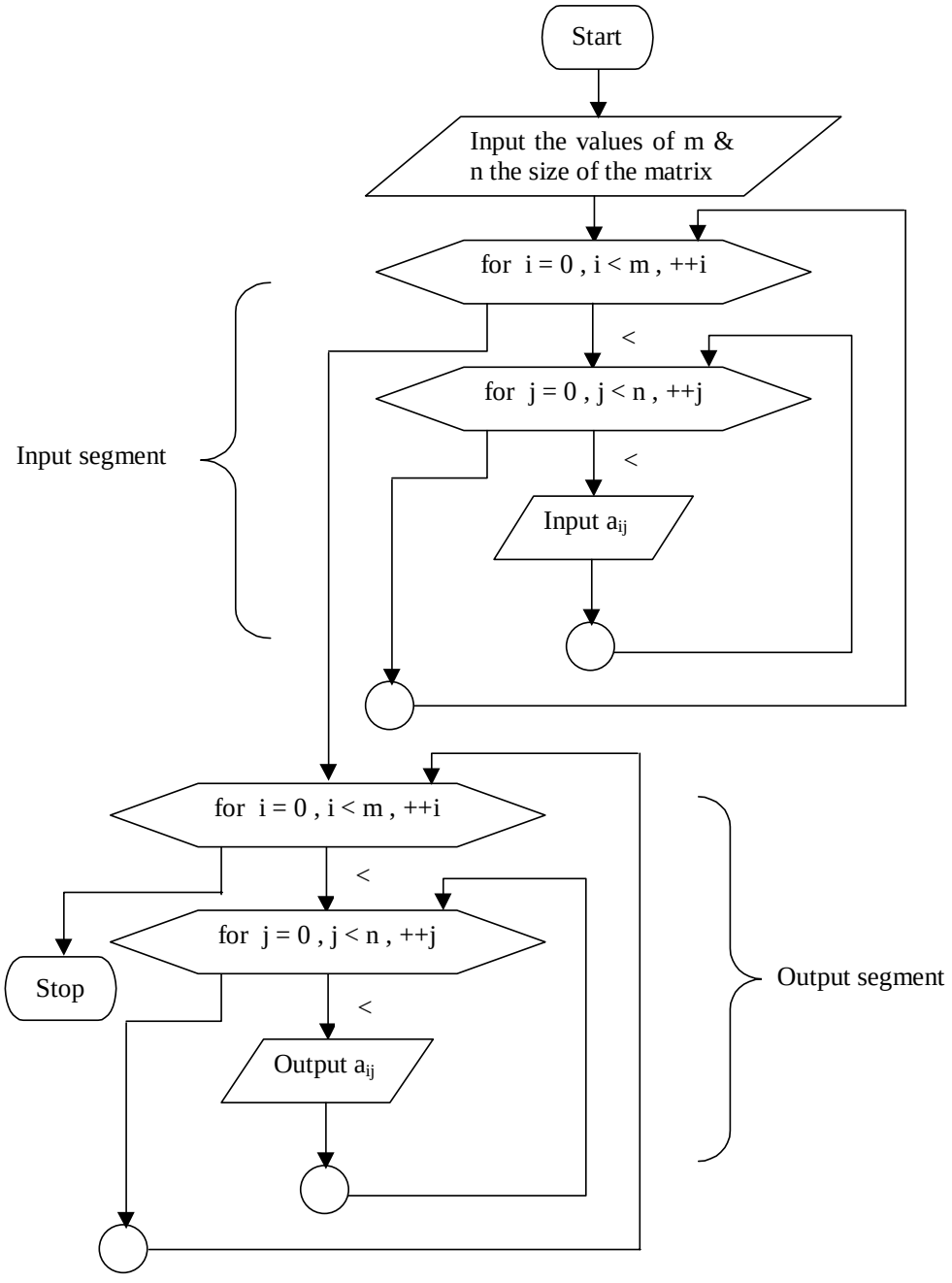
$$\text{Let } A = \overrightarrow{\begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}}$$

The general element of this matrix is a_{ij}. Here the first subscript ***i*** represents a row and the second subscript ***j*** represents a column. This matrix could be *read in row-wise,* i.e. in the order a_{00}, a_{01}, a_{02}, a_{10}, a_{11} and so on, or could be *read in column-wise,* i.e. in the order a_{00}, a_{10}, a_{01}, a_{11} and so on. But a matrix is always printed row-wise only.

By carefully going through the discussion in the above paragraph we see that, while *handling the array row wise* the *first subscript* (i.e. *i*) *varies slowly* and the *second subscript* (i.e. *j*) *varies faster*. Therefore, the *fast varying subscript j must be taken care of before the slow varying subscript i*. In other words, the loop under the control of *j* must constitute the inner loop and the loop under the control of *i* must constitute the outer loop. The matrix can be handled *column-wise by first taking care of the i variable and then the j variable*. It may be noted that the matrices are generally *read in* and *printed out row-wise*.

The flow chart below shows the process of reading a given matrix and printing the same.

Flowchart



The Program

```
/*  To read in a given matrix and to print out the same  */

#include<stdio.h>
#include<conio.h>

main()
{
   int a[10][9],i,j,m,n;
   clrscr();

   printf("Enter the values of m and n\n");/* Size of the matrix */
   scanf("%d%d",&m,&n);

   printf("Enter the matrix\n");
   for(i=0;i<m;i++)                        /* Outer loop begins */
   {
     for(j=0;j<n;j++)                      /* Inner loop begins */
       scanf("%d",&a[i][j]);
   }

   printf("\tThe elements of the array are\n");
   for(i=0;i<m;i++)                        /* Outer loop begins */
   {
     printf("\t\t\n\n");
     for(j=0;j<n;j++)                      /* Inner loop begins */
       printf("\t%3d",a[i][j]);
   }

}
```

Note: We are accustomed to the evaluation of *nested* arithmetic expressions like (7 + (5 – 2)). Because of the parenthesis, (5 – 2) is evaluated first, which results in 3 and then (7 + 3) is evaluated. In other words, when nested, inner parenthesis is taken care of first and then the outer parenthesis. A similar situation exists with nested `for` loop (or any loop control) statements, i.e. inner `for` loop will be taken care of first and then the outer `for` loop.

6.11 TO COMPUTE THE SUM/DIFFERENCE OF GIVEN MATRICES

The Method

In order to obtain the sum or difference of two given matrices, *the size of the given matrices must be same*. The sum of two matrices is obtained by summing the corresponding elements of the given matrices. The difference of two matrices is obtained by finding out the difference between the corresponding elements of the given matrices. For example:

If $A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$ and B = $\begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$

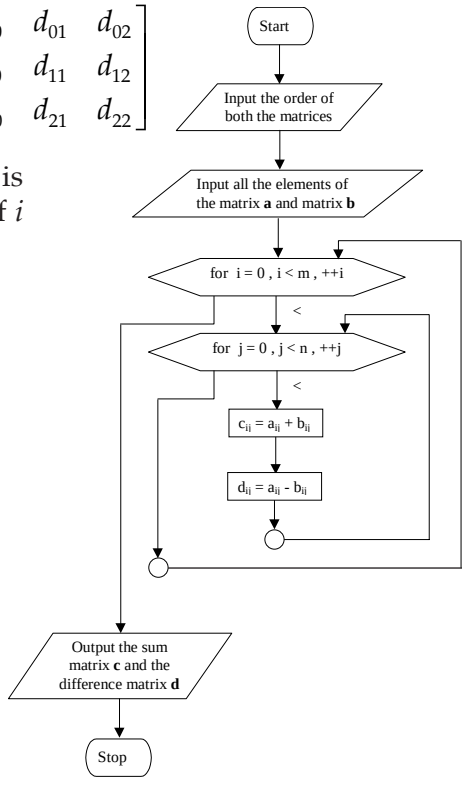
then sum, $C = \begin{bmatrix} a_{00}+b_{00} & a_{01}+b_{01} & a_{02}+b_{02} \\ a_{10}+b_{10} & a_{11}+b_{11} & a_{12}+b_{12} \\ a_{20}+b_{20} & a_{21}+b_{21} & a_{22}+b_{22} \end{bmatrix}$ $\begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix}$

Thus every element of the sum matrix C is obtained using the relation $c_{ij} = a_{ij} + b_{ij}$ for all values of i and j.

Difference, $D = \begin{bmatrix} a_{00}-b_{00} & a_{01}-b_{01} & a_{02}-b_{02} \\ a_{10}-b_{10} & a_{11}-b_{11} & a_{12}-b_{12} \\ a_{20}-b_{20} & a_{21}-b_{21} & a_{22}-b_{22} \end{bmatrix}$ $\begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$

Thus every element of the difference matrix D is obtained using the relation $d_{ij} = a_{ij} - b_{ij}$ for all values of i and j.

Flowchart



The Program

```
/* To compute the sum and difference of given matrices */

#include<conio.h>
#include<stdio.h>

main()
{
   int a[10][10], b[10][10], c[10][10], d[10][10], i, j, m, n;
   clrscr();
   printf("Enter value for m and n ie., the order of matrices A and
           B\n");
   scanf("%d%d", &m, &n);
   printf("Order of Matrix A and B is %d X %d\n", m, n);

   printf("Enter the elements of matrix A\n");

   for(i=0; i <m; ++i)             /* Reading matrix a */
     for(j=0; j<n; ++j)
       scanf("%d", &a[i][j]);

   printf("Enter the elements of matrix B\n");
   for(i=0; i<m; ++i)              /* Reading matrix b */
     for(j=0; j<n; ++j)
       scanf("%d", &b[i][j]);

   printf("\nMatrix A\n");
   for(i=0; i <m; ++i)             /* Printing matrix a */
   {
       for(j=0; j<n; ++j)
       {
       printf("%d\t", a[i][j]);
       }
       printf("\n");
```

```
}

printf("\nMatrix B\n");
for(i=0; i <m; ++i)             /* Printing matrix b */
{
    for(j=0; j<n; ++j)
    {
       printf("%d\t", b[i][j]);
    }
    printf("\n");
}

/* Finding the sum and difference */

for(i=0; i<m; ++i)
    for(j=0; j<n; ++j)
    {
       c[i][j] = a[i][j]+b[i][j]; /* Addition of matrices */
       d[i][j] = a[i][j]-b[i][j]; /* Subtraction of matrices */

    }

printf("\nResultant Matrix C (Sum)\n");
for(i=0; i <m; ++i)             /* Printing sum matrix c */
{
    for(j=0; j<n; ++j)
    {
       printf("%d\t", c[i][j]);
    }
    printf("\n");
}


printf("\nResultant Matrix D (Difference)\n");
for(i=0; i <m; ++i)             /* Printing diff matrix d */
```

```
   {
      for(j=0; j<n; ++j)
      {
         printf("%d\t", d[i][j]);
      }
      printf("\n");
   }


}
```

Note: While developing the flowchart as well as coding, i.e. programming it has been assumed that the two matrices have *same dimensions* so that they can be added or subtracted. Sometimes it may be necessary to check for the dimensions. This checking must be done immediately after reading in the dimensions of the two matrices. If A is an $(m \times n)$ matrix and B is a $(p \times q)$ matrix then for addition or subtraction the relational expression $\{(m = p)$ and $(n = q)\}$ must be true and is therefore tested.

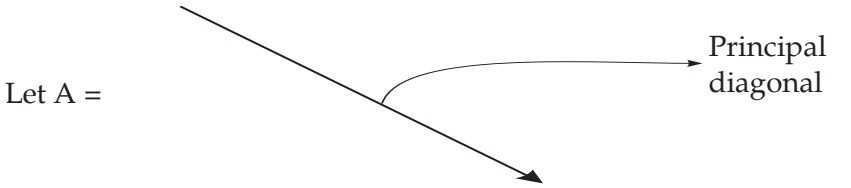
6.12 TRACE OF A GIVEN MATRIX

By definition the *trace* of a matrix is given by the *sum of all the elements on its principle diagonal*. Obviously the matrix has to be a *square matrix*.

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0(n-1)} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1(n-1)} \\ . & & & & \\ a_{(n-1)0} & a_{(n-1)1} & a_{(n-1)2} & \cdots & a_{(n-1)(n-1)} \end{bmatrix}$$

The Method

First all the elements of the given matrix are read in. Then whether the *given matrix is a square matrix* or not is found out. This is done by comparing the two dimensions of the matrix. *If number of rows is equal to number of columns then the trace can be found out.* Otherwise the *trace* cannot be computed.



From the above figure it may be observed that the control moves down on the *principal diagonal* when both the subscripts are same. Thus the *trace* can be obtained using the following relation :

$$trace = a_{00} + a_{11} + a_{22} + \ldots + a_{n-1,n-1}$$

The above relation can be mathematically written as follows :

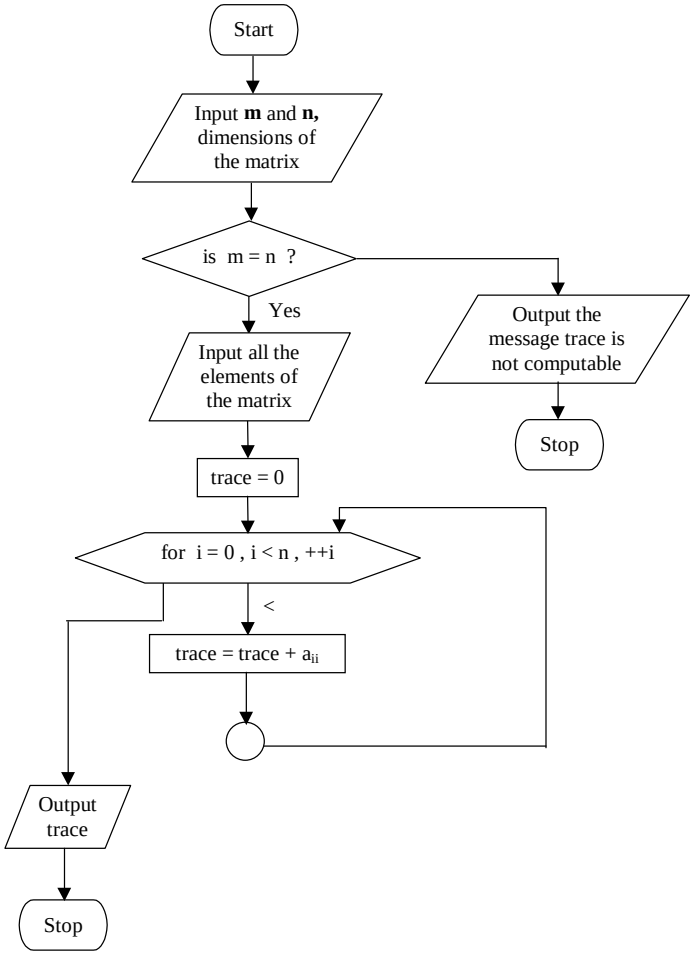
$$trace = \sum_{i=0}^{n-1} a_{ii}$$

Obviously, a `for` statement is used to obtain the above *sum,* i.e. the *trace*.

C-Tips

When the given matrix is not a *square matrix* a proper message has to be displayed and the computation must stop. For this purpose the `exit()` function explained in Section 6.15 is used.

Flowchart



The Program

```
/* Trace of a given matrix */

#include<stdio.h>
main()
{
  int m,n,i,j,a[10][20],trace;
  clrscr();

  printf("Enter the order \n");
  scanf("%d%d",&m,&n);

  if(m!=n)                        /* Is it a square matrix ?*/
{
  printf("Trace Not Possible\n"); /* Not a square matrix */
  exit(1);
}

  printf("Enter a matrix\n");
  for(i=0;i<m;i++)
  {
    for(j=0;j<n;j++)

    scanf("%d",&a[i][j]);
  }

  printf("The given matrix is\n");
  for(i=0;i<m;i++)
  {
    for(j=0;j<n;j++)
    printf("%d\t",a[i][j]);
    printf("\n");
  }
```

```
  trace=0;

  for(i=0;i<m;i++)                /* Computing the trace */
  trace=trace+a[i][i];

  printf("Trace of the given matrix is = %d",trace);

}
```

6.13 NORM OF A GIVEN MATRIX

By definition *norm* of a matrix is the *square root of the sum of the squares of all its individual elements.*

The Method

Since *all the elements* are to be considered, one has to consider the general element $a_{ij,}$ where ***a*** is a given matrix of *m rows* and *n columns*. The *square of every element* is obtained by simply *multiplying it by itself once,* i.e. by using $(a_{ij} * a_{ij})$. The sum of all $(a_{ij})^2$ is obtained by the relation:

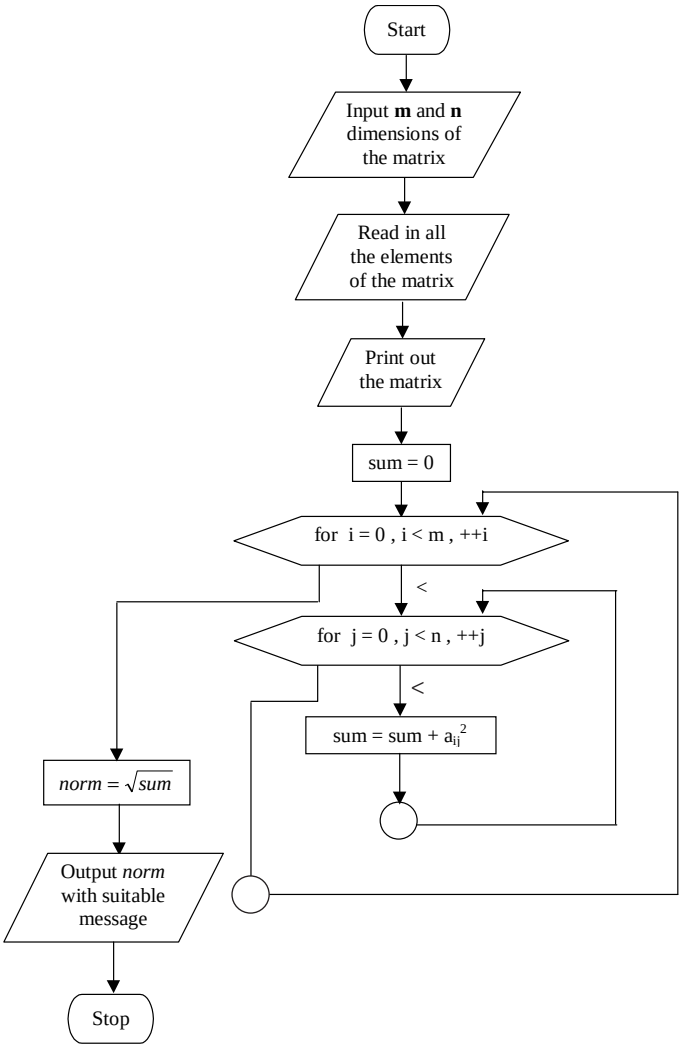
$$sum = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} a_{ij}^2$$

After finding the *sum* the norm is obtained by finding the square root of the *sum*. using the relation

$$norm = \sqrt{sum}$$

The flowchart is given in the adjoining figure.

Flowchart



The Program

```
/* The norm of a matrix */

#include<stdio.h>
#include<math.h>

main()
{
   int m,n,i,j,a[10][10],sum;
   float norm;
   clrscr();

   printf("Enter the values of m and n \n");
   scanf("%d%d",&m,&n);

   printf("Enter a matrix\n\n");
   for(i=0;i<m;i++)
   {
     for(j=0;j<n;j++)

     scanf("%d",&a[i][j]);
   }

   printf("The given matrix is\n\n");
   for(i=0;i<m;i++)
   {
     for(j=0;j<n;j++)
     printf("%d\t",a[i][j]);
     printf("\n");
   }

   sum=0;

   for(i=0;i<m;i++)
   {
```

```
    for(j=0;j<n;j++)

    sum=sum+a[i][j]*a[i][j];
  }

  norm=sqrt(sum);

  printf("\nNorm of the given matrix is = %.2f",norm);

}
```

6.14 TRANSPOSE OF A GIVEN MATRIX

The Method

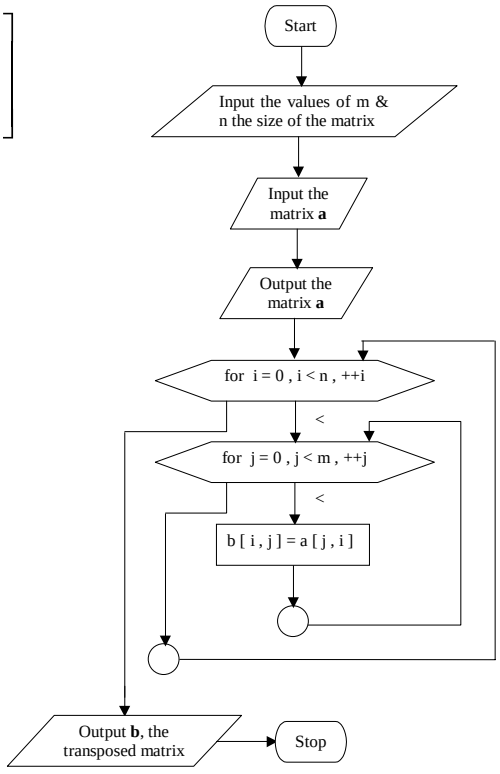
Let ***A*** be the given matrix of size ***m*** by ***n***. Also let ***B*** is the transpose of the given matrix. The transpose of a matrix is obtained *by interchanging its row elements* and *column elements*. For example :

If $A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$ Then $B = \begin{bmatrix} a_{00} & a_{10} & a_{20} \\ a_{01} & a_{11} & a_{21} \\ a_{02} & a_{12} & a_{22} \end{bmatrix}$

Also, $B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$

By comparing the two representations of B given above we see that any element of B i.e. b_{ij} is equal to a_{ji}. Therefore, the ***relation*** $\boldsymbol{b_{ij} = a_{ji}}$ is used *repeatedly* to generate all the elements of the *transposed matrix*. Observe that if the size of matrix is $(m \times n)$ then the size of its transpose will be $(n \times m)$. The adjoining flowchart shows the above procedure.

Flowchart



The Program

```
/* Given a matrix A ( m * n ), to find the transpose of the same and to
output both the input matrix and the transposed matrix. */

#include<stdio.h>
#include<conio.h>

main()
{
   int a[10][10], b[10][10],i, j, m, n;
   clrscr();
   printf("Enter value for m and n i.e.,the order of matrix A \n");
   scanf("%d%d", &m, &n);
   printf("Order of Matrix A is %d X %d\n\n", m, n);
   printf("Enter the elements of matrix A\n");


   for(i=0; i <m; ++i)             /* Reading matrix a */
       for(j=0; j<n; ++j)
           scanf("%d", &a[i][j]);

   printf("  Matrix A\n\n");


   for(i=0; i <m; ++i)             /* Printing matrix a */
   {
       for(j=0; j<n; ++j)
       {
           printf("%d\t", a[i][j]);
       }
       printf("\n");
   }
```

```
    for(i=0; i<n; ++i)
        for(j=0; j<m; ++j)
            b[i][j] = a[j][i];      /* Transposing */


    printf("\nTranspose of Matrix A\n\n");

    for(i=0; i <n; ++i)             /* Printing transposed matrix */
    {
        for(j=0; j<m; ++j)
        {
            printf("%d\t", b[i][j]);
        }
        printf("\n");
    }

}
```

6.15 PRODUCT OF TWO GIVEN MATRICES

The Method

Let A and B are two given matrices of dimensions $(m \times n)$ and $(p \times q)$ respectively. Also let C be the *product matrix*. For the multiplication of given two matrices, *number of columns of the first matrix must be* equal to *number of rows of the second matrix,* i.e. n must be equal to p. The product matrix C will be of size $(m \times q)$. The generation of each element of the *product matrix* (C [i, j]) can be easily understood by going through the following illustration.

Let $A = \overrightarrow{\begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}}$ Let $B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix} \downarrow$

Obviously, A is a (2 * 3) matrix, i.e. $m = 2$ and $n = 3$

B is a (3×3) matrix, i.e. $p = 3$ and $q = 3$

Since $n = p$, *multiplication is possible*. The resultant *product matrix C* will be a $m \times q$, i.e. 2×3 matrix.

Let $C = \begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \end{bmatrix}$

The row elements of C are obtained as follows:

$c_{00} : a_{00} * b_{00} + a_{01} * b_{10} + a_{02} * b_{20}$

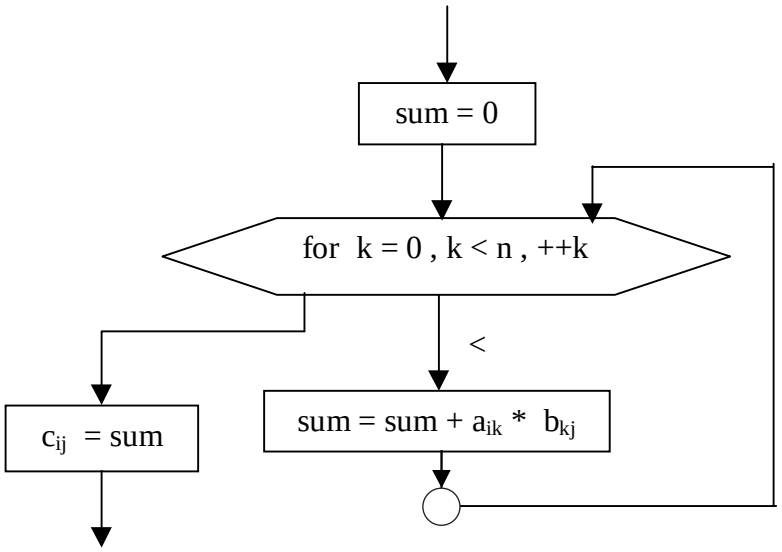
$c_{01} : a_{00} * b_{01} + a_{01} * b_{11} + a_{02} * b_{21}$

$c_{02} : a_{00} * b_{02} + a_{01} * b_{12} + a_{02} * b_{22}$

That is, moving on the row of the first matrix and on the column of the second matrix, as indicated by arrows in the above illustration.

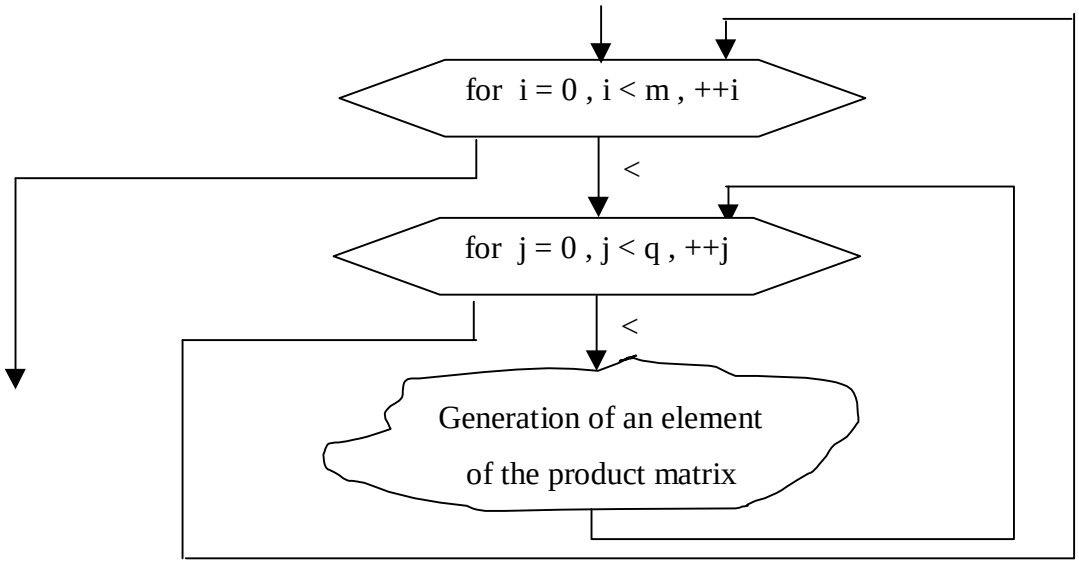
By inspecting the above relations we find that any element of the product matrix *C* can be obtained using the relation :

The first subscript of ***a*** will be the first subscript of ***c*** and the second subscript of ***b*** will be the second subscript of ***c***. *Observe that the second subscript of* ***a*** *and the first subscript of* ***b*** *are same and vary from 0 upto* $(n - 1)$. Let this *common subscript be k*. Thus this subscript *k* varies from 0 to $(n - 1)$. Since every element is a *summation of certain product terms*, one has to use a variable like *sum* in order to have the *generated elements* summed up and stored in it. The flowchart segment for this is given below.

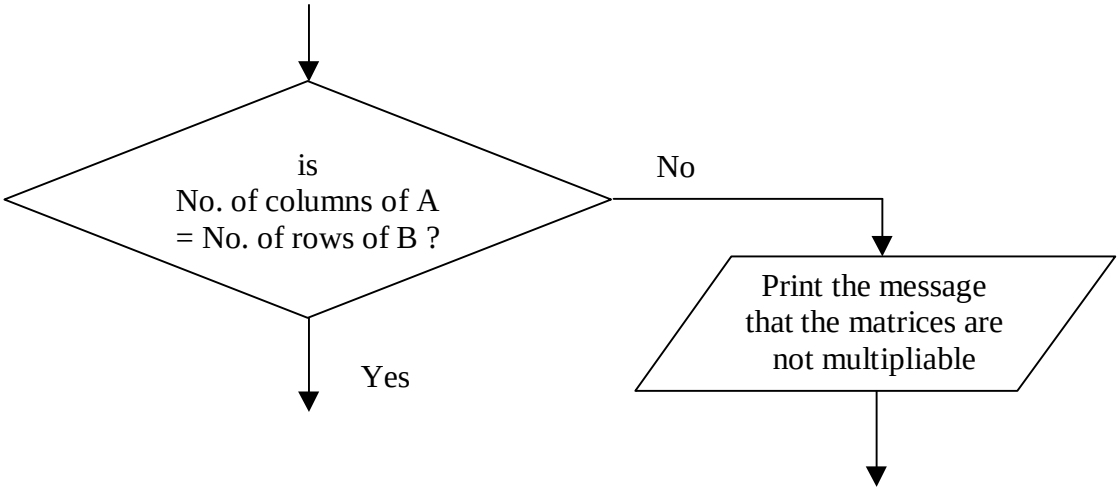


All the elements of the product matrix *C* are generated by *repeating* the above process for all the values of *i* and *j*. *Observe that* ***i,*** *the first subscript of C, should vary from 0 to* $(m - 1)$ *and j, the second subscript of C, should vary from 0 to* $(q - 1)$. This can be shown in the form of a flow chart segment in page 117.

From the flowcharts we see that *nested* `for` loops are used. It may be noted that whenever nested `for` loops are used, the inside `for` is taken care of first. In other words, here, *j varies faster* than *i*. Since *j* varies faster than *i*, the elements of the product matrix are generated *row-wise*. If, by chance, the above two loops are interchanged, then the elements of *C* are generated *column-wise*.



Though care may be taken during the input operation itself to see that the input matrices are multipliable, it is advisable to check for the multipliability of the matrices by checking whether *"number of rows" of the first matrix is equal to "number of columns" of the second matrix* or not. The following flowchart segment may be used for this purpose.

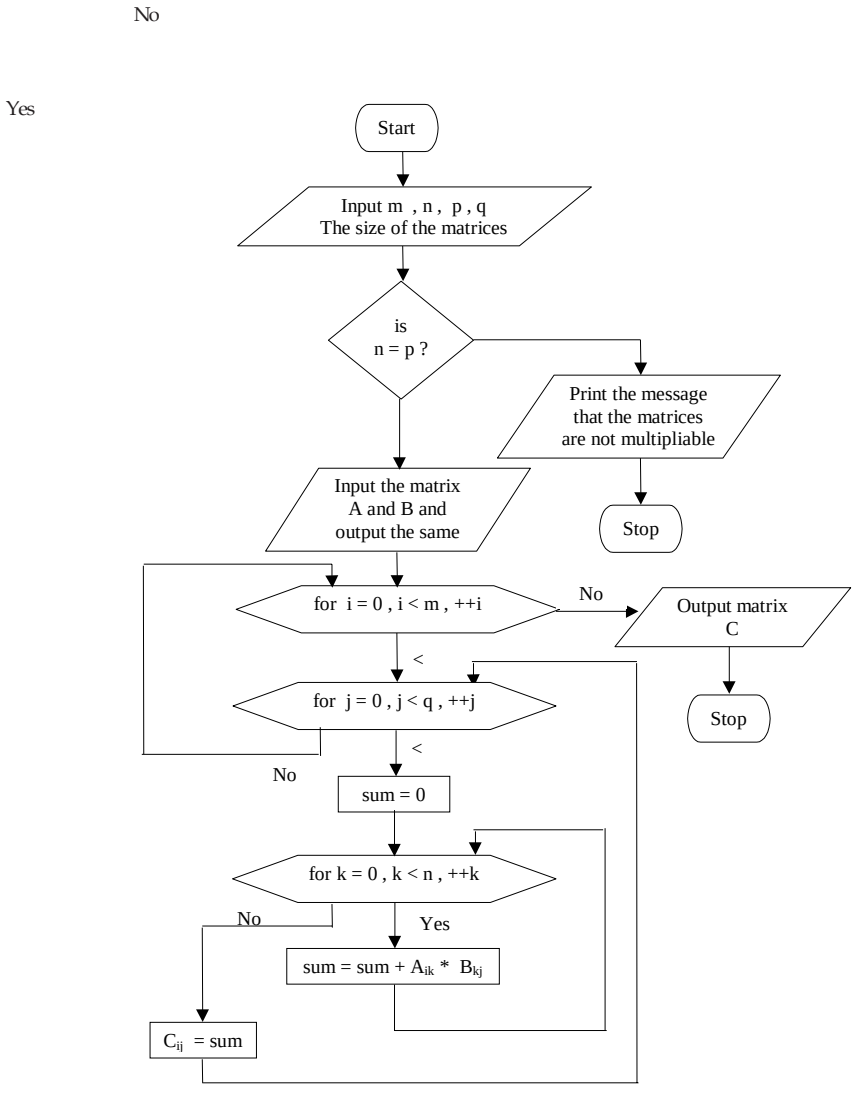


C-Tips

In this program, the program execution has to terminate when the two given matrices are not multipliable, i.e. the number of columns of the first matrix is *not equal* to the number of rows of the second matrix. For such terminations we can use either the ***exit*** function or a `goto` statement. In general, the use of *exit* functions is recommended. Whenever an *exit*

function is called the program terminates. Thus one can think of t*wo types of program execution terminations*. One is the *normal termination* and another is *termination under error conditions*. Normal terminations takes place after all the statements are executed. Abnormal termination, i.e. termination under error condition, takes place with the help of a call to a function like `exit`. In C termination of program execution *returns a numeric value* as *status* indication. This *status* value for normal termination will be zero (0) and it will be a non zero value for abnormal terminations. Therefore, to indicate error conditions the `exit( )` function is always written with a *non-zero value* as its argument. Whenever an `exit( )` function of this form is executed, the *non-zero value is returned* as the *status* value. If necessary, these *status* values returned by a program after the execution of an `exit( )` function *can be trapped* and used or *further processing*. At this stage the reader is advised not to bother much about the *status* values returned. It is sufficient if one uses the `exit( )` function by writing a statement like **`exit(1);`** whenever abnormal termination is required.

Flowchart



Note: Matrices are input and output under the control of a *for* loop. For further details Section 6.1 may be referred to.

The Program

```
/* To compute the product of two given matrices */

#include<stdio.h>
#include<conio.h>

main()
{
   int m, n, i, j,k,p, q,sum,a[10][10],b[10][10],c[10][10];
                              /* a[10][10],b[10][10],c[10][10] are
                                 2D arrays,each of size 10 * 10 */
   clrscr();
   printf("\nEnter the order of the first matrix A \n");
   scanf("%d%d", &m, &n);
   printf("\nEnter the order of the Second matrix B \n");
   scanf("%d%d", &p, &q);

   printf("Order of matrix A = %d X %d  \t Order of matrix B = %d X
          %d\n", m,n,p,q);

   if(n != p)
   {
       printf("\n Matrices are not multipliable \n");
       exit(1); /* The use of an exit statement to terminate the program */
   }

   printf("\nEnter elements of Matrix A\n");
   for(i=0; i<m; ++i)
       for(j=0; j<n; ++j)
           scanf("%d", &a[i][j]);
```

```
printf("\nEnter elements of Matrix B\n");
for(i=0; i<p; ++i)
    for(j=0; j<q; ++j)
        scanf("%d", &b[i][j]);

printf("\nMatrix A\n");
for(i=0; i<m; ++i)
{
    for(j=0; j<n; ++j)
        printf("%4d\t", a[i][j]);
    printf("\n");
}

printf("\nMatrix B\n");
for(i=0; i<p; ++i)
{
    for(j=0; j<q; ++j)
        printf("%4d\t", b[i][j]);
    printf("\n");
}

/* Finding the product */
for(i=0; i<m; ++i)              /* for each row */
{
    for(j=0; j<q; ++j)          /* for each column */
    {
        sum = 0;
        for(k=0; k<n; ++k)      /* for each matrix element */
             sum += a[i][k]*b[k][j];
        c[i][j]= sum;
    }
}

printf("\nProduct Matrix \n");
for(i=0; i<m; ++i)
```

```
   {
          for(j=0; j<q; ++j)
             printf("%4d\t", c[i][j]);
          printf("\n");
   }

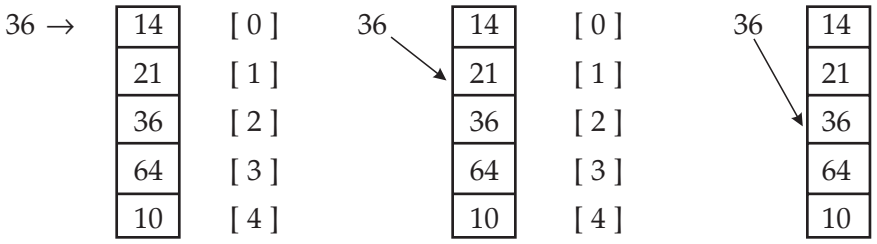

}
```

6.16 SEARCHING A GIVEN ELEMENT IN A GIVEN LIST OF INTEGER ELEMENTS (LINEAR SEARCH ALGORITHM)

The Method

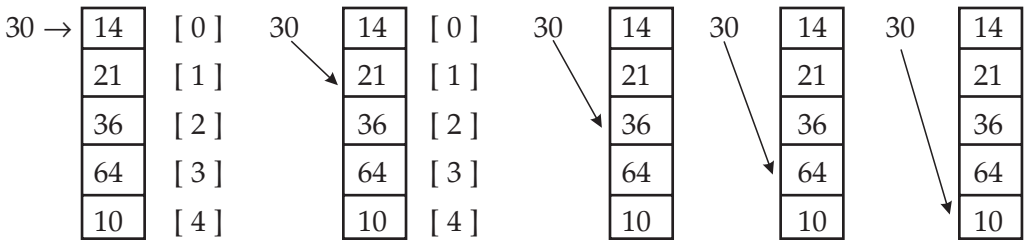
As a set of integer elements are to be handled, it is taken as an array, say **a**. Here the problem is to *search* for a required element in the given array. The required element is generally known as the *search key.* Let this search key be represented by a variable called **key**. The searching process for the key's presence in the array starts from the beginning of the array and continue till either the **key** is *available* or till the *end of the array* is reached. Therefore, this method is known as *linear search* or s*equential search method.* The following illustration and discussion gives an idea about the method used.

Search key = 36



Successful search

Search key = 30



Unsuccessful search

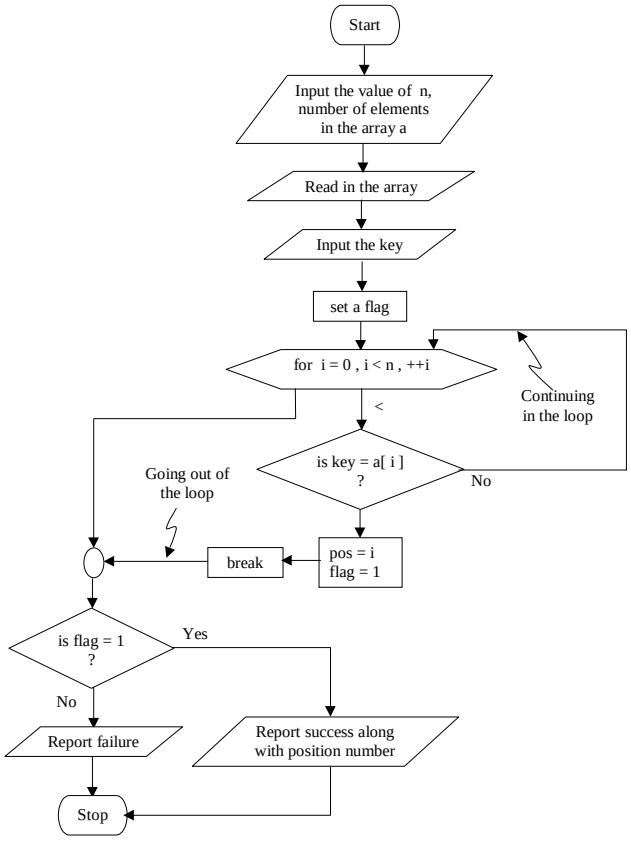
During a search for the required **key,** *the search* ***key*** is compared with each element in the array (for matching i.e. for equality), element by element, starting from the very first element. In otherwords the **key** is first compared with *a*[0], next with *a*[1], next with *a*[2], and so on. If a *match is found* for the **key**, the *loop terminates*; otherwise the comparison process continues till all the elements in the array are compared.

Under the loop control, whenever a *match is found* the control has to come out of the loop. Generally, leaving the loop prematurely is referred to as *breaking* and is achieved with a `break` statement. However, just before one *breaks* and tries to *go out of the loop prematurely*, following two things should be done:

1. The *index value,* i.e. the subscript value of the array, must be saved in some variable (say, **pos**) which helps in reporting the result.
2. A *flag* must be set which indicates the *success or failure of the search*.

If a *match is not found*, the loop gets past the last item of the array and the control comes out of the loop *normally*. When the loop control comes out of the loop normally the *flag* (success/failure indicator) remains unaltered i.e. remains the same as it was in the beginning. Obviously*, a flag has to be initialized in the beginning itself*. This flag is normally set to 0 (zero) initially, which signals *failure*. Whenever the *search succeeds*, the *flag is set to 1* (one). Thus the *final output depends upon the contents of this flag*. At the end, if the flag value still remains 0 (zero), *failure is reported* where as if the flag has become 1 (one), *success is reported* along with the position value.

Flowchart



C-Tips

The `break` statement is used to come out of the loop *prematurely* after satisfying some condition. Generally, under all situations where the program control may come out of a loop, it is necessary to identify whether the control has come out normally or *prematurely*, and then act accordingly. A *flag* is used under all such situations. Here, *flag* means *any variable* used for the purpose of just flagging, i.e. indicating, the success or the failure, not necessarily the name *flag.*

The Program

```
/* Searching a given element in a given list of elements. */
/* Linear Search Problem */


#include<stdio.h>
#include<conio.h>

main()
{
   int i, n, flag = 0,pos=0;
   float a[20], key;              /* Declaring array a and key */
   clrscr();
   printf("\nEnter the  value for n\n");
   scanf("%d", &n);
   printf("n = %d\n", n);

   printf("Enter the elements of array a\n");
   for(i=0; i<n; ++i)
     scanf("%f", &a[i]);          /* Reading elements of the array */

   printf("Elements of the given array are \n");
   for(i=0; i<n; ++i)
      printf("%.2f\t", a[i]);    /* Printing elements of array */

   printf("\nEnter key number \n");
   scanf("%f", &key);         /* Reading  the key number */
   printf("Key number = %.2f\n", key);


   for(i=0; i<n; ++i)             /* Repeating for each element */
   {
      if ( a[i] == key)           /* Whether element is equal to
                                     key number or not ? */
```

```
      {
         flag=1;                /* if yes, flag is set to 1 to
                                   indicate key is found */
         pos = i;
         break;                 /* break from for loop */

      }
   }


   if ( flag )                  /* If flag is true (i.e,one) then key
                                   is found */
     printf("%.2f is found in position number %d\n", key, pos+1);
   else                         /* Otherwise key is not found */
     printf("%.2f is not found in the array\n", key);


}
```

6.17 SEARCHING FOR A GIVEN ELEMENT IN AN ARRAY USING BINARY SEARCH METHOD

The binary search method is applicable only for sorted arrays. Infact, if number of elements are more (how many?) they must be *first sorted* and then the binary search method has to be used.

The Method

In this method, the *search starts* by testing the data element at the *middle* of the array. If the required element *is available* at *that position*, then the *search is complete*. If it *is not available* at that point (i.e at the mid position) it *may be available* either in the *first half* or in the *second half* of the array. The *actual half* (of the array) in which the required data item may be present is determined by making the following test. If the *search key is larger* than the *element at the middle* of the array then the required element may be available in the *second half* of the array. If it is *smaller than* the element at the middle of the array then it may be available in the *first half*. ***If it is in the second half, first half is not searched and vice versa***.

The above process is *repeated* until the required element is *found* or *we get satisfied* that the element is *not available* in the array. Let *low* represents the beginning of the *search*

portion and *high* represents the ending of the search portion. Initially, the *entire array* will be the *search portion*. Therefore the value of *low* will be zero (0) and value of *high* will be (n – 1), where n is the number of elements to be sorted. The *middle* of the array is computed using the relation:

mid = (low + high) / 2 .... integer division.

Whenever the required element is in the second half (*mid + 1*) position is made as *low* position and the search is continued.

Whenever the required element is in the first half (*mid - 1*) is made as *high* position and the search is continued.

The search process is terminated when we come across one of the following conditions

1. search key = element at the *mid* position

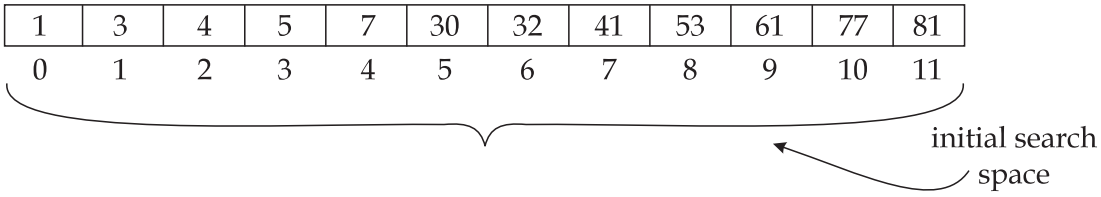
   or

2. value of *low* > value of *high*

Below is given an illustration, the study of which gives a better understanding of the process involved.

Illustration

Consider a set of elements sorted in ascending order :

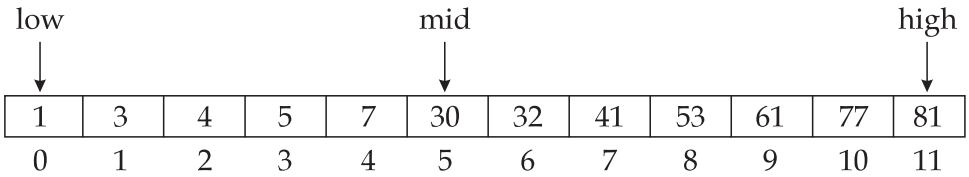


Let search key = 32

Step1

low=0, high=11

mid = (low + high)/ 2 = (0 + 11)/2 = 5.

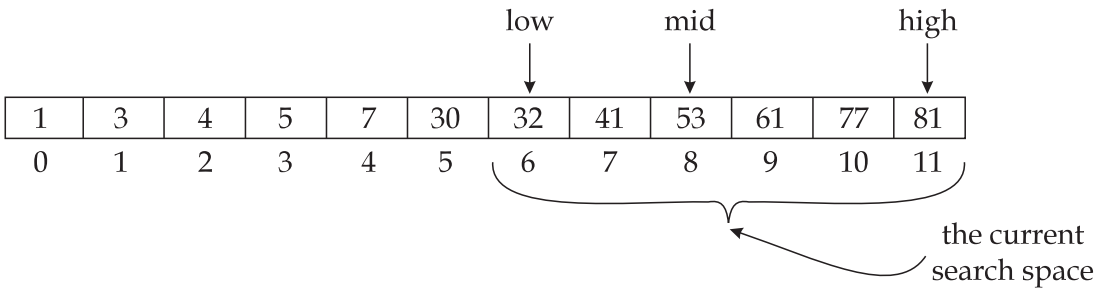


The element at the mid position i.e., at 5 is 30, which is not the required key.

Step 2

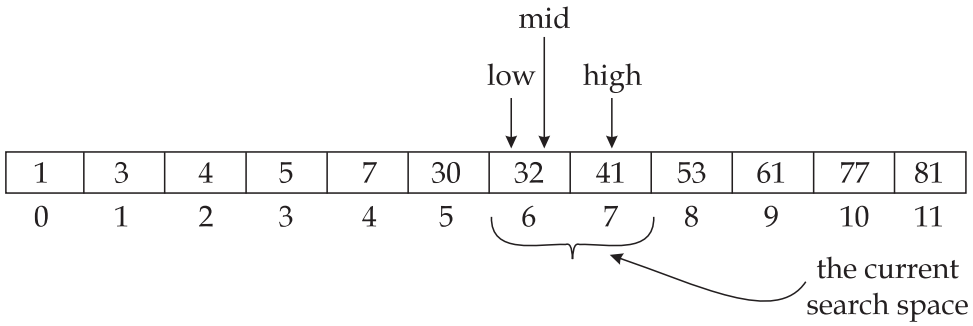
Since the search key *(32) is > the middle element* (30), the search key (32) may be present in the *second half*. Therefore, *low = mid + 1,* i.e. 5 + 1 = 6. The value of *high* remains 11, unaltered. The new value of mid is:

mid = (low + high) / 2 = (6 + 11)/2 = 8.



Now the element at the new mid position, i.e. at 8 is 53. Since the *search key (32) is < than the mid element (53)*, the search key (32) may be present in the *first half* of the current search space. Therefore, *high = mid − 1* = 8 − 1 = 7. The value of *low* remains 6, i.e., unaltered.

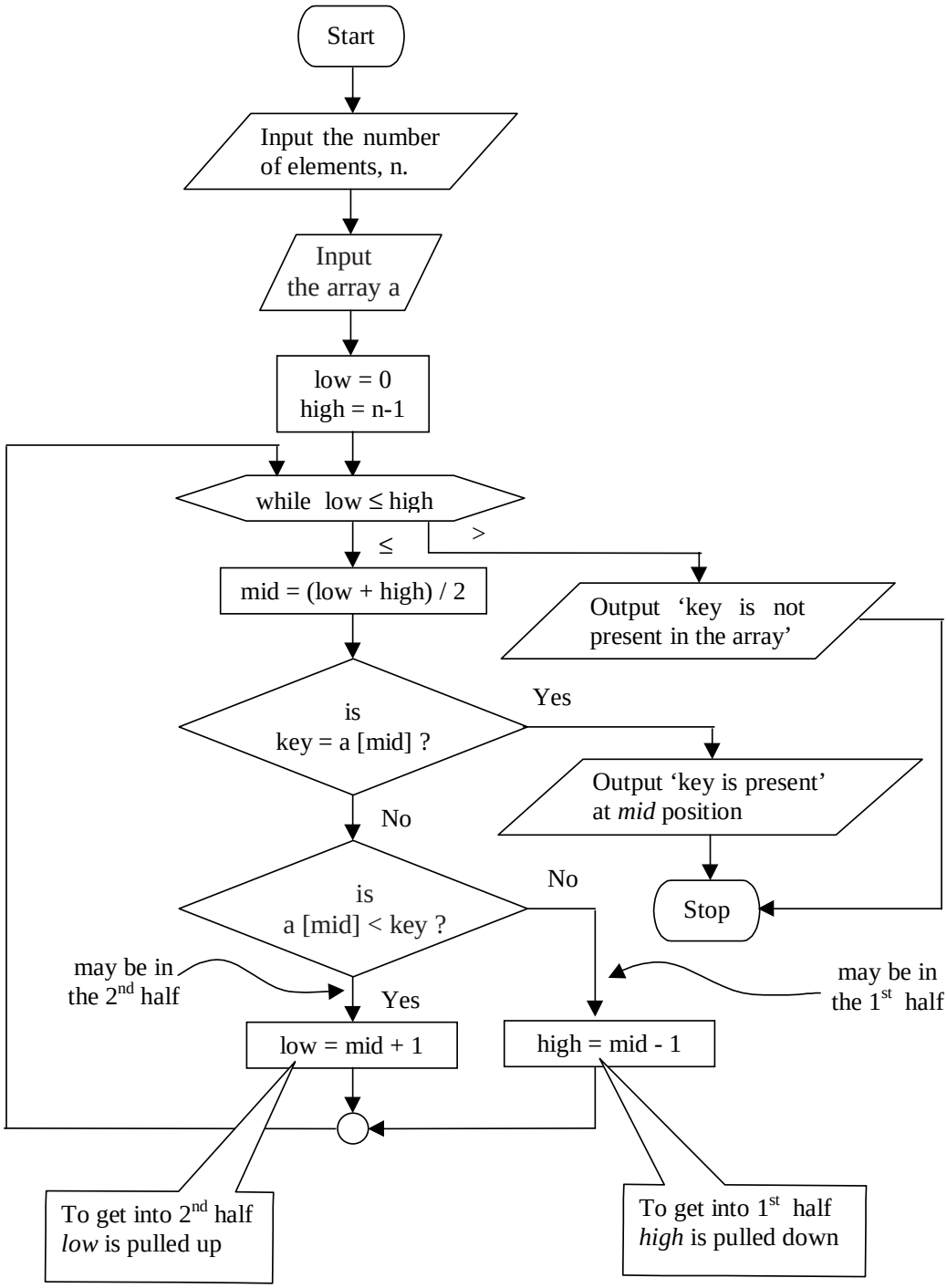
Step 3



mid = (low + high) / 2 = (6 + 7)/2 = 6.

Here we find that the value at position 6 is 32 which *is equal to the search key*. Hence, at this stage *success is reported* along with the position value. Incase, even at this stage, if the search was not a success, the procedure discussed in the earlier steps is repeated. As already mentioned, the procedure is terminated when the condition *low > high* appears.

Flowchart


Start
Input the number of elements, n.
Input the array a
low = 0
high = n-1
while low ≤ high
≤
>
mid = (low + high) / 2
Output 'key is not present in the array'
is key = a [mid] ?
Yes
Output 'key is present' at *mid* position
No
Stop
is a [mid] < key ?
No
may be in the 2nd half
Yes
may be in the 1st half
low = mid + 1
high = mid - 1
To get into 2nd half *low* is pulled up
To get into 1st half *high* is pulled down

The Program

```
/* Searching for a given element in an array using binary search
method. */

#include<conio.h>                /* Including  header files */
#include<stdio.h>

main()
{
   int i, n, a[20],key,low, high, mid;
   clrscr();
   printf("\n\nEnter value for n\n");
   scanf("%d", &n);
   printf("n = %d\n", n);
   printf("\nEnter the elements of array a in ascending order \n");
       /* Reading elements of array */
   for(i = 0; i < n; ++i)
       scanf("%d", &a[i]);
   printf("Elements are \n");     /* Printing elements of array */
   for(i = 0; i < n; ++i)
       printf("%d\n", a[i]);
   printf("\nEnter key element to be searched\n");

   scanf("%d", &key);             /* Reading Key element */
   printf("key = %d\n", key);     /* Printing key element */

   high = n-1;
   low = 0;                       /* Initializing high and low */

   while(low <= high)
   {
       mid = (low + high)/2;      /* mid is mean of low and high */
       /* if key is equal to middle element then exit from loop */
       if( key == a[mid] )
         break;
```

```
      else  if( key > a[mid] )
                /* if key is greater than middle element then it
                   must be in second half. Make low=mid+1 */
                low = mid + 1;
           else
               high = mid - 1;

  }

  if (a[mid] == key)
  {
      printf("Search is successful \n");
      printf("%d is found in position number %d\n", key, mid+1);
  }
  else
      printf("Search is unsuccessful, %d is not found \n", key);


}
```

6.18 SORTING A GIVEN ARRAY OF ELEMENTS USING SELECTION SORT

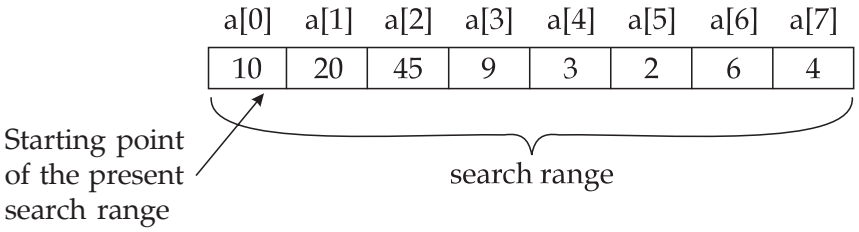
The problem here is to *sort a given set of unsorted integer elements* into a required order, i.e. ascending or descending order.

The Method

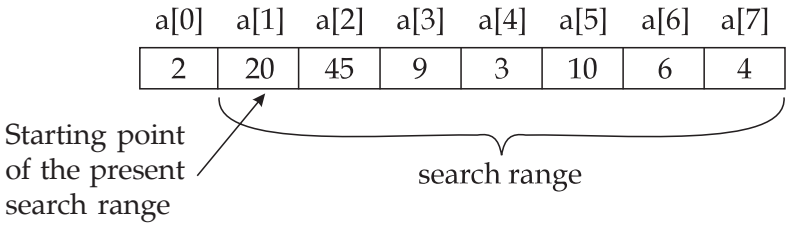
By default it is assumed that we need to arrange the elements in ascending order, i.e. in the increasing order of their numeric values. A study of the following illustration gives an idea of the procedure to be followed in *sorting by selection*. Here **'by selection'** means selecting the first minimum and storing it in the first place, selecting the second minimum and storing it in the second position, selecting the third minimum and storing it in the third position and so on. However, *care should be taken not to loose the already existing values whenever we are storing the minimum values in their proper slots,* i.e. locations. One of the general techniques employed *not to loose the value* of the element already existing in the place where a minimum value is stored, is to *save the value of the element already existing in that location.* This *saved value* is stored in the location from where the minimum value will be picked up.

Illustration

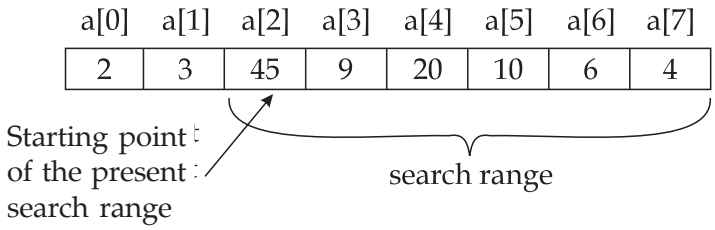
Consider a set of integers stored in an array, as shown in the figure below.



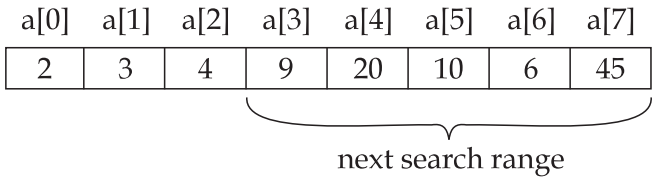
In the *first step select a minimum value* among *a*[0], *a*[1], *a*[2], *a*[3]….. *a*[7] and *store it at a[0].* Here, the minimum is 2 and is at *a*[5]. The values at *a*[0] and *a*[5] are *exchanged*.



In the *second step*, *select a minimum* value among *a*[1], *a*[2], *a*[3]….. *a*[7] and store it at *a*[1]. Here, the minimum is 3 and is at *a*[4]. The values at *a*[1] and *a*[4] are *exchanged*.



In the *third step, select a minimum* value among *a*[2], *a*[3]… *a*[7] and store it at *a*[2]. Here minimum is 4 and is at *a*[7]. The values at *a*[2] and *a*[7] are *exchanged*. Now the array will be as shown in the following figure.



From the above illustration following conclusions are drawn. Let ***i*** is used as subscript for searching the *minimum*. In the *first step* search is made using a_i where *i varies from 0 to 7* and the *minimum is stored* at the *starting point of the search range*, i.e. at a_0.

In the *second step* the search is made using a_i where *i varies from 1 to 7* and the *minimum is stored* at the *starting point of this search range*, i.e. at a_1.

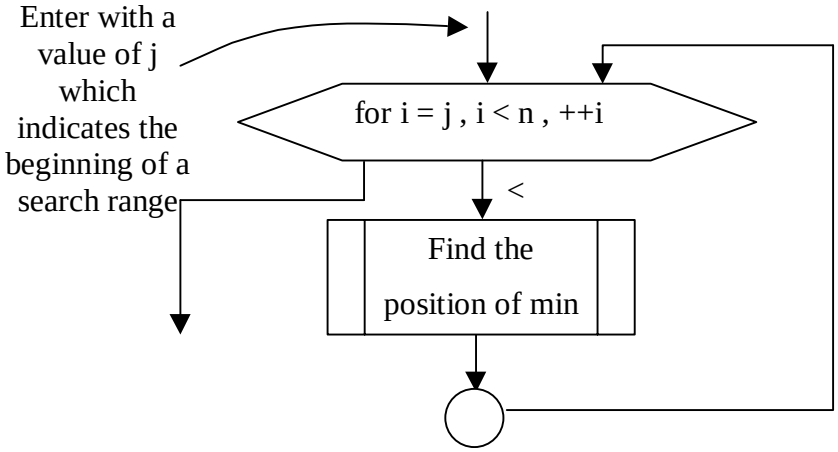
In the *third step* the search is made using a_i where *i varies from 2 to 7* and the *minimum is stored* at the *starting of the search range* i.e., at a_2.

Thus, during first search *i* varies from 0 to 7

During second search *i* varies from 1 to 7

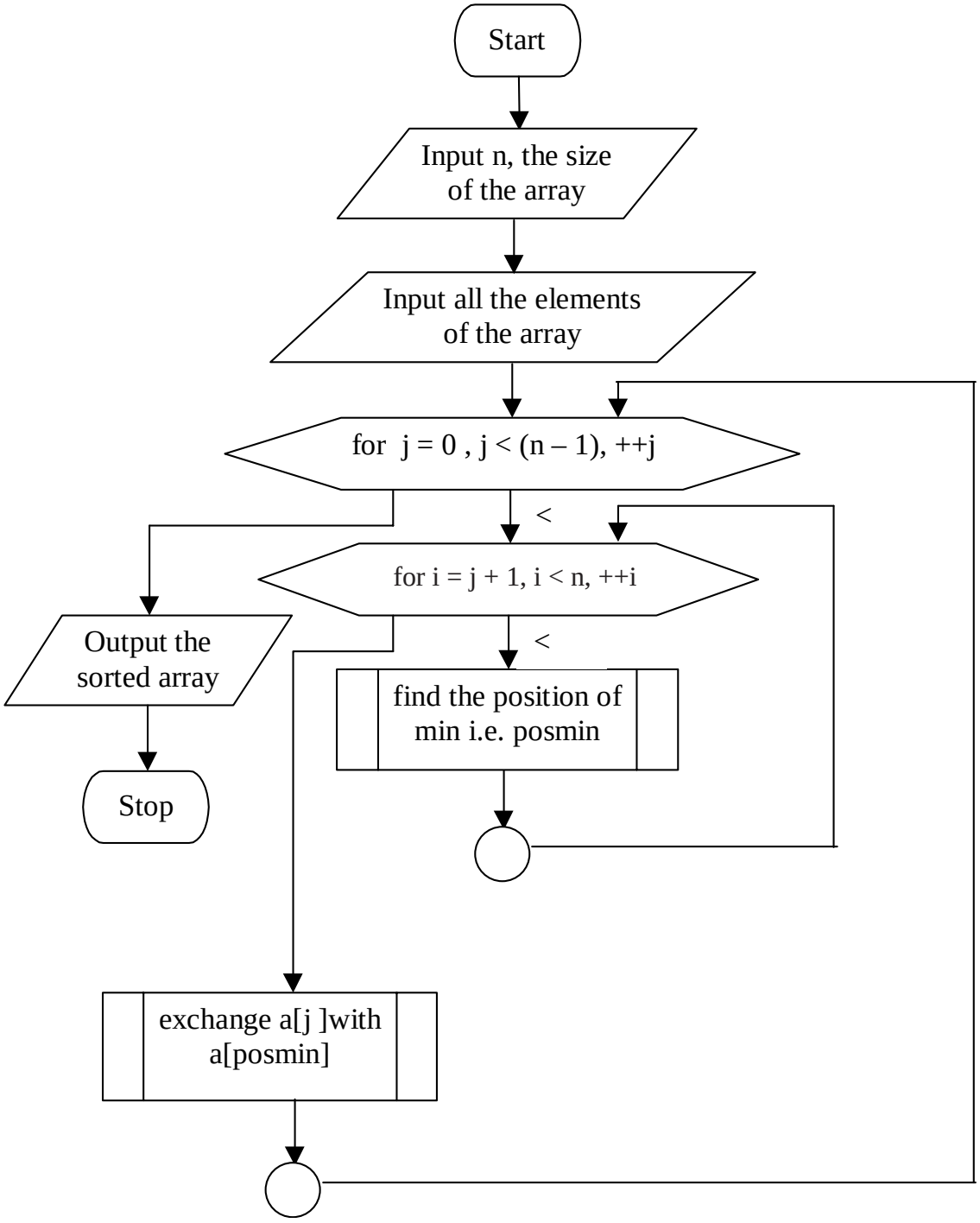
During third search *i* varies from 2 to 7 and so on.

It may be noted that during every search the control *traverses* from a specific starting point till the end of the array. Each of these traversal is called a *pass.* If number of elements to be sorted are '*n*', then a minimum element is searched during every pass, using a *counted loop structure* as follows:



Also it may be noted that here it is necessary and sufficient to search for the position value where the minimum exist. During the first *pass* or *iteration j* has to be 0 (zero), during second *pass* or *iteration j* has to be 1, during third *pass* or *iteration* it has to be 2 and so on. After finding a minimum it has to be *exchanged* with the *first element in that particular search range*. Noting that during every iteration or pass *j* indicates the starting point of the present search range. We may use a statement like a_j = *minimum* to put the *minimum* at its proper position. Thus the technique discussed till now can be pictorially shown in the form of a flowchart as follows.

Flowchart


Start
Input n, the size of the array
Input all the elements of the array
for j = 0 , j < (n – 1), ++j
<
for i = j + 1, i < n, ++i
<
Output the sorted array
find the position of min i.e. posmin
Stop
exchange a[j]with a[posmin]

The Program

```
/* Sorting the given set of elements in ascending order using
selection sort */

#include<stdio.h>
#include<conio.h>

main()
{
   /* declaring variables */
   int a[20], n, i, j, posmin,temp;

   clrscr();

   printf("Enter the number of elements\n");
   scanf("%d", &n);
   printf("Number of elements = %d\n",n);

   printf("Enter the elements\n");

   /* Accepting elements */
   for(i=0; i<n; ++i)
     scanf("%d", &a[i]);

   /* Printing elements */
   printf("\nGiven list of elements\n");
   for(i=0; i<n; ++i)
     printf("%d\t", a[i]);


   for(j=0; j<n-1; ++j)
   {
      posmin=j;
```

```
      for(i=j+1; i<n ; ++i)
      if(a[i]<a[posmin])
       posmin=i;

      temp=a[j];
      a[j]=a[posmin];
      a[posmin]=temp;

  }

  /* Printing sorted array */
  printf("\nSorted list of elements\n\n");
  for(i=0; i<n; ++i)
    printf("%d\t", a[i]);


}
```

EXERCISES

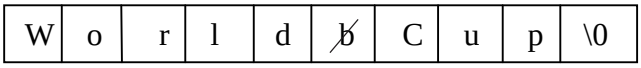
1. Develop a flowchart to find the smallest element in a given set of elements. Code the same using C.
2. Develop a flowchart to find the first biggest and the second biggest elements in a given set of elements. Code the same using C.
3. Develop a flowchart to compute the average of given set of integers (both +ve and -ve) and to count the numbers above the average and below the average. Code it using C.
4. Develop a flowchart to check whether a given set of elements is sorted or not with suitable messages. If sorted, print the order in which it is sorted. Code the same using C.
5. It is necessary to prepare a merit list of 100 students given their total marks. Develop a flowchart for the same and code it using C.
6. Develop a flowchart to compute the row sums of a given matrix and code it using C.
7. Develop a flowchart to compute the column sums of a given matrix and code it using C.

8. Develop a flowchart to convert a decimal integer number into binary number. Code the same using C.
9. Develop a flowchart to generate a square matrix of given order with
   i) 0's as principal diagonal elements.
   ii) 1's as upper diagonal elements.
   iii) -1's as lower diagonal elements of the matrix.
10. Develop a flowchart to insert a given integer element into its proper position in a given sorted array. Code the same using C.

CHAPTER 7

Characters and String Handling

Handling characters and *sequences of characters* with computers is one of the very important activities. *Sequences of characters* are generally referred to as **strings**. In C, strings are written within a *pair of double quote marks (")* and are *stored* in the form of **character arrays with a null character** (`\0`) as it's *last element*. This null character will be *automatically* inserted by the C compiler. For example, "World Cup" is a string and is stored as shown in the following figure.



We have been using character arrays or *strings* right from the very first example to display messages along with the `printf( )` functions. However, none of the examples so far have discussed the *processing* of either a *character* or a *sequence of characters,* i.e. strings. For processing of characters and strings it is necessary to know how the characters are stored inside computers. In computers characters are stored using *numeric codes*. The most popular text codes that have been employed hitherto are EBCIDIC (pronounced as *Eb-si-dic*), ASCII (*As-key*) and Unicode. Among these ASCII code is by far the most widely used, particularly with personal computers. ASCII stands for American Standard Code for Information Interchange and is the contribution of American National Standards Institute (ANSI). In it's original form ASCII has 128 codes ranging from 0 to 127. Among these codes, 48 to 57 are used to represent decimal digits, codes 65 to 90 are used to represent the upper case English alphabets, codes 97 to 122 are used to represent the lower case English alphabets and the remaining are used to represent various control, special and white space characters. The point the reader has to understand here is that *numeric codes are used to represent the characters.* Therefore, processing of characters is somewhat similar (not exactly same) to the processing of integer numbers, which has been discussed in the

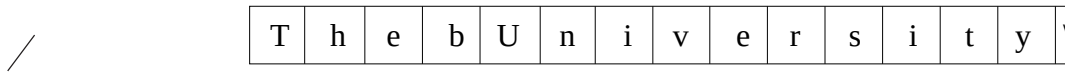
preceding chapters. For example, two character strings can be compared just like integers, names in the form of strings can be sorted using Bubble sort or any other sorting procedures, upper case alphabets can be converted into lower case alphabets by adding a numeric digit (32) to each one of them, and so on.

Typically, character processing also deals with the techniques used for counting number of characters, number of words and/or number of lines in a given text. However, while dealing examples of these types, one has to *carefully identify* the **source**, from where the *input stream* is coming and how *its end is identified*. The source could be either a *string* or a *file*. In the case of strings, a *null* character indicates the *end* of the input stream, whereas in the case of a file, an *end-of- file (EOF)* character indicates the *end* of the input stream. Here, it may be carefully noted that the C compiler, by default, considers the *keyboard* as a file. The *stream of characters input via a keyboard* is therefore considered to be coming from a file. It is important to note that in the case of input via a keyboard, the `Ctrl-z` character is taken as the EOF character. Of course, when the input comes from a string the *null character* (`\0`) indicates the end. This chapter discusses some examples like the ones cited in this as well as the previous paragraph.

7.1 FINDING THE LENGTH OF A GIVEN STRING

The Method

In C, a *string* is stored as an *array* with a null character (`\0`) *as its last element*. For example, the string "The University" is stored in an array as shown in the following figure.



The **length** of the string is equal to the *actual* 'number of characters' in the string (i.e. the character '`\0`' is not considered as a part of the string). Thus the *length* of the string shown in the figure above is 14. Obviously, in order to find the length of the string one has to *count the number of characters* from the beginning till '`\0`'. As in the case of an array each element of a *character array,* i.e. *string*, is also accessed using a subscript. The flowchart on page 138 gives the procedure for determining the length of the string.

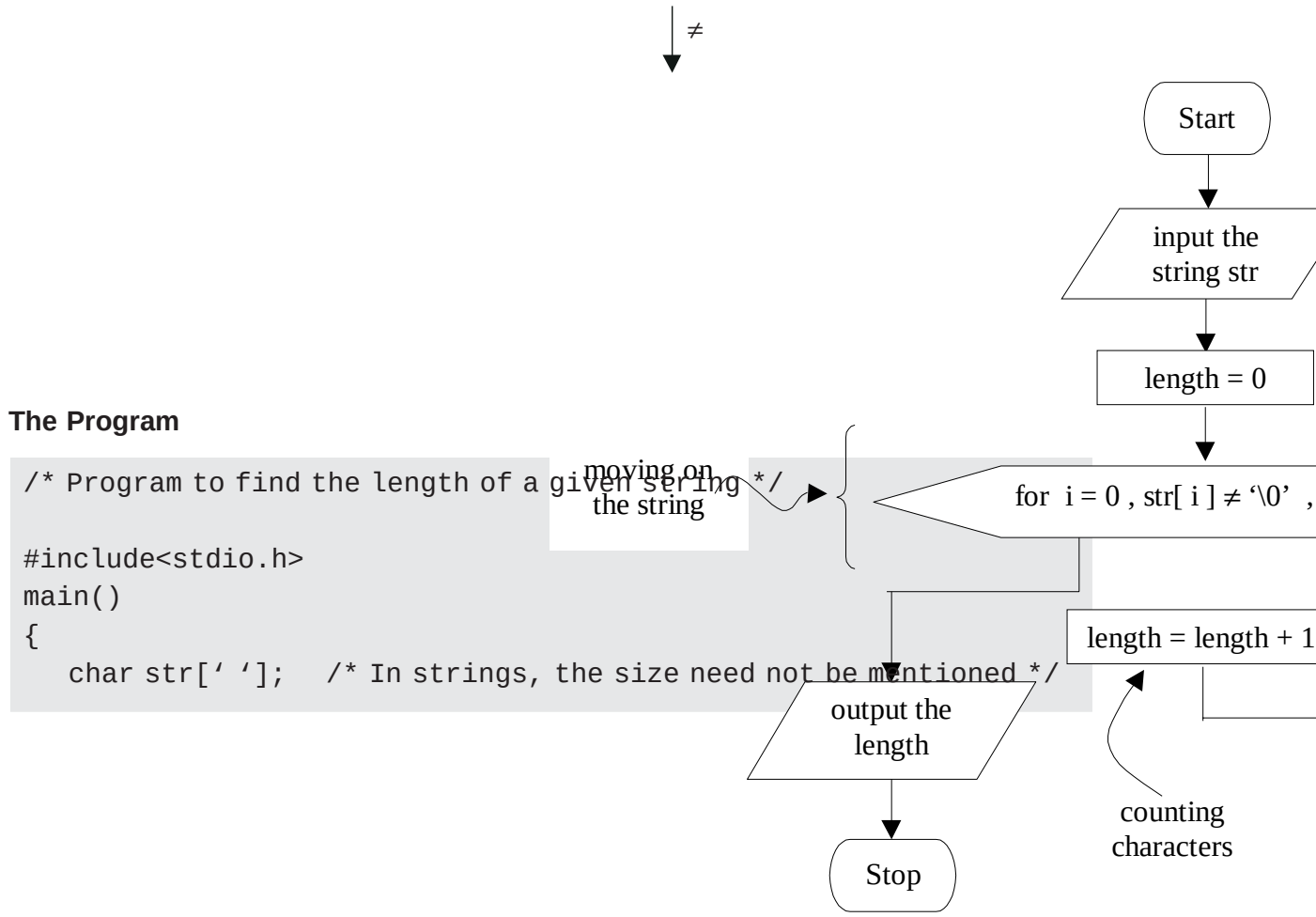
C-Tips

The contents of a string are considered as characters. Therefore a string is declared both as of `char` data type and as an *array* by mentioning the *size of the array.* As strings are of *variable lengths*, the size of the string need not be mentioned explicitly.

The function **`gets( )`** is used to input a string *terminated by a* '`\n`'*, the newline character,* via the keyboard. The input string will be stored in the *variable name* that appears as the *argument* of the **`gets( )`** function in the form of an array with a '`\0`' as its last character. Here it may be noted that *blanks* input as part of the input *string*, till a *new line character* (`\n`) are also taken as valid characters.

As *finding the length* of a string is one of the quite frequently used activity, it is available as a *predefined* or *library function* in C. The function `strlen( )` is this predefined function. It is available in a header file called `<string.h>` which has to be *included* in the beginning of a program in order to use it.

Flowchart



The Program

```
/* Program to find the length of a given string */

#include<stdio.h>
main()
{
   char str[' '];   /* In strings, the size need not be mentioned */
```

```
   int length,i;
   clrscr();

   printf("Enter a string :\n");
   gets(str);

   length=0;

   /* Characters are counted, one by one, by moving along the string
      with the help of the subscript i, till the null character '\0' */

   for(i=0; str[i] != '\0';++i)
   length++;

   printf("The length of the given string is : %d\n",length);

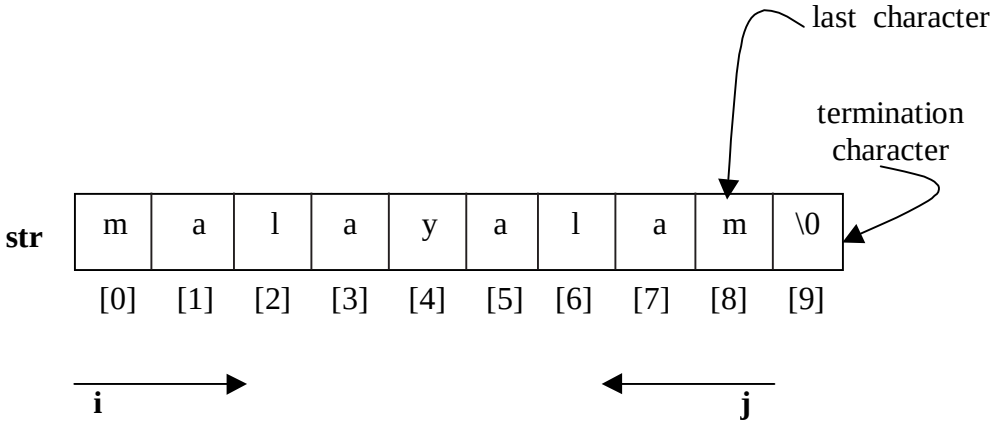
}
```

7.2 DETERMINING WHETHER A GIVEN STRING IS PALINDROME OR NOT

By definition a palindrome is a string that *reads alike backward* and *forward.* For example, the string "malayalam" is a palindrome.

The Method

The given sentence is first read in as a string, say **str**. As already mentioned it gets stored as a *character array* with a null character (\0) at the end, as shown in the following figure.

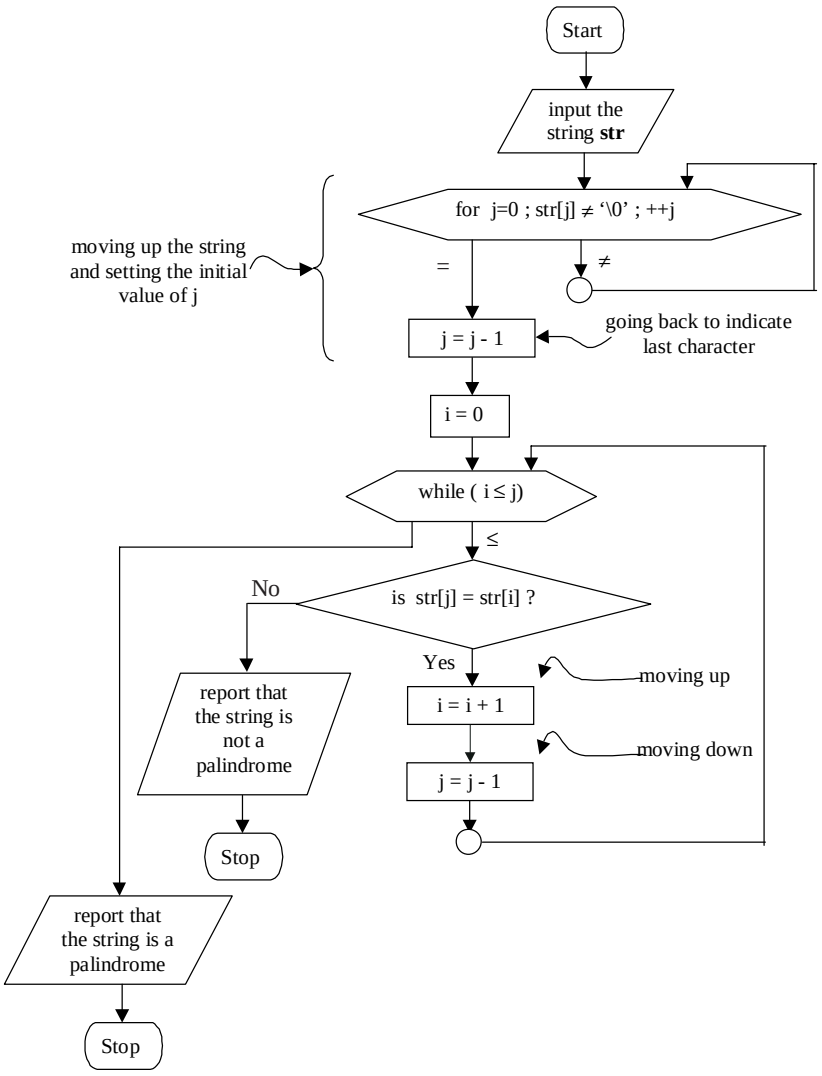


In order to *identify* whether the given string is *a palindrome* or not, the *first character* at `str[0]` is *compared* with the *last character* at `str[8]`. If they are same, then the character at `str[1]` is compared with the character at `str[7]`. Again, if they are same, the character at `str[2]` is compared with the character at `str[6]`, and so on.

Obviously, it is recommended to use *two subscripts*, like ***i*** and ***j***, to move along the character array.

- Subscript ***i*** is used to move up the string starting from 0 (zero).
- Subscript ***j*** is used to move down the string from the last character's position value. For example, here, the initial value of ***j*** is 8.

Flowchart



The Program

```
/* Program to test whether a given string is Palindrome or not */

#include<stdio.h>
main()
{
   int i,j;
   char str[‘ ‘];
   clrscr();

   printf(“Input a string :\n”);
   gets(str);

   printf(“\n\nThe input string is :\n”);
   puts(str);

   for(j=0; str[j]!=’\0' ; j++);

   j=j-1;

   i=0;

   while(i<=j)
   {

     if(str[i]==str[j])
     {

       i=i+1;                    /* Moving up the string */

       j=j-1;                    /* Moving down the string */

     }
     else
     {
```

```
      printf("The given string : %s IS NOT A Palindrome",str);
      exit(1);
    }

  }
  printf("The given string : %s IS A Palindrome",str);

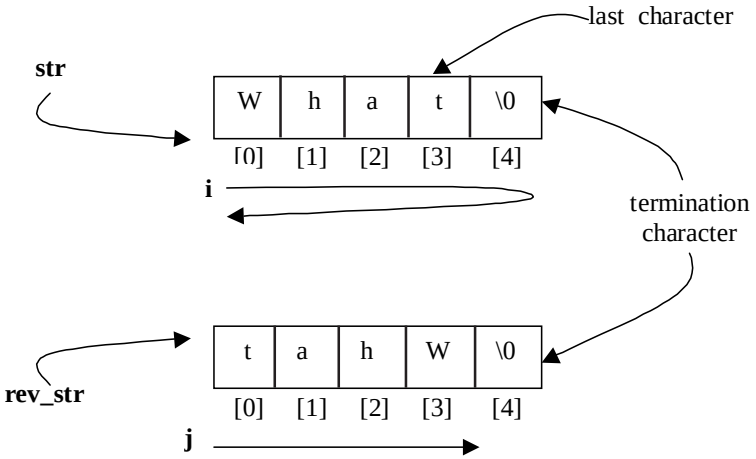
}
```

7.3 REVERSING A GIVEN STRING

The Method

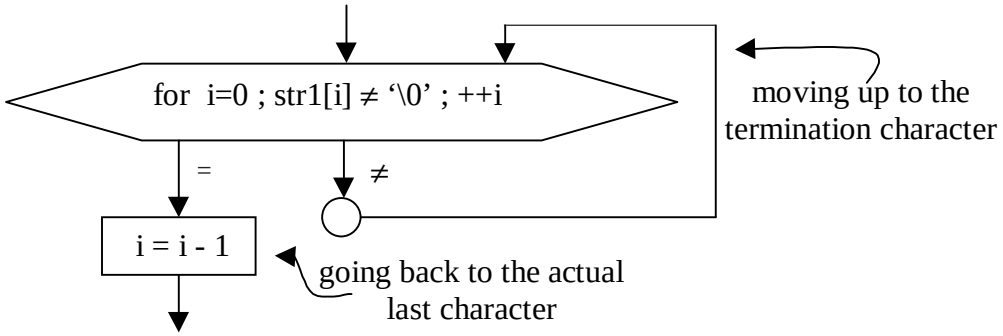
The procedure of obtaining the *reverse of a given string* is explained through a simple illustration as follows.

Illustration Consider a string **`str`** of length equal to 4. The reverse of the string **`str`** is stored in a string called **`rev_str`**.



In order to ***reverse*** the given string the control has to ***move*** on both the strings **`str`** and **`rev_str`**. The subscript ***i*** is used to *move* on the given string **`str`** and the subscript ***j*** is used to move on the reversed string **`rev_str`**.

In this example, the reversing process starts with the initial value of ***i*** as 3, i.e. the position value of the *last character* of the string **`str`**. Here, it may be noted that the *last character* is the one that appears just before the string *termination character* **`'\0'`**. The subscript ***i*** is set to the *necessary initial value* (i.e. 3) by incrementing (i.e. by moving) its value from 0, as shown in the following flowchart segment.

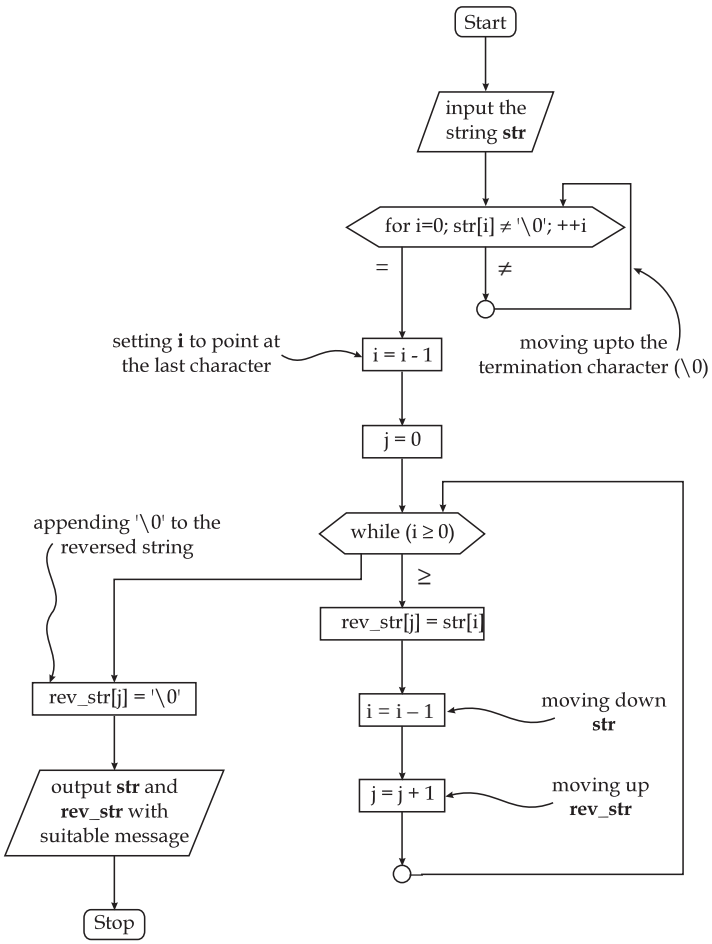


The *next step* is to *copy* the elements of the given string **`str`** onto the reverse string **`rev_str`** in the *reversed* order. This is done using the relation ***`rev_str[j]=str[i]`*** with the *decreasing values of* ***`i`*** and *increasing values of* ***`j`***. With the decreasing values of the subscript ***i,*** the control is *moving down* on the string **`str`** and with the increasing values of the subscript *j,* the control is *moving up* the **`rev_str`**. The above *copying* process is terminated when the beginning of the **`str`** is reached, i.e. when the subscript ***`i`*** becomes less than 0 (zero). In otherwords, the *copying process is done while* $i \geq 0$. The string reversing process discussed above is shown in the form of a flowchart in the adjoining figure.

C-Tips

A *`for`* statement, without body, may be used just to move along a string or array, as is used in this example.

Flowchart



The Program

```
/* Reversing a given string */

#include<stdio.h>
main()
{
 int i,j;
 char str[‘ ‘],rev_str[‘ ‘];
 clrscr();


 printf(“Input a string :\n”);
 gets(str);

 printf(“\n\nThe input string is :\n”);
 puts(str);

 for(i=0; str[i]!=’\0' ; i++);  /* Moving upto the termination
                                   character */

 i=i-1;                         /* Going back to the last character */

 j=0;

 while(i>=0)
 {
  rev_str[j]=str[i];

  i=i-1;                        /* Moving down str */

  j=j+1;                        /* Moving up rev_str */

 }

 rev_str[j]=’\0';

 printf(“The reverse of the given string is %s”,rev_str);

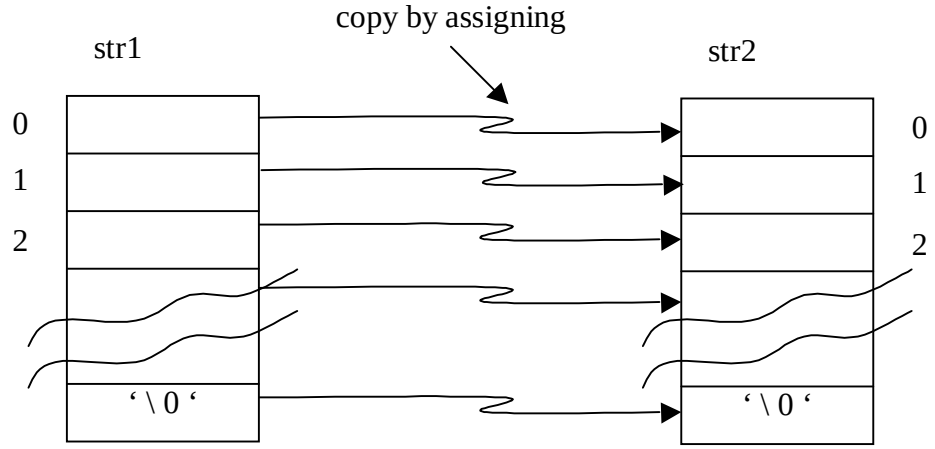
}
```

7.4 COPYING THE CONTENTS OF A GIVEN STRING INTO ANOTHER STRING

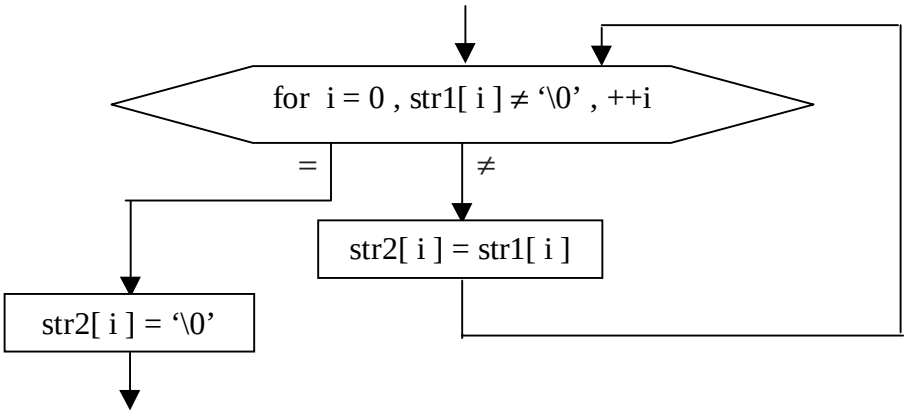
The Method

Let **`str1`** be the given string. The problem here is to *copy its contents* into another string, say **`str2`**. Copying the contents of a variable into another variable is straightforward and is done using an assignment statement. However, for copying the contents of a string (i.e. a character array) into another string, *every element of the first string has to be copied* into the corresponding location on the second string, one by one.

As *copying* process of every element is similar and has to be done *repeatedly* from the beginning (i.e. from index = 0) till the end (i.e. the null character '`\0`' is reached), we can use a loop control structure like '*`for`*' and get the necessary work done. The mechanism of copying each element is illustrated in the figure below.



The flowchart segment given below shows the *copying* process, element by element.



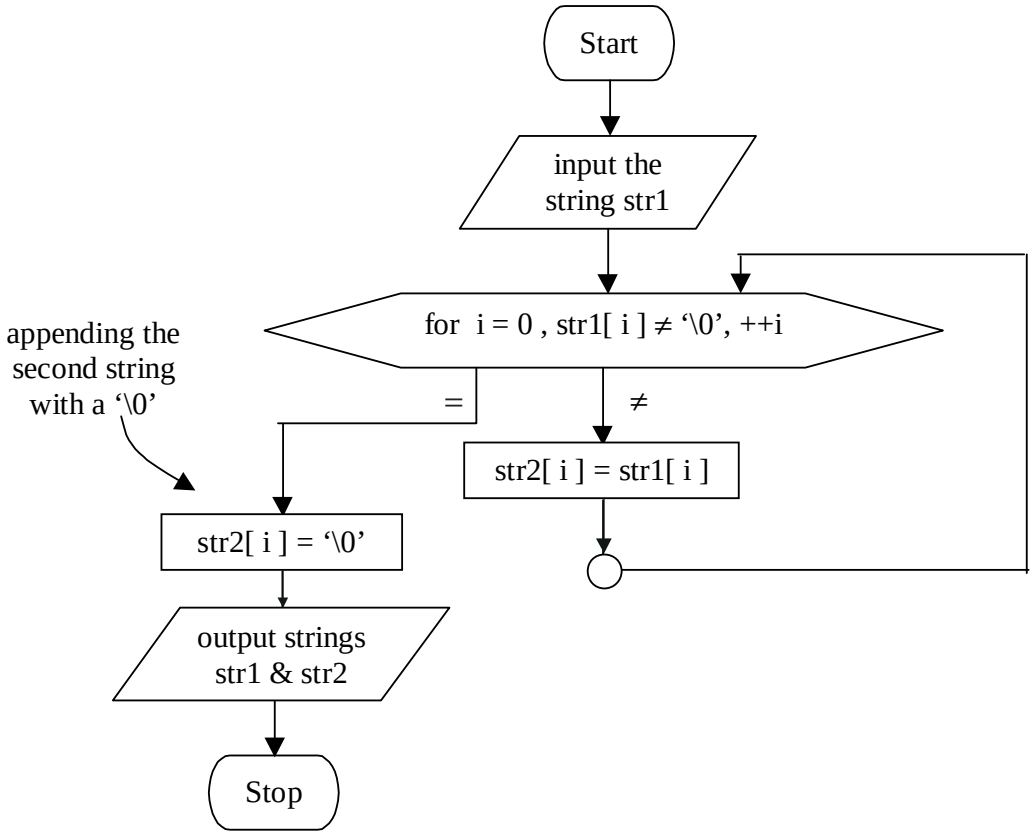
C-Tips

As is known, ordinarily copying is done by just assigning the value of a variable to another variable using an assignment statement like `var1=var2 ;`. However, this is not possible in the case of strings. In this case copying is done element by element. As string copying is one of the frequently used activity in C there is a predefined string copying function called `strcpy()` *function* which takes *two string arguments* and copies the contents of the *second argument* on to the *first argument*. This function automatically puts a *null character* at the end of the copied string.

As already discussed, predefined string functions are available on a *header file* called `<string.h>` which has to be included in the beginning of the program. Though it is possible to input a string using `getchar()` function, it is recommended to use the `gets()` function to input the string, as is done in this example.

When examples like the one discussed here are dealt, care should be taken to see that the null character '`\0`' is the last element on the second string (`str2`) also. Below is given the complete flowchart.

Flowchart



The Program

```
/* Program to copy a string into another string */

#include<stdio.h>
#include<conio.h>
main()
{
   char str1[‘ ‘],str2[‘ ‘];
   int i;
   clrscr();

   printf(“Enter a string\n”);
   gets(str1);                    /* Reading in the given string */

   printf(“The given string is : %s\n”,str1);

   /* following statements copy contents of str1 on to str2, one by one */

   for(i=0;str1[i]!=’\0';i++)
     str2[i]=str1[i];

   /* appending the null character to the end of str2 */

   str2[i]=’\0';

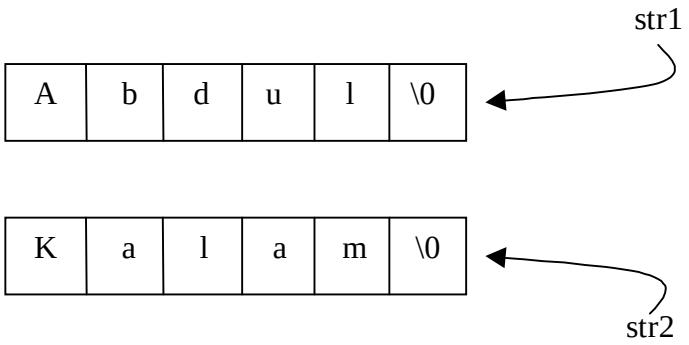
   printf(“\n %s is now copied onto str2\n”,str1);
   printf(“The contents of str2 is : %s”,str2);
}
```

7.5 CONCATENATING TWO GIVEN STRINGS

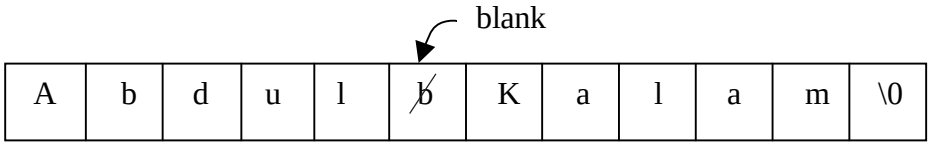
Concatenating two strings means *attaching* the second string to the end of the first string.

The Method

Let ***str1*** and ***str2*** be the two strings to be concatenated. Also let ***str1*** has "`Abdul`" and ***str2*** has "`Kalam`" stored in them respectively as shown in the figure below. It may be observed that both "`Abdul`" and "`Kalam`" have a '`\0`' as their last characters.



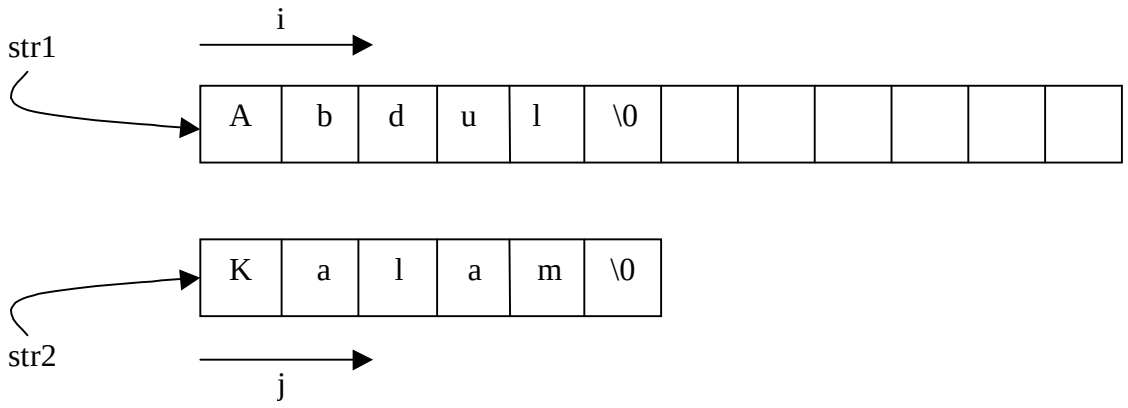
As concatenation 'appends' or 'attaches' the second string (`str2`) to the first string (`str1`), the final string (`str1`, itself) will be as shown in the following figure.



A careful observation of the above figure reveals the following two facts :

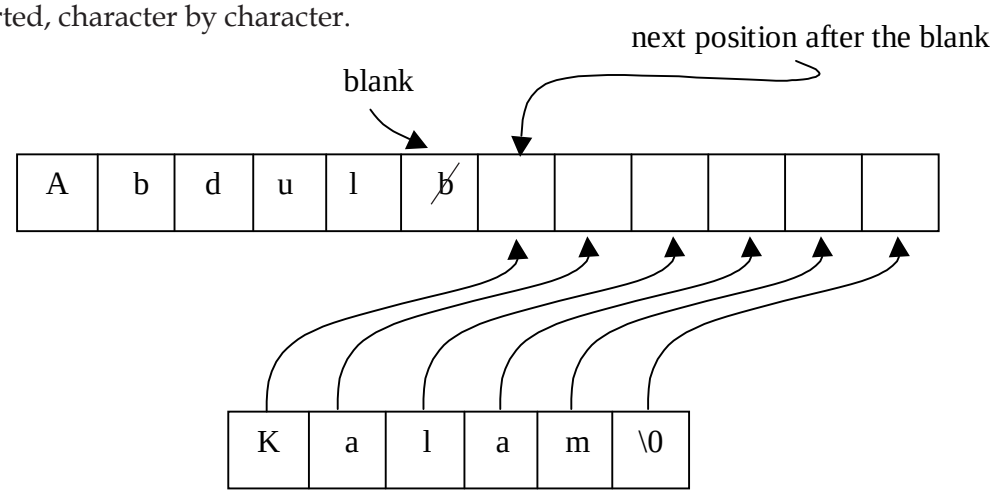
1. The '`\0`' character at the end of the first string which appears in between "`Abdul`" and "`Kalam`" after concatenation *has to be replaced by a blank.*
2. Since "`Kalam`" (i.e. `str2`) has to be attached to "`Abdul`" (i.e. `str1`), there should be *sufficient space* allotted along with the declaration of "`Abdul`", i.e. ***str1***.

The first figure given above is redrawn as shown in the figure on page 149 with *sufficient space* on the first string "`Abdul`" to accommodate the second string "`Kalam`".



The *concatenation* process needs the control to *move along both the character arrays,* i.e. strings. Let the subscript ***i*** is used to move along ***str1*** and the subscript ***j*** is used to move along ***str2***.

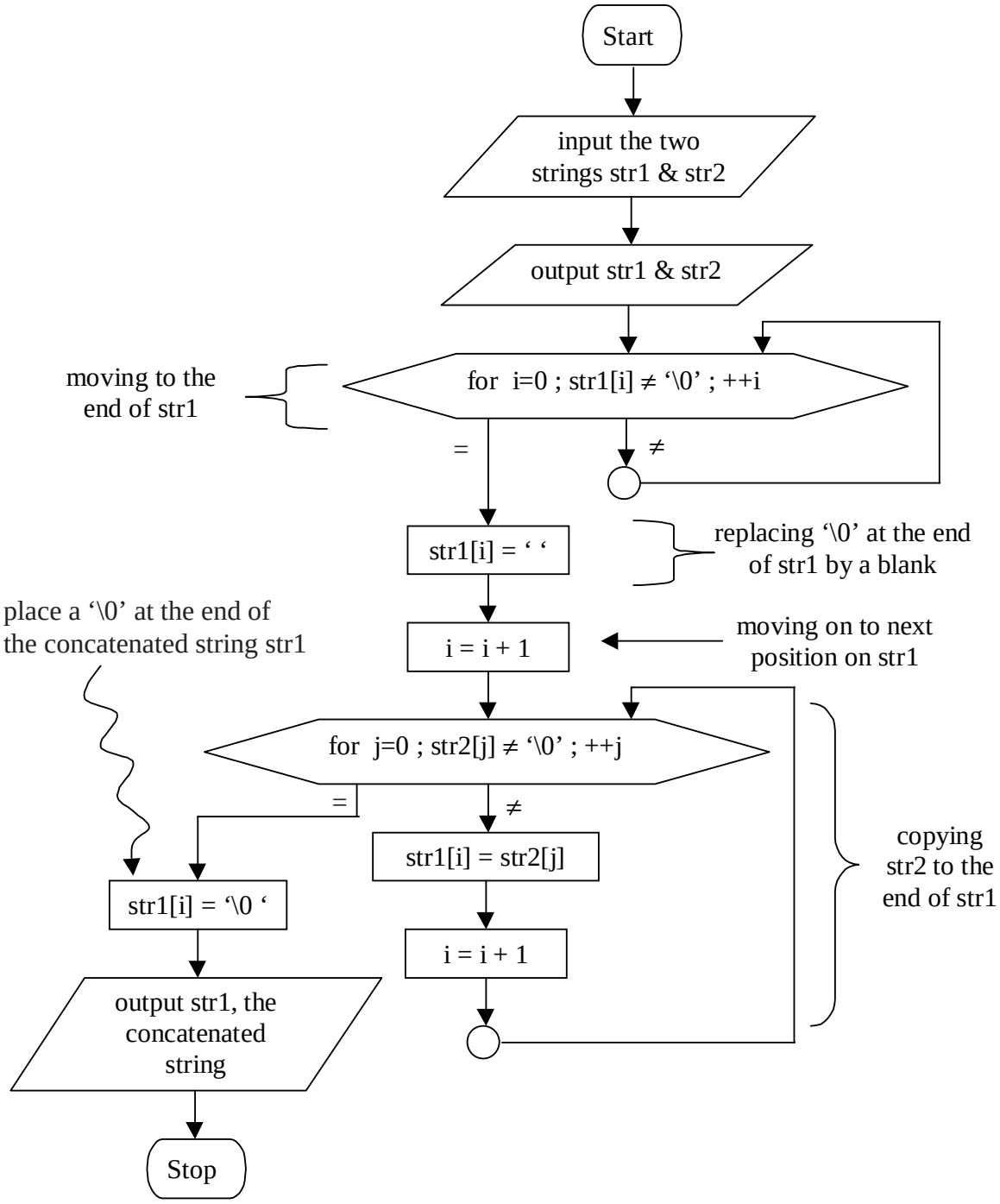
Since '\0' at the end of ***str1*** (i.e. "Abdul") has to be *replaced* by a blank, let us start moving on ***str1*** with the help of ***i***. We have to keep moving on ***str1*** as long as ***str1[i] ≠ '\0'***, As soon as str1[i] is *equal* to '\0', a blank character is placed at that position on ***str1***, i.e. at *current value* of str1[i]. Now ***str2*** is *copied* on to ***str1*** starting from the *next position* on ***str1***, i.e. the position which is next to the one where a blank was inserted, character by character.



C-Tips

In C there is a predefined function called ***strcat()*** which is used to concatenate two given strings i.e. to copy the contents of second string on to the end of the first string. As already mentioned, in order to use this function the *header file* ***<string.h>*** has to be *included* in the beginning of the program.

Flowchart


Start
input the two strings str1 & str2
output str1 & str2
moving to the end of str1
for i=0 ; str1[i] ≠ '\0' ; ++i
=
≠
str1[i] = ' '
replacing '\0' at the end of str1 by a blank
place a '\0' at the end of the concatenated string str1
i = i + 1
moving on to next position on str1
for j=0 ; str2[j] ≠ '\0' ; ++j
=
≠
str1[i] = str2[j]
str1[i] = '\0 '
i = i + 1
copying str2 to the end of str1
output str1, the concatenated string
Stop

The Program

```
/* Concatenating two given strings */

#include<stdio.h>
main()
{
   int i,j;
   char str1[‘ ‘],str2[‘ ‘];

   clrscr();

   printf(“Input the first string :\n”);
   gets(str1);                    /* input str1 */
   printf(“First string is :% s\n”, str1);
   printf(“Input the second string :\n”);
   gets(str2);                    /* input str2 */
   printf (“Second string is :% s/n”, str2);
   for(i=0; str1[i]!=’\0' ; i++); /* Moving to the end of str1 */

   str1[i]=’ ‘;        /* replacing ‘\0’ at the and of str1 by a blank */

   i=i+1;              /* moving to the next position on str1 */

   for(j=0; str2[j]!=’\0' ; j++)
   {
     str1[i]=str2[j]; /* Copying str2 to the end of str1 */
     i=i+1;
   }

   str1[i]=’\0';       /* Placing a ‘\0’ at the end of the concatenated
                          string str1 */

   printf(“The concatenated string is:%s”,str1);

}
```

7.6 CONVERTING THE UPPERCASE ALPHABETS INTO LOWERCASE ALPHABETS IN A GIVEN STRING AND VICE – VERSA

As discussed in the introduction of this chapter, in a computer all the characters, including alphabets, are stored in some form of *integer codes*. The one that is widely used is the ASCII code. In this code the *uppercase alphabets A* to *Z* are represented using the codes ranging from 65 to 90 and the *lower case alphabets a* to *z* are represented using codes ranging from 97 to 122. A careful observation of these codes reveals the fact that *uppercase alphabets are converted into their lowercase equivalents by* ***adding 32***. Obviously, *lowercase alphabets are converted into their uppercase equivalents by* ***subtracting 32***.

The Method

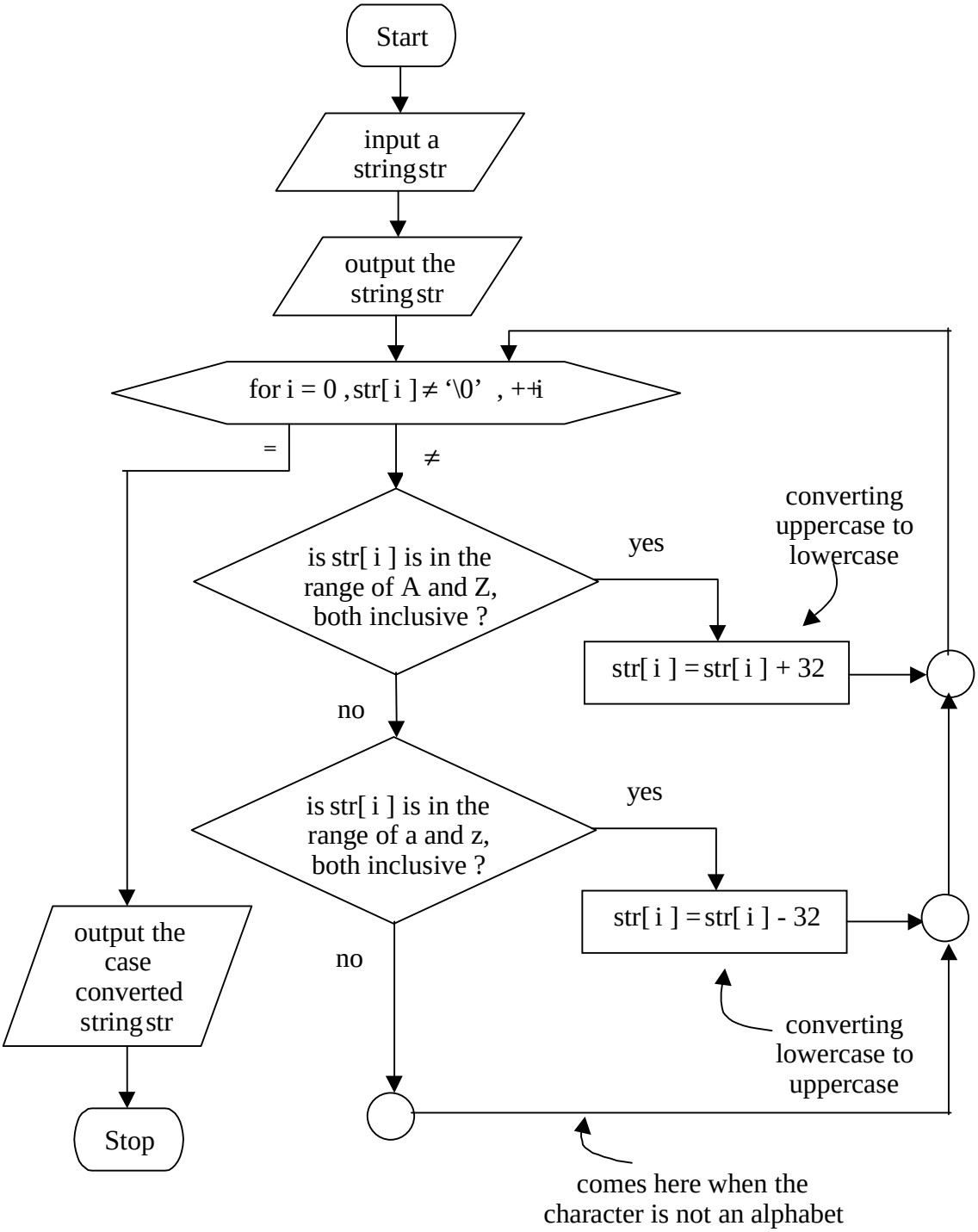
The given string is first read in. Then every character on this string is accessed, starting from the beginning till the end, i.e. till the null character (`\0`) that appears at the end. This accessed character is tested to find out whether it is an *uppercase alphabet* or a *lowercase alphabet*.

A statement of the form "is the character *greater than or equal to* ***'A' and*** *less than or equal* to ***'Z'*** ?" is used to *test* for an uppercase alphabet. Similarly a statement of the form "is the character *greater than or equal to* ***'a'*** **or** *less than or equal* to ***'z'*** ?" is used to *test* for a lowercase alphabet.

Once the *character* under consideration does not belong to any one of the above two groups of characters, *it will not be an alphabet*. Under such circumstances nothing is done with the current character and the *next character* on the string is *accessed* and *tested*. If, at any point of time, the *current character* belongs to the upper case alphabets range, then it is converted in to its lower case equivalent by adding a 32 to it. However, if the *current character* belongs to the lowercase alphabets range, then it is converted to its uppercase equivalent by subtracting a 32 from it.

The above process of *accessing characters* of the string one after the other, '*testing for its range and either adding or subtracting a 32* to/from the current character' is *repeated* till the null character (`\0`) is reached. This process is shown pictorially in the form of a flowchart as follows.

Flowchart


Start
input a string str
output the string str
for i = 0 , str[i] ≠ '\0' , ++i
=
≠
is str[i] is in the range of A and Z, both inclusive ?
yes
converting uppercase to lowercase
str[i] = str[i] + 32
no
is str[i] is in the range of a and z, both inclusive ?
yes
str[i] = str[i] - 32
converting lowercase to uppercase
no
output the case converted string str
Stop
comes here when the character is not an alphabet

The Program

```
/* Converting the uppercase alphabets into lowercase alphabets in a
given string and vice - versa. */

#include<conio.h>                /* Including  header file */
#include<stdio.h>

main()
{
   char str[‘ ‘];
   int i;
   clrscr();

   printf(“\n\nEnter a string\n”);
   gets(str);                    /* Reading a string */

   printf(“\nGiven string is : %s\n”, str); /* Printing a string */


   for(i=0; str[i]!=’\0'; ++i)    /* Repeat till the end of string */

   {
       /* Converting upper to lower */

       if( str[i]>=’A’ && str[i]<=’Z’)
            str[i] = str[i] + 32;

       /* Converting lower to upper */

       else if ( str[i]>=’a’ && str[i]<=’z’)
            str[i] = str[i] - 32;

   }

   printf(“\nCase converted string is : %s\n”, str);

}
```

7.7 COUNTING THE NUMBER OF VOWELS AND CONSONANTS IN A GIVEN LINE

The Method

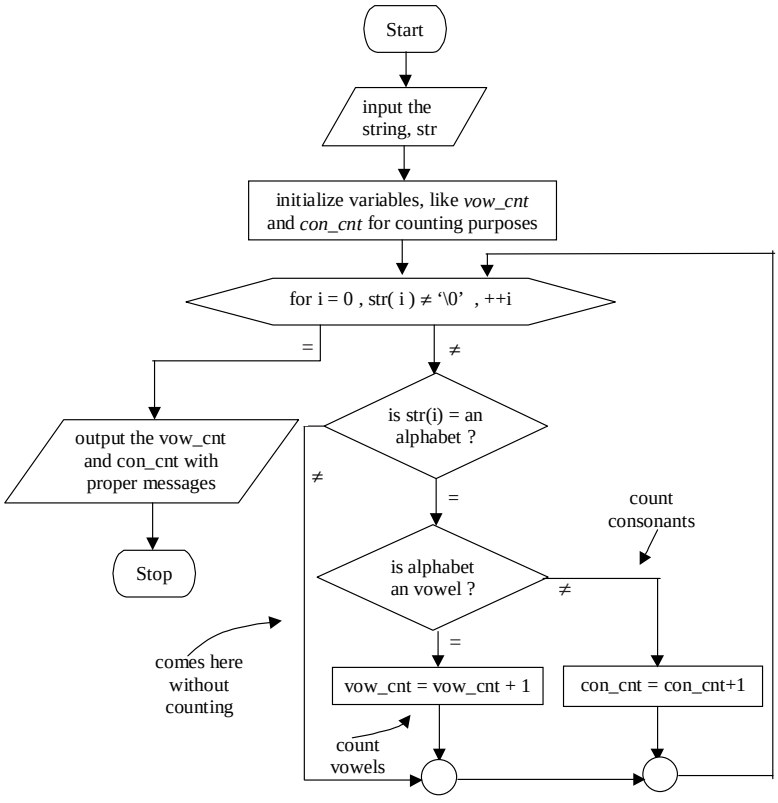
First the given line is input with an *appropriate input function*. It is recommended to use the ***gets( )*** function and to store the given line in the form of a string. We know that the string will be stored in the form of a character array with the null character (`\0`) as its termination character. In order to count the *required type of character*, every character of the line, i.e. string, is *accessed* starting from the beginning till the end of the string is reached. In this example, every accessed *character* is *verified* to find out whether it is an *alphabet* (lowercase or uppercase) or not. *Whenever it is an alphabet* it is *tested* to find out whether it is an *vowel* (*a, e, i, o, u, or A, E, I, O, U*) or not. If it is a *vowel,* the *vowel count* (`vow_cnt`) is incremented. Otherwise, the alphabet has to be a *consonant* and hence the *consonant count* (`con_cnt`) is incremented. Obviously, whenever the *character under current consideration* is *not an alphabet*, the next character on the string is accessed *without counting*. The entire process is *repeated* till the end of the string is reached, i.e. the null character '`\0`' is reached.

C-Tips

A function like ***gets()*** is used to input the line of text as *blanks* will be present between consecutive words. A statement like " is the *current character* (`str[i]`) is *within the range* of *a* to *z* **or** *A* to *Z* ?" is used to find out whether the *current character* is an alphabet or not" A *relational* expression like: ***str(i) ≥ 'a' and str(i) ≤ 'z'*** is used to find out whether a character at the position indicated by the subscript ***i*** on the string ***str*** is a *lower-case alphabet* or not.

It may be observed that in this example a ***switch*** statement is used *repeatedly* under the control of a ***for*** loop control structure.

Flowchart



The Program

```
/* Counting the number of vowels and consonants in a given line */

#include<stdio.h>
#include<conio.h>

main()
{
   char str[‘ ‘];
   int i,vow_cnt=0, con_cnt=0;     /* Initializing vowel and consonant
                                      counters to zero */

   clrscr();
   printf(“\nEnter the line: \n”);
   gets(str);                      /*Reading a string*/

   printf(“Given line is: \n%s\n”, str); /* Printing a string */

   for(i=0; str[i]!=’\0'; ++i) /* Repeat till the end of the string */
   {
                                   /* Checking only letters */
     if((str[i]>=’a’ && str[i]<=’z’) || (str[i]>=’A’ && str[i] <=’Z’))
     {

       switch ( str[i] )
       {

         case ‘a’:
         case ‘e’:
         case ‘i’:
         case ‘o’:
         case ‘u’:
         case ‘A’:
         case ‘E’:
         case ‘I’:
         case ‘O’:
```

```
        case 'U': vow_cnt++; /* If character is vowel increment
                             vowel counter */
                  break;

        default:  con_cnt++; /* Otherwise increment consonant
                             counter */

      }

    }

  }
  printf("Number of vowels  = %d\n", vow_cnt);
  printf("Number of consonants = %d\n", con_cnt);

}
```

7.8 SORTING A GIVEN SET OF NAMES

The Method

Names may be made up of just a single word (like *Vasanth*) or more than one word (like *George Bush*). In this example, *names made up of more than one word* have been considered. The *technique* employed here is to *input all the names* and store them in an *array of* some suitable name (say, **name**). It may be remembered that all strings (i.e. **names**) are stored with ASCII codes, which are nothing but *integers*. For *sorting names* they are mutually *compared*, to find out which one is *smaller* or *bigger*. Actually this comparison takes place on ASCII values of the names. While sorting names in alphabetical order we have to see that names with lower ASCII values appear first. Any commonly available sorting method may be used. Here, the *Bubble sort* method (discussed in Section 6.8) has been used.

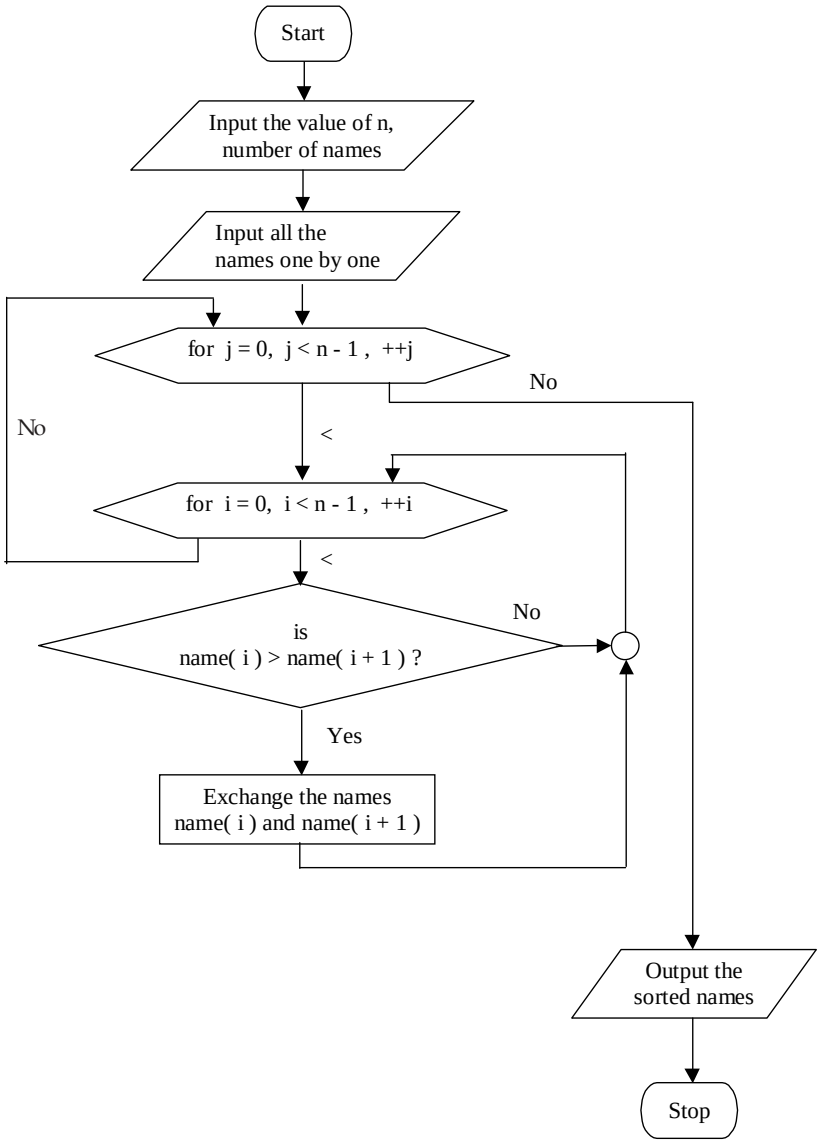
C-Tips

Since we have assumed *names of more than one word* the ***gets()*** function is used for reading in the names. If names of only single word are involved we may use the **scanf()** function. For *comparing strings,* i.e. *names*, the *string-compare function* **strcmp()** is used. In this program, it is necessary to *exchange the positions of two names*. In the case of sorting numeric values, *exchanging the values* of two variables was managed using *assignment* statements. The use of assignment statements to exchange values of strings is not possible. Here, *string-copy function* **strcpy()** has to be used to *exchange* two names.

String handling functions like `strcmp()`, `strcpy()` are available in the *header file* `<string.h>`. Therefore, this file has to be *included* in the beginning. For printing out names the function `puts()` is used.

The string compare function `strcmp()` takes two strings as its arguments and *returns a numeric value* as a *result of the comparison*. If the two strings are *equal*, a zero (0) is returned. If the first string is *alphabetically above* the second string then a *negative value* is returned. If the first string is *alphabetically below* the second string, a +ve value is returned.

Flowchart



The Program

```
/* Program to read n names and to sort them in an alphabetical order
*/

#include<stdio.h>
#include<conio.h>
#include<string.h>

main()
{
   char name[‘ ‘][‘ ‘],temp[‘ ‘];
   int i, j, n;
   clrscr();

   printf(“How many names \n”);
   scanf(“%d”, &n);

   printf(“Enter %d names\n”, n);
   getchar();  /*  To absorb the new line character  */

   for(i=0; i<n; ++i)
       gets(name[i]);

   printf(“Given names are:\n”);
   for(i=0; i<n; ++i)
       puts(name[i]);

   for(j=0; j<n-1; ++j) /* Performing n-1 passes */

     for(i=0; i<n-1; ++i) /* Performing n-1 comparisons in each pass */

       if(strcmp(name[i], name[i+1]) > 0) /* Comparing adjacent names */
       {
           /* Exchanging names */
           strcpy(temp ,name[i]);
```

```
            strcpy(name[i], name[i+1]) ;
            strcpy(name[i+1], temp);

        }

    printf("\nGiven Names in sorted order:\n");
    printf("————————————————\n");
    for(i=0; i<n; ++i)
        printf("%s\n", name[i]);


}
```

7.9 COUNTING THE NUMBER OF CHARACTERS IN AN INPUT

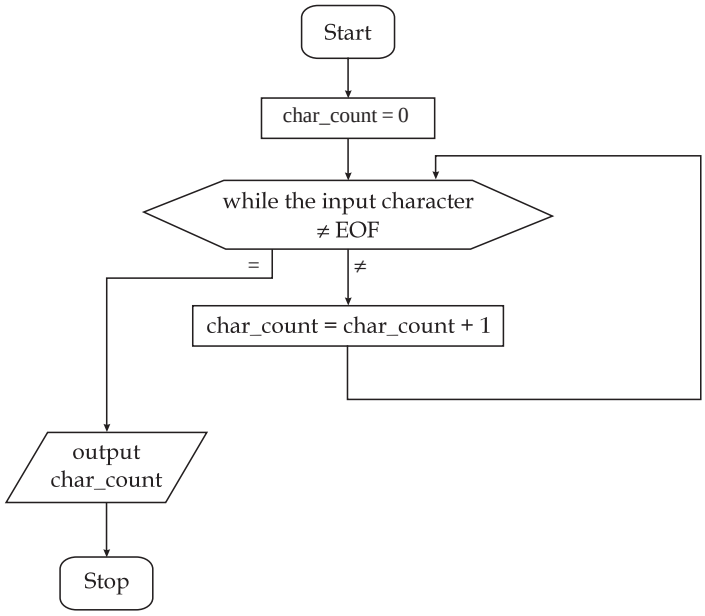
The Method

Characters are input via the *standard input,* i.e. the *keyboard*, one by one. As the characters are input they are counted. The counting is stopped whenever the input character is a *end of file* [EOF] character. Conventionally –1 is taken as the EOF character. However, this is implementation dependent.

C-Tips

Whenever text form of inputs are given via keyboard, *ctrl-Z* is used as the end of file character. As there could be large number of characters, it is recommended to use a *long* type variable to store the characters count. The format specifier **%ld** is used to handle *long* data type variable used to count the total number of characters.

Flowchart



The Program

```
/* To count the number of characters in an input */

#include<stdio.h>
#include<conio.h>

main()
{
   long char_count=0;      /* Notice the use of long data type */
   clrscr();

   printf("Enter text: Press Ctrl-Z to terminate \n");

   /* Keying-in a character and testing for EOF */

   while(getchar()!= EOF)
   {
     char_count = char_count + 1;  /* Counting */

   }

   printf("\nNumber of characters = %ld", char_count);
}
```

Note: In this example, the characters are counted as and when they are input through the keyboard. The input character stream is not stored anywhere. If required, it may be stored in a string or a separate file. Also it may be noted that the EOF character is not counted.

7.10 COUNTING THE NUMBER OF WORDS IN A GIVEN LINE

A line will be made up of many number of words and *ends* with a *newline* character (\n). Two successive words will be separated by one or more blank characters. *Normally* only *one blank* character appears between two consecutive words. Therefore the following discussion *assumes only one blank* between two consecutive words.

When a line is the *very first line* of a given text, the general practice is to *indent it* by using one or more number of white space characters. These are called *leading* white spaces. The procedure discussed here takes care-of these leading white spaces. In other words, it neglects the leading white spaces, if any, *and starts actual word counting* only after a non-white space character is input.

The Method

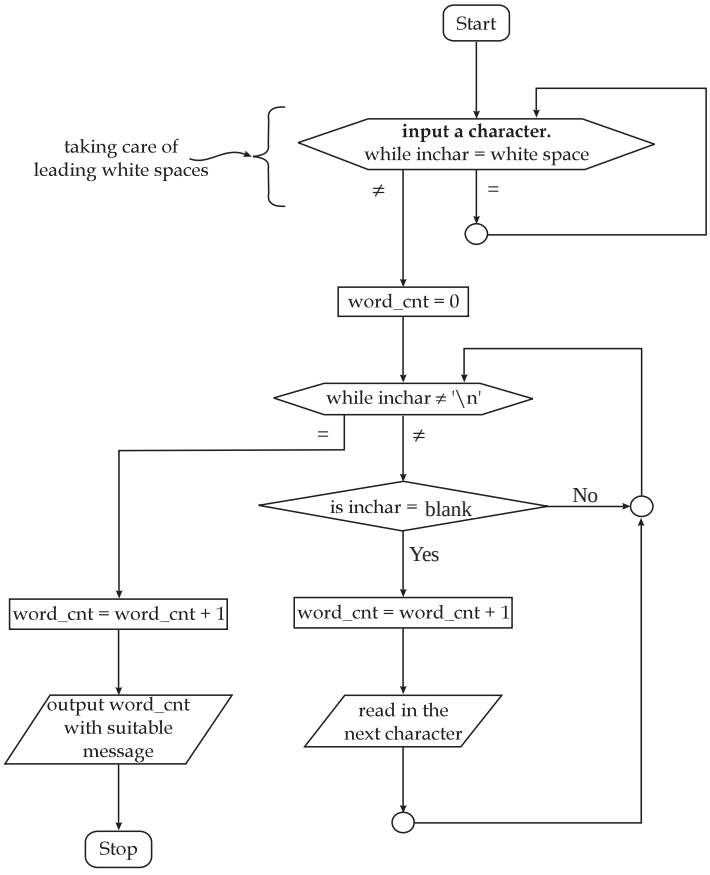
Words can be counted by counting them as and when the characters of the input are *keyed* in from the standard input i.e. the *keyboard*. Also we can input an *entire line into a string* and count the words by going through that string, character by character, till the null character (`\0`) is reached. The following flowchart is based on the *former type* of input, i.e. the *input keyed in from a keyboard*. If any white space characters are input in the beginning, the *idea is to recognize it* and *loop back* to *accept the next input character*, neglecting the present one. The control *flows forward,* i.e. the counting starts, *only after* the input character is a non-white space character. Here afterwards, the *word count is incremented* as and when the input character is a *blank character*. The counting process *stops* after *a newline character (`\n`)* is input. It may be noted that the *line terminating character (`\n`)* not only signals the *end of a line* but also *signals the end of last word*. Therefore, care should be taken to count the last word also. The method discussed is shown diagrammatically in the following flowchart.

C-Tips

White space characters mean the blank (`' '`), the tab (`'\t'`) and the newline character (`'\n'`). As one can easily guess these white space characters are generally used to *improve the readability* of any text. Thus, we have been using these characters in the *control strings* of statements like *`printf( )`* statements all along.

Sequences of characters like **`'\n'`** and **`'\t'`** which have *special meanings* attached to them are called *escape sequences*. In C there are many more such escape sequences. For example, `'\b'` is used for backspacing, `'\a'` is used for ringing a bell, and so on. The input characters are *read-in* using the ***`getchar( )`*** function and stored in some convenient variable like *`inchar`*.

Flowchart



The Program

```
/* Counting the number of words in a line */

#include<stdio.h>
#include<conio.h>
main()
{
   char inchar;
   int word_cnt=0;
   clrscr();

   printf("Enter a line of text :\n");

   /* Following while takes care of leading white spaces */

   while((inchar=getchar())==' ' || inchar=='\t' || inchar=='\n');


   while(inchar != '\n')       /* Actual word counting begins here */
   {
       if(inchar == ' ')
       word_cnt++;

       inchar=getchar();     /* Accepts the remaining characters
*/

   }

   word_cnt++;

   printf("\nThe number of words in the given line is: %d",word_cnt);

}
```

7.11 COUNTING THE NUMBER OF LINES IN A GIVEN TEXT

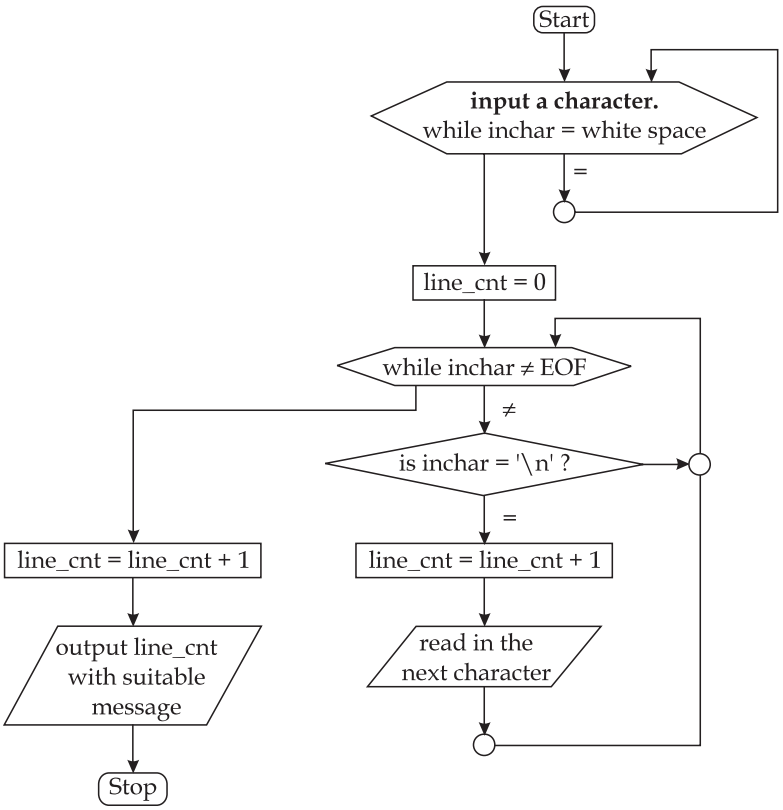
It is known that a text consists of many numbers of lines and every line is terminated by a *newline character* ('\n'). This fact is used to count the number of lines.

The Method

The given text is *keyed* in via the standard input i.e. the *keyboard*. If *white space* characters are input as *leading* characters of the text they are just overlooked. As in the previous example, the actual counting begins from the appearance of the first *non-white space* character. A variable like `line_cnt` is used to count the number of lines and to store the count. As already mentioned the arrival of a *newline* character ('\n') *signals* the *end of a line*. The text input operation is terminated by using the *ctrl-z,* the end-of-file character.

After typing in the last line of the text, the *normal practice* is to close the file i.e. the input operation. In other words, a new line character is not used after keying in the last line. By keeping in mind this *normal practice* the `line_cnt` must be incremented after signaling the last line input i.e. the entry of the entire given text. Keying in the text is terminated by using the `ctrl-z` (EOF) keys.

Flowchart



The Program

```
/* Counting the number of lines in a given text */

#include<stdio.h>
#include<conio.h>
main()
{
   char inchar;
   int line_cnt=0;
   clrscr();

   printf("Enter a passage of text :\n");

   /* To take care of leading white spaces */

   while((inchar=getchar())==' ' || inchar=='\t' || inchar=='\n');


   while(inchar != EOF)   /* Actual line counting begins here */
   {
     if(inchar == '\n')
     line_cnt++;

     inchar=getchar();   /* Accepts the remaining characters */

   }

   line_cnt++;
   printf("\nThe number of lines in the given passage is: %d",
   line_cnt);


}
```

EXERCISES

1. Develop a flowchart to read in a line and count the number of words beginning with a specific letter, say 'E' or 'e'. Code the same using C.
2. Develop a flowchart to find whether a required substring exists in a given line of text or not. Code the same using C.
3. Develop a flowchart to count the number of characters belonging to the alphabets group, digits group and special characters group. Report the total number of characters also. Code the same using C.
4. Develop a flowchart to count the number of words in a given line of text assuming that there could be more than one blank or a tab in between the consecutive words. Code the same using C.
5. Develop a flowchart to count the number of characters, number of words as well as number of lines in a given text. The procedure should handle leading white space characters, if any, properly. Code the same using C.

CHAPTER 8

Solving with Modules

In order to solve a problem, with or without using a computer, one has to know the procedure or *the method* of solving the problem. Once *the method* is known, to solve the problem using a computer, it has to be *coded* using some computer language. These codes which have *the method* of solving a problem *embedded* within them are called *computer programs*. During the early days of programming, programmers used to write a *single program* to solve a problem completely, regardless of the fact how much big they are. This style of writing complete programs having a continuous sequence of computer codes is generally known as *procedural programming*. As the days passed by programmers started facing problems like error fixing, eliminating errors, logic checking etc., particularly with the larger programs. Here, a larger program means a program having *too many code lines*. As a result those programmers used to *spend lot of time* in *identifying the errors* and *correcting* them. The process of finding out the errors and correcting them is known as *debugging*. In order to circumvent such time consuming debugging activities, they naturally thought of *dividing* larger programs into smaller and independent segments called *subprograms* and to solve the given problem part by part. Developing subprograms was possible because a number of problems *naturally permit* their division into smaller and independent ones. Almost all the present computer languages provide facilities using which these subproblems can be coded, tested and used.

For example, consider the problem of finding *mean, variance* and *standard deviation* of a given set of numeric elements (discussed in Section 6.6). Though this is not a very big problem, it is considered here as the idea of subproblems and hence the concept of subprograms can be grasped very easily. A careful study of this problem indicates that it may be divided into *three* smaller problems, viz., *finding the mean*, *finding the variance* and *computing the standard deviation*. In order to solve the complete problem, independent small programs *to find mean, to find variance and to compute standard deviation* are first written, test

run and finally put together in the *main program*. This *strategy of dividing a problem* into smaller ones and getting a solution to the entire problem by tailoring them together is called the *Divide and Conquer strategy*. This strategy has been already discussed very briefly in Chapter 1.

These independent programs of smaller problems which are a part of some bigger problem are called *subprograms* in FORTRAN, *procedures* in Pascal and *functions* in C.

Though many claim that the real power of C lies with *pointers* and its manipulation strategies, facilities available in C for the development of *functions* has also contributed significantly to its power and popularity. Developing independent functions provide certain additional *advantages*. *Functions can be developed, tested and debugged independently as well as, if necessary, they can be used by others.*

Functions are developed not only for subproblems of some larger problem, but are also written for *smaller problems* that are *very frequently used*.

The style of developing complete programs with the aid of smaller *functions* is generally known as *functional programming.* It may be noted that C is not a functional language.

One or more *interrelated independent functions*, after testing, can be stored in a *separate file*. Such files are called *modules*. These files are used, by *including* them in a proper directory, as and when they are required. This technique of developing individual *modules* and using them as and when required is known as *modular programming.* Modular programming is one of the very popular current software development practices.

This chapter discusses some examples, which involve the development of *one or more functions* and using them in a *main program*. Many of these examples have been already discussed in earlier chapters and have been reconsidered here for the sake of better understanding. The important aspect one has to really understand in this chapter is to "*know* when an *user defined function* is *called* from another function, *how* the program execution control *goes to* the *called function* and *comes back* to the *calling function.*" These *control flow paths* have been *visually* shown using *thin dotted lines* with arrowheads on flowcharts. These *arrowheads* indicate the *direction* of control flow. Also, in this chapter, with some examples, certain additional information about C beyond the scope of C-Tips have been presented in a section called *… beyond.*

8.1 FUNCTION TO PRINT A LINE MADE UP OF ONLY HASH (#) CHARACTERS

As one could see this is not a very big problem. A simple program for this problem could be written straight away. Yet, development of a function for this is recommended as one might need to print *hash lines* quite often. In other words, *functions are* not only *written for independent parts of a larger problem* but are also *written based on its* frequency of usage, i.e. the *utility*.

The Method

Normally a line on a display will be made up of 80 characters. In this example 80 hash (#) characters are printed in a line using a `printf( )` function. Note that `printf( )` itself is a C library function, which is put into use by *including* the `<stdio.h>` file in the beginning. Though, developing flowcharts for such a simple example is not a necessity, they have been developed for the sake of continuity. These flowcharts will really *help in understanding* how *the* ***link*** between a *function* and the *one which uses that function gets developed*.

C-Tips

Though, theoretically, *functions can* physically *appear* either before or after the `main( )` function, the *normal practice* is to place the functions *after* the `main( )`. Note that the word `main` is a C keyword and is used to name the `main( )` *program*. It is very important to note that:

1. Every program should have a `main( )` function.
2. The execution of a C program *begins* from the `main( )` function and *ends* with the `main( )` function.

During the execution of the `main( )`, when the *control* comes across a *function name*, the corresponding function is *invoked* or *called*. Automatically, the program execution control goes to the *called function*. This shifting of the control takes place *via* the *function prototype* along with the *arguments* of the invoking or *calling function.*

Once the control *shifts* to the *called function*, the *called function* is executed as usual. After the execution of the *called function*, the *control goes back* to the *calling function*. Infact, the control *goes back* to the point on the *called function* from where the control had come to the *calling function*. The *completion of the execution* of a function is *signaled* by using one or more `return` statements in it.

A `return` statement can be used with an expression or without an expression associated with it as follows.

1) `return;` **2)** `return expression;`

or

`return (expression);`

When a `return` statement without an expression with it is used, the *control simply goes back* to the *calling function* immediately after its execution. However, when a `return` statement with an expression is used, the *associated expression* will be first *evaluated* and then the *control goes back* to the *calling function* with the *value of the expression*. Though a *return* statement need not be present in a function, it is recommended to use one.

… beyond

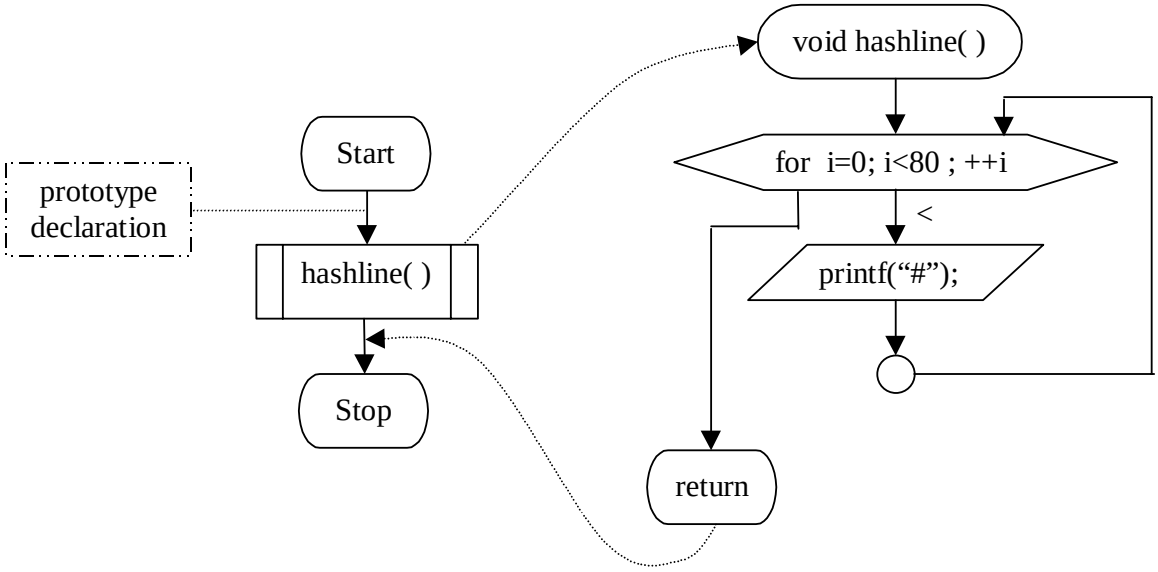
As seen from the flowchart and the program that follows, there are *no arguments* associated with the *function definition.* As a result *no arguments* are used while *calling* the

function from within the `main( )` function. In other words, *no parameters* are *passed* from the *calling function* to the *called function*.

In this example, when the function is called, the control goes to the function, prints the required number of hash characters in a hashed line and *returns back without any values*. In other words, the *function does not return any value*. When no values are returned the function must be declared as of the data type *void*.

Every *user defined function* will have a *name*, *optional list of arguments* along with their *data type*, and the *type of the value returned* by the function. All these will appear in the *first line* of a *function definition*. This first line is called the *function header*. In this example *void* `hashline( )` is the *function header*. The *pattern* of this first line is known as the *function prototype*. In order to use a *user defined function*, its *prototype must be included* in the beginning of the function that intends to use it. A careful observation of the program that follows gives a clear idea about the *function prototype*, the position where it has to be included and the *function body.*

Flowchart



The Program

```
/* Printing a line made up of #(hash) characters using function */

#include<stdio.h>
#include<conio.h>
```

```
main()
{
   void hashline();       /* Function prototype */
   clrscr();

  hashline();             /* Invoking the function hashline() */
}



/* Function to print a line made up of #(hash) characters */

void hashline()           /* Function definition made here */
{                         /* Function body starts here */

   int i;                 /* Declaring local variable i */
   for(i=0; i<80; ++i)
     printf("#");

   return;                /* Returning control back to main() */
}                         /* Function body ends here */
```

8.2 FUNCTION TO MULTIPLY TWO GIVEN INTEGER NUMBERS

The Method

A function, `multiply( )`, for the multiplication of any two given integers, say *x* and *y*, is first developed. While developing functions it is *assumed* that the *input values are readily available*. In other words, generally functions *do not deal* with input statements. Thus, in this example, values of *x* and *y* are *assumed* to be available and hence the *product* is obtained using the relation *product* = *x* * *y*. Finally, this computed *product* is *returned* using a `return` statement. As this *returned value* is an integer value, the *return type* of the function should be declared as `int`.

Then the main program is written. The *prototype of the function* is *included* along with the declaration of other variables. The numbers to be multiplied are input. Let these numbers be `num1` and `num2`. The required *product* is obtained by *invoking* the function `multiply( )` in a statement of the form `product = multiply(num1,num2)`. Invoking a required function is also referred to as *calling the function*. Finally the *returned value*, i.e. the *product*, is output.

C-Tips

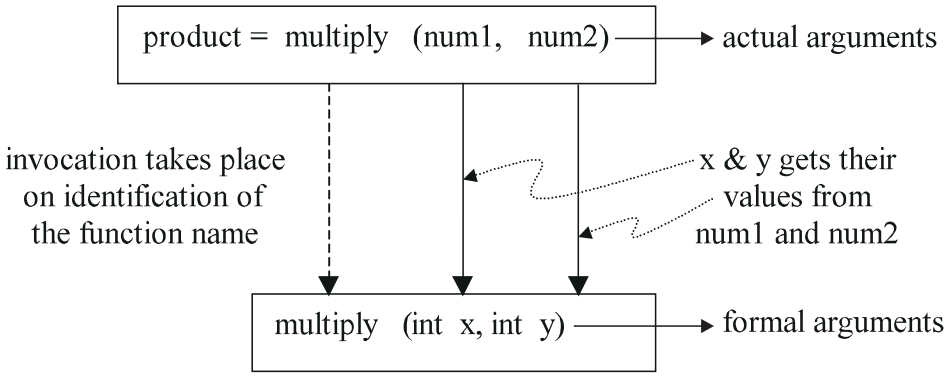
The execution of the program *begins* from the `main( )` function. When the control comes across the *function name* `multiply(num1,num2)`, the *control shifts to the function* `multiply(num1,num2)`. Along with the control the *values of* `num1` *and* `num2` *are also passed* on to the function. Infact, the values of *num1* and *num2* will be *copied* on to the arguments of the *called function*. In other words, the values of `num1` and `num2` will be *copied* into the variables ***x*** and ***y*** respectively. With these values for ***x*** and ***y***, the function `multiply( )` gets executed and the value of the variable *product* will be *returned* to the *calling function,* i.e. the `main( )` function. The execution of the `main( )` function resumes from the point from where it had gone to the *called function.*

… beyond

Within a function any required *function* is *used* by writing its *name* at the point required *along with the arguments* or parameters. Arguments or parameters that are used along with a function name in the calling program are called *actual arguments* or *actual parameters.* The *arguments* that are *assumed to be available* during a *function definition* are called *formal arguments* or *dummy arguments*: The *names* of the variables used as *actual arguments* and as *formal arguments* may be *same* or *different*.

In this example, *different variable names* have been used as *actual arguments* and *formal arguments,* deliberately. It must be noted that the *number*, *type* and *order* of the *actual arguments* and *formal arguments* must match. *Other variable names* used in the `main( )` function and in the *user-defined function* also could be either same or different. Ordinarily, *variable names* and the *values* associated with them are *valid* only within those functions in which they are *declared*. Such variables that are *valid only in the function* in which they are *declared* are called **local variables**. In this example, variables `num1`, `num2` and *product* are *local* to the `main( )` function and the variables *x, y* and *product* are *local* to the *user defined function* `multiply( )`. It may be observed that *product* is a *local variable* both in `main( )` as well as in `multiply( )` functions. *This is permitted*. Required memory locations for the variables `num1`, `num2` and *product* are allocated under the `main( )` function. Similarly, required memory locations for the variables *x, y* and *product* are allocated under the `multiply( )` function. Thus memory location allocated for the variable *product* in the `main( )` will be *totally different* from the memory location allocated for the variable *product* in the `multiply( )`. In C *local variables* are also called `auto` variables. By default, all variables are *local variables.*

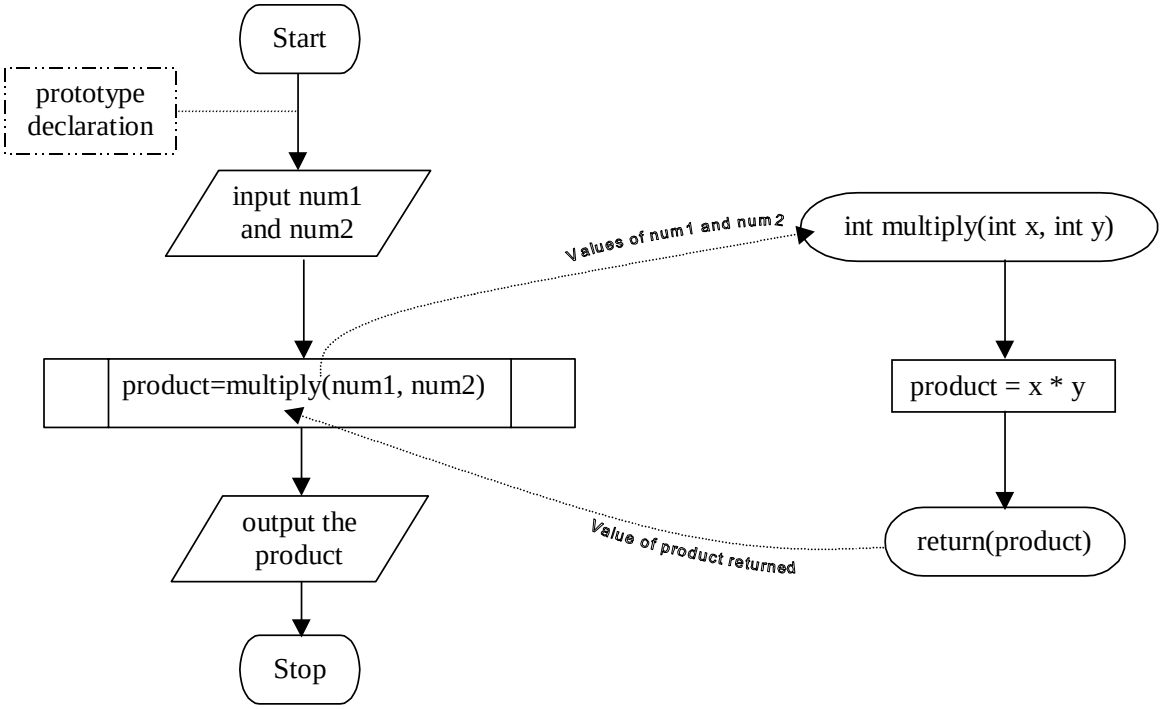
The method of *invoking* or *calling* a function by using the *function name along with real or actual arguments* is known as ***call by name***. When a function is invoked or called in this manner, *formal arguments* in the *function header* will get the values of the *actual arguments*. Infact, the values of *actual arguments* will be copied into the *dummy* or *formal* variable names as shown in the following figure.



This technique of *calling a function* by *passing* on the values to the *dummy* or formal arguments of the function via *actual arguments* is also known as ***call by value.*** It should be carefully noted that all the variables used here are single valued, i.e. *scalars.*

The `multiply( )` function defined in this section is a *function with two arguments* and one *`return value`*.

Flowchart



The Program

```
/* Finding product of two numbers using function */

#include<stdio.h>
#include<conio.h>
main()
{
   int num1, num2, product;    /* Declaring the local variables */

   int multiply(int num1, int num2); /* Function prototype */
                                     clrscr();

   printf("Input two integer numbers\n");
   scanf("%d%d", &num1, &num2);   /* Reading two numbers */

   /* Printing two numbers */
   printf("Given two numbers are : %d and %d.\n", num1, num2);

   /* Calling the function multiply() */
   product = multiply(num1, num2);

   printf("\nThe product is : %d\n", product);

}


/* Function to multiply two given numbers */

int multiply(int x, int y)         /* Function header */
{                     /* The body of the function begins here...*/

   int product;                    /* Declaring local variable */

   product = x * y;
```

```
   return(product);       /* Returning product to main() */

}               /* The body of the function ends here...*/
```

8.3 FUNCTION TO COMPUTE THE GCD OF GIVEN THREE NUMBERS

The Method

Finding the ***GCD*** of ***two*** given ***integers*** has been already discussed in Section 4.6. A function named **`gcd(int num1,int num2)`** is written first using ***the method*** discussed there. The ***flowchart*** and the ***program*** for this are given in the following page. It may be noted that the necessary input to the **`gcd`** ***function*** is ***passed on*** from the **`main( )`** function during the ***function call.*** The computed result is ***returned back*** using the **`return`** statement.

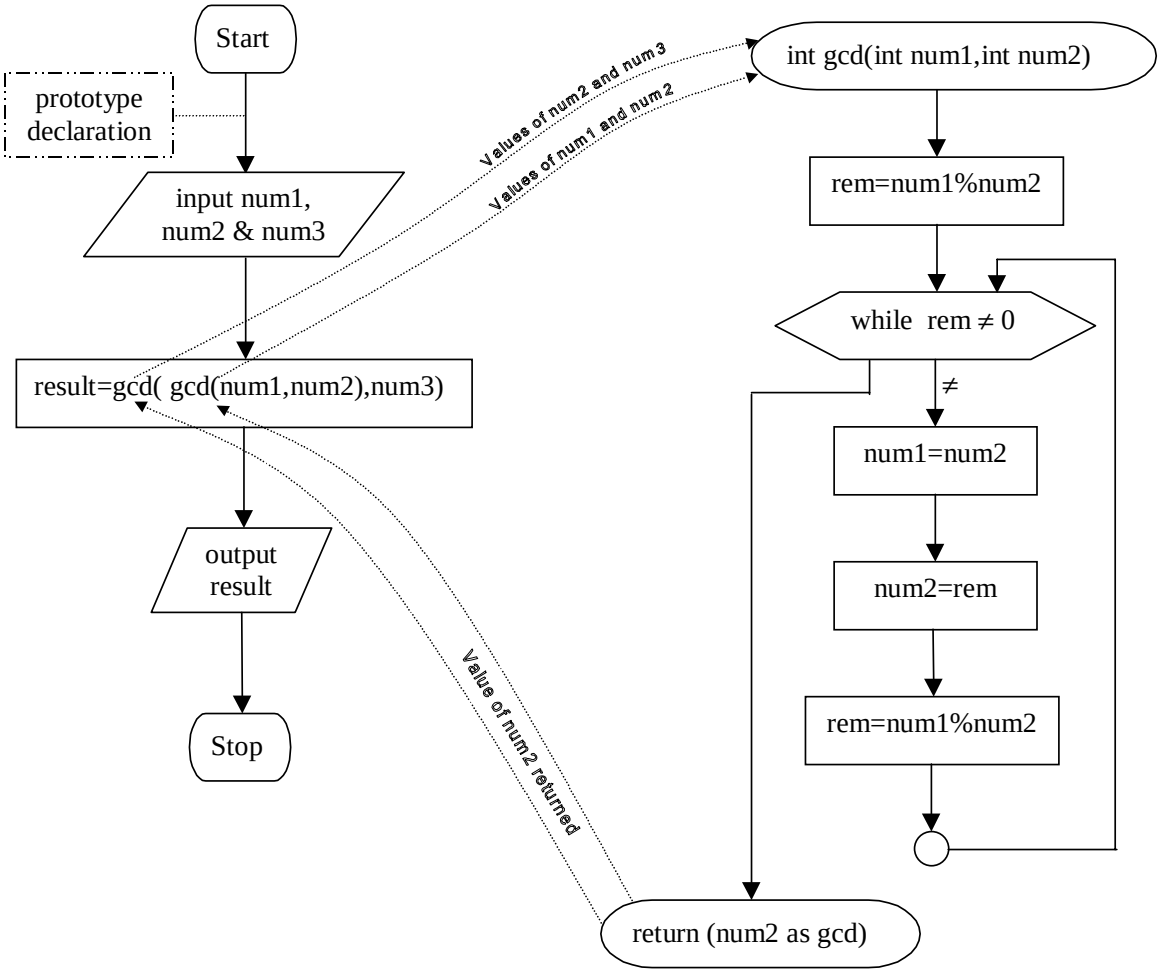
All the three given integers, say `num1`, `num2` and num3, are input in the `main( )` function. The user defined function `gcd( )` is called twice: once with `num1` and `num2` and next with the `gcd` *of the first two integers* and `num3`. A statement of the form `result=gcd( gcd(num1,num2),num3)` is used for this purpose. After the execution of these statements, the *gcd* of the given three numbers will be available in the variable *result.*

C-Tips

As already mentioned the `gcd( )` function is *invoked* or *called* twice. It may be noted that a function may be *called* or used any number of times. As it could be observed, here the *function call* is nested. First the `gcd( )` is invoked with `num1` and `num2` as arguments. It may be observed that *this call itself is an argument* of the outer `gcd( )`. After getting the gcd of `num1` and `num2`, the function `gcd( )` is once again called or invoked, now with the `gcd( )` of `num1` and `num2` as first argument and *num3* as the second argument.

It may be observed that `num1` and `num2` have been used both as *actual arguments* as well as *formal arguments*. Whenever the `gcd( )` function is invoked, values associated with the *actual arguments* `num1` and `num2` will be *copied* onto the *formal argument* variables `num1` and `num2`. The `gcd( )` function is a function with *two arguments* and *one return value*.

Flowchart



The Program

```
/* Finding gcd of three non zero integer numbers using function */

#include<stdio.h>
#include<conio.h>
main()
{

   int num1, num2, num3,result;    /* Declaring the variables */
```

```
   int gcd(int num1, int num2);    /* Function prototype */

   clrscr();

   printf("Input three non zero integers.\n");

   scanf("%d%d%d", &num1, &num2, &num3); /*Reading three integer
   numbers*/

   printf("Given three integer numbers are:\n\n");

   printf("num1 = %d\t num2 = %d\t num3 = %d\n\n", num1, num2, num3);

   result = gcd(gcd(num1,num2), num3);

   printf("GCD of %d, %d and %d is %d\n", num1, num2, num3,result);

 }
```

```
/* Function to find the gcd of two integers */

int gcd(int num1, int num2)    /* Function header */
{                          /* Function body starts here */

   int rem;               /* Declaring local variable */

   rem = num1 % num2;     /* Computing remainder */

   while (rem != 0)       /* Repeat while remainder is not zero */
   {
     num1 = num2;
     num2 = rem;
```

```
    rem = num1 % num2;
  }
  return(num2);          /* Return gcd to main()*/

}                        /* Function body ends here */
```

8.4 FUNCTION TO SUM FIRST *N* TERMS OF SINE SERIES EXPANSION AND TO TABULATE SINE VALUES FOR DIFFERENT ANGLES

The method to generate the different terms of the *sine* series and hence to find the sum of the generated terms has been already discussed in Section 4.11.

The Method

Assuming the values for the variable x of sin(x) and the value for 'n', the number of terms to be summed, the function `sinevalue( float x , int n)` is developed. This function returns a `float` *type* answer. The `sinevalue( )` function is *repeatedly* used for various values of `deg` in the `main( )` function in order to tabulate the sine values for different angles.

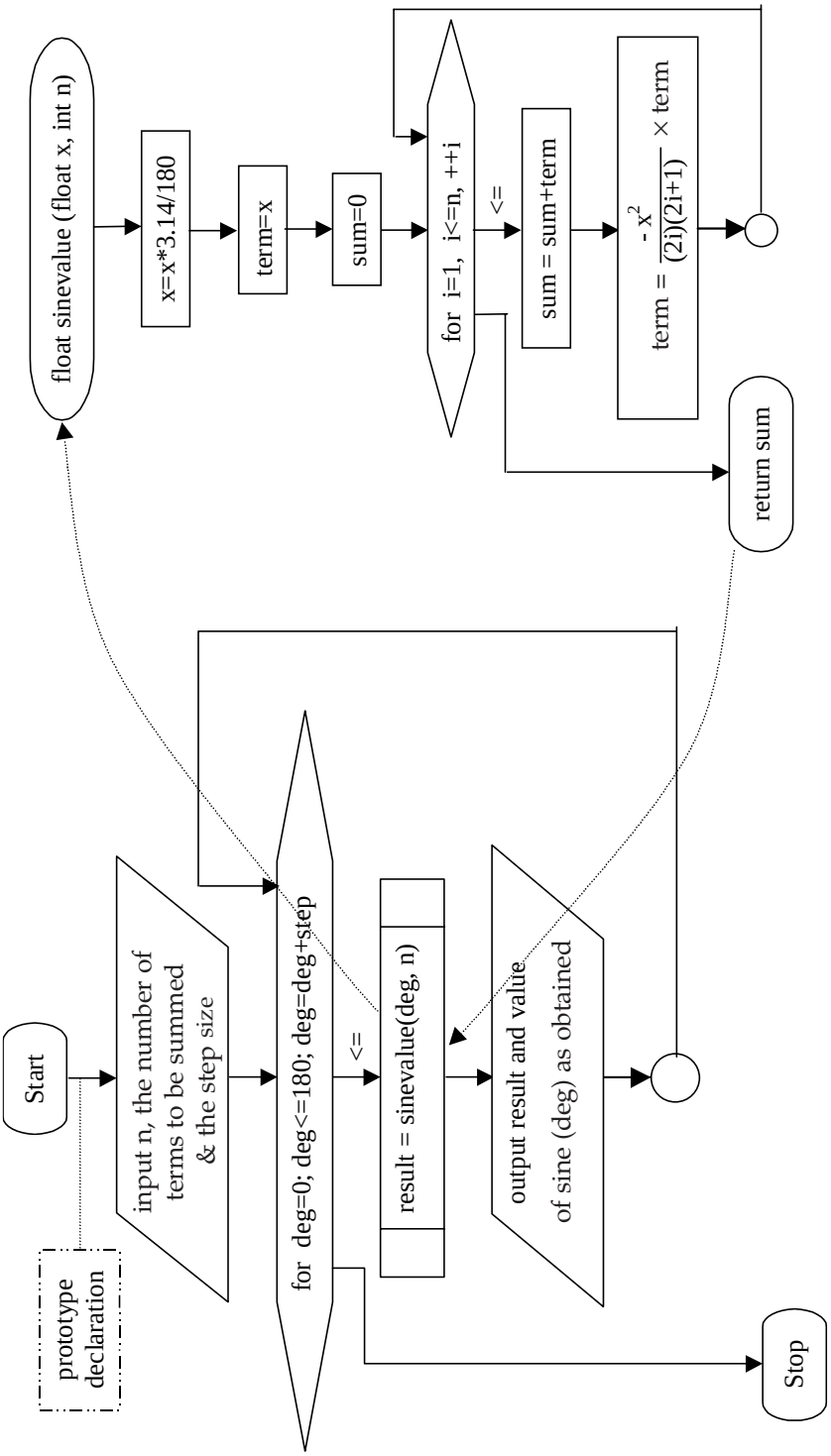
In the `main( )`, the value of n, the number of terms to be summed, and the value of a variable `step` are input. Different values of the angle (i.e. variable `deg`) beginning from 0 up to an upper limit, say 180°, are generated in given *steps* under the control of a `for` statement. The user-defined `sinevalue( )` function is *repeatedly* called with each one of these *generated angles*. Every call *gets back* a value for `sin(deg)` from the user-defined `sinevalue( )` function. The number of `sin(deg)` values 'obtained back' will be equal to the *number of calls made*. All these values are displayed or printed out as and when they arrive at the *called function*. The values for `sin(deg)` are also obtained and output using the C library function for the purpose of verification.

C-Tips

It is known that the value of sin(x) will be of `float` *type*. Hence the *type* of the *returned* value of `sinevalue( )` function has to be a `float` *type* value. Therefore, the function `sinevalue( )` has been *declared* as `float` type.

When the `main( )` comes across the name of the function `sinevalue( )`, the control *automatically shifts* to the `sinevalue( )` function *along with the values of the arguments*. In other words, value of *deg* gets passed on to x and is *copied* onto it. Similarly, the value of 'n' in `main( )` gets passed on to 'n' in the `sinevalue( )` function. As the variables involved are *scalars,* i.e. single valued, the function is *called by value.*

Flowchart


Start
prototype declaration
input n, the number of terms to be summed & the step size
for deg=0; deg<=180; deg=deg+step
<=
result = sinevalue(deg, n)
output result and value of sine (deg) as obtained
Stop
float sinevalue (float x, int n)
x=x*3.14/180
term=x
sum=0
for i=1, i<=n, ++i
<=
sum = sum+term
term = - x^2 / (2i)(2i+1) × term
return sum

The Program

```
/*   Tabulating the results of the Sine function for different
     angles */

#include<stdio.h>
#include<math.h>
#include<conio.h>
main()
{
   int n, step;                      /* Declaring variables */
   float deg, result;
   float sinevalue(float x, int n);  /* Function prototype */
   clrscr();

   printf(“Read number of terms to be added\n”);
     scanf(“%d”, &n);                /* Accepting number of terms */
   printf(“Read the step value in numeric degrees\n”);

   scanf(“%d”, &step);               /* Accepting step value */

   /* Calling function sinevalue() in printf statement */
   for(deg=0; deg<=180; deg = deg+step)
   {
     result = sinevalue(deg, n);
     printf("sin(%3.0f)=%f, %f\n" deg, result, sin(deg * 3.14/180));

   }

}



/* Function to calculate sine value of x upto n terms */

float sinevalue(float x, int n)
```

```
{
   int i;
   float term, sum=0;       /* Declaring local variables */
   x = x*3.14/180;          /* Converting x value to radians */
   term = x;                /* term is initialized to x */

   for(i = 1; i <= n; ++i)  /* Finding sum of n terms */
   {
     sum += term;
     term *= -x*x/((2*i)*(2*i+1));
   }

   return(sum);             /* Returning sum to main()*/

}
```

8.5 FUNCTIONS TO COMPUTE MEAN AND VARIANCE AND HENCE TO COMPUTE STANDARD DEVIATION OF GIVEN SET OF NUMBERS

The *mean, variance* and *standard deviation* are computed using the relations

$$\text{Mean} = \frac{\sum_{i=0}^{n-1} x_i}{n} \qquad \text{Variance} = \frac{\sum_{i=0}^{n-1} (x_i - \text{mean})^2}{n}$$

$$\text{Standard deviation} = \sqrt{\text{Variance}}$$

The Method

Procedures to compute *mean, variance* and *standard deviation* using the above given relations have been already discussed in Section 6.6. Function `meanvalue( )` that computes the *mean* or *average* and *returns,* it is developed assuming that the *entire array* (say, *a*) and the information about the *number of elements* (*n*) are available. The value *returned* from this function will be of type `float`. Function `varvalue( )` that computes the *variance* and *returns* its value is developed assuming that the *entire array* (say, *a*), information about the number of elements in the array (*n*) and the *mean* are available. The *type* of the value *returned* by this function will be of type `float`.

In the `main( )` the value for *n*, and all the '*n*' elements of the array are input. The `meanvalue( )` and `varvalue( )` functions are called one after the other, in that order.

The *standard deviation* is computed using the value of *variance.* Finally, the values of *mean, variance* and *standard deviation* are output.

C-Tips

As the *mean* of all the '*n*' elements of an array (say, *a*), is to be found out, the *array a* and '*n*' are taken as the *formal arguments* of the function to be defined. Also, as the type of the *returned mean* value will be of `float` type, the *type of the defined function* must be taken as of type `float`. Thus the *function header* and hence the *function prototype* have been taken as: `float meanvalue(float a[ ], int n)`.

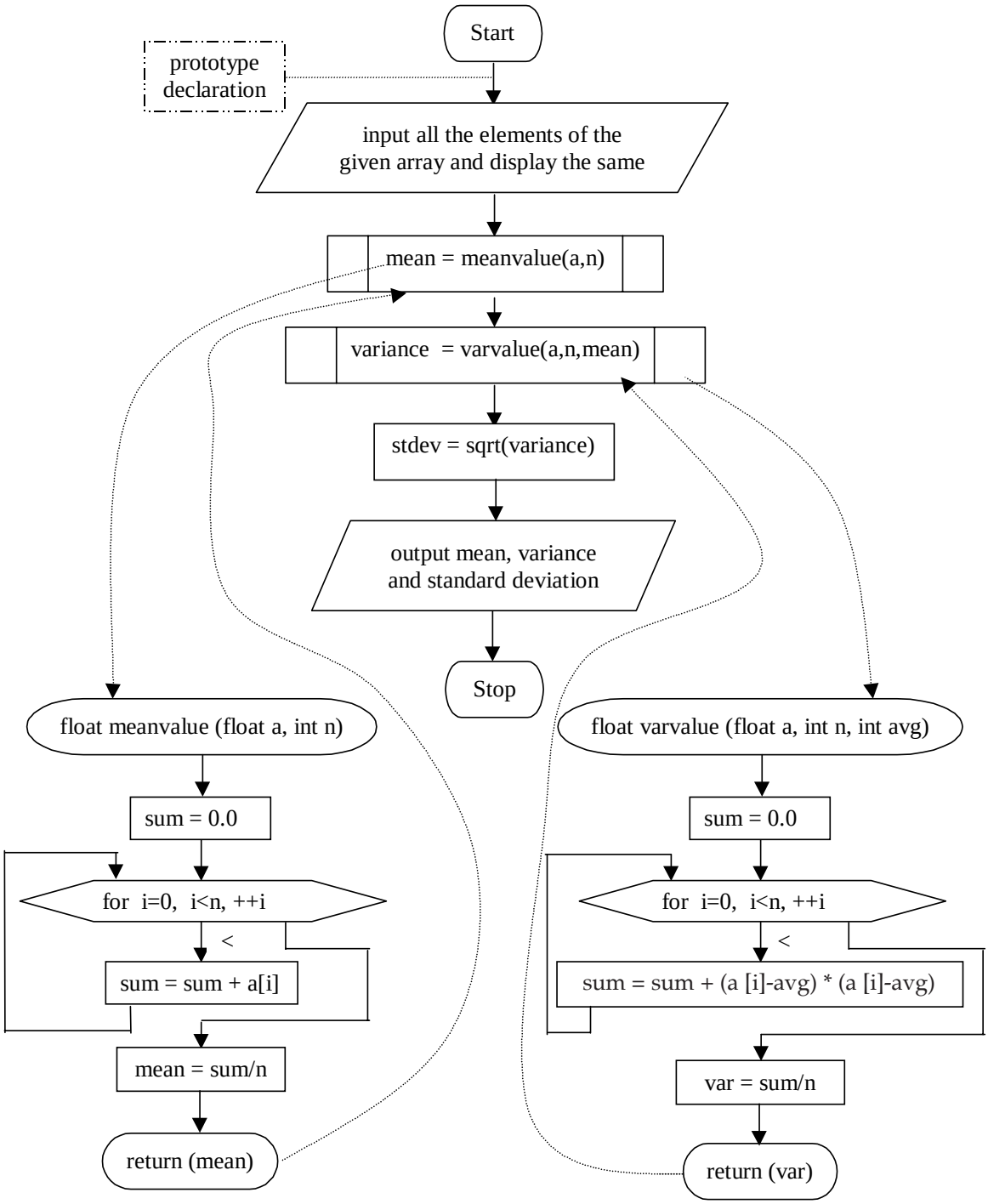
As the computation of *variance* needs an array (say, *a*), information about the number of elements in the array '*n*' and the *mean* or *average* of all the array elements, all these three are taken as the *formal arguments* of the function developed to compute the *variance.* Since the *type* of the *returned variance* value will be of `float` *type*, the type of the defined function must be taken as of type `float`. Thus the *function header* and the *function prototype* of this function have been taken as:

```
float varvalue(float a[ ], int n, float avg)
```

… beyond

A careful study of the examples discussed till now in this chapter reveals an important fact that whenever *user-defined functions* are used, *they are first* ***recognized*** *by their* ***names*** and the *values* of their *(actual) arguments* are *passed* on and made as the *values* of the *formal arguments* of the *invoked* or *called function*. All the arguments or parameters used till now were *scalars* (i.e. single valued) in nature. It is important and interesting to note **how** *arrays* (i.e. multivalued variables) are handled during *function calls*. Here it may be recalled that when single valued variables are used as *actual arguments* as well as *formal arguments*, they will be *local to their functions*. Memory locations of these local variables will be allocated at the point of their declarations within their respective functions and hence will be different. Therefore, the values of actual arguments are *transmitted* or passed on and copied into formal arguments. But in the case of arrays, *memory locations to the arrays are allocated only in the main function*. When these arrays are used in user-defined functions the starting *address* of these allocated memory locations is *transmitted* or *passed* on to the *user-defined functions*. The C- compiler does this ***implicitly*** by passing on the *address of the array* to the *called function*. Thus the arrays inside user-defined functions will be accessing the *same memory locations* as that of the corresponding arrays in the `main( )` and *manipulating* the values stored in them. The *size of the array* is *explicitly* mentioned *only in the* `main( )` during its declaration in the `main( )`. It is *not necessary* to mention the *size* of the arrays explicitly in the *function header,* in the *function prototype* declaration as well as *during the calling of a user-defined function.* Also, it may be noted that the *array names* used in the *calling function* and the *called function* may be *different*. Even then they refer to the *same memory location*s. In other words, different *array names pointing to same memory locations* behave like *alternate names,* i.e. ***aliases***.

Flowchart


Start
prototype declaration
input all the elements of the given array and display the same
mean = meanvalue(a,n)
variance = varvalue(a,n,mean)
stdev = sqrt(variance)
output mean, variance and standard deviation
Stop
float meanvalue (float a, int n)
sum = 0.0
for i=0, i<n, ++i
<
sum = sum + a[i]
mean = sum/n
return (mean)
float varvalue (float a, int n, int avg)
sum = 0.0
for i=0, i<n, ++i
<
sum = sum + (a [i]-avg) * (a [i]-avg)
var = sum/n
return (var)

The Program

```
/*  Computing mean, variance and standard deviation of a given set of
    numbers using functions */

#include <stdio.h>
#include <math.h>
#include <conio.h>
main()
{
    int i, n;                 /* Declaring variables */
    float  a[20], mean, variance, stdev;

    float meanvalue(float a[], int n);  /* Function prototype */
    float varvalue(float a[], int n, float avg);
    clrscr();
    printf("\nRead value of n\n");
    scanf("%d", &n);          /* Accepting n */

    printf("n = %d\n", n);
    printf("Read elements of array \n");
    for(i=0; i < n; ++i)
       scanf("%f", &a[i]);   /* Reading elements of array */
    printf("Elements are \n");
    for(i = 0; i < n; ++i)
       printf("%f\n", a[i]);  /* Printing elements of array */

    /* Calling function meanvalue() to find the mean of given list of
       elements */
    mean = meanvalue(a, n);

    /* Calling function varvalue() to find the variance of given
       list of elements */
    variance = varvalue(a, n, mean);

    /* Computing standard deviation */
    stdev = sqrt(variance);

    printf("Mean = %f\nVariance = %f\nStandard Deviation = %f\n",
            mean, variance,stdev) ;
```

```
}

/*Function to compute mean */
float meanvalue(float a[], int n)
{
    int i;
    float sum=0,mean;        /* Declaring local variables */

    for(i=0; i<n; ++i)
       sum += a[i];          /* Computing sum of all the elements */

    mean = sum/n;            /* Computing mean */
    return(mean);
}


/* Function to compute variance */
float varvalue(float a[], int n, float avg)
{
    int i;
    float sum=0, var;        /* Declaring local variables */

    for(i = 0; i < n; ++i)     /* Computing difference sum */
       sum += (a[i] - avg) * (a[i] - avg);

    var = sum / n;           /* Computing variance */

    return(var);             /* Returning variance to main()*/
}
```

8.6 FUNCTIONS TO READ AND WRITE ONE DIMENSIONAL ARRAY

The Method

This example has been already discussed in Section 6.1. The array reading function *readarr( )* has been developed assuming that the array, say *a*, and *n*, the number of elements, are available. In *readarr( )* function, all the elements of the array are input

under the control of a `for` loop. This function does not *return* any value. Therefore the type of this function is declared as `void`.

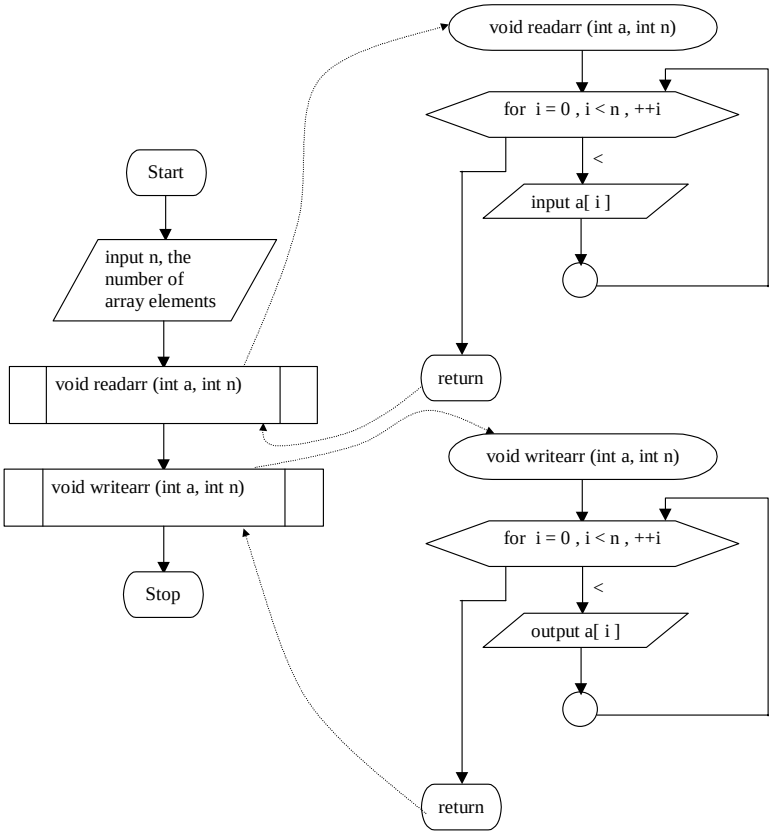
The array reading function `writearr( )` has been developed assuming that the array, say *a*, and *n*, the number of elements, are available. In `writearr( )` function all the elements of the array are output under the control of a *for* loop. This function also does not return any value and therefore has been declared as of type `void`.

In the `main( )` function, first the value of *n*, number of elements is read in. Then the `readarr( )` and `writearr( )` functions are used, in that order, to input and output the array.

C-Tips

In this example two parameters, one the *array(a)* and the other *n*, number of elements, are passed on to both the functions `readarr( )` and `writearr( )`. The scalar quantity *n* is passed on *by-value* and the array is passed on *by-address* or *call by-reference*. These techniques have been already discussed in detail in Sections 8.2 and 8.5, respectively.

Flowchart



The Program

```
/* Functions to read and write an one dimensional array. */

#include<stdio.h>
#include<conio.h>
main()
{
   int i, n, a[20];              /* Declaring variables */

   void readarr(int a[], int n);  /* Function prototypes */
   void writearr(int a[], int n);
   clrscr();

   printf("\nRead n\n");
   scanf("%d", &n);
   printf("n = %d\n", n);
   printf("Input elements of the array \n");

   /* Reading elements of array by calling readarr() function */
   readarr(a,n);

   printf("\nPrinting Elements of the array \n");
   /* Printing elements of array by calling writearr() function */
   writearr(a,n);

}



/* Function to read array elements */
void readarr(int a[], int n)
{
   int i;                        /* Declaring local variable */
   for(i = 0; i < n; ++i)
     scanf("%d", &a[i]);         /* Reading array elements */
```

```
}


/* Function to write array elements */
void writearr(int a[], int n)
{
   int i;                          /* Declaring local variable */
   for(i = 0; i < n; ++i)
     printf("%d\n", a[i]);         /* Printing array elements  */
}
```

8.7 FUNCTION TO SORT AN ARRAY USING BUBBLE SORT METHOD

The Method

The procedure used to *sort* a given set of integer elements is discussed in Section 6.8. In this example a function called `sort( )` to sort a given array '*a*' of '*n*' integer elements is developed. Also a `main( )` function has been developed that using this user-defined function `sort( )`. The *unsorted* array of integer elements is input in the `main( )` function and the *sorted* array is also output in the `main( )` function. The corresponding flowchart is given in the next page.

C-Tips

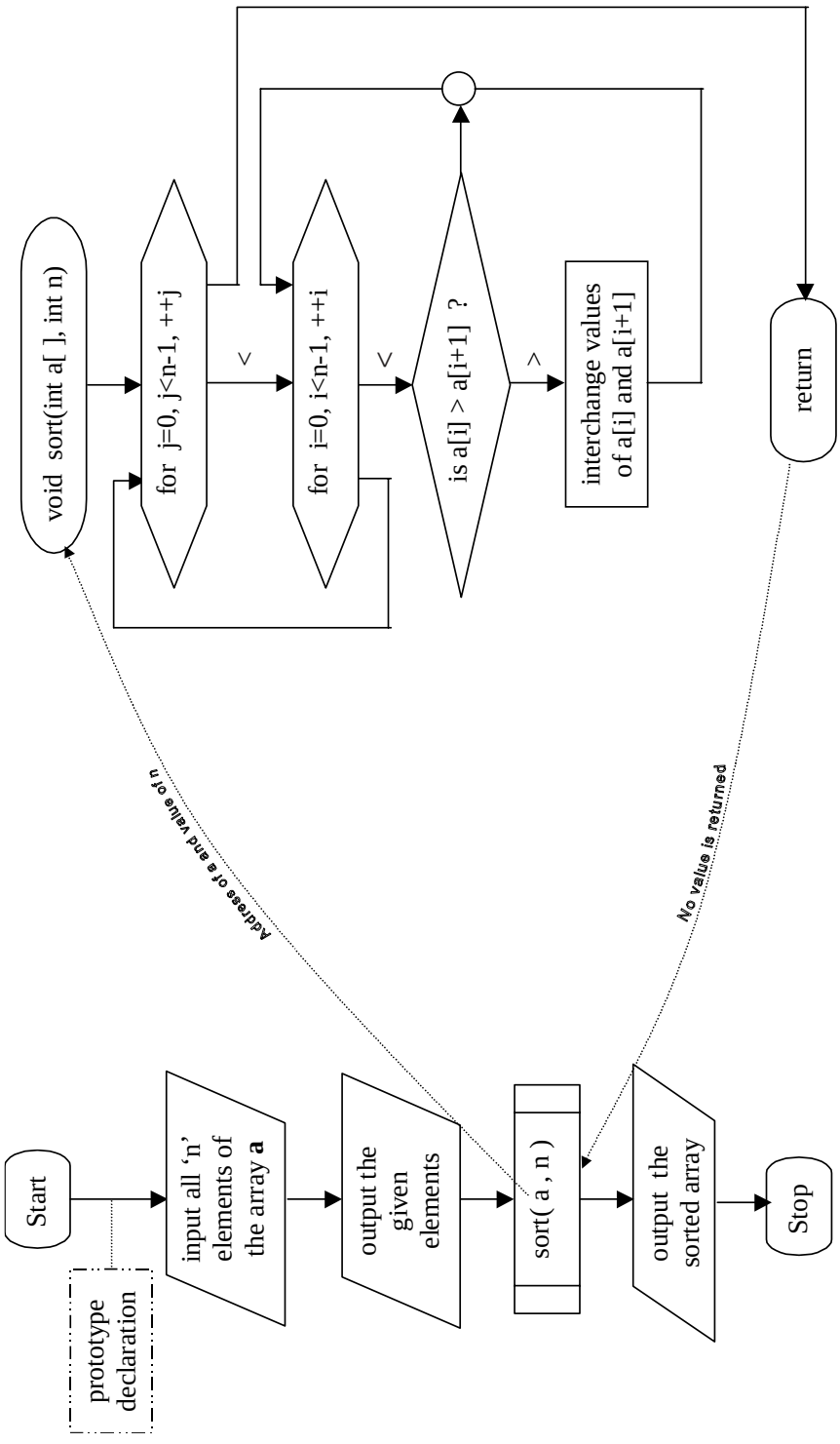
The user defined function `sort( )` is developed assuming all the '*n*' elements of the array '*a*' are available. Thus the array '*a*' and the number of elements '*n*' are taken as the *formal arguments* of the `sort( )` function. The *passing of array* from the *calling function* to the *called function* takes place *implicitly* via *address passing*. Since the *user-defined function* works on the *same memory locations* of the array that are allocated within the main, the elements of the array are *sorted* in those memory locations itself. Thus, in this case, *no values* are *returned* from the *called function* to the *calling function.* Hence the *type* of the user-defined `sort( )` function has to be taken as `void`. Thus *the function header* and the *function prototype* have been taken as:

```
void sort(int a[ ], int n)
```

… beyond

Though an explicit `return` statement is shown in the flowchart of the `sort( )` function, it has not been used *explicitly* in the program. Under such a situation the *control* from the *called function goes back* to the *calling function* as soon as it comes across the last closing curly brace('`}`').

Flowchart


Start
prototype declaration
input all 'n' elements of the array **a**
output the given elements
sort(a , n)
output the sorted array
Stop
void sort(int a[ ], int n)
for j=0, j<n-1, ++j
for i=0, i<n-1, ++i
is a[i] > a[i+1] ?
interchange values of a[i] and a[i+1]
return
<
<
>
Address of a and value of n
No value is returned

The Program

```
/* Sorting given list of n numbers in ascending order using bubble
sort */

#include<stdio.h>
#include<conio.h>
main()
{
   int i, n, a[20],            /* Declaring variables */

   void sort(int a[], int n);  /* Function prototype */
   clrscr();

   printf("\nRead n\n");
   scanf("%d", &n);
   printf("n = %d\n", n);

   printf("Read elements of array\n");
   for(i = 0; i < n; ++i)
      scanf("%d", &a[i]);     /* Reading elements of array */

   printf("Elements are \n");
   for(i = 0; i < n; ++i)
      printf("%d\n", a[i]);   /* Printing elements of array */

   sort(a, n);                /* Calling function sort() */

   printf("Sorted array is:\n");
   for(i = 0; i < n; ++i)
      printf("%d\n", a[i]);

}
```

```
/* Function to sort the array elements in ascending order */

void sort(int a[], int n)
{
   int i,j,temp;              /* Declaring local arguments */

   /* This loop performs required number of passes */
  for(j=0; j<n-1; ++j)
     /* This loop performs all comparisons in a pass */
     for(i=0; i<n-1; ++i)
       if (a[i] > a[i+1] )
       /* If first among two adjacent elements is greater  than the
          other then it is exchanged using temporary  variable
          temp */
       {
          temp  = a[i];
          a[i]  = a[i+1];
          a[i+1] = temp;
       }

}
```

8.8 FUNCTION TO SEARCH FOR A REQUIRED ELEMENT USING BINARY SEARCH METHOD

The method employed to *search* for a required element (i.e. a `key`) in a *sorted* array using *binary search* method has been already discussed in Section 6.17. The same is re-presented here in the form of a *user defined function*. It should be noted that this method is applicable only to *sorted* arrays.

The Method

A function called `binsearch( )` is developed assuming that the array **a**, the information about the number of elements in the array 'n' and the 'key' to be searched are available. Naturally these three, i.e. `a[ ]`, n and `key`, are taken as the *formal arguments* of the function `binsearch( )`. In this example, the *result of searching* is output within the `binsearch( )` function itself. As such this function *does not return* any value to the *calling function*. Therefore, the function is declared as of type `void`.

In the `main( )` first all the '*n*' elements of the *array* are input. Next the '`key`' to be searched is input. Then the function `binsearch( )` is invoked with *actual arguments*.

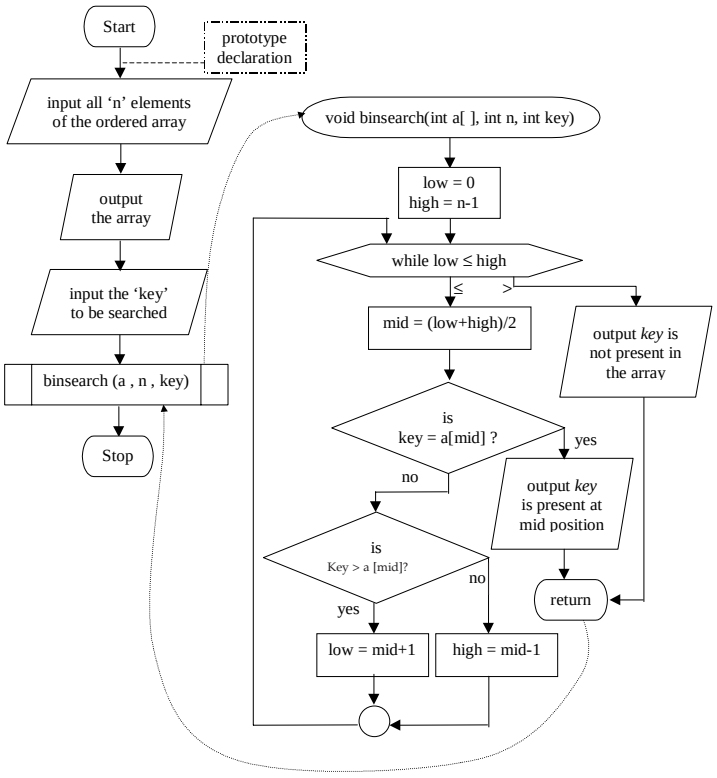
C-Tips

When the `binsearch( )` function is called the value of '*n*', the number of elements in the array, and the `key` to be searched get transmitted to it *by value*. The array is transmitted to the *called function* by *implication*. In otherwords, the *address* of the array gets transmitted automatically from the *calling function* to the *called function*. Since the *success* or *failure* is reported suitably within the `binsearch( )` function itself, *no value* will be returned to the *calling function*. Therefore, the *type* of the function `binsearch( )` has been taken as of type `void`. As already mentioned the *array size* is mentioned only during the array declaration in the `main( )`. In other words, there is *no need* to mention the *array size* in the *function header, function prototype* and during a *function call*. The *function prototype* must be included during the beginning of the `main( )` function. Thus the *function header* and the *function prototype* have been taken as

```
void binsearch (int a[ ], int n, int key)
```

It may be observed that no `return` statement has been used *explicitly*.

Flowchart



The Program

```
/* Searching for the specified number in a sorted array using binary
   search */

#include<stdio.h>
#include<conio.h>
main()
{
   int i, n, a[20],key;       /* Declaring local variables */

   void binsearch(int a[], int n, int key);  /* Function prototype */
   clrscr();
   printf("\nRead n\n");
   scanf("%d", &n);
   printf("n = %d\n", n);

   printf("Input elements of the array in ascending order\n");
   /* Reading elements of array in  ascending order */
   for(i = 0; i < n; ++i)
      scanf("%d", &a[i]);
   printf("Elements are \n");
   /* Printing elements of array */
   for(i = 0; i < n; ++i)
      printf("%d\n", a[i]);  /* Printing elements of array */

   printf("Enter key element to be searched\n");
   scanf("%d", &key);         /*Reading Key element to be searched*/
   printf("key = %d\n", key); /* Printing key element */

   binsearch(a,n,key);        /* Calling function binsearch() */

}
```

```
/* Function to search for the key element */
void binsearch(int a[], int n, int key)
{
   int low, high, mid;
   high = n-1;
   low = 0;
   while(low <= high)
   {
      mid = (low + high)/2;    /* mid is the mean of low and high */

      /* If key is equal to middle element then exit from loop */
      if( key == a[mid] )
         break;
      else
         if( key > a[mid] )
            /*  If key is greater than middle element then it must be
                in second half. Make low = mid+1 */
            low = mid + 1;
         else
            /*  If key is lesser than middle element then it must be in
                first half. Make high = mid-1 */
            high = mid - 1;
  }
         if (a[mid] == key)
            printf("Search is successful, %d is found in position
            number %d\n", key, mid+1);
         else
            printf("Search is unsuccessful, %d is not found \n", key);


}
```

8.9 FUNCTIONS TO SORT A GIVEN UNSORTED ARRAY USING SELECTION SORT

The procedure involved in *sorting* an *unsorted* array of elements using *selection sort* has been already discussed in Section 6.18. As pointed out in that section, this *sorting procedure* has *two distinctly independent* actions to be taken, viz. "finding the *position* of a *minimum value*" and "*exchanging* the *minimum value at that position* with a *value at the beginning* of the *current search space*."

The Method

As this method involves finding the *position* of a minimum value and *exchanging* it with the value at the *beginning* of the *current search space* on the array, two *independent functions* are developed to implement these actions. The two user-defined functions developed here are:

```
int min(int a[ ], int n, int i ) and
void exch(int a[ ], int i, int m)
```

These two functions are *called* one after the other *repeatedly* for *(n-2)* times. During each such repetition the *search space is reduced by one count*. Finally the sorted array is output.

C-Tips

As two independent *subtasks* have been identified two functions

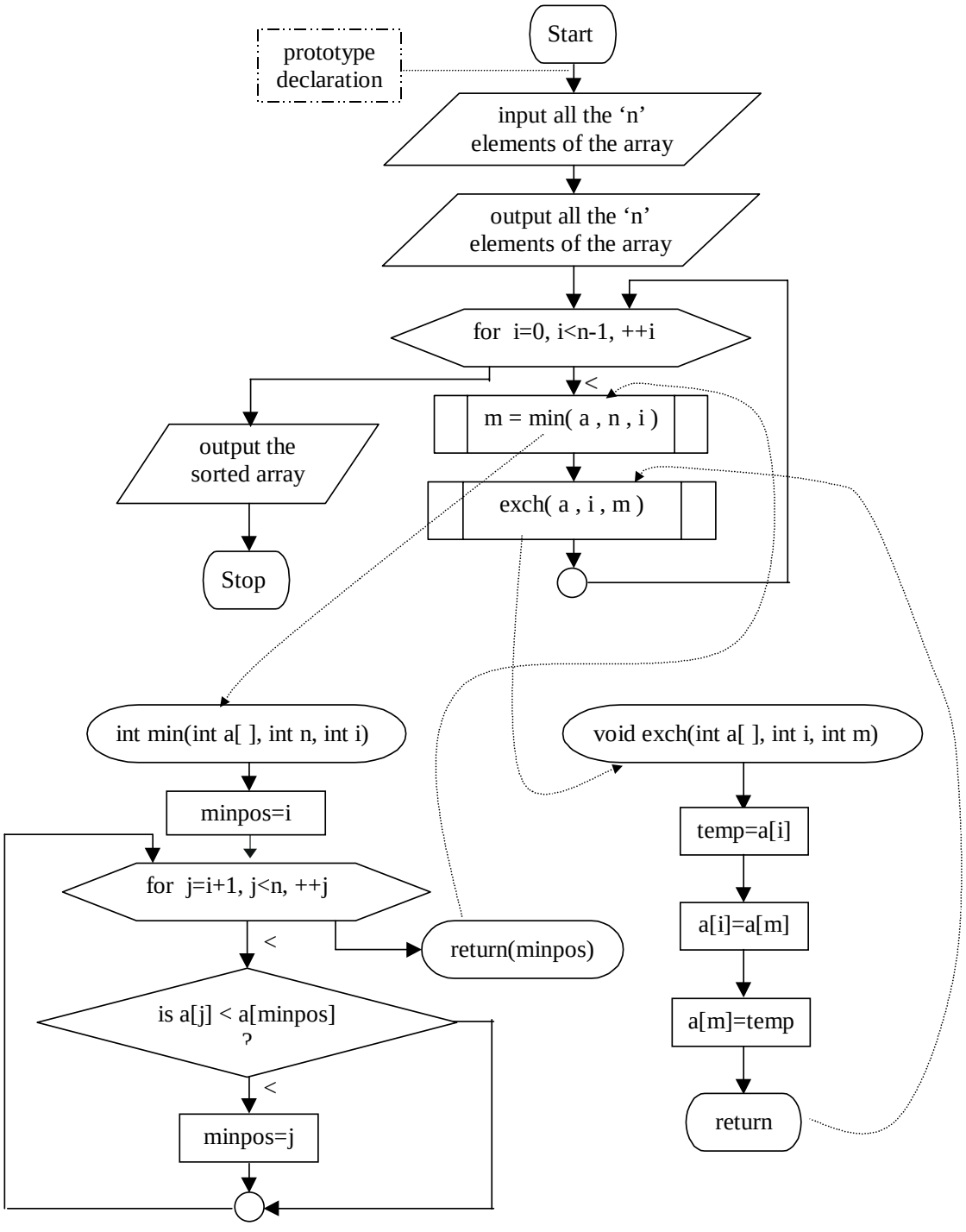
`int min(int a[ ], int n, int i)` and
`void exch(int a[ ], int i, int m)` have been developed.

The given array `a`, information about the total number of elements in the array *(n)* and information about the beginning of the *current search space* `i` are assumed to be available and have been used as *formal arguments* of the function `min( )`. Since this function is expected to *return* a *position value* which will be an *integer* value, the function `min( )` has been declared as of *type* `int`. As it could be seen from the *flowchart* and the *program*, a local variable `j` is used to *move* along the array.

The given array `a`, information about the beginning of the *current search space* `i`, and *value of the position* of the minimum element *m* are all assumed to be available and have been used as *formal arguments* of the function `exch( )`. Since this function just *exchanges* the values stored in *two locations* of an array, *no value* will be returned from this to the *calling function*. As such the function `exch( )` has been declared as of type `void`.

Of course, the values of `m`, `n` and `i` are being passed by *call by value* method and the arrays are passed by *implication*.

Flowchart


Start
prototype declaration
input all the 'n' elements of the array
output all the 'n' elements of the array
for i=0, i<n-1, ++i
<
m = min(a , n , i)
exch(a , i , m)
output the sorted array
Stop
int min(int a[ ], int n, int i)
minpos=i
for j=i+1, j<n, ++j
<
return(minpos)
is a[j] < a[minpos] ?
<
minpos=j
void exch(int a[ ], int i, int m)
temp=a[i]
a[i]=a[m]
a[m]=temp
return

The Program

```
/* Sorting given list of n numbers in ascending order using selection
sort */

#include<stdio.h>
#include<math.h>
#include<conio.h>
main()
{

   int i, n, m, a[20];                /* Declaring variables */

   int min(int a[], int n, int i);
   void exch(int a[], int i, int m);  /* Function prototype */
   clrscr();

   printf("\nRead n\n");
   scanf("%d", &n);
   printf("n = %d\n", n);

   printf("Read elements of array\n");
   for(i = 0; i < n; ++i)
     scanf("%d", &a[i]);              /* Reading elements of array */

   printf("Elements are \n");
   for(i = 0; i < n; ++i)
     printf("%d\n", a[i]);            /* Printing elements of array */

   /* Calling functions */
   for (i = 0; i<n-1; ++i)
   {
     /* Calling function min() to find the position of smallest
        element */
     m = min(a, n, i);
     /* Calling function exch() to exchange a[i] and a[m] */
        exch(a, i, m);
```

```
   }

   printf("Sorted array is:\n");
   for(i = 0; i < n; ++i)
     printf("%d\n", a[i]);
}

/* Function to find the position of smallest element */
int min(int a[],int n, int i)
{
  int j,minpos=i;                    /* Declaring local variables */

   /* Finding position of minimum element */
   for(j=i+1; j<n; ++j)
     if (a[j] < a[minpos])
        minpos = j;

   return(minpos);   /* Returning position of minimum element */

}
/* Function to exchange a[i] and a[m] */
void exch(int a[], int i, int m)
{
   int temp;

   temp = a[i];
     a[i] = a[m];     /* Exchanging */
   a[m] = temp;

   return;           /* Returning control back to main() */

}
```

8.10 FUNCTIONS TO READ, WRITE AND COMPUTE TRACE OF GIVEN MATRIX

As already mentioned in Section 8.12 the *trace* of a *square* matrix is the *sum* of all the elements on its *principal diagonal*.

The Method

The dimensions of the matrix is first input and then all the elements of the matrix, say *a*, are read in using a *user defined function* `readmatrix( )`. Since *square matrix* is assumed, *testing* for the *equality* of the *row size* and the *column size* is not done. The input matrix is printed out using another *user defined function* `writematrix( )`. The *trace* of the matrix is obtained by using another user defined function `tracevalue( )` and output.

C-Tips

In order to read and write matrices an array `a`, of dimensions `m` rows and `n` columns are assumed to be available. As such, array `a`, `m` and `n` are used as *formal arguments* of the *user-defined functions* `readmatrix( )` and `writematrix( )`. As these two functions just perform certain task, *no values* will be returned to the *calling function*. Thus these functions are declared as of type `void`.

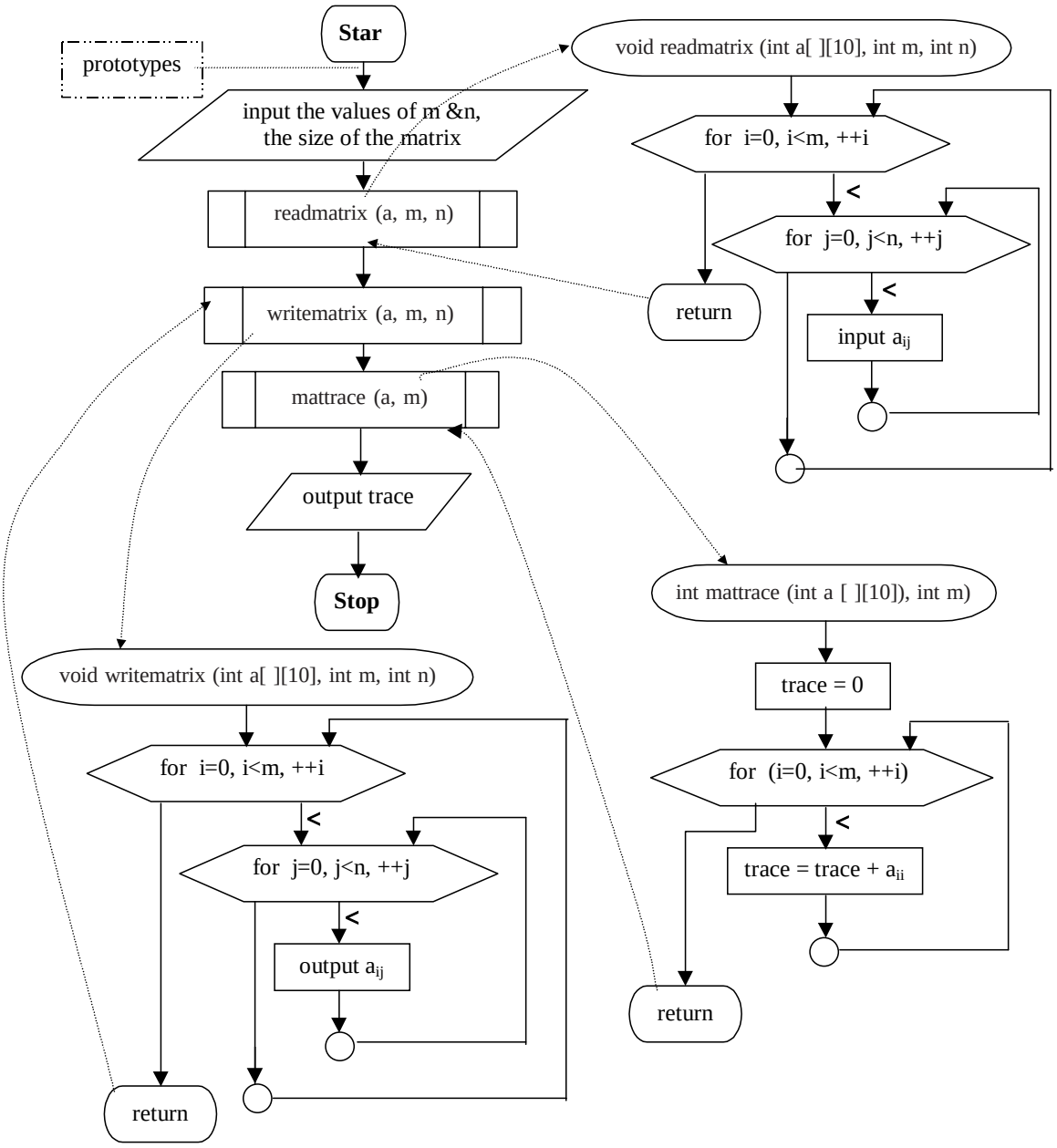
The user-defined function `tracevalue( )` is developed assuming that a *square matrix*, say `a`, along with *one of its dimensions* is available. Thus *array* **a**, and either *row size* **m** or *column size* **n** are taken as its *formal arguments*. The *type of value* returned from this function to a *calling function* depends on the *type of elements* of the array. Since, in this example, *integer type* array elements have been used, the *returned value* from the function `tracevalue( )` will be of type `int`. Thus the type of the function has to be taken as of type *int*. As such, in this example, the following *function headers* and *function prototypes* have been developed and used.

```
void readmatrix(int a[ ][10], int m, int n)
void writematrix(int a[ ][10], int m, int n)
int tracevalue(int a[ ][10],int m)
```

… beyond

In the case of two-dimensional arrays, like in matrices, the *beginning address* is *passed* from a *calling function* to the *called function*. When a two dimensional array of size (**m** × **n**) is declared in a `main( )` function, continuous memory locations for all the **m** × **n** elements will be allocated. Like with one-dimensional arrays all the corresponding arrays, regardless of the fact wherever they are, access only these memory locations. However, first **n** memory locations will be reserved for the first row, second **n** memory locations will be reserved for the second row and so on. In otherwords, elements of this type of array will be *stored row wise*. Here, it is important to note that the second row will be stored starting from the $(n + 1)^{th}$ memory location, third row will be stored starting from the $(2n + 1)^{th}$ memory location and so on. Thus the system expects the value of **n,** i.e. the size of the row. Hence, while handling two-dimensional arrays (infact any multidimensional array) all the sizes of additional dimensions must be provided *explicitly*. However, C allows the user to use a blank '`b`' as the size of various dimensions. This sets a predefined value (usually 32) to the different dimensions of the array automatically.

Flowchart


Star
prototypes
input the values of m &n,
the size of the matrix
readmatrix (a, m, n)
writematrix (a, m, n)
mattrace (a, m)
output trace
Stop
void readmatrix (int a[ ][10], int m, int n)
for i=0, i<m, ++i
for j=0, j<n, ++j
input a_{ij}
return
void writematrix (int a[ ][10], int m, int n)
for i=0, i<m, ++i
for j=0, j<n, ++j
output a_{ij}
return
int mattrace (int a [ ][10]), int m)
trace = 0
for (i=0, i<m, ++i)
trace = trace + a_{ii}
return

The Program

```
/* Finding the trace of a given matrix a(m x n) using function */

#include<stdio.h>
#include<math.h>
main()

{
   int a[10][10], trace=0, i, j, m, n;  /* Declaring variables */
   void readmatrix(int a[][10], int m, int n);
   int mattrace(int a[][10], int m);   /* Function prototypes */
   void writematrix(int a[][10], int m, int n);
   clrscr();

   printf("Read order of the matrix \n");
   scanf("%d%d", &m, &n);          /* Reading order of the matrix */
   printf("Order of the given matrix  is : %d X %d\n", m, n);

   /* Is it a square matrix?  */
   if( m != n)
   {

      printf("Trace not possible \n"); /* Not a square matrix */
      exit();

  }

   printf("Read elements of matrix \n");

   readmatrix(a, m, n);         /* Calling function readmatrix() */

   printf("\nGiven matrix is:\n\n");

   writematrix(a, m, n);          /*  Calling function
   writematrix() */

   printf("\nTrace of the given matrix = %d\n", mattrace(a, m));


}
```

```
/* Function to read matrix */
void readmatrix(int a[][10], int m, int n)
{
   int i, j;
   for(i=0; i<m; ++i)
      for(j=0; j<n; ++j)
         scanf("%d", &a[i][j]);    /* Reading elements of matrix */
   return;                         /* Return control back to main() */
}

/* Function to print matrix */
void writematrix(int a[][10], int m, int n)
{
   int i, j;
   for(i=0; i<m; ++i)
   {
      for(j=0; j<n; ++j)
         printf("%4d\t", a[i][j]);  /* Printing the matrix */
      printf("\n");
   }
   return;                  /* Return control back to main() */
}

/* Function to find the trace of a given matrix */
int mattrace(int a[][10], int m)
{
   int i, trace=0;          /* Declaring local variables */
   for(i=0;  i<m; ++i)
      trace += a[i][i];     /* Computing trace */
   return(trace);           /* Returning trace back to main() */
}
```

8.11 FUNCTIONS TO COMPUTE SUM OF EACH ROW, SUM OF EACH COLUMN AND SUM OF ALL ELEMENTS OF GIVEN MATRIX

In this example, three user-defined functions `rowsum( )` that computes the sum of a row, `columnsum( )` that computes the sum of a column and `matsum( )` that computes the *sum of all the elements* of a given matrix have been developed.

The Method

The *sum of all the elements of a row*, say *i*th row, is obtained by using a relation of the type $sum = sum + a_{ij}$ for a known value of ***i*** and for all the values of ***j*** from 0 to (n-1).

The *sum of all the elements of a column*, say *j*th column, is obtained by using a relation of the type $sum = sum + a_{ij}$ for all the values of ***i*** from 0 to (m-1) and for a known value of ***j***.

The *sum of all the elements* of the matrix is obtained by using a relation of the type $sum = sum + a_{ij}$ for all the values of ***i*** from 0 to (m-1) and for all the values of ***j*** from 0 to (n-1).

The functions ***readmatrix( )*** and ***writematrix( )*** developed in the previous section, i.e 8.10 have been used to input and output the required matrix in the ***main( ).***

C-Tips

The `rowsum( )` function has been developed assuming that the array `a`, the `rowvalue` `i` of which the elements are to be summed, and the `columnsize` `n` are available. Thus, *array* `a`, `rowvalue` `i` and `columnsize` `n` have been taken as the *formal arguments* of `rowsum( )`.

The `columnsum( )` function has been developed assuming that the array `a`, the column value `j` of which the elements are to be summed up, and the `rowsize` `m` are available. Thus the *array* `a`, *column value* `j` and the *rowsize* `m` have been taken as the *formal arguments* of `columnsum( )`.

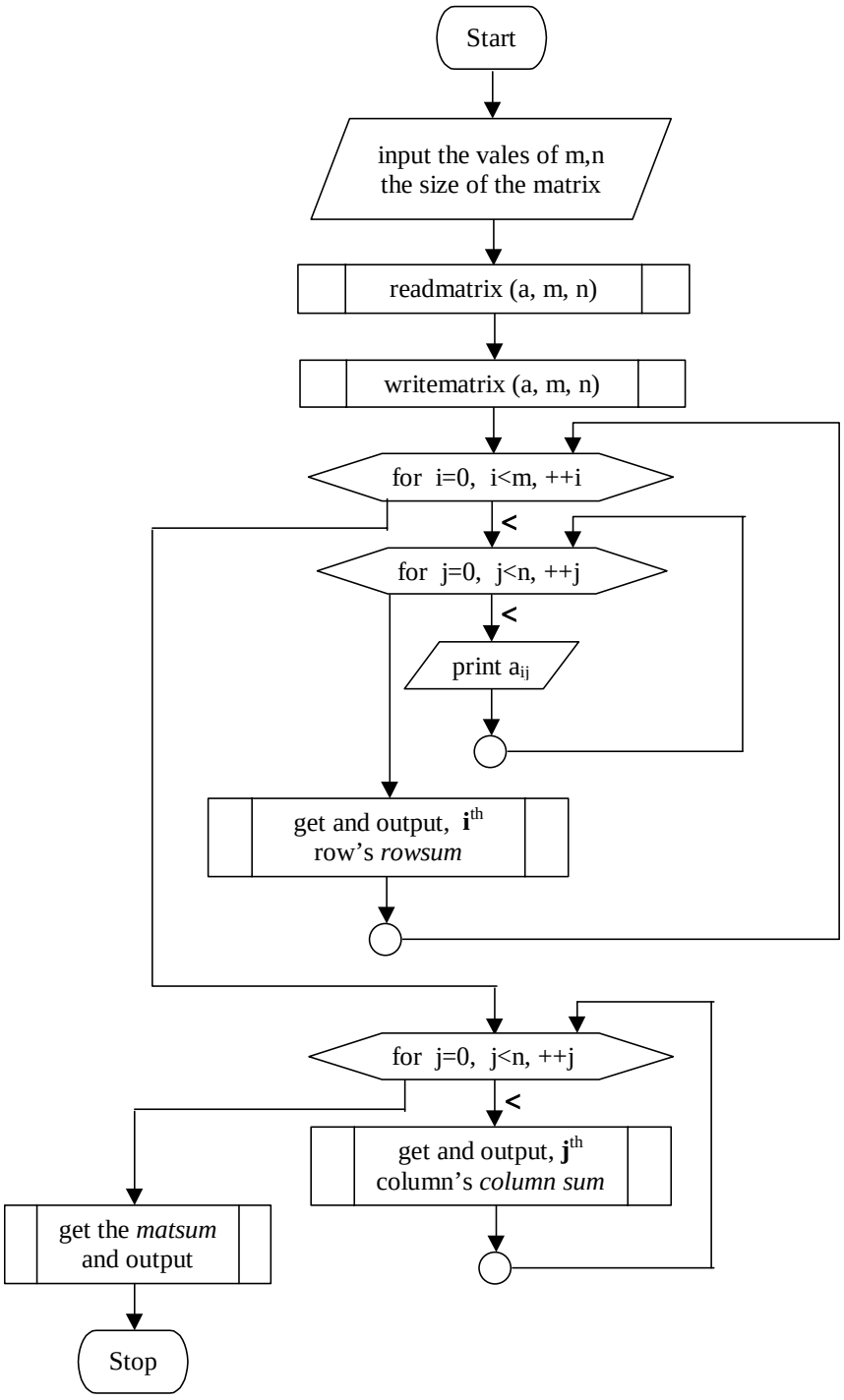
The `matsum( )` function is developed on the same lines using the array `a` and its dimensions `m` and `n` as *formal arguments*.

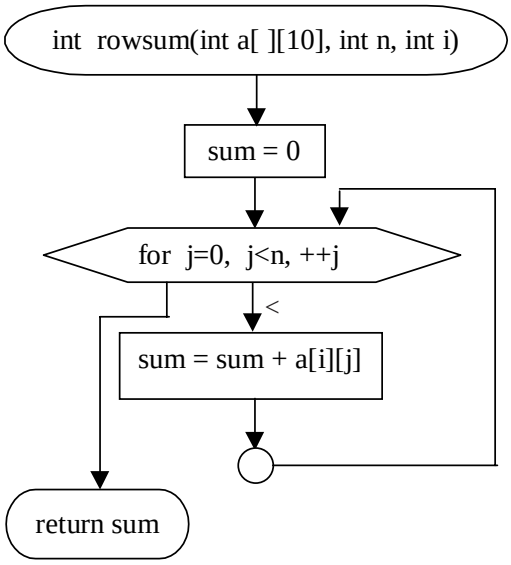
Since all the three functions developed here *return* a sum, an *integer* value, all of them have been *declared* as of *type* `int`. The *function headers* and the *function prototypes* of the functions developed are :

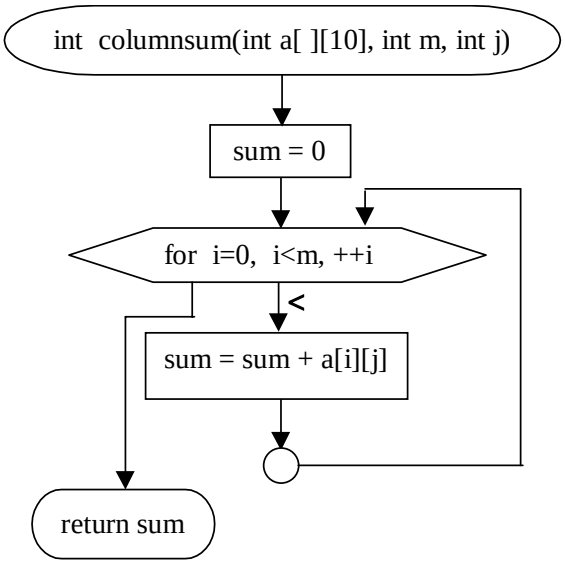
```
int rowsum (int a[ ][10], int n, int i)
int columnsum (int a[ ][10], int m, int j)
int matsum (int a[ ][10], int m, int n)
```

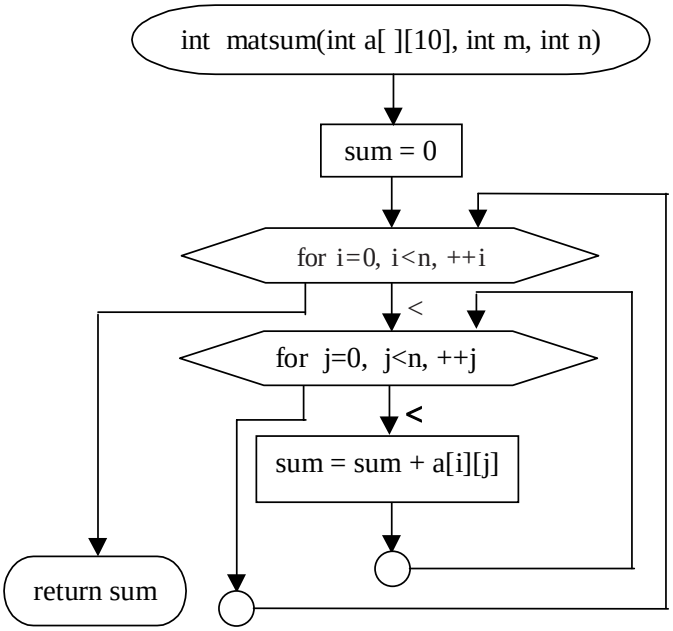
As is known, values of the *scalars* i, j, m and n are transmitted *by-value* and matrices are transmitted by *implication*.

Flowchart


Start
input the vales of m,n
the size of the matrix
readmatrix (a, m, n)
writematrix (a, m, n)
for i=0, i<m, ++i
<
for j=0, j<n, ++j
<
print a_{ij}
get and output, **i**th
row's *rowsum*
for j=0, j<n, ++j
<
get and output, **j**th
column's *column sum*
get the *matsum*
and output
Stop


int rowsum(int a[ ][10], int n, int i)
sum = 0
for j=0, j<n, ++j
<
sum = sum + a[i][j]
return sum


int columnsum(int a[ ][10], int m, int j)
sum = 0
for i=0, i<m, ++i
<
sum = sum + a[i][j]
return sum


int matsum(int a[ ][10], int m, int n)
sum = 0
for i=0, i<n, ++i
<
for j=0, j<n, ++j
<
sum = sum + a[i][j]
return sum

The Program

```
/* Computing row sum, column sum and the sum of all the elements of a
matrix using functions */

#include<stdio.h>
#include<conio.h>
main()
{
   int m, n, i, j, a[10][10];      /* Declaring variables */
   void readmatrix(int a[][10], int m, int n);
   void writematrix(int a[][10], int m, int n);
   int rowsum(int a[][10], int n,int i);  /* Function prototypes */
   int columnsum(int a[][10], int m,int j);
   int matsum(int a[][10], int m ,int n);

   clrscr();
   printf("\nRead order of matrix \n");
   scanf("%d%d", &m, &n);
   printf("Order of matrix is : %dX%d  \n", m,n);

   printf("Read elements of Matrix \n");
   readmatrix(a,m,n);     /* Calling function readmatrix() */

   printf("Printing given matrix\n");
   writematrix(a,m,n);     /* Calling function writematrix() */

   printf("\nGiven matrix with row sum and column sum\n\n");
   for(i=0; i<m; ++i)
   {
      for(j=0; j<n; ++j)
         printf("%4d\t", a[i][j]);
      /* Calling function rowsum() to find the sum of elements  of row i */
      printf("| %4d\n", rowsum(a, n, i));

   }

   printf("————————————————\n");

   for(j=0; j<n; ++j)
      /* Calling function columnsum() to find the sum of elements  of
         column j */
```

```
      printf("%4d\t", columnsum(a, m, j));

  /*  Calling function matsum() to find the sum of all elements in
      the matrix */
   printf("\n\nSum of all the elements of the matrix is %d\n",
      matsum(a, m,n));

}



/* Function to compute row sum */
int rowsum(int a[][10], int n,int i)
{
   int j, sum=0;

   for(j=0; j<n; ++j)
      sum += a[i][j];     /* Computing row sum */

   return(sum);           /* Returning row sum back to main()*/
}


/* Function to compute column sum */
int columnsum(int a[][10], int m,int j)
{
   int i, sum=0;

   for(i=0; i<m; ++i)
      sum += a[i][j];     /* Computing column sum */

   return(sum);           /* Returning column sum back to main()*/
}


/* Function to compute sum of all elements of a matrix */
int matsum(int a[][10], int m ,int n)
```

```
{
   int i,j, sum=0;

   for(i=0; i<m; ++i)
      for(j=0; j<n; ++j)
         sum += a[i][j];  /* Computing sum of all the elements */
   return(sum);          /* Returning sum back to main()*/
}
```

8.12 FUNCTIONS TO READ, WRITE AND COMPUTE PRODUCT OF TWO GIVEN MATRICES

The conditions for the *multipliability* of two given matrices and the process of multiplication have been elaborately discussed in Section 6.15.

The Method

In this example, it is assumed that the two given matrices are multipliable. Functions `readmatrix( )` and `writematrix( )` developed in Section 8.10 have been used to input and output the given matrices as well as the *product matrix*. A *user defined function* called `matprod( )` is developed in this section and is used to *compute the product* of the given two matrices. The fact that functions developed by someone else or somewhere could be used has been demonstrated in this example. Here only the `matprod( )` function has been developed. The complete flowchart for the `main( )` and the `matprod( )` are given in the following pages.

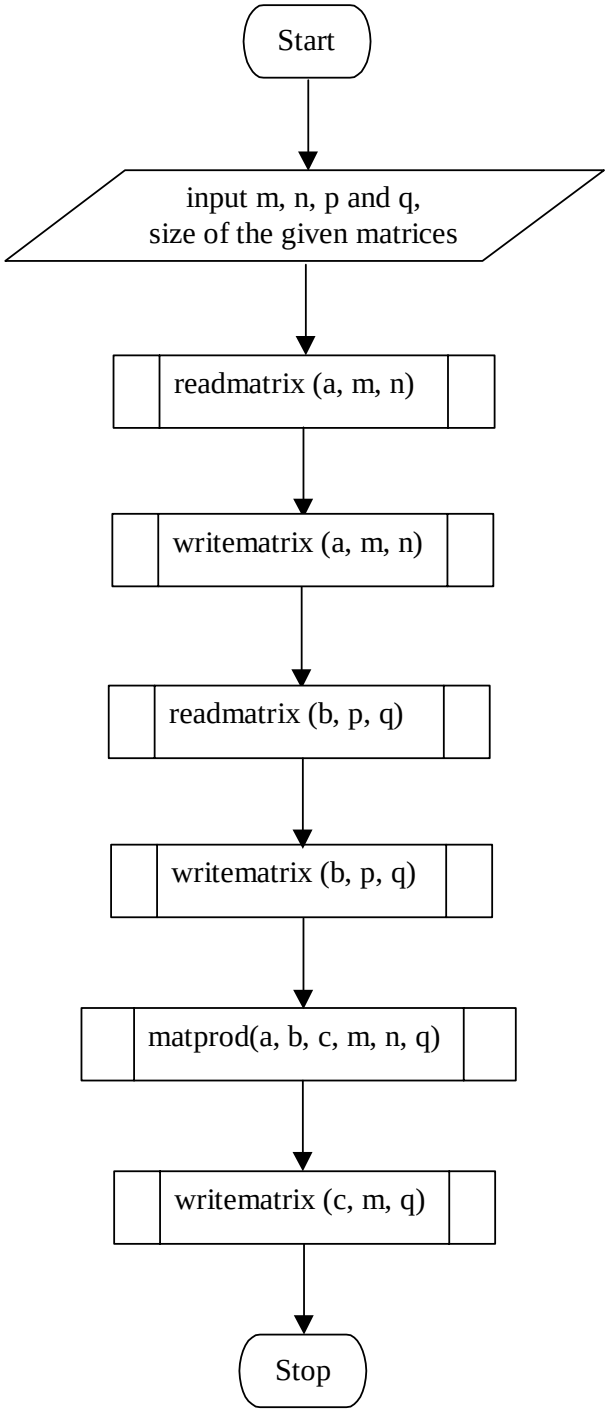
C-Tips

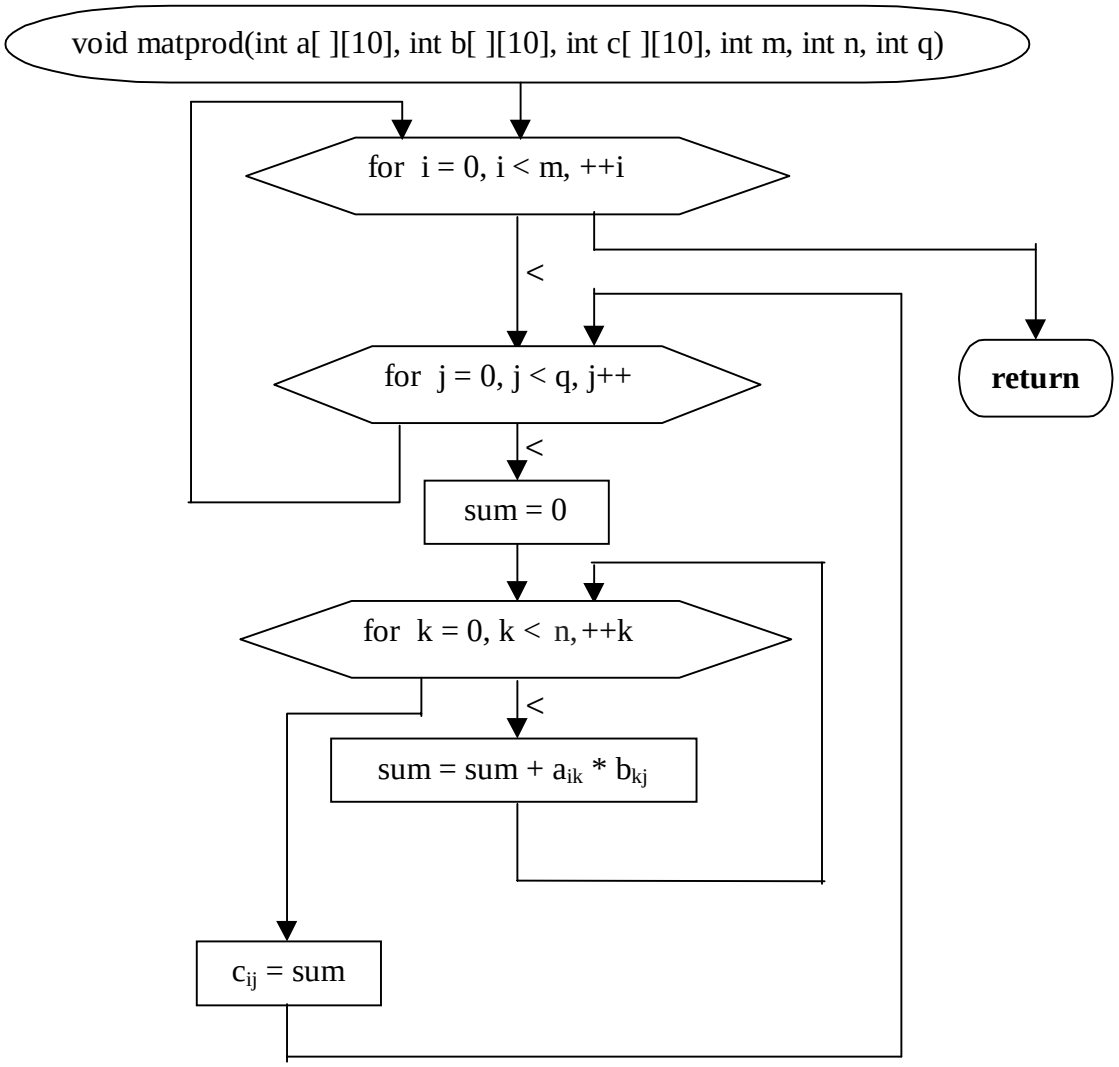
While developing the `matprod( )` function, it is assumed that the two given matrices *a, b* and the product matrix *c*, information about the *number of rows* of the *first matrix*, information about the *columns* of *both* the *first and second matrices* are available. As such, all these have been taken as the *formal arguments* of the `matprod( )` function. This function computes the product of the matrices *a* and *b* and *stores* it in the matrix *c*. Then the *control goes back* to the *calling function without any values*. Therefore the function `matprod( )` has been declared as of type `void`. Thus the function header and the function prototype will be as given below:

```
void matprod (int a[ ][10], int b[ ][10], int c[ ][10], int m, int n, int q)
```

Obviously the values of the *scalar quantities* viz. *m, n,* and *q,* are *transmitted on call,* i.e. by *call by value* method and the *arrays* are *transmitted by implication*.

Flowchart


Start
input m, n, p and q,
size of the given matrices
readmatrix (a, m, n)
writematrix (a, m, n)
readmatrix (b, p, q)
writematrix (b, p, q)
matprod(a, b, c, m, n, q)
writematrix (c, m, q)
Stop



The Program

```
/* Functions to read, write and compute the product of two given
matrices. */
#include<stdio.h>
#include<conio.h>
main()
{
   int m, n, p, q,i, j, a[10][10],b[10][10],c[10][10];
   /* a[10][10], b[10][10], c[10][10] are 2D  arrays, each of size
      10 * 10 */
```

```
   void readmatrix(int a[][10], int m, int n);
   void writematrix(int a[][10], int m, int n);/* Function prototypes*/
   void matprod(int a[][10], int b[][10], int c[][10],int m,int n,
                int q );

   clrscr();

   printf("\nRead the order of first matrix a\n");
   scanf("%d%d", &m, &n);
   printf("\nRead the order of second matrix b\n");
   scanf("%d%d", &p, &q);

   printf("Order of matrix a = %d X %d  \n Order of matrix b = %d X %d
              \n",m,n,p,q);

   printf("\nRead elements of Matrix a\n");
   readmatrix(a, m, n);           /* Calling readmatrix( ) */

   printf("\nMatrix a\n");
   writematrix(a, m, n);          /* Calling writematrix( ) */

   printf("\nRead elements of Matrix b\n");
   readmatrix(b, p, q);           /* Calling readmatrix( ) */

   printf("\nMatrix b\n");
   writematrix(b, p, q);          /* Calling writematrix( ) */

   matprod(a,b,c,m,n,q);          /* Calling matprod( ) */

   printf("\nProduct Matrix c\n");
   writematrix(c, m, q);          /* Calling writematrix( ) */


}

/* Function to compute product */
void matprod(int a[][10], int b[][10], int c[][10],int m,int n,int q)
{

   int i, j, k, sum;

   for(i=0; i<m; ++i)             /* For each row */
   {

      for(j=0; j<q; ++j)          /* For each column */
      {
```

```
            sum = 0;
            for(k=0; k<n; ++k)       /* For each matrix element */
               sum += a[i][k]*b[k][j];
            c[i][j] = sum;
         }
      }
      return;                        /* Return */
}
```

8.13 PROGRAM TO SEARCH FOR REQUIRED ELEMENT IN A GIVEN UNSORTED ARRAY USING FUNCTIONS ON SEPARATE FILE

This example has been discussed to illustrate the meaning of a *module* and hence the *modular programming*.

The method

By carefully studying the given problem one can identify the following activities that could be *performed independently*.

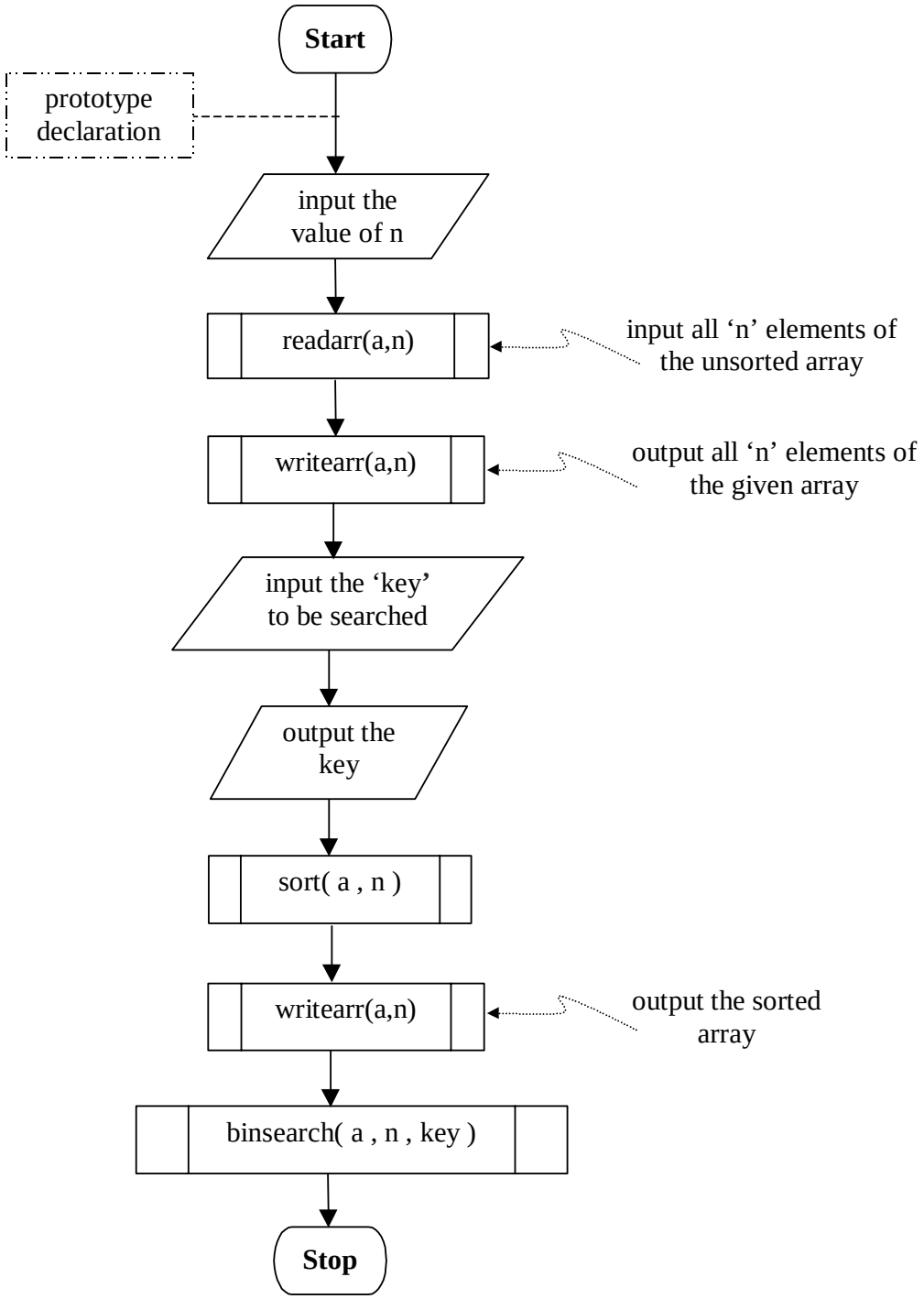
- Reading in a one-dimensional array.
- Writing out a one-dimensional array.
- Sorting an array.
- Searching for a key element in the sorted array.

Independent functions that perform the above mentioned activities have been already developed in Sections 8.6, 8.7 and 8.8 respectively. A `main( )` function has been developed here that uses the previously developed functions to *search* for the required *key* in a given *unsorted array*. A flowchart that depicts the behaviour of the main program is presented in the following page.

C-Tips

All the required functions that have been already developed are put onto a separate file, say `rwss.h`. Now this *file* is made available to every one who works with the same C compiler by storing it in the `bin` directory of the compiler. (At this stage if the reader is not able to understand the meaning of a `bin` directory, do not worry, just proceed.) These separate files are called *modules*. Developing any useful program with the help of one or more modules is known as *modular programming*. Any required *module* is *included* in a main program as usual i.e. with the help of the *preprocessor directive* `#include`. Here the reader is advised to observe *its usage* in *the program* presented at the end of this section.

Flowchart


Start
prototype declaration
input the value of n
readarr(a,n)
input all 'n' elements of the unsorted array
writearr(a,n)
output all 'n' elements of the given array
input the 'key' to be searched
output the key
sort(a , n)
writearr(a,n)
output the sorted array
binsearch(a , n , key)
Stop

The Program

```
/* Program to search for a required element in a given unsorted array
using functions on a separate file. */

#include<stdio.h>
#include<conio.h>
#include “rwss.h”                  /* Module included here */

main()
{

   int i, n, a[20],key;            /* Declaring variables */
   clrscr();
   printf(“\nRead n\n”);
   scanf(“%d”, &n);                /* Value of n is input */
   printf(“n = %d\n”, n);

   printf(“Read elements of array\n”);

   /* Reading elements of array by calling readarr() from rwss.h
      header file */
   readarr(a,n);

   printf(“Elements are \n”);

   /* Printing elements by calling writearr() from rwss.h header
      file */
   writearr(a,n);

   printf(“Enter key element to be searched\n”);
   scanf(“%d”, &key);              /* Reading Key element */
  printf(“key = %d\n”, key);       /* Printing key element */

   /* Calling function sort() from rwss.h header file */
   sort(a, n);
```

```
   printf("Sorted array is:\n");
   /* Printing sorted elements by calling writearr() from rwss.h
      file */
   writearr(a,n);

   /* Calling function binsearch() from rwss.h header file */
      binsearch(a,n,key);

}
```

8.14 UNDERSTANDING THE MEANING OF LOCAL AND GLOBAL VARIABLES

One of the *advantages* that the solving of problems *using functions* has brought in is that different persons can *develop* programs for *different functions* that are parts of a common problem. In such situations, there are every chances that *same variable names* might be used across different functions. Such *usage of common variable names* across different functions *is permitted*. This is because a variable name in a function is *valid* or *alive* only in that specific function. Such variable names that are *valid* or *alive* in which they are declared and used are called *local variables*. However, we need certain variable names to be *valid throughout a program* i.e. across various constituent functions of a single program. Such variable names that are valid across an entire program are called *global variables*. As already mentioned, *local variables* are declared within the *body of a function* in which they are used. However, *global variables are declared* in the beginning of a program i.e. even before the *main( )* function, but just after the *preprocessor directives.*

Global variables provide certain advantages, like a *value* required by *more than one function* can be *propagated* to them, *more than one value* can be returned back to a *calling function* from a *called function* etc.

The Method

In this example a `main( )` function and a user-defined function `illustration( )` without any arguments are developed. `x` and `y` variables have been declared as *global variables*. `x` and `z` variables have been used as *local variables* in `illustration( )`. As it is required to check the validity of values of the variables, they have been output from various parts of the program.

C-Tips

Under certain situations names of *local variables* and some *global variable* names could be *same*. Under such circumstances, *local variables* of a function will be valid within that function. In other words, when there exists a *conflict* between a *local variable* and a *global*

variable, local variable wins over the global variable. However, values of global variables remain intact. This fact is demonstrated in the program.

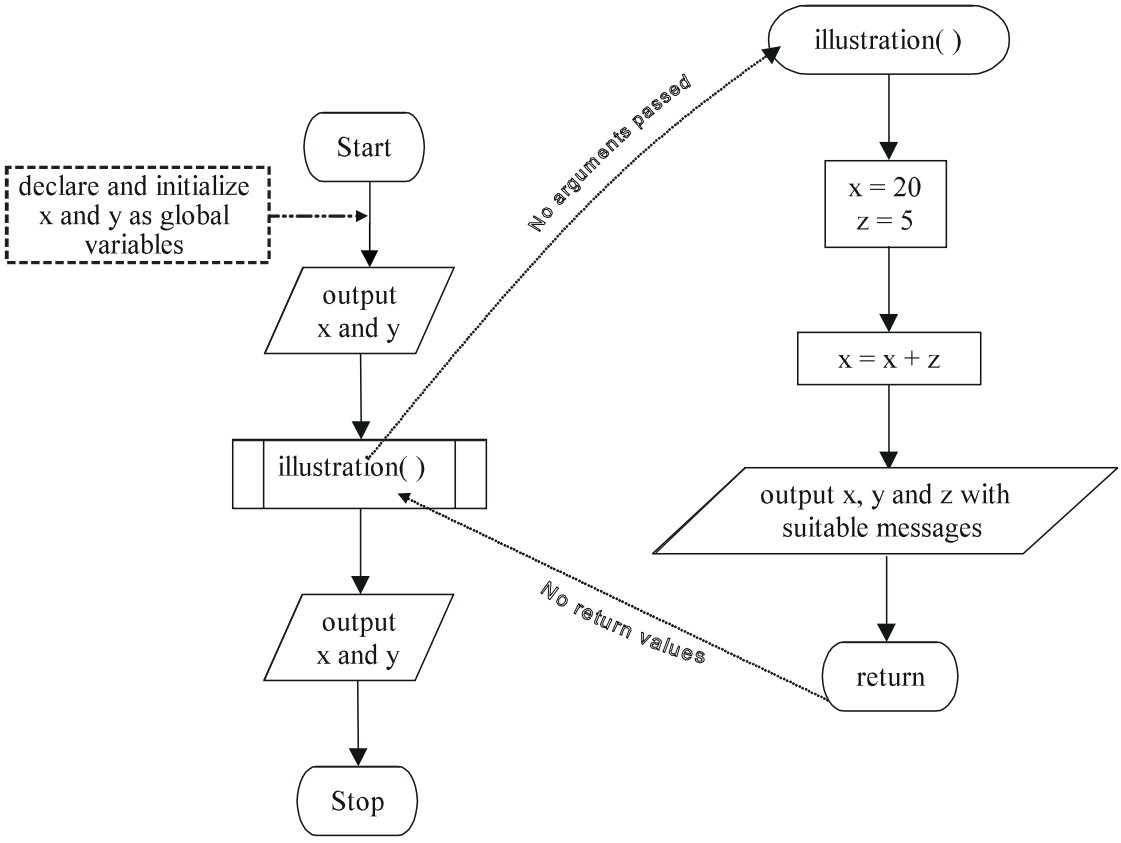
… beyond

Local variables (i.e. auto variables) are always declared within a function and are valid/active only in the function in which they are declared; i.e. their *scope* is limited to that function only.

Global variables (i.e. external variables) are always declared outside a function. Generally they are declared before the `main( )` and their values will be available to all the functions that follow their declarations. Thus, the *scope* of *global variables* extends to all the functions in a program.

In addition to the above two types of variables, C provides two more types of variables called *static variables* and *register variables.*

Flowchart



The Program

```
/* Understanding the meaning of local and global variables */

#include<stdio.h>
#include<conio.h>

/* Declaration and initialization of global variables */
int x=13,y=15;

main()
{
   void illustration();       /* Function prototype */

   clrscr();

   /* Printing the values of x and y before calling the
      illustration() function */
   printf("\nThe values of x = %d & y = %d before the function is
           called. \n",x,y);

   illustration();            /* Calling illustration() */


   /* printing the values of x and y after calling the illustration()
      function */
   printf("\nThe values of x = %d & y = %d after the function is
             called. \n",x,y);

}


void illustration()
{
   int x,z;                   /* Declaring the local variables */
```

```
   x=20;
   z=5;
   x=x+z;

   /* printing the values of variables within the function */
   printf("\nThe values of x = %d , y = %d & z = %d within the function.
          \n",x,y,z);

   return;                    /* Returning */

}
```

8.15 FUNCTION TO SWAP CONTENTS OF TWO VARIABLES—THE WRONG ONE

As explained in Section 2.2 *swapping* of the contents of two given variables is one of the most frequently used activities. As this is a *frequently used activity* one is naturally tempted to develop a *user-defined function* for the same and use it as and when required.

By now the reader is aware of the fact that most of the examples discussed in this chapter are the migrated ones from the earlier chapters. A careful observation of all these examples reveal the fact that while developing functions, the input and output operations are not done, normally. Rather it is assumed that the values for the required variables are available, *which will be obtained during program execution* and if there are any *computed results* they are *returned back* to the *calling program*.

The Method

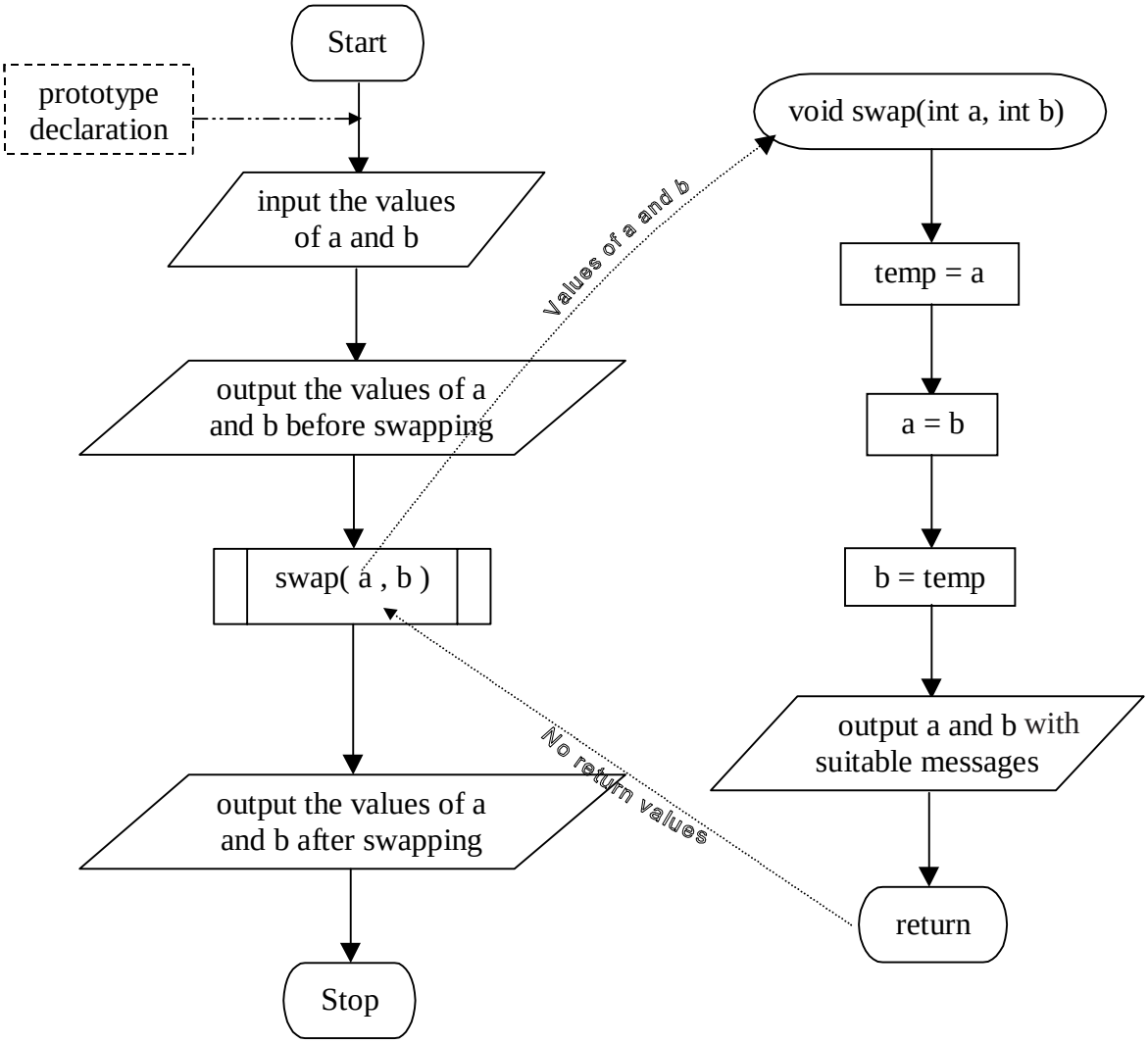
The flowchart and program segments developed in Section 2.2 have been migrated and re-presented here in the form of a *user-defined function* and a `main( )` function. This program does not work, i.e. *the wrong* one. However, the *correct one* is presented in the next chapter on pointers.

C-Tips

It may be observed that variables `a` and `b` have been used as local variables both in the `main( )` function as well as in the `swap( )` function. Since they are local to their respective functions, `a` and `b` in `main( )` and `a` and `b` in `swap( )` will have different memory locations. When `main( )` calls `swap( )`, values stored in the memory locations of `a` and `b` pertaining to `main( )` will be *copied* in to the memory locations of `a` and `b` pertaining to `swap( )`. Here, it should be carefully observed that the `swap( )` function manipulates the contents of the memory locations of its own a and b. In other words, the contents in the memory locations of `a` and `b` of the `main( )` function does not get

manipulated. Thus, when the control goes back to the *calling function,* it finds that the contents of the local variables of `main( )` *have not got swapped*. In other words, the technique of *call-by-value* method has *failed*. In practice such situations are circumvented by using another very popular technique known as *call-by-reference* technique. More about this type of *parameters passing* is discussed in the next chapter.

Flowchart



The Program

```
/* To swap the contents of two variables using function */

#include<stdio.h>
#include<conio.h>

main()
{
   void swap(int a , int b);  /* Function prototype */

   int a,b;

   clrscr();

   printf("Enter the values of a & b\n");
   scanf("%d %d",&a,&b);

   /* printing the values of a & b before swapping */
   printf("\nBefore swapping (in the calling function). \na = %d & b
            = %d\n",a,b);

   swap(a,b);               /* Calling swap() function */

   /* printing the values of a & b after swapping */
   printf("\nAfter swapping ( in the calling function).\na = %d & b =
            %d\n",a,b);


}



/* The swap() function */
```

```
void swap(int a, int b)
{
   int temp;                /* Declaring the local variable */

   /* interchanging the contents of variables a & b */
   temp=a;
   a=b;
   b=temp;

   /* printing the values of a & b within the function */
   printf("\nThe values of a = %d & b = %d within the swap
            function\n", a,b);

   return;           /* Returning back to the main() function */

}
```

EXERCISES

1. Develop flowcharts to print the message "`Programming Techniques using C`" using a function and a main function to print the message using the above developed function. Code the same in C.
2. Develop flowcharts to find the biggest among three given numbers using a function and a main function to print the biggest. Code the same in C.
3. Develop flowcharts to compute real and equal roots, real and distinct roots and complex roots of a given quadratic equation using functions depending upon the three different cases of the discriminant. Also, develop a flowchart to use the above developed functions in a main function. Code the same in C.

   [*Hint:* Use global variables to return the roots to the calling function.]
4. Develop flowcharts to count the number of occurrences of a given integer in a given set of integers using a function and a main function to print out the same. Code the same in C.
5. Develop flowcharts to transpose a given matrix using a function and a main function to output the same. Code the same in C.
6. Develop flowcharts to find the norm of a given matrix using a function and a main function to output the same. Code the same in C.

CHAPTER 9

Pointers

Pointers are one of the most sophisticated features of the C language. A *pointer is just another data type* available in C. Understanding pointers needs the understanding of the meaning of an *address.* Below is given a brief discussion, the study of which helps in understanding the meaning of *addresses* and *pointers*.

We have been using *variables declaration statements* with almost all the functions developed hitherto. It is time to *look closely* to see what happens when a declaration statement is used. For example, consider the statement

```
int temp = 10;
```

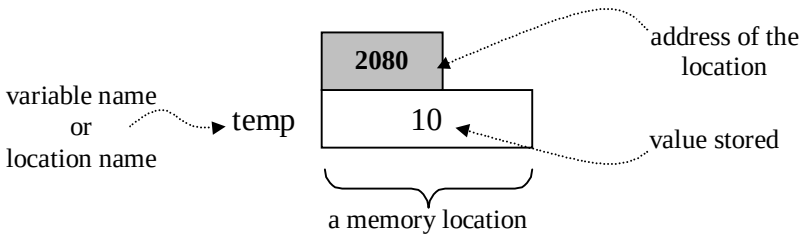
The above statement:

1. *Reserves* a memory location (of 2 bytes size).
2. *Names* the memory location with the name *temp*.
3. Stores an *integer* value 10 in the location.

At the machine level *every byte* of memory will have a *numeric address.* Obviously, the *memory location reserved* during a declaration will have a *numeric address*. Since every memory location will have an address, this *reserved memory* location will also *have an address.* Let this address be 2080. It may be noted that the *address values* will be *unsigned integers*.

The size of the *memory location* reserved depends upon the *data type* that is expected to be stored in it. For example, `char` data type uses 1 byte, `int` data type uses 2 bytes, `float` data type uses 4 bytes and so on. Thus in our discussion a *memory location* means one or more *continuous bytes*. When a memory location having *more than one byte* is dealt with, the *address of the first byte* is taken as the *address of the location*.

The *mutual relationship* between the various attributes of a variable name mentioned above is shown diagrammatically in the following figure.



Knowing address values: the & operator As mentioned in the previous paragraph, *every memory location* will have an *address*. The **&** operator is used to obtain the address of any required memory location. Therefore this operator is also known as the *address operator*. The value of `&temp`, in the example given above, is 2080. Thus the *address* of the memory location with the name `temp` is 2080. In other words, the name of the memory location with an address 2080 is `temp`.

It may be noted that we have been using the **&** operator all along, with `scanf( )`. Thus, during an input operation using `scanf( )` the required memory location is accessed using *its address*.

Pointers: the * operator In its simplest form "a pointer is defined as *something* which indicates or *points* towards *something else.*"



Let **x** and **y** are two variables and **x** has the *address value* of **y** stored in it. Obviously, the variable **x** is *looking-at* or *pointing* towards **y** and hence is known as a *pointer*.

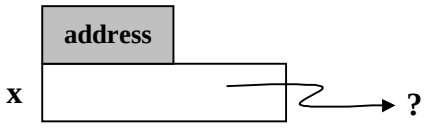
As a variable like **x** could have *address values* of different memory locations stored in it at different points of time, they are also known as *pointer variables*.

In C, a *variable is declared as a pointer* by using the ***** (asterisk) operator. This operator ***** is known as the *indirection operator* or the *dereferencing operator.* Like all other variables, *pointers are also declared* in the beginning. For example, the variable **x** mentioned above is *declared as a pointer* using a declaration statement as shown below

```
int *x;
```

When declaration statement like the one given above is used, the following action takes place.

1. A memory location is *reserved*.
2. It will be of pointer type pointing *no-where.*
3. The memory location is *named* as **x**.
4. This memory location will have an address.



Thus the more formal way of defining a pointer is that "a variable (like x) that holds the address of another variable (like y), using which one can access the contents of the other variable y". The other variable is generally called "the *target* variable" of the pointer.

Thus y is the *target* variable of the pointer variable x. This definition of a pointer can be diagrammatically shown as follows.



It is time to understand the need of a *data type* ***qualifying*** *a pointer*. It is known that the contents of *pointers* are always *address* values that are (unsigned) integers. Then why is that pointers are *type declared ?*: as in int *x; during the declaration of variables. This is because, every pointer will have a *target* variable which can have any *type* of value stored in it. It is the *data type* of this *target* variable of the *pointer* which is used during a pointer declaration. Thus in our example *int* refers to the *data type* of the possible *target variable* of the pointer x.

A *pointer* variable, unless otherwise not initialized, will be always looking at some or the other variable. Whenever a *pointer* variable is *not initialized* it will be pointing *nowhere* (shown with a "?" in the diagrams). It is recommended that a pointer should be initialized. If the user is not sure that where should the pointer be looking at initially, it is customary to make it *to look at* or *point towards* the 0th ($zero^{th}$) memory location. This is done by assigning the NULL to the pointer. A pointer that is initialized with a NULL and is, therefore, pointing to the 0th location is known as a **NULL** *pointer*.

Normally variables within a function access memory locations of the variables within the same function. However, using pointers memory locations of variables in *some other function* can be accessed. Thus pointers help variables to *access more* than what they normally could. Pointers have many number of other uses. Some of them are:

1) they are used in called functions to return *two or more values* to the calling function
2) they are used to access as well as to manipulate the data items of arrays, strings and structures
3) they are used in *dynamic memory management*.

9.1 UNDERSTANDING PRELIMS OF POINTERS

In this example, a number of declaration statements and assignment statements that *extract addresses* of certain variables has been discussed. Different memory locations have been *accessed* both in the *classical way* (i.e. by using names of the memory locations) and the *pointer way* (i.e. by using addresses of the memory locations). The output of the example is also given for verification. A detailed discussion regarding what the various statements in the program are doing has been given at the end.

The Program

```
/* Understanding the prelims of pointers */

#include<stdio.h>
#include<conio.h>
main()
{

   int a = 10;                                          /* Line 1 */
   float b = 5.4;                                       /* Line 2 */
                  /* p is a pointer pointing to an int type data */
   int *p;                                              /* Line 3 */
                /* q is a pointer pointing to a float type data */
   float *q;                                            /* Line 4 */
   clrscr();

                                 /* Assigning address of a to p */
   p = &a;                                              /* Line 5 */

                                 /* Assigning address of b to q */
   q = &b;                                              /* Line 6 */

   printf("The address of a is : %u\n",&a);             /* Line 7 */
   printf("The address of b is : %u\n",&b);             /* Line 8 */

   printf("\nThe address of a as stored in p = %u\n",p);  /* Line 9 */
   printf("The address of b as stored in q = %u\n\n",q); /* Line 10 */

   printf("The value in a = %d\n",a);                  /* Line 11 */
   printf("The value at address in p = %d\n",*p);      /* Line 12 */

   printf("\nThe value in b = %.2f\n",b);              /* Line 13 */
   printf("The value at address in q = %.2f",*q);     /* Line 14 */
}
```

```
----------OUTPUT----------
The address of a is : 65524
The address of b is : 65520

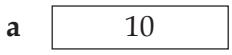
The address of a as stored in p = 65524
The address of b as stored in q = 65520

The value in a = 10
The value at address in p = 10

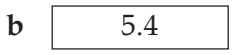
The value in b = 5.40
The value at address in q = 5.40
```

Explanation of the behavior of every statement during the program execution is given below.

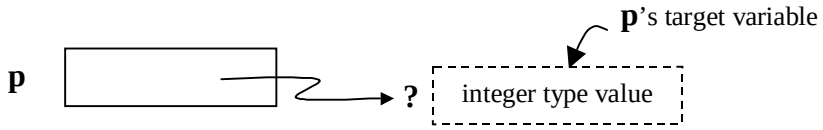
When line 1 is executed a memory location named **a** is allocated. The *data type* of its contents will be *int*eger. An initial value of 10 is stored in it.



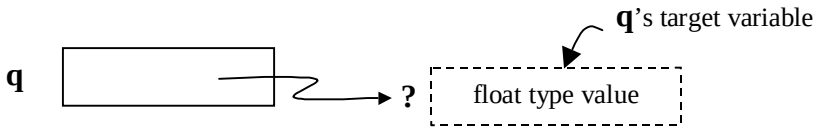
When line 2 is executed a memory location named **b** is allocated. The *data type* of its contents will be *float.* An initial value of 5.4 is stored in it.



When line 3 is executed a memory location named **p** is allocated. **p** being a pointer variable, this memory location can store only unsigned integers that are addresses. As this is uninitialized it could be *pointing* anywhere. However, the *target* data type has to be `int`.



When line 4 is executed a memory location named **q** is allocated. **q** being a pointer variable, this memory location can store only unsigned integers that are addresses. As this is also uninitialized it could be *pointing* anywhere. However, the *target* data type has to be *float.*



When line 5 is executed the address of the *integer* variable **a** is stored in **p**. Therefore, here, **p** is said to be *pointing* towards **a,** i.e. the memory location with the address 65524.



When line 6 is executed address of the `float` type variable **b** is stored in **q**. Therefore, here, **q** is said to be *pointing* towards **b,** i.e. the memory location with the address 65520. Actions carried out at line number 5 and 6 are in-line with the anticipated actions at lines 3 and 4.



When line 7 is executed the address of **a** is *extracted* using the *address operator* (&) and printed out. This address is 65524. When line 8 is executed the address of **b** is *extracted* using the *address operator* (&) and printed out. This address is 65520.

When line 9 is executed the contents of **p** i.e. 65524, which has to be the *address of* ***a,*** is printed out. On verification with the output of the line 7 it is found correct. When line 10 is executed the contents of **q** i.e. 65520, which has to be the *address of* ***b,*** is printed out. On verification with the output of the line 8 it is found correct.

When line 11 is executed the value of **a** is accessed *by name,* as usual, and printed out. When line 12 is executed the value of **a** is accessed *via* ***p,*** i.e. *the pointer* **p** and printed out. On comparison with the output of line 11 it is found correct.

When line 13 is executed the value of **b** is accessed *by name*, as usual, and printed out. When line 14 is executed the value of **b** is accessed *via* ***q,*** i.e. the *pointer* **q** and printed out. On comparison with the output of line 13 it is found correct.

A careful observation of the lines 7 to 10 reveals the fact that we have used the *format specifier* **%u.** It may be noted that **%u** is used to handle *unsigned integer values*. This is because memory *addresses* will be always *unsigned integers.*

Further, observe the different *format specifiers* used in lines 11, 12, 13 and 14. In lines 11 and 12, **%d** is used. This is because the variable **a** is of *type int* and the *target data type* of the pointer **p** is *int.* In lines 13 and 14, **%f** is used. This is because the variable **b** is of *type float* and the *target data type* of the pointer **q** is `float`.

9.2 UNDERSTANDING POINTER EXPRESSIONS, POINTER-TO-POINTER AND NULL POINTER

It is possible to manipulate the contents of pointers. In other words, pointers can be used in certain types of expressions. The only *permitted operators* for such pointer

manipulations are + (plus), – (minus), ++ (increment), –– (decrement), = (assignment) and the *relational operators*. Expressions that involve pointers and additions or subtractions of integers to them and *comparison* of contents of pointers are called *pointer expressions*. For the purpose of illustration certain statements that involve pointer expressions have been considered in this example. The use of the NULL pointer is also shown in this example. The meaning of '*pointer to pointer*' is also demonstrated. Below is given an illustrative example.

The Program

```
/* Pointer Expressions */

#include<stdio.h>
#include<conio.h>

main()
{
   int a = 10, *p, **r, x;                                  /* Line 1 */
   float b = 5.4, *q = NULL, y;                             /* Line 2 */
   clrscr();

   printf("The contents of the pointer q = %u \n",q);       /* Line 3 */
   /* Assigning address of a to the pointer p */
   p = &a;                                                  /* Line 4 */
   /* Assigning address of b to the pointer q */
   q = &b;                                                  /* Line 5 */
   printf("The contents of the pointer p = %u \n",p);       /* Line 6 */
   printf("The contents of the pointer q = %u \n",q);       /* Line 7 */

   p = p + 3;                                               /* Line 8 */
   q ++;                                                    /* Line 9 */
   printf("\nThe contents of the pointer p=%u \n",p);       /* Line 10 */
   printf("The contents of the pointer q = %u \n",q);       /* Line 11 */

   p --;                                                    /* Line 12 */
   q = q - 3;                                               /* Line 13 */
   printf("\nThe contents of the pointer p=%u \n",p);       /* Line 14 */
   printf("The contents of the pointer q = %u \n",q);       /* Line 15 */

   r = &p;                                                  /* Line 16 */
   printf("\nThe contents of the pointer p=%u \n",p);       /* Line 17 */
   printf("The contents of the pointer r = %u \n",r);       /* Line 18 */
```

```
   if(p == q)                              /* Line 19 */
      printf("\np and q are pointing to the same memory location");
   else
      printf("p and q are pointing to different memory locations");

   p = &a;                                              /* Line 20 */
   x = *p;                                              /* Line 21 */
   printf("\nThe contents of the pointer p=%u \n",p); /* Line 22 */
   printf("x=%d\n",x);                                  /* Line 23 */

   r = &p;                                              /* Line 24 */
   printf("x=%d",**r);                                  /* Line 25 */

}
```

Explanation for each of the executable statements used in the above program is given below.

When **line 1** is executed, an integer variable ***a*** with an integer value of 10 is created, a pointer ***p*** with a *target data type* of `int` and *pointing nowhere* is created, a 'pointer to pointer' with a *target data type* of *int*eger and *pointing nowhere* is created and an uninitialized integer variable ***x*** is also created. When **line 2** is executed, a `float` type variable ***b*** with an initial float value of 5.4 is created, a null pointer ***q*** with a *target data type* of `float` and pointing towards the 0th (zeroth) memory location is created and an uninitialized `float` type variable ***y*** is created. When **line 3** is executed, the value printed out will be the *address* of the location pointed to by ***q*** which is 0 (zero). This is found to be the anticipated result. When **line 4** is executed, the address of the variable ***a*** is extracted and assigned to the pointer ***P***. When **line 5** is executed, the address of the variable ***b*** is extracted and assigned to the pointer ***q***.

When **line 6** is executed, the contents of the pointer ***p,*** i.e. the address of the variable ***a*** (65524) is printed out. When **line 7** is executed, the contents of the pointer **q,** i.e. the address of the variable ***b*** (65516) is printed out. When **line 8** is executed, the contents of ***p*** (which is an address) is increased by (3x size of the location) 6 and is assigned to ***p***. Thus the contents of ***p*** now has to be (65524 + 6) 65530. When **line 9** is executed, the contents of ***q*** is incremented by 1-memory location size, i.e. 4 bytes. Thus the new contents of ***q*** must be 65520. When **line 10** is executed, the recently stored contents of the *pointer* ***p,*** i.e. 65530 is printed out. This is found to be correct.

When **line 11** is executed, the recently stored contents of the *pointer* ***q,*** i.e. 65520 will be printed out. This is found to be correct. When **line 12** is executed, the contents of the *pointer variable* ***p*** will be decreased by 1-memory location size, i.e. 2 bytes. Thus the new contents of ***p*** must be now 65528. When **line 13** is executed, the contents of the pointer variable ***q*** will be decreased by 3-memory location size, i.e. 12 bytes. Thus the new

contents of ***q*** must be now 65508. When **line 14** is executed, the recently stored contents of the pointer ***p,*** i.e. 65528 is printed out. This is found to be correct. When **line 15** is executed, the recently stored contents of the pointer ***q,*** i.e. 65508 is printed out. This is found to be correct.

When **line 16** is executed, the address of the pointer ***p*** is stored in the "pointer to pointer" ***r***. When **line 17** is executed, the contents of pointer ***p*** is printed out (65528). This is found to be correct. When **line 18** is executed, the contents of "pointer to pointer" ***r*** is printed out. This has to be the address of the pointer ***p***. Comparison for equality of two pointers is being made in **line 19**. Addresses of ***p*** and ***q*** being different, the printed out result is correct. When **line 20** is executed, the address of the variable ***a*** is extracted and assigned to the pointer variable ***p***.

When **line 21** is executed, the contents of the *target* location pointed by ***p,*** i.e. the variable ***a*** is assigned to the variable *x*, i.e. ***x*** value becomes 10. When **line 22** is executed, the contents of the pointer ***p,*** i.e. the address of ***a*** is printed out. This is 65524 and is found to be correct. When **line 23** is executed, the value of ***x*** is printed. This is found to be correct. When line 24 is executed, the address of the *pointer* ***p,*** i.e. 65522 is assigned to the "pointer-to-pointer" ***r***. When **line 25** is executed, the value of the *target* location (*i.e.* ***a***) of the "pointer-to-pointer" ***r*** is printed out. This is found to be correct.

9.3 FINDING SUM OF ALL ELEMENTS OF ARRAY USING POINTERS

The Method

First all the '*n*' elements of the array, say **a**, is *read-in*. The *sum* of all the elements of the array is obtained by *repeatedly adding* every element of the array to the *sum* using a statement of the form `sum = sum + *p;` where **p** is a *pointer variable*. This pointer variable **p** should be *looking at* or *pointing-to* the *successive memory locations* of the array, one after the other, starting from the first one to the last one. The pointer **p** is made to *look-at* the successive memory locations by *adding* `a 1` to its contents, i.e. by incrementing its contents. Here a `for` loop structure has been used for adding the contents of the memory locations of the array as well as to *point* towards successive memory locations. The *sum* obtained is output finally.

C-Tips

Variable **p** is declared as a *pointer* in the declaration statement. *Address of the first element* of the array may be assigned to ***p*** by using a statement of the form `p = &a[0];` or a statement of the form `p = a;`. Using the statement of the *former type* is obvious. However, the *latter type* statement is also permitted because as soon as an array **a** is declared, the C compiler *assigns* the *first address* of the allocated *memory area* to its *name,* i.e. to **a**, here.

Pointer **p** itself has been used as the *loop control variable.* Obviously its *initial value* has to be the *address of the first location,* i.e. the contents of the array name **a** or *address of* `a[0]`, the *upper limit* has to be the *last address* of the allocated memory area, i.e. `(a + n)`, and the *next address* is obtained by incrementing the value of **p**.

The *beginning address* of an array or the *address of an array* can be obtained either *by* using the *array name* directly or by using the address-of-operator (&) along with the name of the *first location,* i.e. `&a[0]`.

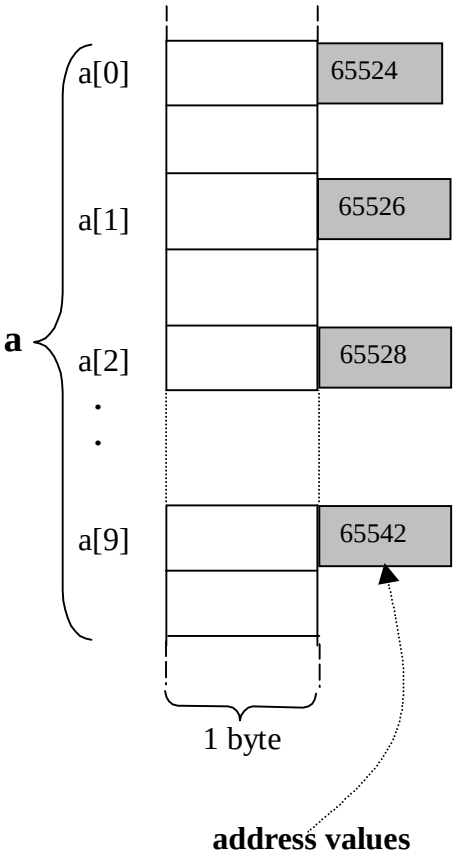
With many number of examples involving arrays, we have seen that the *arrays are transmitted* between different functions by using *only the array names,* i.e. without the use of any additional effort by the user. Since *array names* hold the *first address* of their memory area, when *array names* are used as *arguments* of functions, their beginning address will be *handed over* to the *called function*. This *automatic transfer* of *array addresses* which is equivalent to the classical *call-by-reference* technique is generally referred to as *call by implication*.

Further, it may be noted that the *size of every memory location* and hence *the total number of bytes* allocated for an array depends on the *data type* of the elements in the array. Thus, if an array of `char` data type with 10 elements is declared, then the *size* of every memory location will be *1 byte* and the total number of bytes allocated to the array will be 10 bytes. Similarly, if an array of `float` data type with 10 elements is declared then the *size* of every memory location will be *4 bytes* and the total number of bytes allocated to the array will be 40.

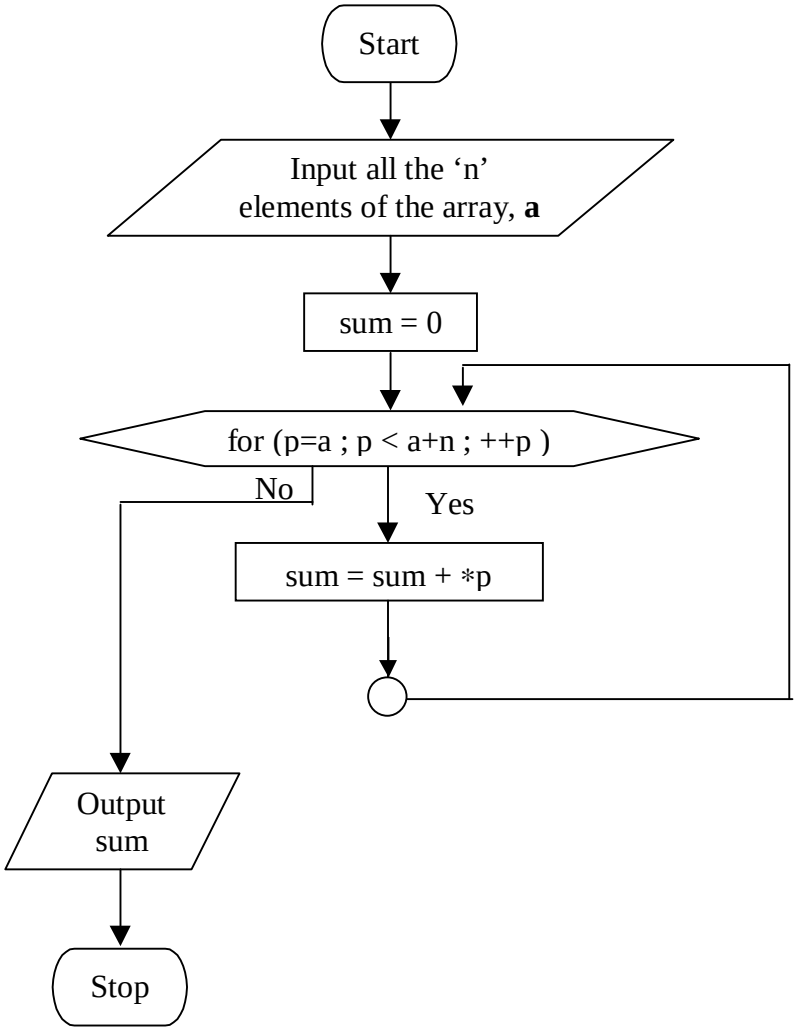
… beyond

When a declaration statement like `int a[10];` is declared, 10 continuous *memory locations* will be allocated to store all the 10 integer elements of the array **a**. The *size* of each memory location depends upon the *data type* of the elements to be stored in these locations. Note that in an array all the elements will be of same *data type*. Here, as the *data type* of the array is `int`, each location will be 2 bytes long. Thus the memory area reserved for the array **a** will be 20 bytes long, as shown in the adjoining figure.

Let the address of the first memory location, i.e. `a[0]` be 65524. Then the address of the second memory location `a[1]` will be 65526, the address of the third memory location `a[2]` will be 65528 and so on. It is important to note that the contents of an array name, like **a,** without any subscript will be the *first address* of the *memory area* reserved for all its elements. Thus in the above example, the *value* of **a** will be 65524, same as that of `& a[0]`.



Flowchart



The Program

```
/* Program to find the sum of all the elements of an array using
pointers */

#include<stdio.h>
#include<conio.h>
main()
{
```

```
   int i, n, sum=0, a[10];                /* Declaring variables */
   int *p;                        /* Declaring a pointer variable */
   clrscr();
   printf("Read the value of n : ");
   scanf("%d", &n);
   printf("%d\n",n);

   printf("\nThe address of a = %u\n",&a[0]);          /* Line # */
   printf("The value of a = %u\n\n",a);               /* Line # */

    printf("Read the elements of the array \n");
   for(i=0; i<n ;i++)
   {
      scanf("%d",&a[i]);              /* Reading array elements */
}

   printf("\nElements are \n");
   for(i=0; i<n ;i++)
   printf("%d\n",a[i]);             /* Printing array elements */

   for(p=a; p<a+n ; ++p)
   sum += *p;                                      /* Finding sum */

   printf("\nSum = %d",sum);

}
```

Note: Lines with comments `/* Line # */` have been included in order to demonstrate the fact that the address stored in the name **a** will be same as that of the element **a[0]** .

```
————————OUTPUT————————
Read the value of n : 4

The address of a = 65458
The value of a = 65458
```

```
Read the elements of the array
23   12   -12   4
Elements are
23
12
-12
4

Sum = 27
```

9.4 FUNCTION TO SWAP CONTENTS OF TWO VARIABLES USING POINTERS

Unusually a program that *does not work* was discussed in Section 8.15. The intention of the discussion there was to make the reader understand why the function `swap( )` *failed?*, whereas *it worked* when it was developed as a (main) *program* in Section 2.2. At this stage the reader is advised to *browse* those examples *once again* and try to find out the reasons.

The Method

For this example to work, we must see that the concerned variables in the `swap( )` function *directly access and work* on the relevant memory locations in the `main( )` function. This is achieved by *passing on the address* of the concerned variables from the `main( )` to the corresponding variables in the `swap( )`. Thus the concerned variables in the `swap( )` function should be holding address values, and hence **pointing** towards the *appropriate memory locations* in the `main( )`.

As soon as the *called function* is recognized, the *address values* of **a** and **b** are transmitted and made as the *contents* of the *pointers* **p** and **q**. As a result, a sort of *coupling* takes place between the *called function* and the *calling function*. Thus *swapping* (or any other activity) done with the *pointers* inside the *called function* `swap( )` actually takes place in the memory locations of the *calling function* and hence *this program works*. Notice that the `temp` is a *local variable* in the *called function* and the actual swapping is managed with the *variables* **a**, **b** {in `main( )`} and `temp` in `swap( )`. As this `swap( )` function *does not return any value* its type has been declared as *void*.

…beyond

When developed as a `main( )` function, the *swapping* activity was a *success* because the function was *manipulating* the contents of the *memory locations* of its *local variables* ***a*** *and* ***b*** *directly*. However, when developed as a user-defined function (as in Section 8.15), the *swapping* was a *failure* because *memory locations* of variables **a** and **b** *in the* `main( )` and the *memory locations* of the variables **a** and **b** *in the user-defined function* `swap( )` were

different. This is because **a** and **b** in `main( )` are *local to main( )* and **a** and **b** in *swap( )* are *local to* `swap( )`. Memory locations for the *local variables* of a function will be *allocated* inside that function itself and are *directly accessible* only *within that function*. As a result when the contents of **a** and **b** inside the *swap( )* were *exchanged* only memory locations of **a** and **b** pertaining to `swap( )` got *manipulated*. In other words, the memory locations of **a** and **b** in *main( ) remained unaltered*. Hence the failure.

The above *reason for failure* can be circumvented by providing an ability to the variables inside the `swap( )` function {called function} to access the corresponding variables inside the `main( )` {calling function} and *work on those variables directly*. The required *accessing ability* to the variables inside a *called function* can be granted by providing the *address of the memory locations of the variables in a calling function {*`main( )`*} to the corresponding variables in the called function {*`swap( )`*}.*

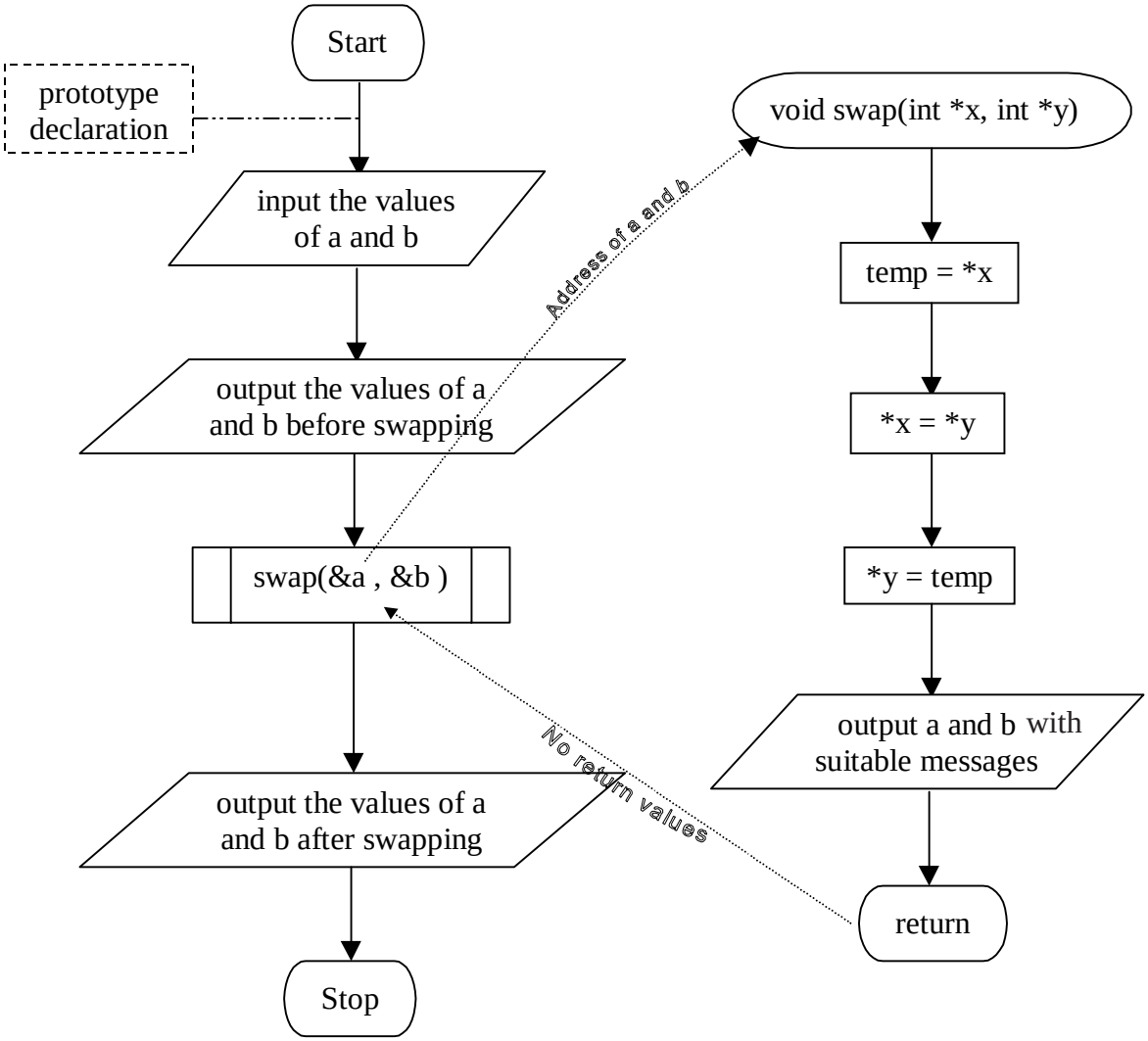
In the present example, inside the `swap( )` function, *corresponding values* of the *local variables* **a** and **b** of `main( )` are **x** and **y**. As these variables **x** and **y** of `swap( )` should be able to *access* the memory locations of the variables **a** and **b** of `main( )` they should *hold the addresses* of **a** and **b,** respectively**.** Obviously, variables **x** and **y** have to *hold address* values of **a** and **b** and hence should be *pointer variables.* Thus, the method used in solving this example is to *pass* "*the address of the variables*" from the *calling function* to the *called function* and "*store them in pointer variables*." This enables the *called function* to *work directly* on the memory locations of the *calling function*. This technique of *invoking* or *calling* a function by *passing address values*, storing them in "pointer variables" and hence providing the *ability* to these variables to access corresponding variables in the *calling function* is known as **call by address** or **call by reference**. The entire mechanism that depicts the *address flow* and *direct accessing through pointers* is schematically shown in the next page.

A study of the schematic diagram given in the next page gives an insight to the *mechanism* involved in the *call by address* (i.e. call by reference) method of passing the parameters. When the function `swap(&a, &b)` inside the `main( )` is executed the *control* goes to the `swap( )` function, *recognizing* it *by name.* Along with the control, the *addresses of* ***a*** *and* ***b*** *also get transmitted* to the `swap( )` function and become the values of the pointers **x** and **y** respectively. It is because address values are to be passed on, the *actual arguments* are **&a** and **&b**, rather than **a** and **b**. As *address values* arrive at the *formal arguments* of the *called function*, these *formal arguments* must be *pointers*. This is why the *formal arguments* of `swap( )` have been taken as pointers **x** and **y,** i.e. as ***x** and ***y**. As the contents of the *target* memory locations of both these pointers are of *integer* type, the *qualifying type* of both the pointers ***x** and ***y** have been taken as of type `int`.

Note: **1**,**2** and **3** indicate the order of action during swapping

Schematic diagram showing the mechanism of the flow of address and accessing the memory locations in a different function.

Flowchart



The Program

```
/* Function to swap the contents of two variables – Using Pointers */

#include<stdio.h>
#include<conio.h>

main()
```

```
{
   void swap(int *x, int *y);              /* Function prototype */
   int a,b;                                   /* Local variables */
   clrscr();

   printf("Enter the values of a & b\n");
   scanf("%d %d",&a,&b);

   /* printing the values of a& b before swapping */
   printf("\n Before swapping (in the calling function). \na = %d & b
           = %d\n",a,b);


   swap(&a,&b);                        /* Calling swap() function */

   /* printing the values of a & b after swapping */
   printf("\n After swapping ( in the calling function).\na = %d & b
           = %d\n",a,b);

}



/* The swap() function */

void swap(int *x, int *y)
{
   int temp;                     /* Declaring the local variable */

   /* interchanging the contents of variables a & b */

   temp=*x;/* Contents of the target location    of x is put in temp */
   *x=*y;                        /* Contents of the target location
                                            y is made as the contents
                                     of the target location of x */
```

```
   *y=temp;                    /* Contents of temp is put into the
                                          target location of y */

   /* printing the values of a & b within the function */

   printf("\n The values of a = %d & b = %d within the called
        function\n",*x,*y);

   return;      /* Control going back without any return values! */

}
```

9.5 SOLVING QUADRATIC EQUATION—USING FUNCTIONS AND POINTERS

The procedure involved in solving a given quadratic equation has been already discussed, in detail, in Sections 3.6 and 5.1. The reader is advised to browse through those sections once again for better understanding.

The Method

The *procedure* discussed in Sections 3.6 and 5.1 is used here once again. Here, three functions `quad_re( )`, `quad_rd( )` and `quad_comp( )` have been developed. The function `quad_re( )` *has been developed to compute* the *real* and *equal roots*. The function `quad_rd( )` *has been developed to compute* the *real* and *distinct roots*. The function `quad_comp( )` *has been developed to compute* the *real* and *imaginary parts* of the *complex roots*. All the functions have been developed assuming that the co-efficients **a** and **b** of the equation, are available. Obviously, **a** and **b** have been taken as the *formal arguments* of all the functions developed here. Computations of *real* and *distinct roots* and *complex roots* require the value of the discriminant `disc`. Therefore, `disc` has also been taken as *one of the formal arguments* while developing `quad_rd( )` and `quad_comp( )` functions. The *two roots* computed inside the functions are returned back to the `main( )`, by using two `float` type *pointers* **p1** and **p2**. Thus all the functions have the pointer type variables **p1** and **p2** as their *formal arguments*. (Here, one has to carefully notice that why the last two formal arguments have been taken as `float *p1` and `float *p2)`. Thus the *function headers* and the *function prototypes* of the functions developed here are

```
void quad_re(float a,  float b,  float  *p1, float *p2)
void quad_rd(float a, float b,  float disc, float *p1,  float *p2)
void quad-comp(float a, float b, float disc, float  *p1, float *p2)
```

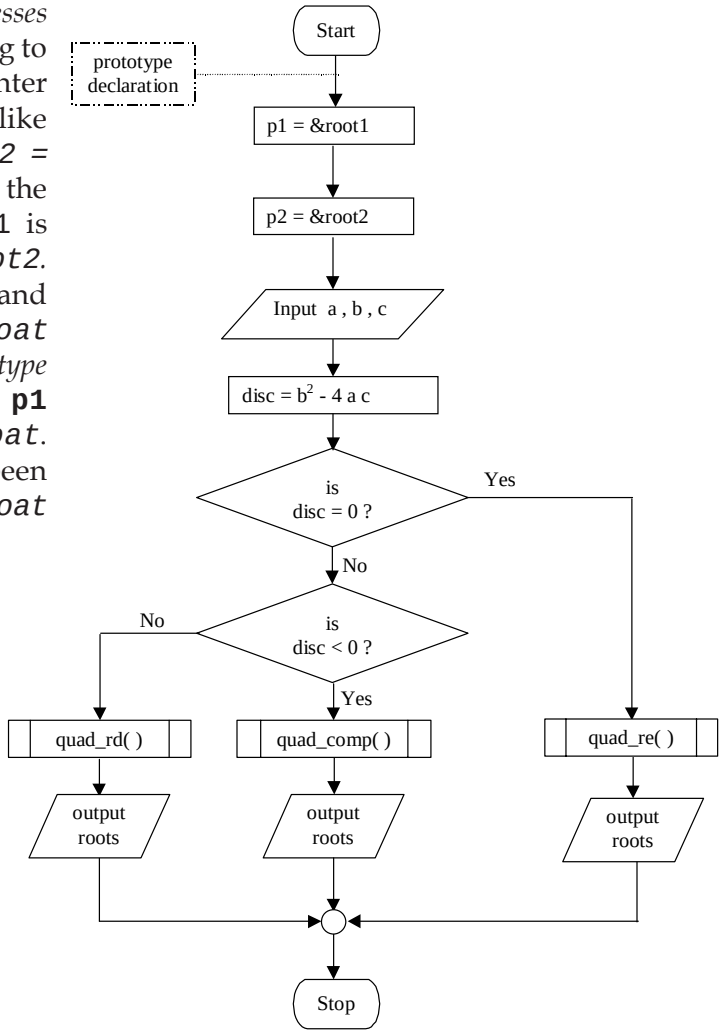
A `main ( )` function is developed in which after inputting the co-efficients *a, b* and *c* the discriminant `disc` is computed. Depending upon whether this `disc` is equal to 0 (zero), is +ve or –ve, one of the functions `quad_re( )`, `quad_rd( )` and `quad_comp( )` is *invoked* or *called*. After getting the required computed values for the *roots*, the roots are printed out. Flowcharts for all the functions as well as the `main( )` function are given in this page as well as the next page.

C-Tips

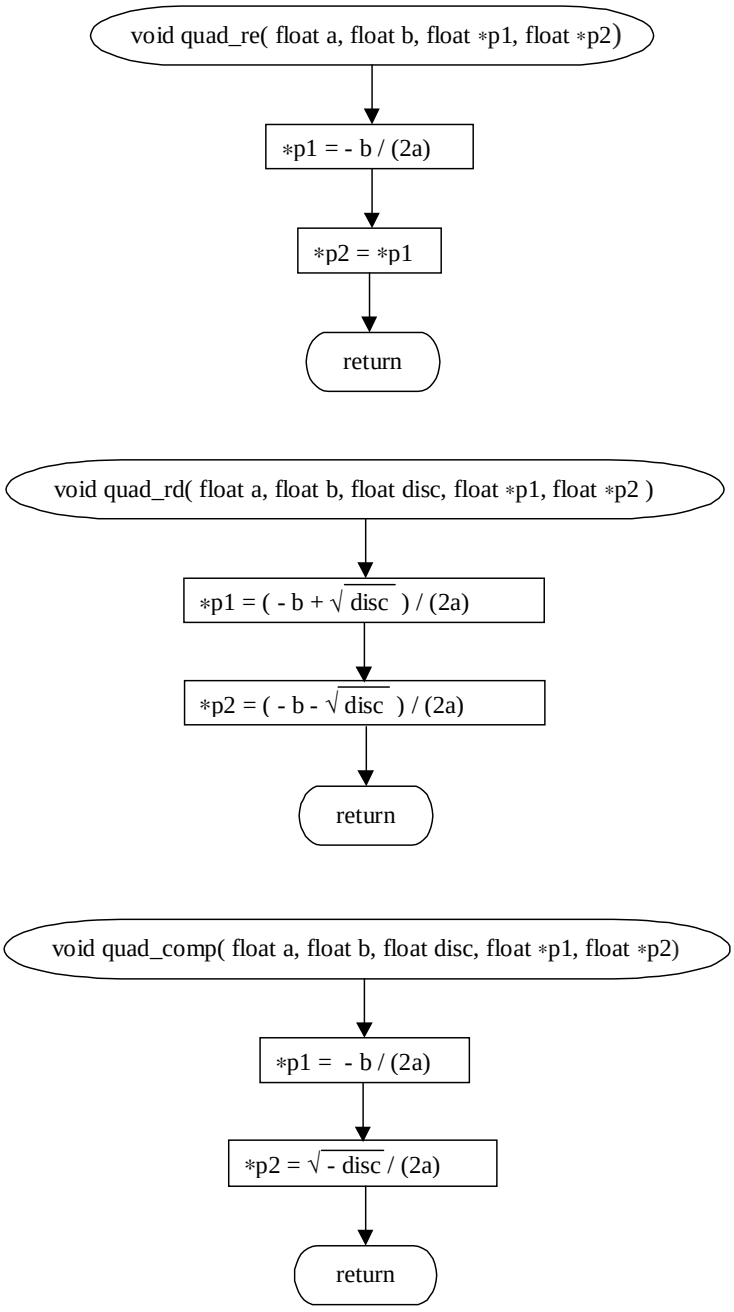
As functions cannot *readily return more than one value* to their *calling functions*, pointers have been used. **p1** and **p2** are the two pointers used. These pointers are expected to *look at* or *point* towards the memory locations of `root1` and `root2` in `main( )`. This is accomplished by storing the *addresses* of the memory locations pertaining to `root1` and `root2` in the pointer variables, using the statements like `p1= p1 = &root1;` and `p2 = p2 = & root2;` in the `main( )`. Thus the *target* variable of the pointer `p1` is `root1` and of the pointer `p2` is `root2`. As these *target* variables `root1` and `root2` are expected to store `float` type values in them, the *qualifying type declaration portion* of the *pointers* **p1** and **p2** have been taken as `float`. Thus the *two pointers* have been declared as: `float *p1;` and `float *p2;`.

When any of the functions to compute roots developed here is *invoked* or *called*, the *values of* `a`, `b` and `disc` are passed on to the *called function by-value* and the *addresses* of `root1` and `root2` are made available via the pointers **p1** and **p2,** respectively. Thus, it may be observed that both the techniques, viz. *call-by-value* and *call-by-reference* are being used simultaneously in this example. This is legal and

Flowchart



permitted. A careful observation reveals the fact that the computed *roots* will be *directly available* in the `main( )` function. In other words, no values are returned to the `main( )` by any of the *user defined functions*. Hence, all the functions `quad_re( )`, `quad_rd( )` and `quad_comp( )` have been declared as of type `void`.



The Program

```
/* To find the roots of a given quadratic equation using pointers */

#include<conio.h>                    /* Including  header files */
#include<stdio.h>
#include<math.h>

main()
{
   float a,b,c,disc, root1, root2;       /* Declaring variables */
   float *p1, *p2;               /* Declaring pointers p1 and p2 */

   /* Function prototypes*/
   void quad_re(float a, float b, float *p1, float *p2);
   void quad_rd(float a, float b, float disc, float *p1, float *p2);
   void quad_comp(float a, float b, float disc, float *p1, float *p2);

   clrscr();                              /* Clearing the Screen */

   p1 = &root1;                          /* Initializing pointers */
   p2 = &root2;

   printf("\n\nEnter co-efficients of the quadratic equation\n\n");
   scanf("%f%f%f", &a, &b, &c);
   printf("\nCo-efficients are a = %f\t b = %f\t c = %f\n\n", a,b,c);

   disc=b*b-4*a*c;                 /* Calculating discriminant */

   if ( disc == 0)
             /* If discriminant is zero roots are real and equal */
     {
       quad_re (a,b, p1, p2);                  /* Function call */

       printf("\nRoot1 = %f\t Root2 = %f\n",root1,root2);
     }
```

```
   else
     {
       if ( disc < 0)
           /* If discriminant is less than zero roots are complex */
     {

       quad_comp(a, b, disc, p1, p2);          /* Function call */
       printf("\nRoot1 = %f + i %f\tRoot2 = %f -i %f\n", root1, root2,
       root1, root2);
     }
   else
       /* If discriminant is greater than zero roots are real and
           distinct */
     {
       quad_rd(a,b, disc, p1, p2);             /* Function call */

       printf("\nRoot1 = %f\t Root2 = %f\n",root1,root2);
     }
   }
}
/* Function to find the real and equal roots of equation */

void quad_re(float a,float b, float *p1, float *p2)
{
   printf("Roots are real and equal\n");
   *p1 = -b/(2.0*a);
   *p2 = *p1;
   return;
}



/* Function to find the real and distinct roots of equation */

void quad_rd(float a,float b, float disc, float *p1, float *p2)
{
```

```
   printf("Roots are real and distinct\n");
   *p1 = (-b+sqrt(disc))/(2.0*a);
   *p2 = (-b-sqrt(disc))/(2.0*a);
   return;
}



/* Function to find the complex roots of equation */

void quad_comp(float a,float b, float disc, float *p1, float *p2)
{
   printf("Roots are complex\n");
   *p1 = -b/(2.0*a);
   *p2 = sqrt(-disc)/(2.0*a);
   return;
}
```

EXERCISES

1. Write a C program with three array variables each of *int*, *float* and *char* data types, each having 5 data elements that prints out the addresses of each memory location in which the different elements of the declared arrays are stored.
2. Write a C program having two variables of different data types and to print the values of these variables without using pointers, using pointers and using pointer-to- pointers.
3. Draw a flowchart to develop a function that finds the *highest* and the *second highest* among a given set of *float* type of data elements and returns them to a calling function {say, main( )}. Code the same using C.
4. Draw a flowchart to develop a function that counts the number of positive elements, the number of negative elements, computes the average of all the elements of a given array and returns the same to a calling function. Code the same using C.

   [*Note:* Consider zero (0) element as positive.]

CHAPTER 10

Structures

Structures are one of the very important features available in C. These are chiefly used to build various types of *data structures* that are generally used in the domain of computers. In this chapter the meaning of structures, defining a structure, declaring structure variables and using structure variables are discussed with the aid of some simple examples.

Things that exist around us in reality are called *entities*. These entities are identified by a *name* and will have a number of *properties* associated with each of them. A book, a student, a car, a date are all some examples of entities. Some of the *properties* or *attributes* associated with each one of these *named entities* are given in the following table.

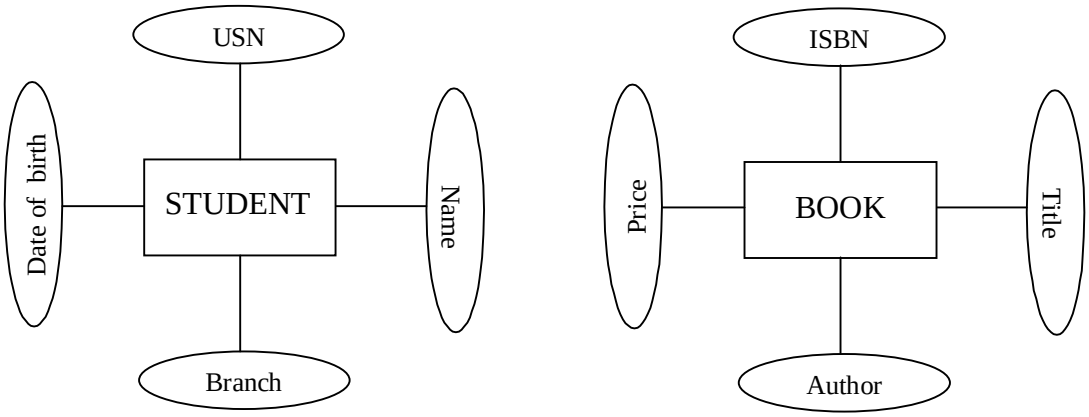
Sl.No.	*Entity name*	*Entity Properties/Attributes*
1	Student	University seat number (USN). Name. Branch of study. Date of birth. Subjects of study. ... ...
2	Book	International standard book number (ISBN). Title. Author. Publisher. Price. ... ...

(Table Contd.)

(Table Contd.)

Sl.No.	*Entity name*	*Entity Properties/Attributes*
3	Table	Material. Size. Number of legs. Cost. ... ...
4	Date	Day. Month. Year. ... ...

The relationship between the entities and their attributes can be diagrammatically represented as shown below. Such diagrams are generally called *entity relationship diagrams* (*ER-Diagrams*).



Processing of these *entities* or *objects* using computers need the *entities* and *their attributes* to be written in *some compact form* having certain *definite pattern*. For example, the entities *student* and *book* can be represented in a *compact form* as shown below.

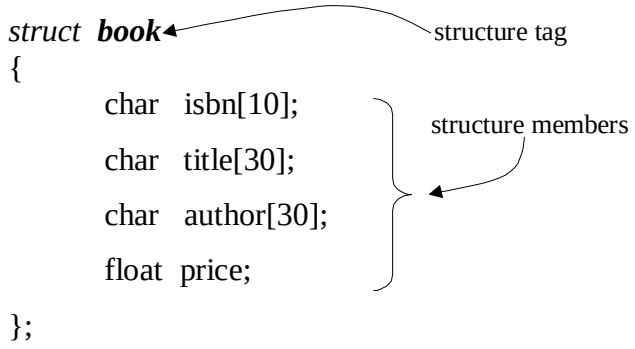
STUDENT (usn, name, branch of study, date of birth)

BOOK (isbn, title, author, price)

It may be observed that an entity representation, as given above, describes its *physical layout* or *format*. These physical layouts or formats of entities with which all the attributes are put together under a common name are referred to by different terminologies in

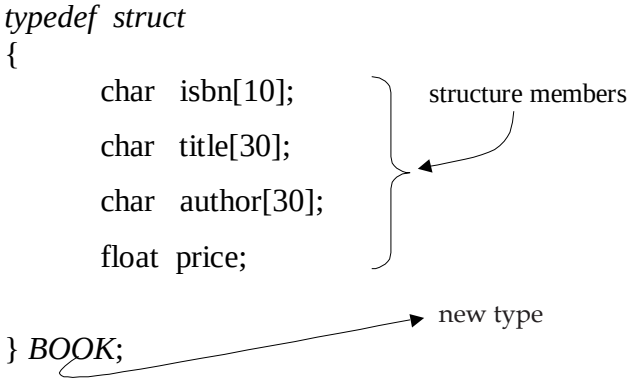
different computer languages. For example, they are called *records* in COBOL, *objects* (of course, along with some procedures attached to them) in C++, *structures* in C and so on.

Thus, in C, a *structure* is defined as the *compact form* representation of an *entity* along with all or some of its *attributes*. For example, the structure of the *entity* ***book*** may be defined as shown in the following figure.



Obviously, the *name* of the above structure is *book*. Often, *name of a structure* like ***book*** is also referred to as the *structure tag* or simply *tag*. As shown in the above figure, *all the attributes* are listed inside a pair of *curly braces* immediately after the *structure name* or the *structure tag*. Every attribute is represented as a *variable name* of some suitable *data type* and *size*. These variable names of the attributes inside the curly braces of structure are called the *members* or the *elements* of a structure. The use of the C keyword `struct` in the beginning and a *semicolon* (;) at the end of a structure definition are *compulsory*. The use of *structure tag* is *optional*.

A structure may, however, also be defined using the C keyword `typedef` as shown below



The structure definition as above creates a *new data type* called *BOOK*. For the sake of convenience, generally the *new data types created* using the *typedef* facility are written using only uppercase alphabets.

Structures are like arrays in that they both group a number of individual elements into a single larger unit. But while a structure usually groups *items of different types*, an array groups *items of the same type*. More importantly, the members of a structure are *accessed by their name* (as will be explained very shortly) while the elements of an array are *accessed by an index number,* i.e. the *subscript*. Another important point to be noted here is that like arrays, structures are also *transmitted* or *passed-on* between different functions by their *starting addresses,* i.e. by using the *call-by-implication* technique. It is interesting to note that we can have *arrays of structures* (i.e. arrays whose elements are structures, as in Section 10.4) and *structures with arrays* as its members.

Before using the variables of the *structure type* (i.e. *structure variables*), structures must be defined. A structure definition may appear either in the *beginning of a function* in which it is used or in the *beginning of a program* immediately after the pre-processor directives, just like global variables.

A mere structure definition describes just a *template,* i.e. the shape of the structure. It will *not allocate any memory area. Memory area will be allocated* only when structure type variables are *declared.*

The required *structure variables* are declared either using or without using the *name* or *tag* of the structure definition. Let `book1` and `book2` be two required structure variables. These two structure variables can be declared in any of the following ways.

A
```
struct
{ ..........
  ..........
  ..........
} book1, book2;
```

B
```
struct book
{ ..........
  ..........
  ..........
} book1, book2;
```

C
```
struct book
{ ..........
  ..........
  ..........
};
struct book book1, book2;
```

D
```
typedef struct
{ ..........
  ..........
  ..........
} BOOK;
BOOK book1, book2;
```

In **A** and **B** forms the required *structure variables* have been declared *along with the structure definition itself.* In the **C** and **D** forms the required variables have been declared at a later stage, *independently of the structure definition*.

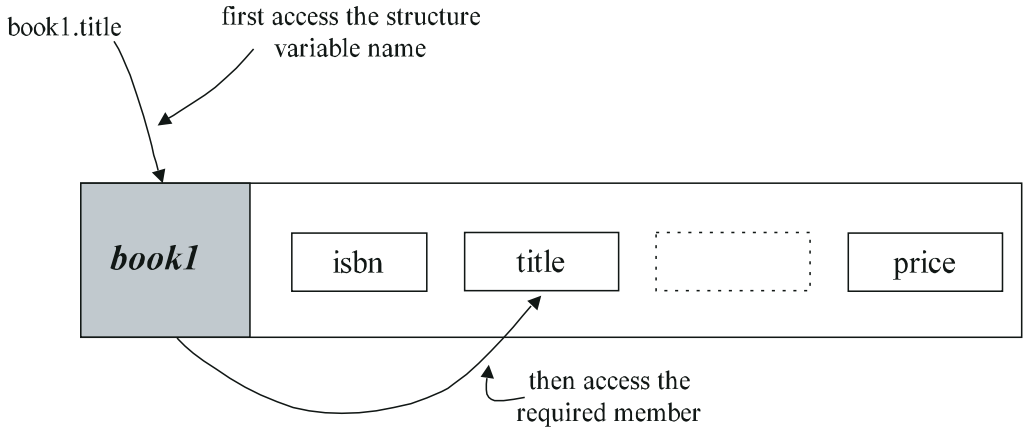
It may be observed that in the **A** form there is no *structure name* or *tag*. The disadvantage with this method is that, one cannot have *structure variables declared* as and when they are required. If required, the *shape of the structure,* i.e. the *structure* has to be

defined once again. The **B** form is alright. However, it is customary to declare the required *structure variables* independently *along with other type of variables* as shown in form **C**. It may be noted that these independent declarations need the `struct` key word to be used during all declarations.

The **D** form uses the key word `typedef` in its definition. As shown in form **D** any *new data type* (like BOOK) may be defined. This *new data type,* i.e. *BOOK* is used to declare the required structure variables like `book1`, `book2`, etc. Here it may be noticed that it is *not necessary* to use the key word `struct` during structure variable declarations and hence is comfortable to use. The reader is advised to practice using either of the forms given in **C** or **D** depending up on his/ her convenience. In fact, if necessary, even form **D** can have a tag.

After defining a structure and declaring structure variables, the structure variables are used just like all other types of variables. As in the case of arrays, with structures also the manipulations are done at individual elements level. It may be noted that an *individual element* in the case of a structure is its *individual member*. In other words, names of the *structure members* are used in computations. As it is known (discussed in Chapter 6) *individual array elements* are accessed using *index numbers* or *subscripts*. *Individual members* of a structure are *accessed* using an operator called the *dot (.) operator* or the *period operator.* This *dot operator (.)* along with the *structure variable name* and the *member name* is used to access any required individual element as shown in the following figure.

For example, the *'title'* of **`book1`** is accessed *by first accessing* the required *structure variable name* [i.e. **`book1`**] and then accessing the *required member* [i.e. title]. Thus the *title* of the *type book*, **`book1`** is accessed using an expression of the form **`book1.title`**.



Accessing the member of a structure variable

It should be noted that the *dot (.) operator* is an *unary operator* and has the *highest precedence* among all operators. The dot expressions are evaluated from left to right.

10.1 DEFINING A STRUCTURE, DECLARING, STRUCTURE VARIABLES, ASSIGNING VALUES TO STRUCTURE MEMBERS AND PRINTING OUT STRUCTURE MEMBER VALUES

The Method

In this example, a structure called *student* is defined. This structure is assumed to have four *members* or *elements*. Let these four members be *university seat number* **(usn)**, *name*, *date-of-birth* and *marks*. Before actually defining a structure, it is necessary to decide about the *size* and/or *type* of the values that are expected to be stored in its *member variable names*. Let the size of the *usn* be 10 *characters*, size of the *name* be 20 *characters*, *date-of-birth* be 10 *characters* and *marks* be of *integer* type. The required structure may be *defined* either within the `main( )` or before the `main( )`. In this example the structure has been defined within the `main( )`.

A *structure variable* **`student1`** is declared in the beginning of the `main( )`, *immediately after* the structure definition. Values to the different members of structure variables may be either *initialized directly* or *assigned* by inputting the values using `scanf( )` and/or `gets( )` functions, as in the case of other types of variables. In this example the necessary values to the structure members have been assigned *interactively* using the above mentioned functions.

The values assigned to the different members of the structure `student1` are displayed or printed out by first *accessing each member* using the *dot (.)* operator and then using the `printf( )` function.

C-Tips

In this example, the keyword `struct` has been used both to *define* a *structure* as well as to *declare* a *structure variable*. Values to the `char` type of members of the structure are input using `gets( )` *function.* Values to the `int` type of members are input using `scanf( )` function. Values of the different members of the structure are output using the `printf( )` statement. The mechanism used to access the different members of the structure is similar to the one shown in the previous page.

The Program

```
/* Defining a structure, declaring a structure variable, reading and
printing its members */

#include<stdio.h>
#include<conio.h>
```

```
main()
{
   struct student                  /* Defining the structure */
   {
      char name[20];
      char usn[10];
      char birth_date[20];
      int marks;
   };

   struct student student1;    /* Declaring a structure variable */

   clrscr();

   /* Accepting the student details */
   printf("\n\t\tEnter the student name : ");
   gets(student1.name);
   printf("%s",student1.name);

   printf("\n\t\tEnter the University Seat Number:  ");
   gets(student1.usn);
   printf("%s",student1.usn);

   printf("\n\t\tEnter the Date of Birth : ");
   gets(student1.birth_date);
   printf("%s",student1.birth_date);

   printf("\n\t\tEnter the marks: ");
   scanf("%d",&student1.marks);
   printf("%d",student1.marks);

   /* Printing the student details */
   printf("\n\t\t————————————————————");
   printf("\n\t\tStudent's Details :\n");
   printf("\t\t ————————————————————\n");
```

```
   printf("\n\t\tName                          : %s",student1.name);
   printf("\n\t\tUniversity Seat Number        : %s",student1.usn);
   printf("\n\t\tDate of Birth                 : %s",
            student1.birth_date);
   printf("\n\t\tMarks                         : %d",student1.marks);

}
```

10.2 ARRAY OF STRUCTURE VARIABLES: LISTING NAMES OF STUDENTS WHO HAVE SCORED MORE THAN 60% OF TOTAL MARKS IN THREE SUBJECTS USING STRUCTURE VARIABLES

The Method

A `student` structure having the university seat number (`usn`), `name`, `sub1`, `sub2`, `sub3`, `total` and `percent` as its *members* is first defined. Marks scored in individual subjects are stored in `sub1`, `sub2` *and* `sub3`. As we have to consider many number of students, a *group of structures* has to be used. Obviously, this *group of structures* will be nothing but an *array of structure variables* of the type `student` structure. In this example, the required structure has been defined before the `main( )` using the `typedef` mechanism. An array of structures (say, `s[ ]`) is declared along with other local variables, as usual, inside the `main( )`.

The values for the different members of all the structure variables are input *interactively* using the `scanf( )` and `gets( )` functions. As usual, the input values are output as and when they are input.

In order to determine the *total* marks and the *percentage* of marks, all the *subject fields* of a structure variable `s[i]` are accessed using *dot (.) operator expressions* of the form

```
s[i].total = s[i].sub1 + s[i].sub2 + s[i].sub3;
s[i].percent = s[i].total/3.0;
```

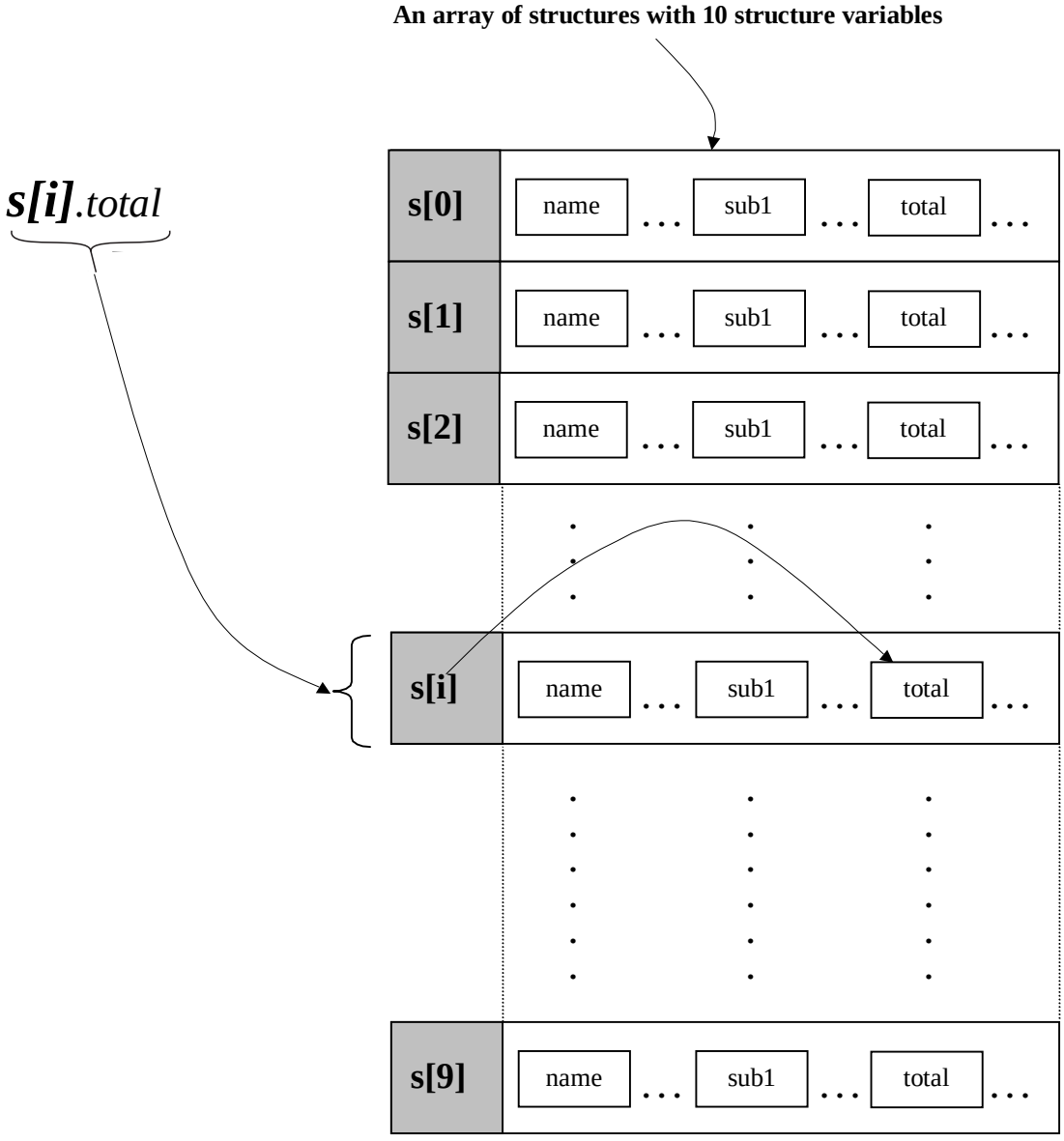
} for all values of `i`

The mechanism of *accessing a structure member* of a structure variable within an array is shown in the figure on page 252. Obviously, the above mentioned type of statements are *repeatedly executed* and the required result is printed out under the control of a `for` loop control structure.

C-Tips

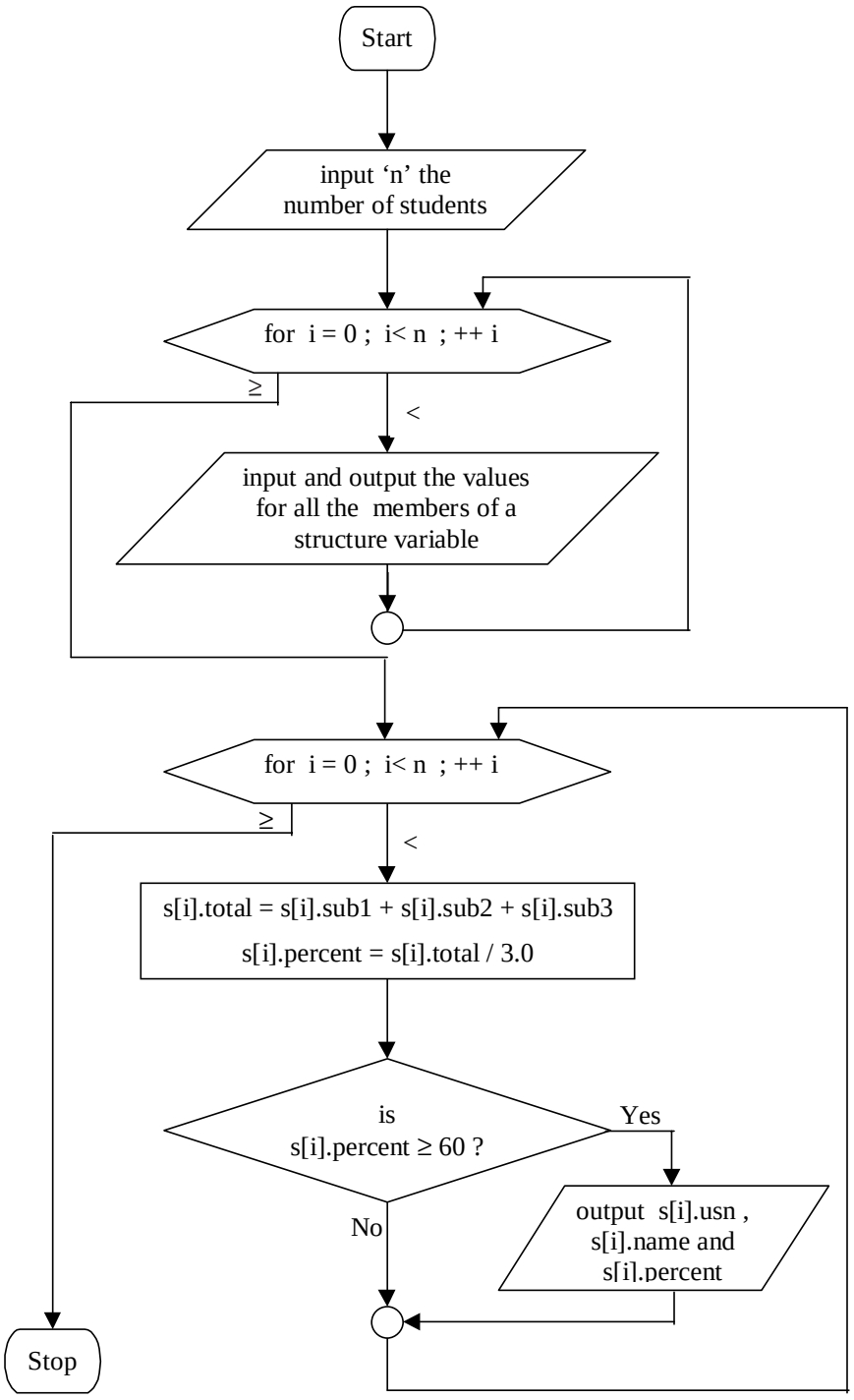
In this example, the `student` *structure* has been defined before the `main( )`. Individual elements of an array, of whatever the type of elements it is made up of, are *accessed* by

using one or more *subscripts*. Since we are dealing with an one-dimensional student array use of only one subscript in this example is sufficient. As such the subscript **`i`** has been used to access individual structure variables in the array **`s[]`**. The individual member of a specific structure variable, say *total*, is accessed using the *dot (.) operator* by writing an expression of the form ***s[i]****.total.*



Accessing the member of a structure variable within an array of structures.

Flowchart


Start
input 'n' the number of students
for i = 0 ; i< n ; ++ i
≥
<
input and output the values for all the members of a structure variable
for i = 0 ; i< n ; ++ i
≥
<
s[i].total = s[i].sub1 + s[i].sub2 + s[i].sub3
s[i].percent = s[i].total / 3.0
is s[i].percent ≥ 60 ?
Yes
No
output s[i].usn , s[i].name and s[i].percent
Stop

The Program

```
/* Program to list the names of students who have scored more than 60%
of total marks in three subjects using structure variables */

#include<stdio.h>
#include<conio.h>

struct student                    /* Defining the structure */
{
   char name[30];
   char usn[10];
   int sub1;
   int sub2;
   int sub3;
   int total;
   float percent;
};

main()
{
   int i,n;
   struct student s[20];    /* Declaring an array of structures */

   clrscr();
   printf("\t\tEnter the number of students: ");
   scanf("%d",&n);
   printf("%d",n);

   for(i=0;i<n;++i)
   {
      getchar();            /* to accept the newline character */

      /* Accepting the students details */
      printf("\n\n\t\tEnter the student name : ");
      gets(s[i].name);
```

```
      puts(s[i].name);

      printf("\n\t\tEnter the student University Seat Number:  ");
      gets(s[i].usn);
      puts(s[i].usn);

      printf("\n\t\tEnter the marks of subject1: ");
      scanf("%d",&s[i].sub1);
      printf("%d",s[i].sub1);
      printf("\n\t\tEnter the marks of subject2: ");
      scanf("%d",&s[i].sub2);
      printf("%d",s[i].sub2);
      printf("\n\t\tEnter the marks of subject3: ");
      scanf("%d",&s[i].sub3);
      printf("%d",s[i].sub3);
   }
      /* printing the students name,USN and percentage who have
         secured more than 60% */
      printf("\n\n\t\tList of students Name ,  University Seat
         Number and Percentage\n\t\twho have scored more than 60%
         marks\n");
      printf("\n\t\t————————————————————————————————");
      printf("\n\t\tName\t\t\tRollnumber\tPercentage\n");
      printf("\t\t————————————————————————————————\n");
      for(i=0;i<n;++i)
      {
         /* calculating the total and percentage */
         s[i].total=s[i].sub1+s[i].sub2+s[i].sub3;
         s[i].percent= s[i].total / 3.0;

         if(s[i].percent>=60)
            printf("\t\t%-20s\t%s\t\t%.2f\n",s[i].name
                 ,s[i].usn,s[i].percent);
      }

}
```

10.3 SUM OF TWO COMPLEX NUMBERS–PASSING STRUCTURE VARIABLE TO FUNCTION

A *complex number* will be of the form ($x \pm i\, y$) where x is called the *real part* and y is called the *imaginary part*. These real and imaginary parts could be either *integers* or *floats*. In mathematics, complex numbers are just another *type of numbers* like integers, reals etc. However, in many computer languages including C there is no *fundamental data* type available to handle these types of numbers directly. One of the methods generally employed to handle complex numbers in C is to use *structures*. The *shape* of the *structure of a complex number* will be as follows:

```
typedef struct
{
     float part_real;
     float part_img;
} COMPLEX;
```

This structure has only *two members*. The first member represents the *real part* and the second member represents the *imaginary part*.

The Method

In order to *sum* any two complex numbers (or to carry out any operations on complex numbers) first we have to have a *new data type* to handle complex numbers. Let this *new data type* be `COMPLEX`. This *new data type* will have the shape as shown in the previous paragraph.

The necessary structure is defined either before the ***`main( )`*** or inside the ***`main( )`*** function. In this example the required structure has been defined before the ***`main( )`***, just like global variables, and three complex numbers **`a`**, **`b`** and **`c`** have been declared inside the ***`main( )`***, as usual, using a declaration statement of the form

```
COMPLEX  a, b, c;
```

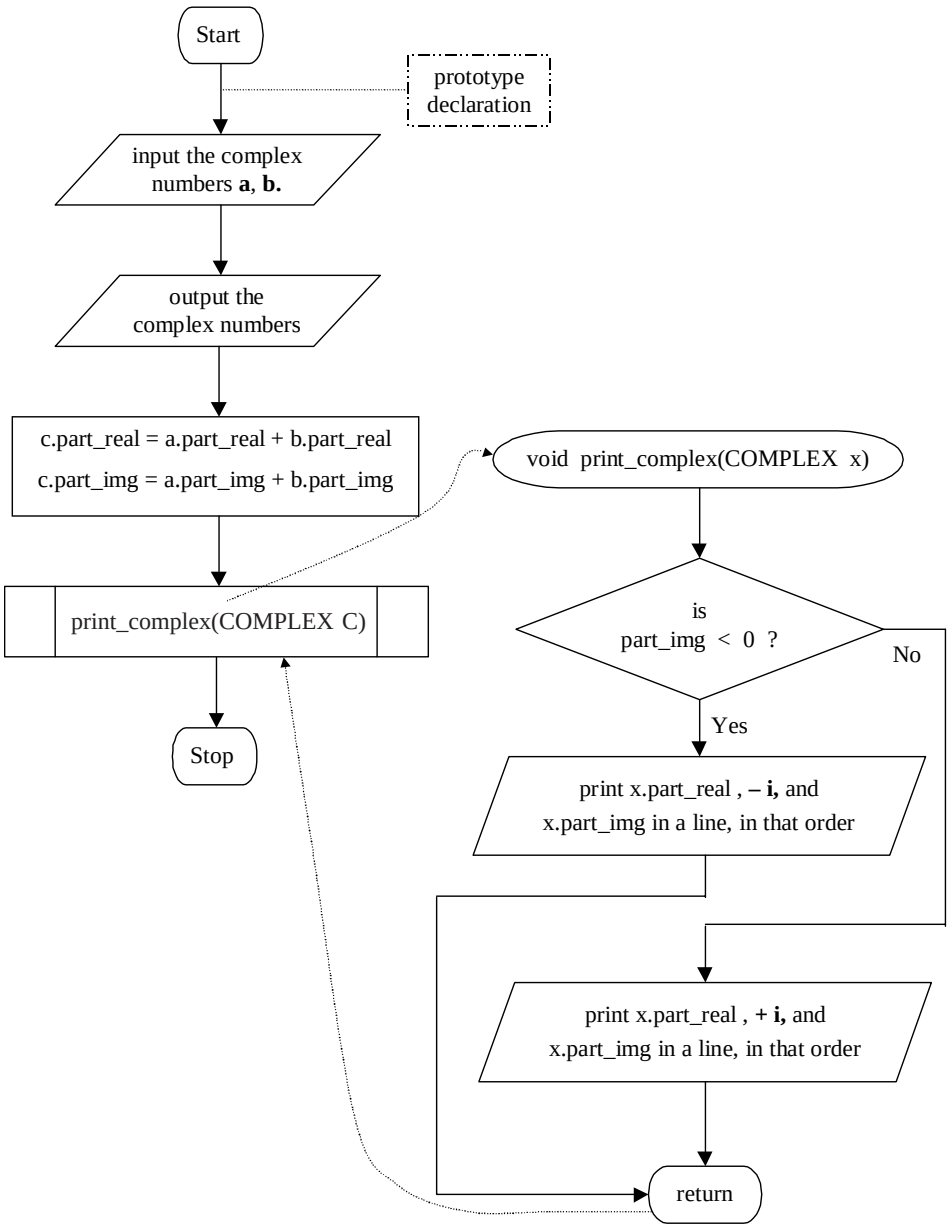
The sum of two or more complex numbers is obtained by finding the *sum* of *real parts* of all complex numbers and the *sum* of *imaginary parts* of all the complex numbers separately. After obtaining the sum of the *individual members*, the sum number is printed out. The output should appear like ***`part_real ± i part_img`***. There is no direct method available for this type of output. As such a complex number print *function* called ***`print_complex( )`*** has been developed and used for the necessary output operation.

C-Tips

As already mentioned *individual members* of a structure are accessed using the *dot (.) operator*. A *function* called ***`print_complex( )`*** has been developed to *print the sum*, which is also a complex number. Therefore, *a structure has to be transmitted to the function*.

Like arrays, structures are also transmitted by *implication*. In other words, when *structures* are used as *actual arguments* in a *calling function*, *address* of the structure is *passed on automatically* to the called function by the C compiler. Also, as usual, the *function prototype* has to be used in the `main( )` function. As the `print_complex( )` function is not returning any value, it has been type declared as `void`.

Flowchart



The Program

```
/* To compute the sum of two complex numbers - passing a structure to
   a function */

#include<stdio.h>
#include<conio.h>
#include<math.h>

typedef struct                          /* Defining a structure  */
{
   float part_real;
   float part_img;
}  COMPLEX;                       /* COMPLEX is the new data type */

main()
{

      COMPLEX a,b,c;           /* Declaring structure variables */
      void print_complex( COMPLEX x);     /* Function prototype */
      clrscr();

      printf("Enter first complex number:  ");
                              /* Accepting first complex number */
      scanf("%f %f",&a.part_real,&a.part_img);
      /* Printing the first complex number by calling
         print_complex()*/
      printf("\nComplex number1 : ");
      print_complex(a);
                             /* Accepting second complex number */
      printf("\nEnter second complex number:  ");
      scanf("%f %f",&b.part_real,&b.part_img);
      /* Printing the second complex number by calling
         print_complex()*/
   printf("\nComplex number2 = ");
   print_complex(b);
```

```
      c.part_real=a.part_real+b.part_real; /* Computing the sum */
      c.part_img=a.part_img+b.part_img;
                        /* Printing the resultant complex number */
      printf("\nAddition of two complex numbers = ");

   print_complex(c);       /* Calling print_complex function() */

}

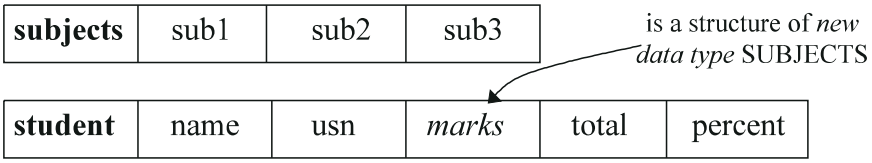
/* Function to print the  complex number  */
void print_complex(COMPLEX x)
{
      if (x.part_img < 0 )
         printf("%.2f – i %.2f\n",x.part_real, fabs(x.part_img));
      else
         printf("%.2f + i %.2f\n",x.part_real,x.part_img);
      return;
 }
```

10.4 ARRAY OF STRUCTURE VARIABLES WITH A STRUCTURE AS MEMBER OF ANOTHER STRUCTURE

A *structure* can be used *as a member* of another *structure*. Such a *member structure* can be referred to as a *substructure*. Obviously, both the *structure* as well as its *substructure* have to be defined before they are used. A *substructure* must be defined before its *main structure* is defined. These definitions can appear either inside the beginning of a `main( )` function or before the `main ( )`.

The Method

Here, one of the examples discussed earlier (in Section 10.2) is considered once again. In this example the three subjects `sub1`, `sub2` and `sub3` which were treated as *individual members* in Section 10.2 have been taken as the *members* of a *substructure* called the *subjects*. The *shape* of the substructure *subjects* and the structure *student* will, now, be as shown in the next page.

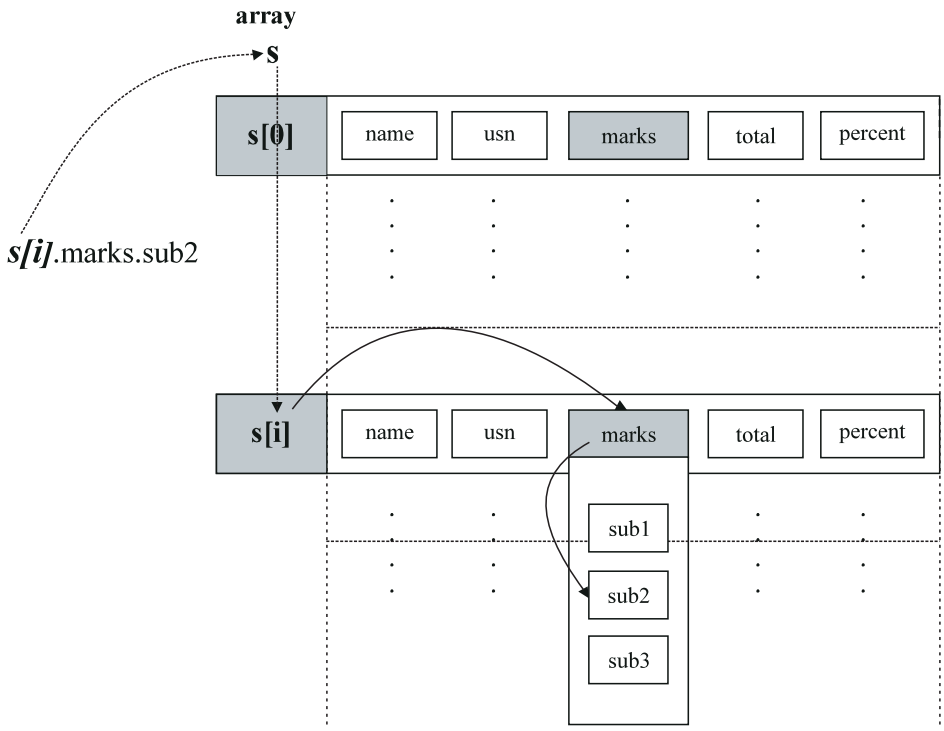


The required *substructure* and its *main structure* have been defined using the ***typedef*** and before the ***main( )***. It may be observed that *marks* is a *substructure* of the *new data type* SUBJECTS. This *new data type* declaration is actually made within the *student* structure definition.

As in Example 10.2, here also an *array of structure variables* is processed. First, necessary declarations of the structures and local variables are made. Then all the *structure members* are given values *interactively*. The *total* marks and the *percentage* of marks are computed using the relations of the form

```
s[i] .total = s[i].marks.sub1 + s[i].marks.sub2 + s[i].marks.sub3;
s[i]. percent = s[i].total / 3.0;
```

The computed results are output using the ***printf( )*** function. The *mechanism* of accessing a substructure member within an *array of structures* is shown in the following figure.



The Program

```
/*Structure within a structure (an array of structure variables)*/

#include<stdio.h>
#include<conio.h>

/* Defining structure subjects*/
typedef struct
   {
    int sub1;
    int sub2;
    int sub3;
   }SUBJECTS;

typedef struct                /* Defining the structure student */
   {
    char name[100];
    char usn[10];
    SUBJECTS marks;         /* Member marks is of structure type */
    int total;
    float percent;
   }STUDENT;
main()
{
    int i,n;
    STUDENT s[100];        /* Declaring array of structure type */
    clrscr();

    printf("\tEnter the number of students: ");
    scanf("%d",&n);
    printf("%d",n);

    for(i=0;i<n;++i)
    {
```

```
        getchar();              /* To absorb the newline character */
                                  /* Accepting the student details */
        printf("\n\n\tEnter the student name: ");
        gets(s[i].name);
        printf("%s",s[i].name);

        printf("\n\tEnter the student University Seat Number: ");
        gets(s[i].usn);
        printf("%s",s[i].usn);

        printf("\n\tEnter the marks of subject1: ");
        scanf("%d",&s[i].marks.sub1);
        printf("%d",s[i].marks.sub1);
        printf("\n\tEnter the marks of subject2: ");
        scanf("%d" ,&s[i].marks.sub2);
        printf("%d",s[i].marks.sub2);
        printf("\n\tEnter the marks of subject3: ");
        scanf("%d",&s[i].marks.sub3);
        printf("%d",s[i].marks.sub3);
                                          /* Computing the total */
        s[i].total = s[i].marks.sub1 + s[i].marks.sub2
                      + s[i].marks.sub3;
        s[i].percent = s[i].total / 3.0;

    }

    printf("\n\n\tStudent Details :\n\n");
    printf("\t————————————————————————  ——————————\n");
    printf("\tName\t\t   USN\t\tSubj1 \tSubj2 \tSubj3 \tTotal\t
              Percent \n");
    printf("\t———————————————————————————————————\n");
    for(i=0;i<n;++i)
    {
        /* Printing the students details */
        printf("\t%-20s",s[i].name);
```

```
        printf("%s",s[i].usn);
        printf("\t\t%d",s[i].marks.sub1);
        printf("\t%d",s[i].marks.sub2);
        printf("\t%d",s[i].marks.sub3);
        printf("\t%d",s[i].total);
        printf("\t%.2f\n",s[i].percent);
    }

}
```

Note: *Arrays* could be the members of a *structure*. For example, the three subjects `sub1`, `sub2` and `sub3` which were considered as individual members of a structure in Section 10.2 and as members of a substructure in Section 10.4 can be taken as the elements of an array `subjects[ ]`. This array can be then used as a member of the structure `student` as shown below.

```
struct  student
{
     char   name[30];
     char   usn[10];
     int    subjetcs[3];
     int    total;
     float  percent;
};
```

A program similar to the one's discussed in Section 10.2 and 10.4 can be written using the above mentioned *shape of the structure*.

EXERCISES

1. Define an *employee structure* having the employee name, employee number, experience (in years) and salary as its members. Develop a C program to print the names and numbers of all the employees who have more than 3 years of experience and salary less than Rs. 5000/-.
2. Define a *student structure* having the name, usn (University Seat Number), marks in five subjects, total and percentage of marks as its members. Marks of all the subjects are to be stored in an array, say marks. Develop a C program to list the names of all the students who have failed.

[A student is declared as failed if he/she has scored less than 40 in any of the subjects and less than 45% in aggregate].

3. Define a *book structure* having title of the book, ISBN, author, price and month and year of publication as its members. Use a *substructure* to store the month and year of publication information. Develop a C program to accept a date (in the form of month and year) and list out all the book titles (along with price and ISBN) published during that date.

Index

`#define` 12
`#include` 7
`%` 15
`%` operator 46
`%+d` 99
`%c` 58
`%d` 7
`%ld` 15
`%ld` 38
`&&` 25
`\0` 136
`\n` 7
`\t` 9
`||` 25
evaluating section 70
repeated execution 31
modulo 2 division 93
`++i` 34
`<conio.h>` 7
`<math.h>` 13
`<stdio.h>` 6, 7
`<string.h>` 137
`==` operator 46
abnormal termination 117
absolute value 30
actual arguments 171, 172
actual parameters 171
address 221
address of the structure 257
address operator 226
address passing 187
advantages 2
algorithm 1
aliases 181
arguments 171
array declaration 77
array name 76
array of structure variables arrays 76
arrays of structures 247
ascending order 132
ASCII 135
attributes 246
`auto` variables 171
average 84

biggest element 80
`bin` directory 211

binary search 123
body of a function 214
`break` 50
bubble sort 95
by selection 128

compiler 211
call by address 234
call by value 172, 194
call-by-reference 218
called function 168, 181
calling function 167, 187
case 63, 67
case values 62
characters 159
characters and string handling 135
`clrscr( )` 7
co-efficients 27
coefficients 98
comment statements 7
common subscript 115
complex number 256
computed goto 4
computer programs 166
concatenating two strings 147
consecutive memory locations 77
consonant 151
cos(x) 61
counter 31
counting 159, 160, 163
current search space 125, 194

data type 221
debugging 166
decision making 65
decision making 27
`default` block 64
defining a structure 248
dereferencing operator 222
descending order 95
discriminant 27
divide and conquer 5
divide and conquer strategy 4, 167
`do…while` 4, 31
`do…while` block 59
dot (.) operator 248, 249
dot expressions 248
dummy arguments 171
dynamic memory management 223

EBCIDIC 135
`else` 19
`else` block 19
end-of- file (EOF) 136
entities 244
even elements 92
exchanging 8
`exit( )` 117
exit(1) 117
exponential series: 61

factorial 35
FALSE Block 19
fast varying subscript 101
Fibonacci sequence 40
file 211
flag 64, 121
`flag` character 99
flow charts 2
flowchart symbols 2
`for` 31
for control structure 36
`for` loop control structure 56
`for, while` 4
formal arguments 171, 174
format 15
format specifier 58, 226
Fortran 5
function prototype 168, 169, 191
function body 169
function definition 168
function header 169, 191
function name 168
functional programming 167
functions 167

GCD 42
`gets( )` 136
global variables 214
goto 116
group of data items 76
group of structures 251

header files 7
high 124

if 4
`if…. else` 18, 20, 21
`if…..else` ladder 67
imaginary part 28
implication 194
increment operator 34
independent functions 211
indirection operator 222
`infix` form 69
initial search space 124
input function 77
insert 134
integer division 15
integer division 10
invoked 168
iteratively 31

key 121
keyboard 136

LCM 42
leap year 25
least significant digit 46
leaving the loop prematurely 121
length of a string 137
line terminating character 161
linear search 120
lines 163
link 168
local to their functions 181
local variables 171, 214
logical AND 25
logical expression 25, 32
logical OR operator 25
`long` 38
loop control structures 31
loop control variable 33, 40
low 124
lowercase alphabets 151

main functions 5
main programs 5
matrix 101
matrix 76
maximum 82
mean 84
memory location 221
Menu 72
middle element 125
minimum 82, 130
modular programming 167
modules 167, 211
modulo division 15
multipliability 207
multivalued variables 181
multi-way decision making 62, 73

names 156
natural numbers 31
nested `for` loops 115
nested ifs 20
new data type 248, 260
norm 220
norm of a matrix 110
normal termination 117
normally 50
null character 135
NULL pointer 223
numeric address 221
numeric codes 135

odd 93
odd elements 92
one-dimensional array 76, 184
operator `==` 20
output function 77

palindrome 46, 138
parameters 171
parameters passing 218
PASCAL 5
pass 130
period operator 248
pictorial representations 2
pointer 221
pointer expressions 227
pointer to pointer 227
pointer variable 228
polynomial 98
preprocessor directives 7, 211
prime number 18
prime number identification procedure 53
principle diagonal 107, 197
`printf( )` 6
procedural programming 166
procedures 167
product matrix 114
program 1

quadratic equation 27, 65
qualifying type of 234

read 101
read in column-wise 101
read in row-wise 101
read-in 70
real part 28
relationship diagrams 245
remainder 15
required accuracy 55
result of searching 190
`return` 168
reverse of a given string 141
Reversing a Number 46
Roots are complex 66
roots are complex and distinct 27
Roots are real and equal 66

scalars 202
`scanf( )` 65
scope of global variables 215
search key 120
search portion 124
search range 129
second half, first half 123
selection Sort 128
sequential search 120
shape of the structure 263
shorthand assignment 34
Simple Calculator 69
simple `if` statement 19
sin (*x*) 55, 56
sine series 55
sine series expansion 177
single subscript 77
size of each memory location *77*
size of the array 136
sorted arrays 123
sorting 95, 156
sorting algorithms 17
specifier 15
`sqrt()` 13
square matrix 108
standard deviation 89
static variables 215
status 117
stored row wise 198
`strcpy()` 145, 148
strings 135
structure name 246
structure tag 246
structure variables 247
structures 244
structures with arrays 247
subproblems 166
subprogram 4, 166
subscript variables 79
substructure 259
sum of a column 202
sum of a row 202
swapping 8
`switch` 4, 50
symbolic constant 12

tab 9
target data type 225
termination under error condition 117
the * operator 222
the & operator 222
trace 107
trace of a square matrix 197
transpose of the given matrix 112
traversal 96, 130
true 23
TRUE block 19
two roots will be equal 27
two subscripts 77
Two way decision 18
two-dimensional 77
type cast 84
unary operator 248
unique names 17
unit's digit 47
unsigned integers 221
uppercase alphabets 151
user defined function 169
user's response. 58

variance 89
vector 76
void 169
vowel 151, 154

`while` 31
`while` loop 32
`while` loop control structure 42
words 160